



ABAP RESTful Application Programming Model

Generated on: 2026-07-06 18:31:40 GMT+0000

ABAP Cloud | ABAP Cross-Product

Public

Original content: https://help.sap.com/docs/ABAP_Cloud/f055b8bf582d4f34b91da667bc1fcce6?locale=en-US&state=PRODUCTION&version=sap_cross_product_abap

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

Extending RAP Business Objects

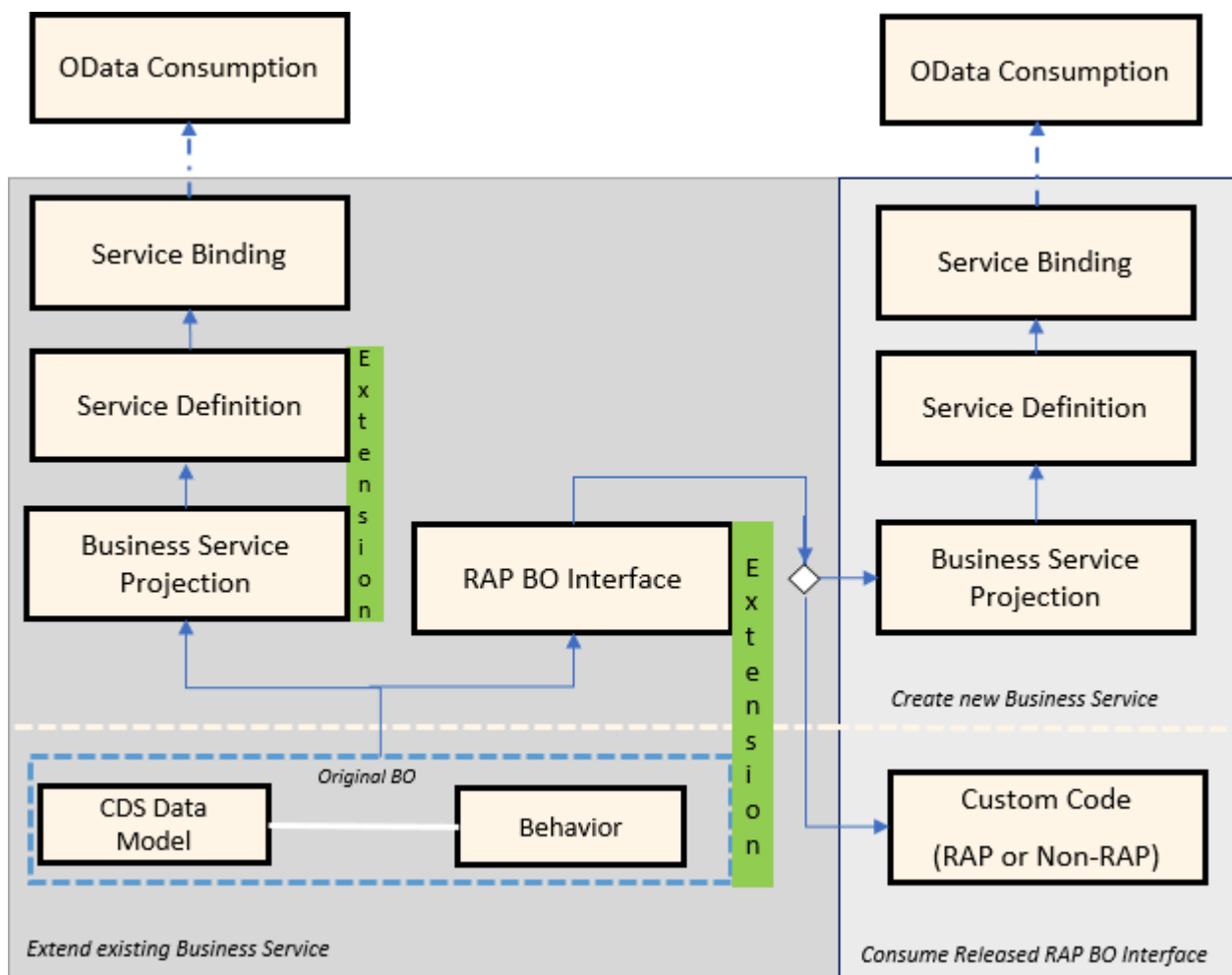
RAP Extensibility Process and Personas

RAP Extensibility offers the possibility to develop semantically rich, upgrade-safe, and lifecycle-stable extensions for RAP business objects on each layer of the RAP stack. An original business object developed by a BO provider is extended with additional functionality to extend the functional scope of the original RAP BO. Using well-defined extension points enabled in the original business object, an extension provider can then extend the original data model and behavior in accordance with the business requirements or create their own business service based on a RAP BO interface.

The overall extensibility process consists of two main parts: The extensibility-enablement in which a RAP BO is prepared and technically enabled for an extensibility use case. This is done by the extensibility-enabler. Then the original RAP BO is extended with data model, behavior, or node extensions by the extension provider.

- **Extensibility-Enabler:** The extensibility-enabler is a developer who technically enables an existing RAP BO and defines the available extensibility options for the extension provider. You can find the relevant information for this persona in the chapter [RAP Extensibility-Enablement](#).
- **Extension Provider:** The extension provider is a developer who extends an original RAP BO with data model or behavior extensions, as well as node extensions. You can find the relevant information for this persona in the chapter [RAP Extensions](#).

RAP Extensibility Big Picture



Generally, the RAP extensibility architecture is based on an **Opt-in** approach to provide fine-granular control over which parts of a business object can be extended. This means that every possible extension point must be defined explicitly in the corresponding development objects in the data model and the behavior definition and implementation. Once enabled, every BO layer can be extended with one or several extensions from extension provider side. In addition, the extensibility architecture ensures a separation of concerns between original BO elements and extensions to limit technical dependencies to public and well-defined extension points. This guarantees lifecycle-stability and upgrade-safety that is also checked and technically enforced by the RAP framework.

Extension providers can extend RAP BOs and their services on-stack or develop and expose their own services based on released RAP BO interfaces.

Extensibility Use Cases

Extensibility Use Cases

Extensibility Use Case	Extensibility Persona	Background Information
Data Model Extension: Build full-stack data model extensions by adding new fields and associations including corresponding behavior characteristics and authorization control.	Extensibility-Enabler: Adds annotations and extension include structures to the original RAP BO to enable data model extensions.	- For more information about enabling full-stack data extensibility, see Extensibility-Enablement for CDS Data Model Extensions. - For an implementation example, see Enabling Data Model Extensions.
	Extension Provider: Extends the original RAP BO with new fields or associations including field characteristics depending on the options defined by the extensibility-enabler.	- For more information about how to develop data model extensions, see CDS Data Model Extensions. - For an implementation example, see Develop Data Model Extensions.
Behavior and Field-Related Behavior Extensions: Build additional behavior like new validations, determinations, or actions including dynamic feature control and other field-related behavior.	Extensibility-Enabler: Enables data model extensibility and behavior extensibility on the original RAP BO.	- For more information about enabling your BO for behavior extensions, see Extensibility-Enablement for Behavior Extensions. - For an implementation example, see Enabling Non-Standard Behavior and Field-Related Behavior and Enabling Standard Behavior Extensions.
	Extension Provider: Extends the original RAP BO with new validations, determinations, or actions depending on the options defined by the extensibility-enabler.	- For more information about how to develop different behavior extensions, see Behavior Extensions. - For an implementation example, see Develop Behavior Extensions.

Extensibility Use Case	Extensibility Persona	Background Information
Node Extensibility: Build additional BO nodes with own behavior and data model with node extensibility.	Extensibility-Enabler: Enables node extensibility on the original RAP BO.	- For more information about node extensibility enabling, see Extensibility-Enablement for Node Extensibility. - For an implementation example, see Enabling Node Extensions.
	Extension Provider: Extend the original BO with new nodes that have their own data model and behavior.	For more information about how to develop node extension, see Node Extensions.

[About RAP Extensibility](#)

This topic gives you an overview of the RAP extensibility architecture, extension development artifacts and all available extensibility features.

[RAP Extensibility-Enablement](#)

This section explains the concepts of how to enable a custom RAP BO for extension development.

[RAP Extensions](#)

This section explains the concepts of how to extend an original RAP business object with data model extensions, behavior extensions and node extensions.

About RAP Extensibility

This topic gives you an overview of the RAP extensibility architecture, extension development artifacts and all available extensibility features.

Extensibility enables application consumers to adapt business requirements and processes of existing applications using RAP extensions. The extensibility options in RAP are designed for lifecycle-stability and a separation of concerns between the original BO and its extensions. Extensions themselves are self-contained and are always RAP managed for both managed and unmanaged BOs.

By default, a RAP BO is not extensible and requires **extensibility enablement** on part of the original BO provider to allow extensions at dedicated extension points. The extension points can be controlled on a fine-granular level to give full control over which parts of a BO can be extended with what kind of extension.

Once a RAP BO has been enabled for extensibility, an extension provider can add different types of extensions to enhance the original data model and behavior. Extension providers can only define extension fields and behavior that the original BO is enabled for. Defining other behavior or field types that the original BO isn't enabled for leads to a syntax error in the extension BDEF and the corresponding data model extension objects.

From the perspective of the consumer, extensions are fully integrated into the BO runtime and design time and act as a whole even though extensions are self-contained and can only define behavior specifically for extension elements and operations. They can not redefine any behavior for any elements of the original BO.

For an architecture overview, see [Extensibility Architecture Overview](#).

Relation Between the Original BO and Extensions

Extensions are self-contained and can only refer to elements from the original BO and elements defined in the extension itself. All derived types and method signatures are automatically extended to include the extension elements in the structures and method signatures. An extension inherits the authorizations from the original BO: If an original RAP BO defines `authorization (global, instance)`, this is automatically inherited by the extension where the extension provider can then implement global and instance authorization for elements of the extension.

❖ Example

If a BO interface has several extensions, a `projection A` can reference an element from `BDEF extension B` and `BDEF extension C`, if they both apply the BO interface `I_interface`.

Extension behavior characteristics and operations behave like they do in the original BO. For example, so there's no difference between an original validation or an extension validation at runtime or design time. In both cases, the RAP framework manages the runtime orchestration for defined trigger conditions in the same way.

For an overview of all available extensibility features, see [Extensibility Feature Overview](#).

From the consumer's perspective, extensions are considered an implementation detail of the respective RAP BO. There's no way to differentiate if an element is part of the original BO or an extension. The BO is always consumed as a whole, including all its extensions that apply a particular BO interface. From original BO perspective, an extension acts as external consumer. The RAP BO runtime differentiates in `local mode` for elements of the original BO or extension elements.

❖ Example

If `BDEF extension B` contains an action with the name `sampleAction (features:instance)`, the `BDEF extension C` can not define an action with the same name. Additionally, it's not possible to redefine `SampleAction (features:instance)` in `BDEF extension C` with additional or different behavior.

For more information, see [Behavior Extensions](#).

Extensibility Lifecycle-Stability and Dependencies

The lifecycle-stability of RAP extensions is decoupled from the technical architecture. The stability for extensions is achieved by releasing dedicated development objects as released API (a so-called **Release Contract**) in the respective language version. Depending on the underlying release contract, the respective development objects undergo additional compatibility and consistency checks that ensure lifecycle-stability between dependent objects. They also impose general restrictions on the development object to avoid incompatible changes in the future. The lifecycle-stability of a RAP BO and its extension essentially relies on the following release contracts for extensibility-enablement and consumption:

- **Extend (C0):** The **Extend** release contract with the visibility **Use in Cloud Development** is required as a basis for extensibility-enablement for all development objects in a RAP stack. If you want to release a database table, a CDS view or a behavior definition for the **Extend** release contract, the relevant development object must be enabled for extensibility as a prerequisite. Depending on the development object, additional technical requirements must be met before stable extension points can be defined.
- **Use System-Internally (C1):** The **Use System-Internally (C1)** release contract with the visibility **Use in Cloud Development** is required for all development objects intended for consumption. You can only access objects released for **Use System-Internally (C1)** in your implementations. Accessing unreleased objects results in a syntax error. After releasing a development object for this release contract, restrictions and checks on further changes are imposed. For example, you can not delete elements used in extensions or you can not make additions to the implementation that would result in incompatible changes and possible upgrade issues..

The combination of both release contracts ensures that extension points and extensions remain stable and consistent during upgrades and lifecycle changes.

For more information, see: [Release Contracts \(ABAP Keyword Documentation\)](#)

Extensibility for Released Business Objects

Released business objects can be used for building extensions that fit customer-specific requirements. If an SAP RAP business object is released for consumption and extension, the business object interface is always released with the release contract C1 to ensure stability for customer extensions. Consequently, all extensions built on a released BO interface inherit the C1 release contract and its resulting limitations on future changes. Furthermore, behavior extensions can only refer fields and associations in their implementations belonging to the same namespace and/or software component. Fields from the original BO can only be used in contexts where the field itself is not modified, e.g., as part of a trigger condition.

For more information about how to build extensions for a released business object, see [RAP Extensions](#). For information about how to consume released business objects in your behavior implementation, refer to [Consuming RAP Business Objects and Events](#).

Extension Layering

It is possible to create behavior definition extensions that extend the behavior which is defined in other behavior definition extensions. This enables customers, for instance, to build a behavior extension for a node extension that has been added to an SAP business object by a partner. Other use cases of extension layering are:

- Extending behavior that is defined in a behavior extension (determine actions and field mapping information)
- Adding a node extension to a node extension

Behavior definition extensions may extend behavior that is defined in behavior definition extensions which are located in a different software component. A CO release contract defined for the original business object is automatically inherited by all of its behavior definition extensions.

Basic Procedure

Extending the behavior of a behavior definition extension follows the same principle as extending the behavior of an original business object: The behavior definition extension addresses the entity node of the business object it intends to extend by using the keywords `extend behavior for`. In the case of a behavior extension to a node extension, the name of the extension node needs to be provided. The behavior defined in a behavior definition extension is extended in the same way as behavior defined in an original business object. A node extension for an existing node extension is defined analogously to a node extension for a node of an original business object.

Once a behavior definition extension for an entity node has been created, it is merged internally with all behavior that is defined for that node in the original business object. So, although extension layering enables the extension of behavior that is defined in other behavior definition extensions, a behavior definition extension always extends the original behavior definition.

Extension layering provides a way to modularize the behavior of entity nodes while preventing dependencies between the involved artifacts. This facilitates the move of behavior from extensions back to the original business object, which may be required when e.g. a feature toggle is removed.

[Extensibility Feature Overview](#)

This topic gives you an overview of all available extensibility features.

[Extensibility Architecture Overview](#)

Extensibility Feature Overview

This topic gives you an overview of all available extensibility features.

Extensibility-Enabler Feature Overview

This table gives you an overview of all available features for extension-enabler in an extensibility scenario. You can find the following information:

- **Feature:** This column lists the available features.
- **Available for Implementation Types:** Not all implementation types can be enabled for all extensibility options. This column lists for which implementation types a feature is available.
- **Further Information:** This column provides links to further conceptual information or implementation examples.

Feature	Available for Implementation Type	Further Information
Enable the original RAP BO for node extensibility.	• Managed	• About extensibility-enablement for extension nodes: Extensibility-Enablement for Node Extensibility • Example for node extensibility-enablement: Enabling Node Extensions
Enable the original data model for data model extensions.	• Managed • Unmanaged	• About extensibility-enablement for data model extensions: Extensibility-Enablement for CDS Data Model Extensions • Example for data model extensibility-enablement: Enabling Data Model Extensions
Enable extensibility for nonstandard and field-related behavior like actions and functions or dynamic feature control.	• Managed • Unmanaged	• About extensibility-enablement for behavior: Extensibility-Enablement for Behavior Extensions • Example for nonstandard and field-related behavior extensibility-enablement:: Enabling Non-Standard Behavior and Field-Related Behavior The extensibility-enablement for data model extensions is a prerequisite for nonstandard and field-related behavior. For more information, see Extensibility-Enablement for CDS Data Model Extensions
Enable extensibility for standard behavior like determinations and validations, additional or unmanaged save.	• Managed	• About extensibility-enablement for behavior: Extensibility-Enablement for Behavior Extensions

Feature	Available for Implementation Type	Further Information
		• Example for standard and unmanaged save extensibility-enablement:: Enabling Standard Behavior Extensions

Extension-Provider Feature Overview

This table gives you an overview of all available features for extension-providers in an extensibility scenario. You can find the following information:

- **Feature:** This column lists the available features.
- **Required Extensibility-Enablement:** All extensibility features depend on the enabled extensibility options in the original RAP BO. This column lists the required extensibility-enablement for the respective feature.
- **Further Information:** This column provides links to further conceptual information or implementation examples.

Feature	Extensibility Scenario	Required Extensibility-Enablement	Further Information
Extend field mapping for extensions fields in your BDEF extension.	• Field Extensibility	The field mapping in the original RAP BO must be defined as extensible. For more information, see Extensibility-Enablement for Behavior Extensions.	• Creating Extension Fields: CDS Data Model Extensions
Add new extension fields to an original RAP BO data model.	• Field Extensibility	The data model must be enabled for data model extension to allow adding new extension fields to the original data model. For more information, see Extensibility-Enablement for CDS Data Model Extensions	• Creating new extension fields: CDS Data Model Extensions • Creating new behavior for the extensions fields: Behavior Extensions and Develop Behavior Extensions
Add new extension actions or extension functions.	• Behavior Extensibility	Adding new functions and actions requires enabling the base BDEF with extensible and defining at least one node as extensible. For more information, see Extensibility-Enablement for Behavior Extensions	• About actions and functions: Nonstandard Operations • Creating new behavior extensions: Develop Behavior Extensions.
Add new extension determinations and validations with their respective trigger conditions in a BDEF extension.	• Behavior Extensibility	Adding new determinations and validations requires explicit enablement in the original behavior definition. Only behavior and trigger conditions specified in the BDEF header can be added in a behavior extension.	• About determinations and validations: Determinations and Validations. • Creating new extension validations or

Feature	Extensibility Scenario	Required Extensibility-Enablement	Further Information
		For more information, see Extensibility-Enablement for Behavior Extensions	determinations: Develop Behavior Extensions .
Add extension validations and determinations to an extension of the determine action Prepare.	Behavior Extensibility	The original RAP BO must be draft-enabled. The draft determine action Prepare must be defined as extensible. For more information, see Extensibility-Enablement for Behavior Extensions.	- Add determinations and validations to Prepare example: Develop Behavior Extensions. - About determine actions: Determine Actions. - State Messages : State Messages.
Add new extension determine actions.	Behavior Extensibility	The original RAP BO must be draft-enabled and enabled for behavior extensions. For more information, see Extensibility-Enablement for Behavior Extensions.	About determine actions: Determine Actions.
Define and implement an additional save in your extension. i Note The cleanup can only be defined and implemented in node extensions, not in behavior extensions of existing nodes.	Behavior Extensibility	The original BDEF must be enabled for behavior extensions and additional save. For more information, see Extensibility-Enablement for Behavior Extensions	About with additional save: Defining Unmanaged Save in the Behavior Definition
Add a business event in your behavior extension.	Behavior Extensibility	The original RAP BO must be enabled for additional save. For more information, see Extensibility-Enablement for Behavior Extensions.	About RAP business events: Business Events.
Add a new extension node to an original RAP BO data model.	Node Extensibility	The original RAP BO must be enabled for node extensibility. For more information, see Extensibility-Enablement for Node Extensibility.	- Node Extensions: Node Extensions. - Develop new extension nodes: Develop Node Extensions.

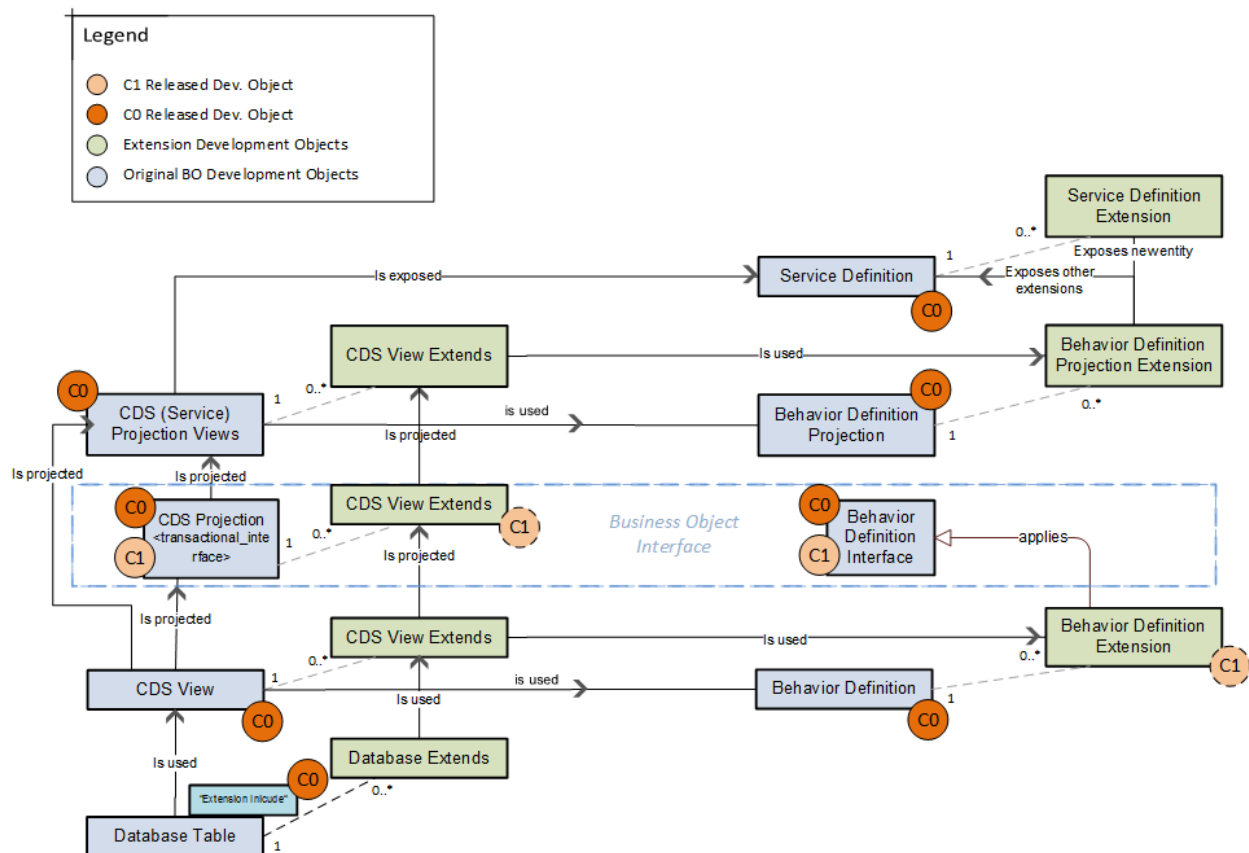
Extensibility Architecture Overview

Depending on which part of business object is extended, different development artifacts are used to store the extensions for the data model, the behavior, or the service definition. The required extension artifacts depend on the individual use case. For an overview of extension options and use cases, refer to [RAP Extensions](#).

The RAP extension artifacts store all extensions to the different parts of a business object from the data model extension to the service definition in a full-stack extension scenario. The individual artifacts ensure that only previously defined and stable extension points can be extended.

The following graphic gives you an overview of all extension development objects throughout the stack of a RAP BO. The depicted architecture is recommended if you want to make use of release contracts in your BO. Each layer in the stack can be extended with the respective extension artifacts:

- **Database Tables:** Database tables are extended with Database Extends. A C0 extension include structure is included in draft and active database tables. These extension include structures can then be extended with the actual database extends from a BO extension provider. This indirect EXTENSION architecture ensures that the C0 release contract requirements are decoupled from the original database tables.
- **CDS Views:** CDS Views are extended with CDS View Extends. The corresponding original artifact for CDS View Extends is released for C0 if release contracts are used and for released RAP BOs.
- **Behavior Definitions:** BDEFs are extended with Behavior Definition Extensions. The BDEF extensions for the base layer can include additional behavior extensions for nodes of the original BO or define new behavior for extension nodes. The BDEF extension on projection layer can only expose behavior defined on the base layer or define new behavior that doesn't require an implementation class.
- **Service Definition Extension:** A service definition extension is required to expose an extension node in an OData Service.



RAP Extensibility-Enablement

This section explains the concepts of how to enable a custom RAP BO for extension development.

About Extensibility-Enablement

Since RAP extensibility is based on an opt-in approach, all possible extension points in the stack have to be explicitly enabled for a RAP BO.

That means that all development objects, like database tables, CDS views, behavior definitions, and behavior projections must be explicitly enabled for extensibility before the respective RAP BO can be extended by an extension provider. The RAP BO provider has full control over which parts of a RAP BO are extensible. The only exceptions are BO interfaces that are required for behavior and field-related behavior extensions. Other than the keyword `extensible`, BO interfaces don't require any specific enablement: They merely serve as a consolidated consumption view of the base BO and always reflect the extensibility-enablement level of the base layer. When a BO interface is used in an extension, the extension behavior of the nodes from the base RAP BDEF is added implicitly to the BO interface. For more information about the implicit extension of BO interfaces, see [Business Object Interface](#).

Extensibility-Enablement for Different Implementation Types with or without Draft Capabilities

The specific enablement steps depend on the implementation type of a RAP BO and whether a BO has draft capabilities or not.

The enablement steps are similar for managed and unmanaged RAP BOs:

1. **Data Model Extensibility Enablement:** Define the technical extension points and extension possibilities in the respective artifacts, e.g. prepare a database table for extension by adding an `extension include` in active and draft tables and defining the technical properties by means of annotation of the extensions, like which extensions are possible and what is the maximum number of available fields. For details, see [Extensibility-Enablement for CDS Data Model Extensions](#).
2. **Behavior Extensibility Enablement:** Define the BDEF as `extensible` and specify the behavior extension scope available for extension providers, like `validations on save`, `determinations on modify`, `determinations on save` and `additional save`. For details, see [Extensibility-Enablement for Behavior Extensions](#).
3. **Node Extensibility Enablement:** Define the annotation `@AbapCatalog.extensibility.allowNewCompositions` in all CDS view entities throughout the stack to enable adding new `toParent/Child` compositions. Define the base BDEF as `extensible` to allow adding the behavior for the extension node. Additionally, define the service definition as `extensible`. If the service definition is to be released with the `C0` contract, add a provider contract to the service definition which specifies the requirements for CDS views that may be added to the service. These requirements of course also apply to CDS views that are already part of the service. For details, see [Extensibility-Enablement for Node Extensibility](#).
4. **Release for Extend (C0) Release Contract:** This release contract ensures the stability of the extension point with additional checks for the extension provider and RAP BO consumer. This release contract inhibits incompatible changes to extension points. Note that it's mandatory to use the highest available `strict` level, if you want to release your development objects for this release contract.

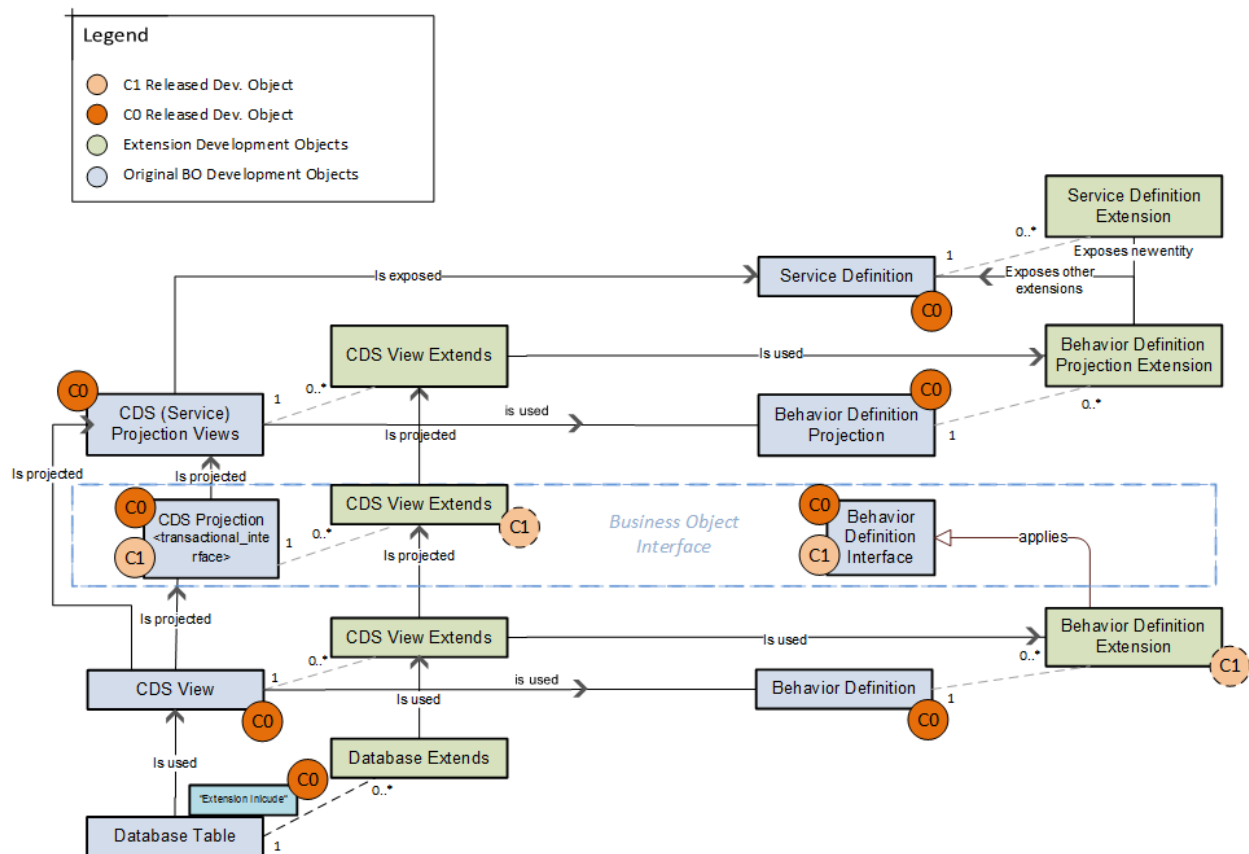
i Note

You can't subsequently draft-enable a RAP BO interface or CDS projection after they have been released for the `Extend (C0)` release contract.

5. **Use System-Internally (C1) (Consumption Enablement):** The `C1` release contract is required for all development objects intended for consumption or use in the behavior implementation. With `ABAP for Cloud Development`, development objects released for `Use System-Internally (C1)` can only consume objects released for the same release contract and visibility in your implementations. If you try to access non-released objects, a syntax error occurs. With the release for `Use System-Internally (C1)`, restrictions and checks on further changes are imposed to ensure end-to-end stability for consumers: For example, you can't delete elements or you can't make additions to the implementation that would result in incompatible changes and possible upgrade issues.

Development Objects with Release Contracts Overview

The following diagram gives an overview of app development artifacts and their release contract. If you want to use release contracts in your BO this is the recommended architecture that is also valid for released RAP BOs:



For a detailed description, see [Extensibility Architecture Overview](#).

Extensibility-Enablement Target Overview

The following overview shows the extensibility-enablement throughout the stack of a generic RAP BO. The depicted extensibility-enablement model enables the RAP BO for field and behavior extensions, as well as node extensibility.

- **Data Model:** The data model consisting of database tables and CDS Views is extensibility-enabled with the required technical properties and released for the C0 release contract. This enables field extensibility and allows the addition of extension fields to the database tables. It also allows the extension of the CDS data model with fields and associations from extension provider side.

For more information about the extensibility-enablement for the data model for extension fields, see [Extensibility-Enablement for CDS Data Model Extensions](#).

- **Behavior:** The behavior definition is defined as `extensible` and the general extensibility scope that specifies which behavior extensions are possible is defined. The individual BO entities are defined as `extensible` to define to which BO entities the behavior extension scope can be applied to. The BDEF is released for the C0 release contract to ensure the stability of the defined extension points.

For more information about extensibility-enablement for the behavior of a RAP BO, see [Extensibility-Enablement for Behavior Extensions](#).

- **Service Extensions:** For adding extension BO entities, the service definition is defined as `extensible` (annotation `@AbapCatalog.extensibility.extensible: true`), it has a specified provider contract and it is released for the

C0 release contract. Node extensibility also requires extensibility-enablement throughout the stack to be able to add the required associations at the appropriate extension points.

For more information about extensibility-enablement for node extensibility, see [Extensibility-Enablement for Node Extensibility](#).

[Learn about Extensibility-Enablement](#)

[Develop Extensibility-Enablement](#)

Learn about Extensibility-Enablement

This section offers you conceptual background about extensibility-enablement for different enablement use cases.

[Extensibility-Enablement for CDS Data Model Extensions](#)

[Extensibility-Enablement for Behavior Extensions](#)

[Extensibility-Enablement for Node Extensibility](#)

Extensibility-Enablement for CDS Data Model Extensions

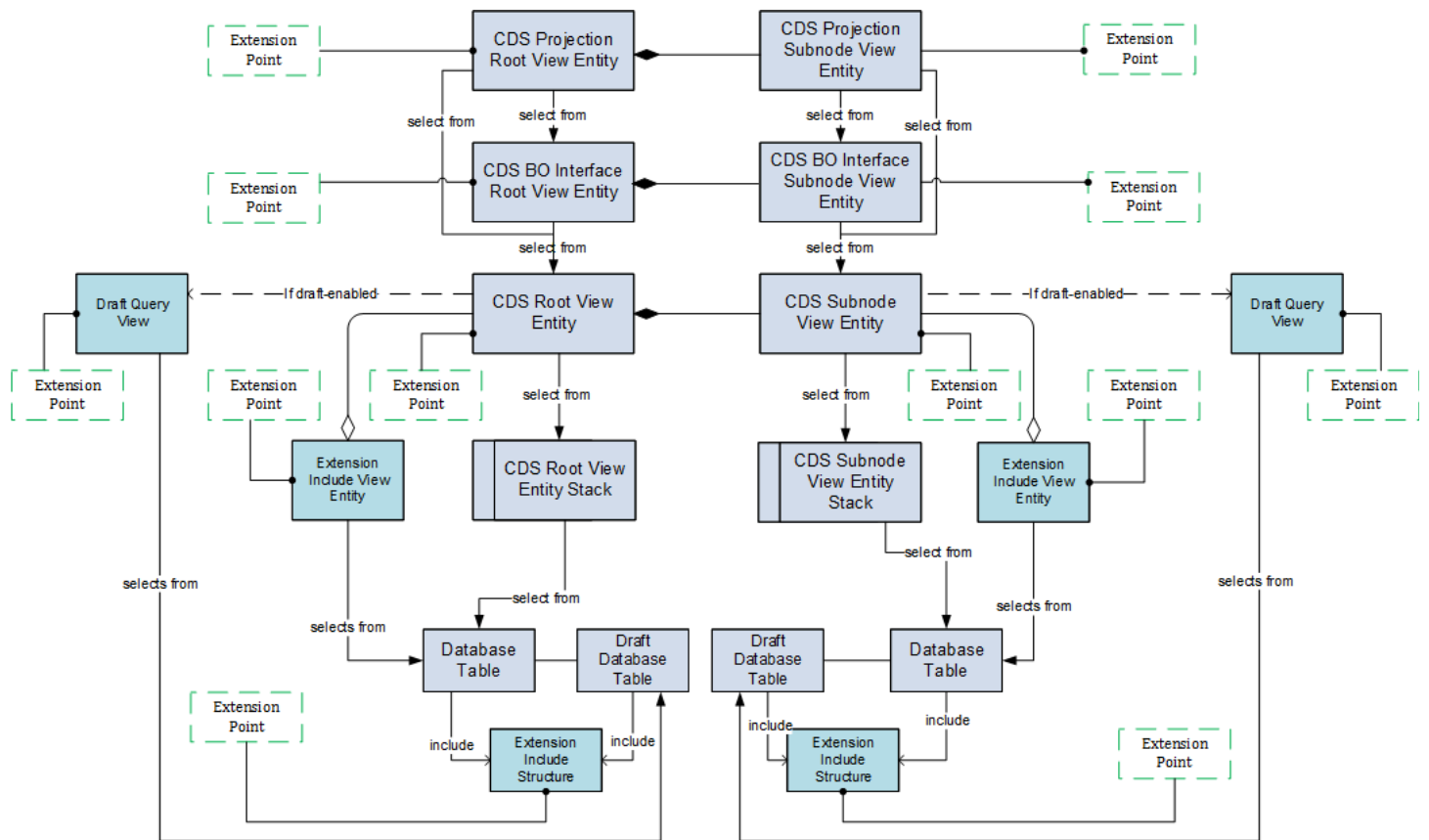
Enabling Extensibility for Field and Association Extensions

i Note

A prerequisite for the extensibility enabling of your CDS data model is the use of CDS View Entities.

You can enable your data model, so that extension providers can add additional fields and associations to the CDS data model and expose them in a business service. The enabling approach focuses on separating the dedicated extension points from the modeling details of the respective tables and CDS views: To achieve this separation of concerns, the data model extensibility is based on extension include structures for the database tables and extension include CDS views for the CDS data model.

The following graphic shows a draft-enabled root node with one subnode including all development objects relevant for the extensibility-enabling of the CDS data model:



Enabling your data model for extensibility consists of the following parts:

1. [Extensibility-Enabling for Database Tables](#)
2. [Extensibility-Enabling for Change Indicator Includes](#)
3. [Extensibility-Enabling for CDS Views](#)

If an application uses change indicator includes to indicate changed fields (for example if BAPIs are used), the change indicator includes must be extensibility enabled, if the application is extensibility-enabled.

Extensibility-Enabling for Database Tables

Database Table Enablement Architecture

To allow adding extension fields to a RAP BO, the same extension include structure is added to the active and also the draft table in case of a draft-enabled RAP BO. This achieves a separation of concerns between extension elements and the original databases: The extension include structure serves as abstraction layer between the extension fields and the base BO database tables to limit the stability requirements of the Extend C0 release contract to the extension include structure and guarantee full flexibility with regards to enhancing the original database tables later:



Technically, it's possible to extend the database table directly. In this case, the database table itself must be Extend C0 released with visibility Extend in Cloud Development. However, to maintain flexibility and separation of concerns, this is only recommended for simple data models.

In unmanaged RAP BOs, this extension includes are required throughout the legacy ABAP application to be able to persist the extension field values on the database. Make sure to use **MOVE - CORRESPONDING** to transfer the extension field values in your application stack.

Basic Extensibility-Enablement for Extension Type, Suffix, and Quota Enablement

Extensibility-Type-Enablement

The annotation `@AbapCatalog.extension.category` determines if and how a database table or structure can be extended. You can define, for example, if a structure can only be extended with character-type fields.

- **#EXTENSIBLE_ANY**: All kinds of extension types are allowed for the structure or table (e.g. deep types).
- **#EXTENSIBLE_CHARACTER**: Only character-like extension types are allowed.
- **#EXTENSIBLE_CHARACTER_NUMERIC**: Only character and numeric like extension types are allowed.
- **#NOT_EXTENSIBLE**: No extensions are allowed.

Element Suffix-Enablement

The extensibility element suffix is defined with the annotation `@AbapCatalog.extension.fieldSuffix`. This annotation allows you to avoid field and association naming conflicts. All fields and associations included in an extension need to end with the given suffix. The suffix should be unique, but this isn't technically enforced. Note that you can only add one extension-include structure for every BO node.

❖ Example

If you've defined `@AbapCatalog.extension.fieldSuffix: 'ABC'`, all fields and associations included in the extensions must end with the specified suffix like in this case with `SAMPLE_EXT_FIELDABC`. If the field name was defined as `SAMPLE_EXT_FIELD` without the specified suffix, an error will occur when release contracts are used.

Extension-Quota-Enablement

You can use the `@AbapCatalog.extension.quotaMaximumFields` annotation to limit the number of extension fields that can be added by an extension provider. The annotation `@AbapCatalog.extension.quotaMaximumBytes` allows you to define a maximum byte capacity that is reserved for extension providers. The maximum byte capacity needs to be chosen in an adequate proportion to the the maximum number of fields, e.g. `quotaMaximumBytes = quotaMaximumFields * 100`.

Quota Definition for different Consumer Groups

The Quota definition shares are used to assign the reserved field counts / byte capacity to a dedicated consumer group.

i Note

For database tables and structures, a quota share of 50% for both parties is technically enforced.

Release for Stability Contract

After you've defined the possible extension types, as well as extensibility quotas if necessary, you can release the extension include structure for the C0 Extend stability contract.

Parent topic: [Extensibility-Enablement for CDS Data Model Extensions](#)

Next: [Extensibility-Enablement for Change Indicator Includes](#)

Extensibility-Enablement for Change Indicator Includes

If an application uses change indicator includes to indicate changed fields (for example if BAPIs are used), the change indicator includes must be extensibility enabled, if the application is extensibility-enabled.

Change indicator include structures are used to identify which fields have been changed. This information is used, for example, to pass the change information to used BAPIs. Change indicator include structures mirror the fields of the applications, but all fields are of type CHAR1. If an SAP application or a custom application uses a change indicator include for processing changes, the data model of the application and the change indicator include structure must be consistently enabled for extensibility. Otherwise, extension fields aren't processed.

Change indicators include structures that are extensibility-enabled like extension include structures. For more details about how to enable extensibility, see [Extensibility-Enabling for Database Tables](#).

Parent topic: [Extensibility-Enablement for CDS Data Model Extensions](#)

Previous: [Extensibility-Enabling for Database Tables](#)

Next: [Extensibility-Enablement for CDS Views](#)

Extensibility-Enablement for CDS Views

The extensibility-enablement for CDS views is based on an opt-in approach allowing for full control of possible extensions and quantity of extensions. By using annotations, you can define whether the addition of new data sources or compositions is allowed or not.

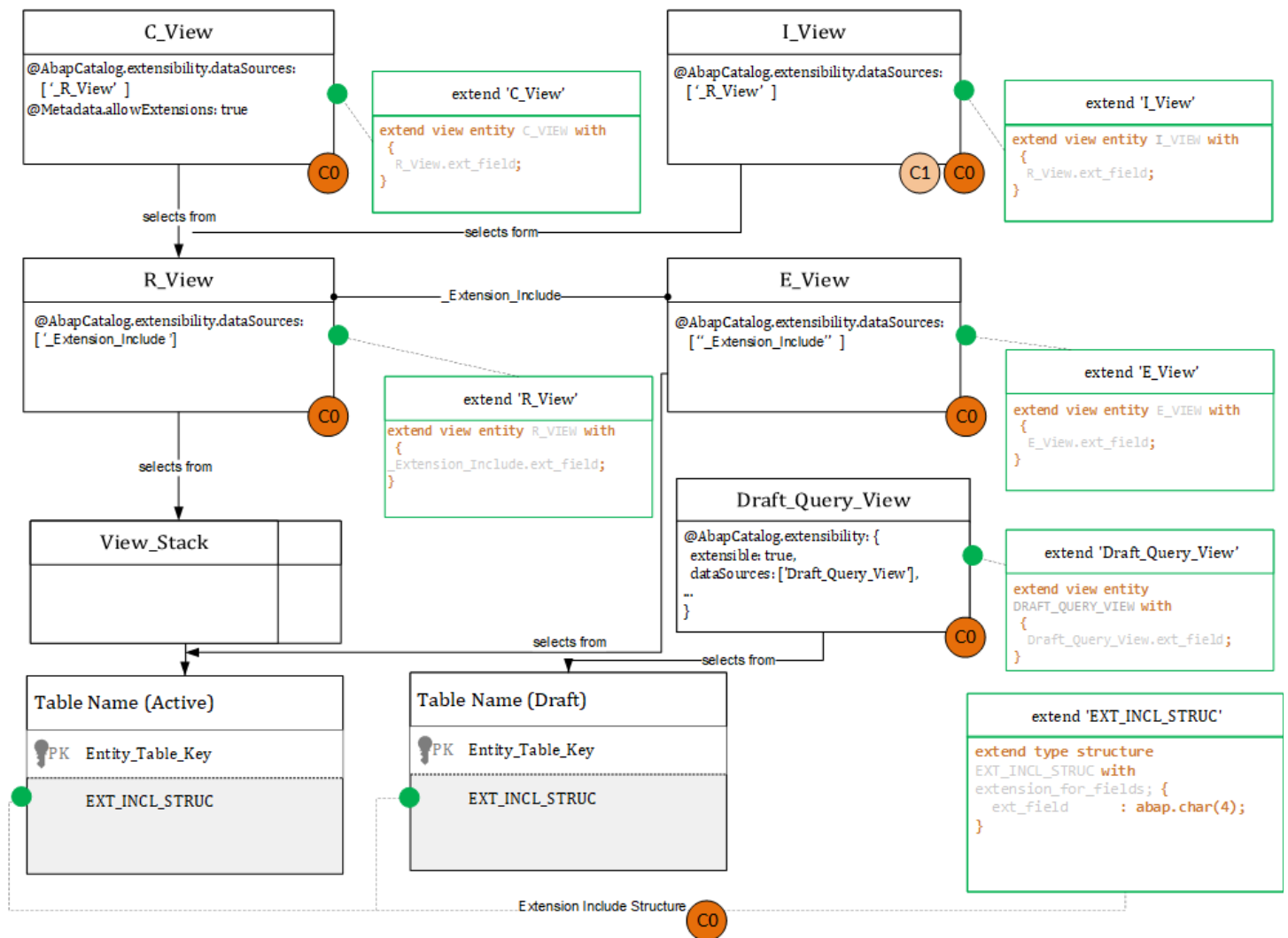
CDS-Enabling Architecture

The following graphic shows the best practice architecture throughout the CDS data model with an example extension field that extends the data model for a RAP BO with draft.

To develop a robust CDS architecture for extensions, an extension include view is used. This limits the constraints resulting from the Extend (C0) release contract to the E_View and R_View, while the CDS View stack below the R_View can still be changed flexibly. The extension include view E_View is associated to the original R_View and selects from the original database bypassing the view stack below the original R_View.

The model contains a CDS view Draft_Query_View, because the BO is draft-enabled. The draft query view is used in the corresponding behavior definition to read the content directly from the draft table. Since the extension include structure is included in both tables, the E_View and the Draft_Query_View must be extended to add the extension field to the data model.

Every CDS view that is later extended with extension fields and associations must be enhanced with the respective annotations before the CDS View can be released for Extend (C0). These CDS views are then extended throughout the stack with extensions fields and associations by the extension provider.



For more information about the syntax, see [ABAP CDS - Extending CDS Views \(ABAP Keyword Documentation\)](#).

Draft Query Views in BOs with Draft

Specifying a draft query view in the behavior definition is mandatory if the BO is draft-enabled and you want to use the Extend (C0) for your RAP BO.

Basic Extensibility-Enablement for the Data Model

The data model needs to be enhanced with extensibility annotations that define the type and scope of possible extensions for extension providers. These annotations are prerequisites if you want to release the CDS views for the Extend (C0) release contract.

Basic Extensibility Enablement

i Note

If the annotation isn't set at all, the CDS View entity is considered extensible by default.

The CDS views need to be defined as extensible to enable any kind of extensions. The annotations `@AbapCatalog.extensibility.extensible` must be set to `true`. Optionally, for CDS Views, the annotation `@AbapCatalog.viewEnhancementCategory` can also be used. For node extensibility, the annotation `@AbapCatalog.extensibility.allowNewCompositions: true` is a mandatory prerequisite.

Displaying Extension Elements on the UI

You need to define the annotation `@Metadata.allowExtensions: true` on the projection layer of your RAP BO, so that the extension elements can be added to UI with metadata extensions.

Suffix Enablement

The CDS Entity needs to define an element suffix, which needs to be used by extension fields and associations. The annotation `@AbapCatalog.extensibility.elementSuffix` allows you to define a suffix. If an extension association doesn't use the Suffix, a syntax error occurs.

Quota Enablement

The CDS Entity needs to define a quota definition, which specifies capacity for extension content. The annotation `@AbapCatalog.extensibility.quota.maximumFields` enables you to define the maximum number of possible extension fields and with the annotation `@AbapCatalog.extensibility.quota.maximumBytes`, you can define a maximum byte capacity that is reserved for extension providers. Both annotations are obligatory if you want to release the datasource for C0. The maximum number of fields should be chosen higher than the number of fields specified on database level in order to consider e.g. denormalized or calculated fields. The maximum byte capacity needs to be chosen in an adequate proportion to the the maximum number of fields, e.g. $\text{maximumBytes} = \text{maximumFields} * 100$.

Stable Data Sources Enablement

It's required to define stable aliases for associations contained in the original CDS view to ensure stability for consumers. The aliases can't be changed after a CDS view has been released for a release contract. You can use the annotation `@AbapCatalog.extensibility.dataSources` to define the respective aliases.

Extension Data Sources and Compositions

An original data source must be explicitly enabled for new associations, new data sources and compositions. If that's not the case, no new data sources or compositions can be introduced to the data model. The annotations `@AbapCatalog.extensibility.allowNewDatasources` allows you to add new data sources. These data sources need to comply with the respective stability requirements.

The annotation `@AbapCatalog.extensibility.allowNewCompositions` allows you to enable the data model for new compositions. This is a mandatory requirement if you want to enable node extensibility. For more information about node extensibility enablement, see [Extensibility-Enablement for Node Extensibility](#).

For more information about what to take into account when introducing extension associations or compositions, see [CDS Data Model Extensions](#).

Release for Release Contract

After you've defined the possible extension possibilities, as well as extensibility quotas if necessary, you can release the CDS Views in the data model for the Extend (C0) release contract.

Field Mapping

Extension fields need to be mapped in an extension of the field mapping. Thus, the field mapping enablement is required for extension fields. For more information, see [Extensibility-Enablement for Behavior Extensions](#)

Parent topic: [Extensibility-Enablement for CDS Data Model Extensions](#)

Previous: [Extensibility-Enablement for Change Indicator Includes](#)

Related Information

[Extensibility Enablement \(ABAP Data Models guide\)](#)

Extensibility-Enablement for Behavior Extensions

Behavior extension-enablement allows an extension provider to enrich the original BO with additional behavior. The following use cases can be covered with behavior extensibility:

- **Field-Related Behavior Extensions:** Extension providers can add additional behavior to extension fields like static or dynamic feature control. This use case only requires basic behavior enablement. This automatically enables extensibility for actions and functions.

i Note

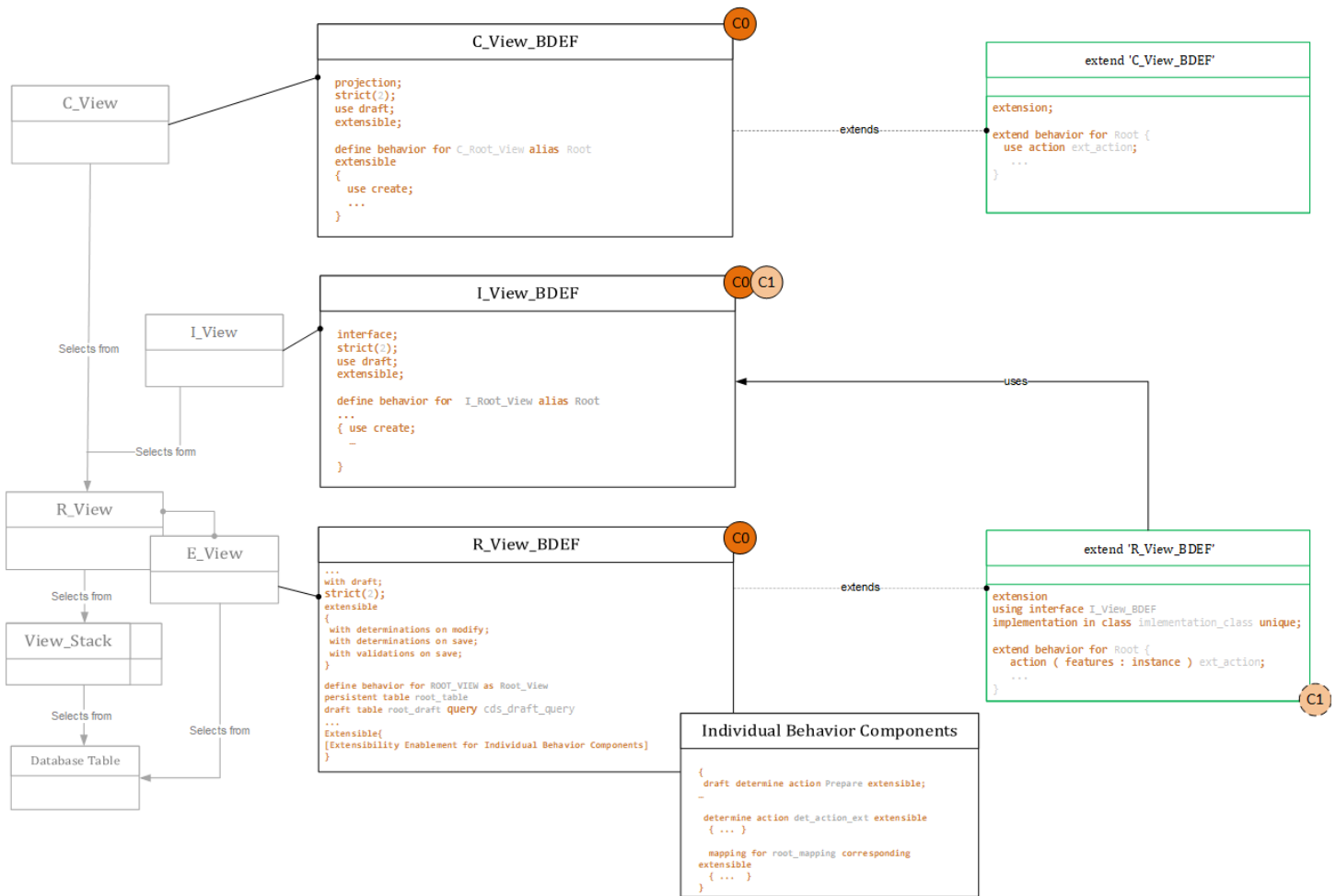
As a prerequisite, the data model of the BO must be extensibility-enabled. For more information, see [Extensibility-Enablement for CDS Data Model Extensions](#).

- **Behavior Extensions:** Extension providers can add additional behavior like validations or determinations. This use case requires basic behavior enablement and extensibility-enablement for behavior extensions. For more information, see [Extensibility-Enablement for CDS Data Model Extensions](#) and [Extensibility-Enablement for Behavior Extensions](#).

Behavior Extension-Enablement Architecture

Offering the possibility for behavior extensibility in a homogeneous and lifecycle-stable way requires extensibility enabling for the corresponding BDEF and the projection BDEFs in which the behavior and extension behavior is defined. Another prerequisite is a BO interface and the use of `strict (2)`. For more information, see [Strict Mode](#) and [Business Object Interface](#).

The opt-in approach allows you fine-granular extensibility enabling on all levels of a RAP BO. The following graphic shows a fully enabled BDEF architecture and uses an action as a behavior extension example to show how an extension is then propagated throughout the entire stack.



Enablement in the Original BDEF

In the BDEF header, you can define whether the BDEF is extensible overall by adding the keyword `extensible` to the header. Furthermore you can specify whether validations and determinations and with which trigger conditions can be added to the entity extension behavior. It's not possible to add determinations or validations with trigger conditions to a BDEF extension, if it's not defined in the BDEF header. This leads to a syntax error in the BDEF extension.

❖ Example

Using the example above, you could add for example a `validation on save` to the `Root` entity node.

On entity behavior definition level, you can define for each individual entity node whether it's extensible or not. The determinations and validations with their trigger conditions defined after `extensible` in the BDEF header apply to all entity nodes defined as `extensible`.

If the original BDEF is draft-enabled and managed, you can define `determine actions` and the `determine action Prepare` as extensible. This is recommended in case you have enabled your BO for validations, since adding validations to the `determine action Prepare` is required for processing state messages. For more information about state messages, see [State Messages](#).

Furthermore, you can define the field mapping as `extensible` which is required for extension fields and their mapping.

For more information about the syntax and the individual options, see [CDS BDL - Extensibility Enabling for RAP BOs \(ABAP Keyword Documentation\)](#).

Enablement in an Abstract BDEF

Deep parameters for non-standard operations (actions, functions, or events) are modeled by using an abstract behavior definition to define the composition hierarchy of the parameter. An abstract BDEF can be enabled for extensibility just like an original BDEF with the keyword `extensible`.

For more information, see [Modeling Parameters for Non-Standard Operations](#).

Enablement in the BO Interface

You can define whether a BDEF interface is extensible by adding `extensible` to the header. The BDEF interface then implicitly inherits the extension properties defined in the original base BDEF, and no further enabling is required on this level.

Enablement in the BDEF Projection

You can define whether a BDEF projection is extensible by adding `extensible` to the header. You can enable each entity node individually as `extensible`, but at least one node must be defined as `extensible`.

An extension provider can then expose extension behavior defined for the base BO in a BDEF projection extension. However, it's not possible to add behavior on the projection level that requires an implementation class.

For more details about the syntax and the individual elements, see [CDS BDL - Extensibility Enabling for CDS Projection BDEFs \(ABAP Keyword Documentation\)](#).

Enablement in a behavior definition extension

You can enable a behavior definition extension for extensibility by adding the keyword `extensible` to the header.

The enablement of a business object for standard behavior extensions (determinations, validations and additional save) can also be done in one of its extensions. Added determine actions and field mapping information can be enabled in an extension by using the keywords `determine action <determine_action_name> extensible` and `extend mapping for <database_table> corresponding extensible` respectively. Ensure that the respective node is enabled for behavior extensibility in its behavior definition as well.

You can enable node extensions defined in the extension by specifying the keyword `extensible` in the definition section of this node. In case you enabled a node extension on the base layer of the behavior extension, you need to do the same on its projection layer.

i Note

Remember that there is no need to release behavior definition extensions with the CO contract. If the original business object has been released with CO, this release contract is automatically inherited by all of its behavior definition extensions. Keep in mind, that extensible behavior definition extensions of course need to meet the specifications of this release contract as well and need to be kept stable.

For more details about the syntax and the individual elements, see [Extensibility Enabling for BDEF Extensions \(ABAP Keyword Documentation\)](#).

Extensibility-Enablement for Node Extensibility

With node extensibility enablement, extension providers can add their own extension node to a data model, including its own data model and behavior.

Node Extensibility Architecture

In this use case, the extension provider wants to add an additional child node with data model and behavior to an original BO and define behavior for that node in a BDEF extension.

To establish the relationship between root node and extension node, the extension provider must introduce a new composition to the data model. Thus, the association target (in this case the root node) must be enabled to allow new compositions with the corresponding annotation as described in [Extensibility-Enablement for CDS Views](#). The BDEF must be enabled to add the extension node, and in the last step, the service definition requires extensibility enabling, since the extension node needs to be exposed for service consumption.

Prerequisite: Extension Node

i Note

As extensibility enabler, you must define a provider contract for a service definition before it can be released with the C0 contract. The provider contract defines the requirements for CDS views that may be added to the respective service. It of course also applies to CDS views that are already part of the service.

i Note

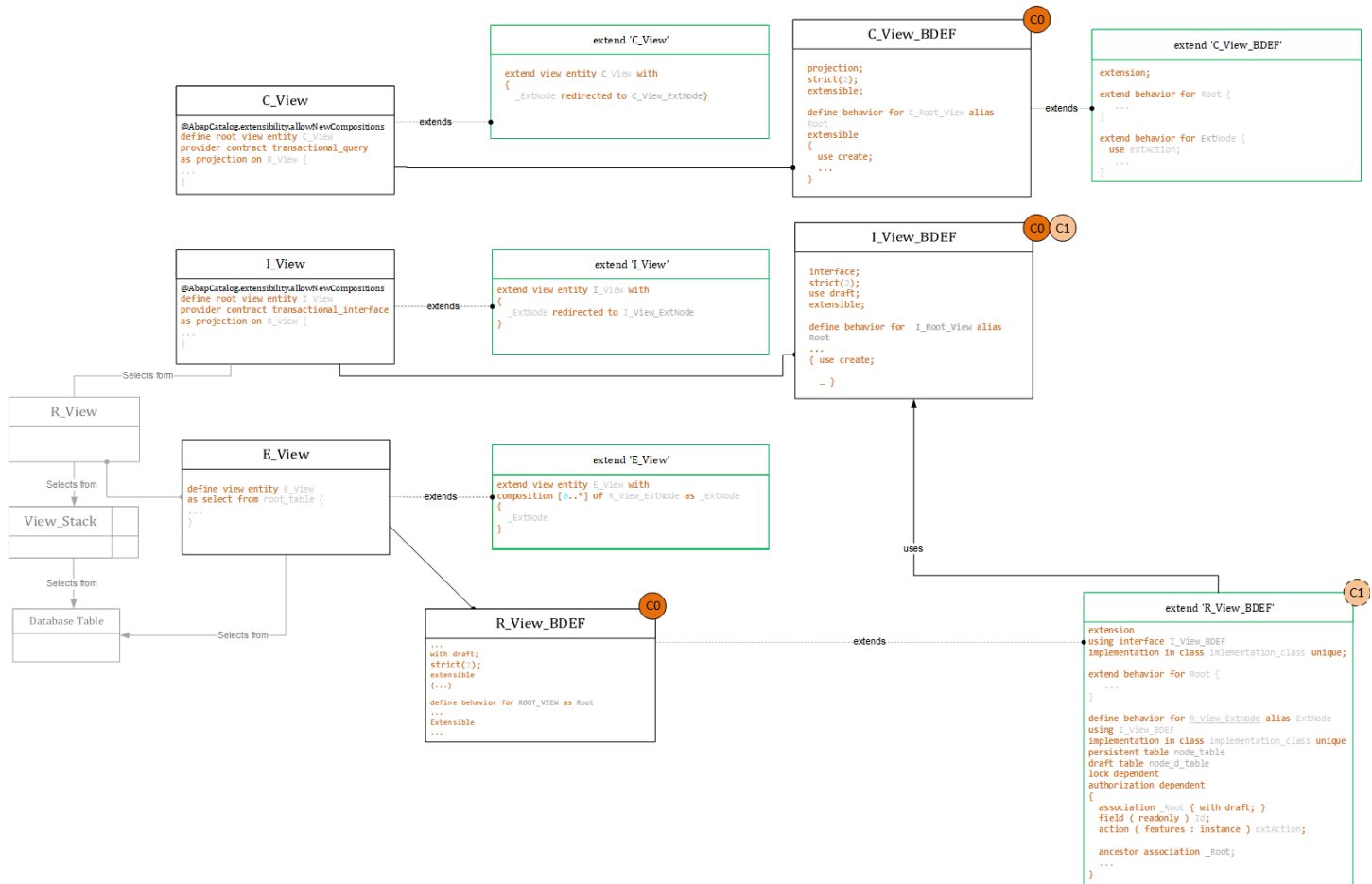
As extension provider, you must use ancestor associations to support locking, authorization, and etag handling later on in the BDEF. For more information about ancestor associations, see [Node Extensions](#).

As a prerequisite, the extension provider has defined an additional node with its own data model and behavior that is to be included in the existing data model. The data model described here is described in more detail in [Node Extensions](#), however the data model of the extension node is not relevant for the enablement itself.

Enabling the Data Model and Behavior

The original data model is enabled for to-parent-associations/compositions by using the annotation `@AbapCatalog.extensibility.allowNewCompositions` in the CDS views `R_View`, `I_View` and `C_View`. This allows the extension provider to create the required composition in a CDS View extension. The `R_View_BDEF` is enabled for extension with the keyword `extensible` in the header, and no further enablement is required on node level. This allows the extension provider to define behavior for the extension node in `extend 'R_View_BDEF'` and reuse the behavior in the BDEF projection `extend 'C_View_BDEF'`

The following graphic shows the enablement throughout the data model:



Enabling the Service Definition

The service definition is enabled by adding the annotation `@AbapCatalog.extensibility.extensible: true` to the header. If the service definition is to be released with the `C0` contract, you need to specify a provider contract for the service definition. The provider contract is defined according to the respective service kind, e.g. `odata_v4_ui`. It ensures that only those CDS views are added to the service that are compatible with the given service kind. The compatibility is of course also checked for CDS views that are already part of the service. The following provider contracts can be specified:

- `ina`
- `odata_v2_ui`
- `odata_v2_webapi`
- `odata_v4_ui`
- `odata_v4_webapi`
- `sql`

Sample Code

```
...
@AbapCatalog.extensibility.extensible: true
define service ExtNodeService
  provider contracts odata_v2_ui, odata_v4_ui {
    expose R_View as Root;
    ...
  }
}
```

Develop Extensibility-Enablement

This section offers how-to information and implementation examples for extensibility enabling a RAP BO.

Reference Scenario

We use an example from the ABAP Flight Reference Scenario to show how to enable a business object for different extensibility use cases. In this example, we work with the business object from the package `/DMO/FLIGHT_REUSE_AGENCY` that can be used for the administration of agency master data like agency names and addresses.

i Note

The following sub packages of the package `/DMO/FLIGHT_REUSE_AGENCY` contain the artifacts to be developed by the extender of the business object as described in chapter [Develop RAP Extensions](#):

- `/DMO/FLIGHT_REUSE_AGENCY_SLOGN`
- `/DMO/FLIGHT_REUSE_AGENCY_CNTRY`
- `/DMO/FLIGHT_REUSE_AGENCY_REV`

For practical reasons, the extension artifacts in these packages are shipped within the same software component as the artifacts of the original business object itself, thus making release contracts obsolete. In a productive environment, the extension artifacts don't belong to the same software component as the original business object, which is why the artifacts of the original business object require release contracts in order to be consumable and extendable from another software component. During the course of the following enablement guide, the artifacts that require release contracts for cross-component access are pointed out respectively. The artifacts of the `/DMO/` reference content themselves, however, are shipped without release contracts for maintenance reasons.

Extensibility Enablement Scenarios

The following guide is structured by the different extensibility use cases and guides you through the enablement processes required respectively.

[Enabling Data Model Extensions](#)

In the following section, you will enable our reference business object `/DMO/R_AgencyTP` for data model extensions that allow extenders to add new elements to the business object.

[Enabling Non-Standard Behavior and Field-Related Behavior](#)

This chapter guides you step-by-step on how to enable a business object for actions and functions as well as for static and dynamic feature control extensions for new fields.

[Enabling Standard Behavior Extensions](#)

To allow the extender to add validations, determinations and additional save implementations, you need to enable these operations in the behavior definition of your business object.

[Enabling Node Extensions](#)

To allow extenders to add their own extension node to the data model, you need to enable node extensions in the original business object.

Enabling Data Model Extensions

In the following section, you will enable our reference business object `/DMO/R_AgencyTP` for data model extensions that allow extenders to add new elements to the business object.

Required Enablement Steps

Data model extensions affect the whole stack of a business object. Hence you will extensibility-enable the database layer as well as every involved layer of the CDS data model. In order to retain flexibility for the original business object and the stability of extensions, you will develop dedicated extension points which decouple the business object from the data model extension artifacts on the respective layers.

For more information and the whole picture of the CDS-enabling architecture, see [Extensibility-Enablement for CDS Data Model Extensions](#).

[Enabling the Database Tables](#)

This chapter guides you through the necessary steps to enable the database tables of our reference business object for extensions to its data model.

[Enabling the CDS Views](#)

This chapter guides you through the necessary steps to enable the CDS views of our reference business object for extensions to its data model.

Enabling the Database Tables

This chapter guides you through the necessary steps to enable the database tables of our reference business object for extensions to its data model.

Required Enablement Steps

In order to enable the database tables for data model extensions, you need to define an extension include structure which can later be used by the extender for structure extends. The extension include structure is then referenced in both the active and in the draft database table. Additionally, you need to define the database tables and the extension include structure as extensible explicitly.

For more information about enabling database tables for extensibility, see [Extensibility-Enabling for Database Tables](#).

Procedure

1. Create a structure called `/DMO/S_EXT_INCL_AGENCY`. This extension include structure serves as an abstraction layer between the original business object and extensions and is added as an include to the database tables in a subsequent step.
2. In the extension include structure, add a dummy field and use the following annotations in the header section to define the enablement scope for the structure:
 - `@AbapCatalog.enhancement.category`: Use the value `#EXTENSIBLE_ANY` to allow any kind of extensions to the extension include structure.
 - `@AbapCatalog.enhancement.fieldSuffix`: Choose the field suffix `ZAG` to avoid field name collisions.
 - `@AbapCatalog.enhancement.quotaMaximumFields`: Choose the number of fields which can be added by the extender of the include structure.
 - `@AbapCatalog.enhancement.quotaMaximumBytes`: Choose the byte capacity reserved for the extender of the include structure.

```
@EndUserText.label : 'Extension Include'
@AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
@AbapCatalog.enhancement.fieldSuffix : 'ZAG'
@AbapCatalog.enhancement.quotaMaximumFields : 350
```

```
@AbapCatalog.enhancement.quotaMaximumBytes : 3500
define structure /dmo/s_ext_incl_agency {

    dummy_field : abap.char(1);

}
```

i Note

We recommended that you add a CO release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

3. In the header of the database tables /DMO/AGENCY and /DMO/AGENCY_D, add the annotation

@AbapCatalog.enhancement.category : #EXTENSIBLE_ANY. It allows any kind of extensions for the database table.

```
...
@AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
define table /dmo/agency {
...

```

4. Include the created extension include structure to the database tables /DMO/AGENCY and /DMO/AGENCY_D.

```
...
    include /dmo/s_ext_incl_agency;

}
```

The resulting database table /DMO/AGENCY looks like this:

```
@EndUserText.label : 'Flight Reference Scenario: Managing Agencies'
@AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/agency {

    key client                : abap.clnt not null;
    key agency_id             : /dmo/agency_id not null;
    name                      : /dmo/agency_name;
    street                   : /dmo/street;
    postal_code              : /dmo/postal_code;
    city                    : /dmo/city;
    country_code             : land1;
    phone_number             : /dmo/phone_number;
    email_address            : /dmo/email_address;
    web_address              : /dmo/web_address;
    attachment               : /dmo/attachment;
    mime_type                : /dmo/mime_type;
    filename                 : /dmo/filename;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_created_by         : abp_creation_user;
    local_created_at         : abp_creation_tstmpl;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_last_changed_by    : abp_locinst_lastchange_user;
    local_last_changed_at    : abp_locinst_lastchange_tstmpl;
    last_changed_at          : abp_lastchange_tstmpl;
    include /dmo/s_ext_incl_agency;

}
```

5. The resulting database table /DMO/AGENCY_D looks like this:

```
@EndUserText.label : 'Draft table for entity /DMO/R_AGENCY'
@AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
```

```

@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/agency_d {

    key mandt          : mandt not null;
    key agencyid       : /dmo/agency_id not null;
    key draftuuid      : sdraft_uuid not null;
    name               : /dmo/agency_name;
    street             : /dmo/street;
    postalcode         : /dmo/postal_code;
    city               : /dmo/city;
    countrycode        : land1;
    phonenumber        : /dmo/phone_number;
    emailaddress       : /dmo/email_address;
    webaddress         : /dmo/web_address;
    attachment         : /dmo/attachment;
    mimetype           : /dmo/mime_type;
    filename           : /dmo/filename;
    localcreatedby     : abp_creation_user;
    localcreatedat     : abp_creation_tstmpl;
    locallastchangedby : abp_locinst_lastchange_user;
    locallastchangedat : abp_locinst_lastchange_tstmpl;
    lastchangedat      : abp_lastchange_tstmpl;
    "%admin"           : include sych_bdl_draft_admin_inc;
    include /dmo/s_ext_incl_agency;

}

```

Result

You have now enabled the database tables for data model extensions. To enable extenders to add new fields, you need to extensibility-enable the CDS data model as well, see chapter [Enabling the CDS Views](#).

Enabling the CDS Views

This chapter guides you through the necessary steps to enable the CDS views of our reference business object for extensions to its data model.

Required Enablement Steps

Analogously to the database level, you need to create an extension include view and define an association to it from the CDS view /DMO/R_AGENCYTP. The extender can then add view extends with the new fields for the extension include view and for the CDS view /DMO/R_AGENCYTP. In the latter one, the extender uses the association to the extension include view in order to extend the CDS view /DMO/R_AGENCYTP with the new fields.

In order to prepare the whole stack of our business object for field extensions, you need to define the CDS views on the base layer as well as on the projection and interface layer as extensible explicitly using CDS annotations.

For more information and the whole picture of the CDS-enabling architecture, see [Extensibility-Enablement for CDS Views](#).

Defining the Extension Include View

Create a CDS view entity with the name /DMO/E_AGENCY which will be used as your extension include view. In the CDS view, select only the key field agency_id from the database table /DMO/AGENCY and provide the alias Agency for the data source. Use the following CDS annotations that define the enablement scope for the extension include view:

- `@AbapCatalog.viewEnhancementCategory: [#PROJECTION_LIST]`: Allows extensions of the select list.
- `@AbapCatalog.extensibility.extensible: true`: Defines the extension include view as extensible.

- `@AbapCatalog.extensibility.elementSuffix: 'ZAG'`: Defines the element suffix to be used in view extends of this extension include view.
- `@AbapCatalog.extensibility.allowNewDatasources: false`: Defines whether associations defined in CDS view extends can be used to address data sources in path expressions. In this example, we do not want any further data sources to be used in the CDS view extends, so we choose the option `false`.
- `@AbapCatalog.extensibility.dataSources: ['Agency']`: Defines that the alias `Agency` can be used in extensions of this CDS view.
- `@AbapCatalog.extensibility.quota.maximumFields: 500`: Defines the maximum number of fields that can be added by extenders.
- `@AbapCatalog.extensibility.quota.maximumBytes: 50000`: Defines the byte capacity reserved for extenders.

The resulting extension include view looks like this:

```
@AbapCatalog.viewEnhancementCategory: [#PROJECTION_LIST]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Agency - Extension Include View'
```

```
@AbapCatalog.extensibility: {
  extensible: true,
  elementSuffix: 'ZAG',
  allowNewDatasources: false,
  dataSources: ['Agency'],
  quota: {
    maximumFields: 500,
    maximumBytes: 50000
  }
}
```

```
define view entity /DMO/E_Agency
as select from /dmo/agency as Agency
{
  key agency_id as AgencyId
}
```

i Note

We recommended that you add a CO release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

Enabling the CDS View /DMO/R_AGENCYTP

1. Define an association to the extension include view `/DMO/E_AGENCY` with the cardinality `[1]`, the association name `_Extension` and the field `Agency ID` as join condition.

```
association [1] to /DMO/E_Agency as _Extension on $projection.AgencyID = _Extension.Agency
```

2. Use CDS annotations that define the enablement scope for the CDS view. Most of the annotation values are identical to the extension include view described above. For the annotation `@AbapCatalog.extensibility.dataSources`, use the association `_Extension`. This enables extenders to use this association in extensions of this CDS view.

The upper section of the resulting CDS view looks like this:

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Agency'
```

```
@Search.searchable: true
```

```
@AbapCatalog.extensibility: {
  extensible: true,
  elementSuffix: 'ZAG',
  allowNewDataSources: false,
  dataSources: ['_Extension'],
  quota: {
    maximumFields: 500,
    maximumBytes: 50000
  }
}
```

```
define root view entity /DM0/R_AgencyTP
  as select from /DM0/I_Agency as Agency
  association [0..1] to I_Country as _Country on $projection.CountryCode = _Country.Country
  association [1] to /DM0/E_Agency as _Extension on $projection.AgencyID = _Extension.AgencyID
{
  key AgencyID,
```

i Note

We recommended that you add a CO release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

Enabling the CDS View /DM0/C_AGENCYTP

Use CDS annotations that define the enablement scope for the CDS view. Most of the annotation values are identical to the views described above. For the annotation `@AbapCatalog.extensibility.dataSources`, use the alias `Agency`. This enables extenders to use this alias in extensions of this CDS view. Additionally, the annotation `@Metadata.allowExtensions: true` is used to enable the addition of metadata extensions for this view.

The upper section of the resulting CDS view looks like this:

```
@EndUserText.label: 'Agency'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Search.searchable: true
@Metadata.allowExtensions: true
@ObjectModel.semanticKey: ['AgencyID']

@AbapCatalog.extensibility: {
  extensible: true,
  dataSources: ['Agency'],
  elementSuffix: 'ZAG',
  quota: {
    maximumFields: 500,
    maximumBytes: 50000
  }
}

define root view entity /DM0/C_AgencyTP
  provider contract transactional_query
  as projection on /DM0/R_AgencyTP as Agency
{
  key AgencyID,
```

i Note

We recommended that you add a CO release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

Enabling the CDS View /DMO/I_AGENCYTP

Use CDS annotations that define the enablement scope for the CDS view. Most of the annotation values are identical to the views described above. For the annotation `@AbapCatalog.extensibility.dataSources`, use the alias `Agency`. This enables extenders to use this alias in extensions of this CDS view.

The upper section of the resulting CDS view looks like this:

```
@Metadata.ignorePropagatedAnnotations: true
@EndUserText.label: 'Agency'
@Search.searchable: true
@AccessControl.authorizationCheck: #NOT_REQUIRED

@AbapCatalog.extensibility: {
  extensible: true,
  dataSources: ['AGENCY'],
  elementSuffix: 'ZAG',
  quota: {
    maximumFields: 500,
    maximumBytes: 50000
  }
}

define root view entity /DMO/I_AgencyTP
  provider contract transactional_interface
  as projection on /DMO/R_AgencyTP as Agency
{
  @ObjectModel.text.element: ['Name']
  key AgencyID,
```

i Note

We recommended that you add a C0 release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

i Note

It is recommended to add a C1 release contract in order to specify that this development object is intended as a stable data source for consumption.

For maintenance reasons, the artifacts of the /DMO/ reference content are not shipped with defined release contracts.

Defining the Draft Query View /DMO/R_AGENCYDRAFT

1. Create a CDS view entity called `/DMO/R_AgencyDraft` which selects the database table `/DMO/AGENCY_D` and provide the alias `Agency` for the data source. Remove all CDS annotations in the header except for `@AccessControl.authorizationCheck: #NOT_REQUIRED` and `@EndUserText.label: 'Draft Query View: Agency'`. The CDS view will be used as the Draft Query View.
2. Use CDS annotations that define the enablement scope for the CDS view. Most of the annotation values are identical to the views described above. For the annotation `@AbapCatalog.extensibility.dataSources`, use the alias `Agency`. This enables extenders to use this alias in extensions of this CDS view.

The resulting CDS view looks like this:

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Draft Query View: Agency'

@AbapCatalog.extensibility: {
  extensible: true,
  dataSources: ['Agency'],
```

```

    elementSuffix: 'ZAG',
    quota: {
      maximumFields: 500,
      maximumBytes: 50000
    }
  }
}

define view entity /DMO/R_AgencyDraft
as select from /dmo/agency_d as Agency
{
  key agencyid                as Agencyid,
  key draftuuid               as Draftuuid,
  name                        as Name,
  street                      as Street,
  postalcode                  as Postalcode,
  city                        as City,
  countrycode                 as Countrycode,
  phonenumber                 as Phonenumber,
  emailaddress                as Emailaddress,
  webaddress                  as Webaddress,
  attachment                  as Attachment,
  mimetype                    as Mimetype,
  filename                    as Filename,
  localcreatedby              as Localcreatedby,
  localcreatedat              as Localcreatedat,
  locallastchangedby          as Locallastchangedby,
  locallastchangedat          as Locallastchangedat,
  lastchangedat               as Lastchangedat,
  draftentitycreationdatetime as Draftentitycreationdatetime,
  draftentitylastchangedatetime as Draftentitylastchangedatetime,
  draftadministrativeatauuid  as Draftadministrativeatauuid,
  draftentityoperationcode    as Draftentityoperationcode,
  hasactiveentity             as Hasactiveentity,
  draftfieldchanges           as Draftfieldchanges
}

```

i Note

We recommended that you add a CO release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

3. In the behavior definition /DMO/R_AGENCYTP, add the reference to the created draft query view.

```

...
define behavior for /DMO/R_AgencyTP alias Agency
persistent table /dmo/agency
draft table /dmo/agency_d query /DMO/R_AgencyDraft
...

```

Result

You have now enabled the business object for data model extensions. Refer to chapter [Develop Data Model Extensions](#) to learn how to add new fields to the database tables and the CDS data model.

Enabling Non-Standard Behavior and Field-Related Behavior

This chapter guides you step-by-step on how to enable a business object for actions and functions as well as for static and dynamic feature control extensions for new fields.

Required enablement steps

In order to enable a business object for non-standard behavior and field-related behavior, you need to enable the base behavior definition and the interface behavior definition. If the extended behavior needs to be available for service consumption, the projection behavior definition needs to be enabled as well.

For more information about enabling business objects for behavior extensions, see [Extensibility-Enablement for Behavior Extensions](#).

Enabling the Base Behavior Definition

1. In the base behavior definition `/DMO/R_AGENCYTP`, add the keyword `extensible`; in the header section to overall enable the base behavior definition for behavior extensions.

```
managed implementation in class /dmo/bp_r_agencytp unique;
strict(2);
with draft;
extensible;
```

2. Enable the node `Agency` for behavior extensions by adding the keyword `extensible` in the section of this node.

```
define behavior for /DMO/R_AgencyTP alias /DMO/Agency
persistent table /dmo/agency
draft table /dmo/agency_d query /DMO/R_AgencyDraft
lock master
total etag LastChangedAt
authorization master ( global )
etag master LocalLastChangedAt
late numbering
extensible
```

i Note

We recommended that you add a C0 release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

Enabling the Interface Behavior Definition

In the interface behavior definition `/DMO/I_AGENCYTP`, add the keyword `extensible`; in the header section. The BDEF interface then implicitly inherits the extension properties defined in the original base BDEF, and no further enabling is required on this level.

```
interface;
use draft;
extensible;
```

i Note

We recommended that you add a C0 release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

i Note

It is recommended to add a C1 release contract in order to specify that this development object is intended as a stable data source for consumption.

For maintenance reasons, the artifacts of the `/DMO/` reference content are not shipped with defined release contracts.

Enabling the Projection Behavior Definition

1. In the projection behavior definition /DMO/C_AGENCYTP, add the keyword `extensible`; in the header section to overall enable the projection behavior definition for behavior extensions.

```
projection;
strict(2);
use draft;
extensible;
```

2. Enable the node Agency for behavior extensions by adding the keyword `extensible` in the section of this node.

```
define behavior for /DMO/C_AgencyTP alias /DMO/Agency
extensible
```

i Note

We recommended that you add a CO release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

Result

You have now enabled the business object for field-related behavior extensions and for non-standard behavior extensions. Besides feature control implementations, actions and functions, this also includes the definition of new field mappings (for own structures) and new foreign entities (only for own entities).

The chapter [Develop Behavior Extensions](#) shows how to develop behavior extensions for the reference business object based on the implemented behavior enablement.

Enabling Standard Behavior Extensions

To allow the extender to add validations, determinations and additional save implementations, you need to enable these operations in the behavior definition of your business object.

The standard behavior enablement allows extenders to add validations, determinations and additional save implementations with the specified trigger conditions via extensions. Note that adding these extensions may change the behavior of the BO at runtime.

Required Enablement Steps

In order to enable the business object for standard behavior extensions, you need to enable the base behavior definition. Since you have enabled the interface behavior definition and the projection behavior definition in the previous step already, no further enablement steps are required for these layers.

Prerequisite

This guide assumes, that you have enabled the business object for non-standard behavior as described in [Enabling Non-Standard Behavior and Field-Related Behavior](#).

Procedure

1. In the header section of the base behavior definition /DMO/R_AGENCYTP, define that determinations, validations and additional save implementations can be added by the extender.

```
managed implementation in class /dmo/bp_r_agencytp unique;
strict(2);
with draft;
extendable {
  with determinations on modify;
  with determinations on save;
  with validations on save;
  with additional save; }
```

In this guide, the extender will add determinations on modify and a validation on save to the original business object.

2. Enable the node Agency for behavior extensions by adding the keyword `extendable` in the section of this node.

```
DEFINE behavior FOR /DMO/R_AgencyTP alias /DMO/Agency
  persistent table /dmo/agency
  draft table /dmo/agency_d query /DMO/R_AgencyDraft
  lock master
  total etag LastChangedAt
  authorization master ( none )
  etag master LocalLastChangedAt
  late numbering
  extendable
```

3. Define the determine action Prepare as `extendable`. This allows extenders to add their validations and determinations to this action, thus enabling adequate handling of state messages.

```
draft determine action Prepare extendable
{
  validation /DMO/validateEmailAddress;
  validation /DMO/validateCountryCode;
  validation /DMO/validateName;
  validation /DMO/validateLargeObject;
}
```

4. Define the field mapping as `extendable`. This allows extenders to add new fields to the data model. In case these new fields have the same name on database and CDS level, no mapping extension is required.

```
mapping for /dmo/agency corresponding extendable
{
  AgencyId      = agency_id;
  Name          = name;
  Street        = street;
  PostalCode    = postal_code;
  City          = city;
  CountryCode   = country_code;
  PhoneNumber   = phone_number;
  EmailAddress  = email_address;
  WebAddress    = web_address;
  Attachment    = attachment;
  MimeType      = mime_type;
  Filename      = filename;
  LocalCreatedBy = local_created_by;
  LocalCreatedAt = local_created_at;
  LocalLastChangedBy = local_last_changed_by;
  LocalLastChangedAt = local_last_changed_at;
  LastChangedAt = last_changed_at;
}
```

i Note

We recommend that you add a CO release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

Result

You have now enabled the business object for standard behavior extensions.

The chapter [Develop Behavior Extensions](#) shows how to develop behavior extensions for the reference business object based on the implemented behavior enablement.

Enabling Node Extensions

To allow extenders to add their own extension node to the data model, you need to enable node extensions in the original business object.

Required Enablement Steps

Extenders need to be able to add their extension node using a composition from the root node of the original business object to the node of the extension. Hence, the root node needs to be enabled for new compositions by using the corresponding CDS annotation in the CDS views `/DMO/R_AGENCYTP`, `/DMO/I_AGENCYTP` and `/DMO/C_AGENCYTP`.

Since extenders want to be able to include behavior in their new node, the behavior definition `/DMO/R_AGENCYTP` needs to be extensibility-enabled in the header section. If you followed the previous implementation steps of this guide, you have already done this as part of chapter [Enabling Non-Standard Behavior and Field-Related Behavior](#).

In the last step, the service definition requires extensibility enabling, since the extension node needs to be exposed for service consumption. Before a service definition can be released with the C0 contract, a suitable provider contract needs to be added.

Prerequisite

This guide assumes, that you have enabled the business object for standard behavior as described in [Enabling Standard Behavior Extensions](#).

Enabling the CDS View Stack for new Compositions

1. In the CDS view `/DMO/R_AGENCYTP`, add the CDS annotation `@AbapCatalog.extensibility.allowNewCompositions: true` to the existing `@AbapCatalog.extensibility` block in the header section.

```
@AbapCatalog.extensibility: {
  extensible: true,
  elementSuffix: 'ZAG',
  allowNewDatasources: false,
  dataSources: ['_Extension'],
  quota: {
    maximumFields: 500,
    maximumBytes: 5000
  },
  allowNewCompositions: true
}
```

2. In the CDS view `/DMO/I_AGENCYTP`, add the CDS annotation `@AbapCatalog.extensibility.allowNewCompositions: true` to the existing `@AbapCatalog.extensibility` block in the header section.

```
@AbapCatalog.extensibility: {
  extensible: true,
  dataSources: ['AGENCY'],
  elementSuffix: 'ZAG',
  quota: {
    maximumFields: 500,
```

```

        maximumBytes: 5000
    }, allowNewCompositions: true
}

```

i Note

It is recommended to add a C1 release contract in order to specify that this development object is intended as a stable data source for consumption.

For maintenance reasons, the artifacts of the /DMO/ reference content are not shipped with defined release contracts.

3. In the CDS view /DMO/C_AGENCYTP, add the CDS annotation

@AbapCatalog.extensibility.allowNewCompositions: true to the existing @AbapCatalog.extensibility block in the header section.

```

@AbapCatalog.extensibility: {
    extensible: true,
    dataSources: ['Agency'],
    elementSuffix: 'ZAG',
    quota: {
        maximumFields: 500,
        maximumBytes: 5000
    }, allowNewCompositions: true
}

```

4. In the behavior definitions /DMO/R_AGENCYTP, /DMO/I_AGENCYTP and /DMO/C_AGENCYTP, make sure that the keyword `extensible` is specified in the header section. Additionally, make sure that the node for which new compositions are allowed is enabled in the behavior definitions /DMO/R_AGENCYTP and /DMO/C_AGENCYTP.

If you have followed the instructions of this guide, then you have done all this already in chapter [Enabling Non-Standard Behavior and Field-Related Behavior](#).

5. In the service definition /DMO/UI_AGENCY, add the CDS annotation @AbapCatalog.extensibility.extensible: true to the header section. Additionally, define the service as an OData V2/V4 UI service using respective provider contracts. By doing so, you ensure that only CDS views that are compatible to these service kinds can be added.

```

@EndUserText.label: 'Agency'
@AbapCatalog.extensibility.extensible: true
define service /DMO/UI_AGENCY
    provider contracts odata_v2_ui, odata_v4_ui {
        expose /DMO/C_AgencyTP as Agency;
    }

```

i Note

We recommended that you add a C0 release contract to specify that this development object is intended as a stable API for extensions. This is mandatory for clean core extensibility use cases.

Result

You have now enabled the business object for node extensions.

The chapter [Develop Node Extensions](#) shows how to extend a business object with an existing node based on the implemented node extensibility enablement.

RAP Extensions

This section explains the concepts of how to extend an original RAP business object with data model extensions, behavior extensions and node extensions.

About Building RAP Extensions

RAP extensions enable you to extend existing RAP BOs from other software components or SAP delivered RAP BOs to optimize and adapt their functionality to your specific business requirements. The RAP extension approach focuses on separation-of-concerns between original BO as basis for your extensions and the actual extension artifacts that contain your additions to the data model and behavior of the original BO. Each layer in the stack of a RAP business object can be extended and adapted depending on your business use case. You can add additional fields to the data model, add new validations or determinations, actions and functions. You can also extend the BO with an additional BO entity and extend the service definition to include the new BO node. The available extensibility options depend on the extensibility-enablement of the original BO.

Generally, the RAP runtime differentiates between elements of an original BO and elements of extensions. Generally, an extension can't reference any elements from the original BO or redefine any behavior defined in the original BO.

For more information about the enablement requirements as basis for extensions and the granularity, see [Learn about Extensibility-Enablement](#).

Building Extensions for Different Implementation Types

Similar to draft capabilities, extensions are always managed by the managed BO provider at runtime, so extensions behave the same way for all implementation types and the described concepts apply to both managed and unmanaged RAP BOs.

i Note

It's not possible to define standard behavior extensions for unmanaged RAP BOs.

Extending Released RAP BOs

Released RAP BOs follow the same extension principles as described in [About Building RAP Extensions](#). Additionally, released BOs are always released with `release` contracts to ensure lifecycle stability and upgrade-safety for customer extensions. For an overview of the release contract architecture of a released RAP BO, see [Development Objects with Release Contracts Overview](#).

Extensions implicitly inherit the release contract of the objects that they belong to. BDEF extensions always use a BO interface that is C1 released. Consequently, a behavior extension can, for example, only reference data sources that are also C1 released. Refer to the respective BO KTD documentation to get more information about available extension options for released RAP BOs.

[Learn about RAP Extensions](#)

[Develop RAP Extensions](#)

Learn about RAP Extensions

This section offers you conceptual background about building RAP extensions for different use cases.

[CDS Data Model Extensions](#)

Data model extensions allow you to add extensions fields to the database tables and corresponding CDS View entities.

[Behavior Extensions](#)

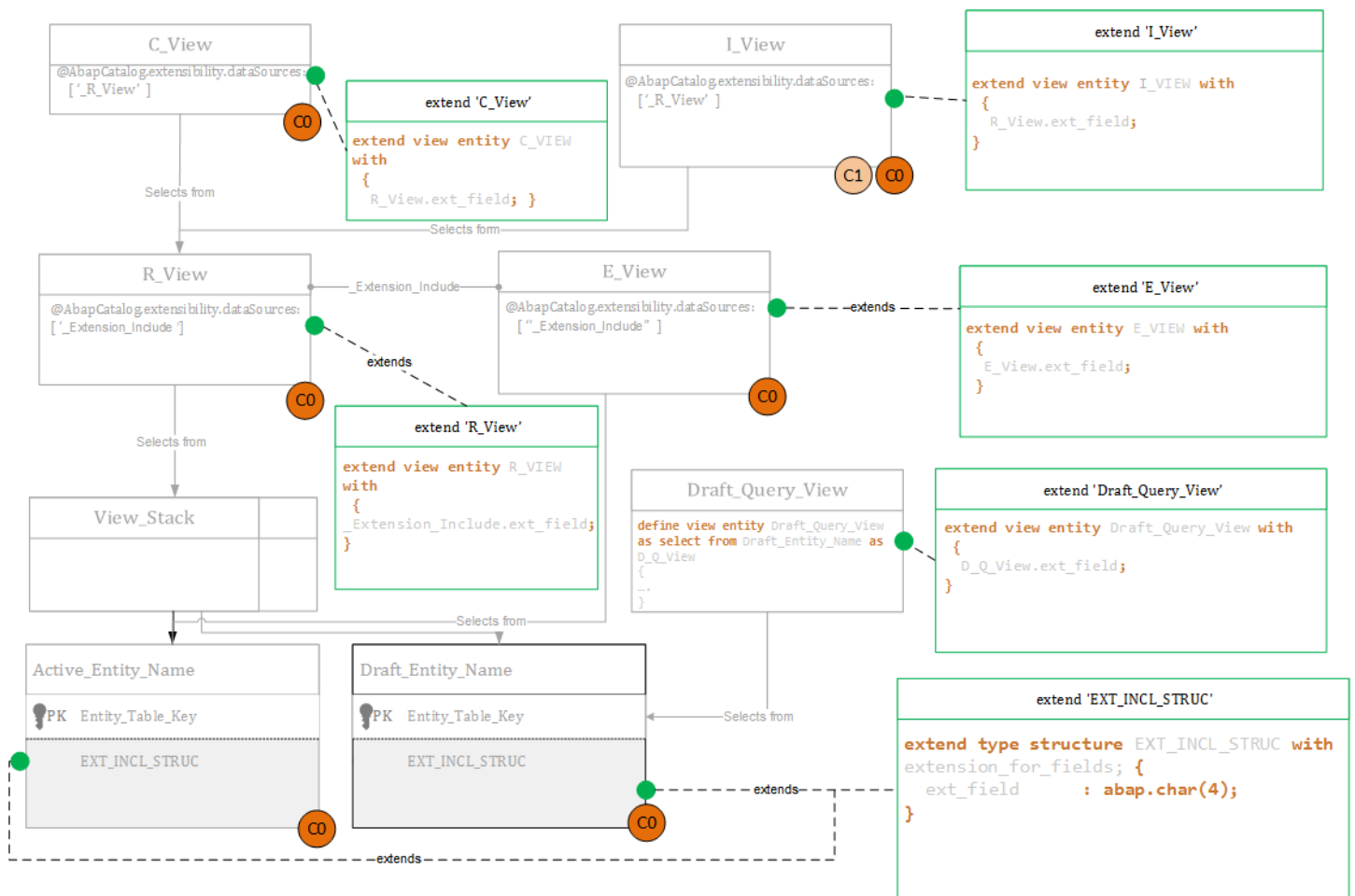
CDS Data Model Extensions

Data model extensions allow you to add extensions fields to the database tables and corresponding CDS View entities.

Data Model Extension Architecture

The data model of a RAP BO consists of database tables for active data and draft tables in the case of a draft-enabled BO. On top of the database tables, the actual data model is defined with CDS View entities on the base and projection layers, which are then exposed for service consumption. Building extensions follows a bottom up approach, meaning you first define your extensions on the base BO layer, and then continue on to the projection layer and the service exposure level. Extension fields are added as extension to extension include structures embedded in the active and draft database tables, so that the extension has to be defined only once also for a draft-enabled RAP BO.

The following graphic extends an existing datamodel with an extension field of type `abap.char(4)` and the name `ext_field`. This field is propagated throughout the stack to the projection level. Each CDS View entity is extended with a `View Extend`, adding the extension field. The `R_View` adds the `ext_field` via the associated extension includeview `E_View`. The `E_View` itself selects directly from the active database table. The draft query view `Draft_Query_View` adds the extension fields via CDS view extend. On the projection layer both the business service projection `C_View` and the BO interface CDS View `I_View` add the field from the base layer via the `R_View`. Since all entities are included in the service exposure, the field is automatically exposed and no extension to the service definition is required.



Extending Database Tables

You extend both active and draft database tables by extending an extension include structure that is included in both tables in the case of a BO with draft capabilities. This is also the standard architecture for released RAP BOs. However, if a BO doesn't use this architecture, it's also possible to directly extend the database table. This is only recommended for smaller BOs and BOs that don't use release contracts.



Extending CDS Views

When you've extended the database tables, you have to propagate the extension fields throughout the stack. This means that you have to add a View Extend to every CDS View entity. The base CDS View extends add the extension fields to the corresponding entities. Once all layers are extended, the new extension fields are automatically exposed in the OData service and no extension of the Service Definition is required.

For more information about the syntax, see [ABAP CDS - Extending CDS Views \(ABAP Keyword Documentation\)](#) and [CDS BDL - BDEF Projection Extension \(ABAP Keyword Documentation\)](#).

Exposing Extension Fields on the UI

If you want to include the new fields not only in the OData Service, but also on the UI, you have to define UI annotations for the extension projection CDS view.

For more information about UI annotations, see [CDS Annotations](#).

Adding Behavior for Extension Fields

You can use the extension fields and add additional behavior like determinations and validations for the extensions fields.

For more information, see [Behavior Extensions](#).

Related Information

[Data Model Extensions \(ABAP Data Models guide\)](#)

Behavior Extensions

Behavior Extension Architecture

i Note

- You can only define behavior extensions on a projection level that doesn't require an implementation class.
- Extensions for determinations, validations, additional save and cleanup are only supported for the implementation type managed.

Behavior Extensions allow you to add additional behavior to an original RAP Business Object or a released RAP BO. This chapter only covers behavior extensions to existing BO entities. For adding a new BO entity with its own behavior and data model, see [Node Extensions](#).

Prerequisites

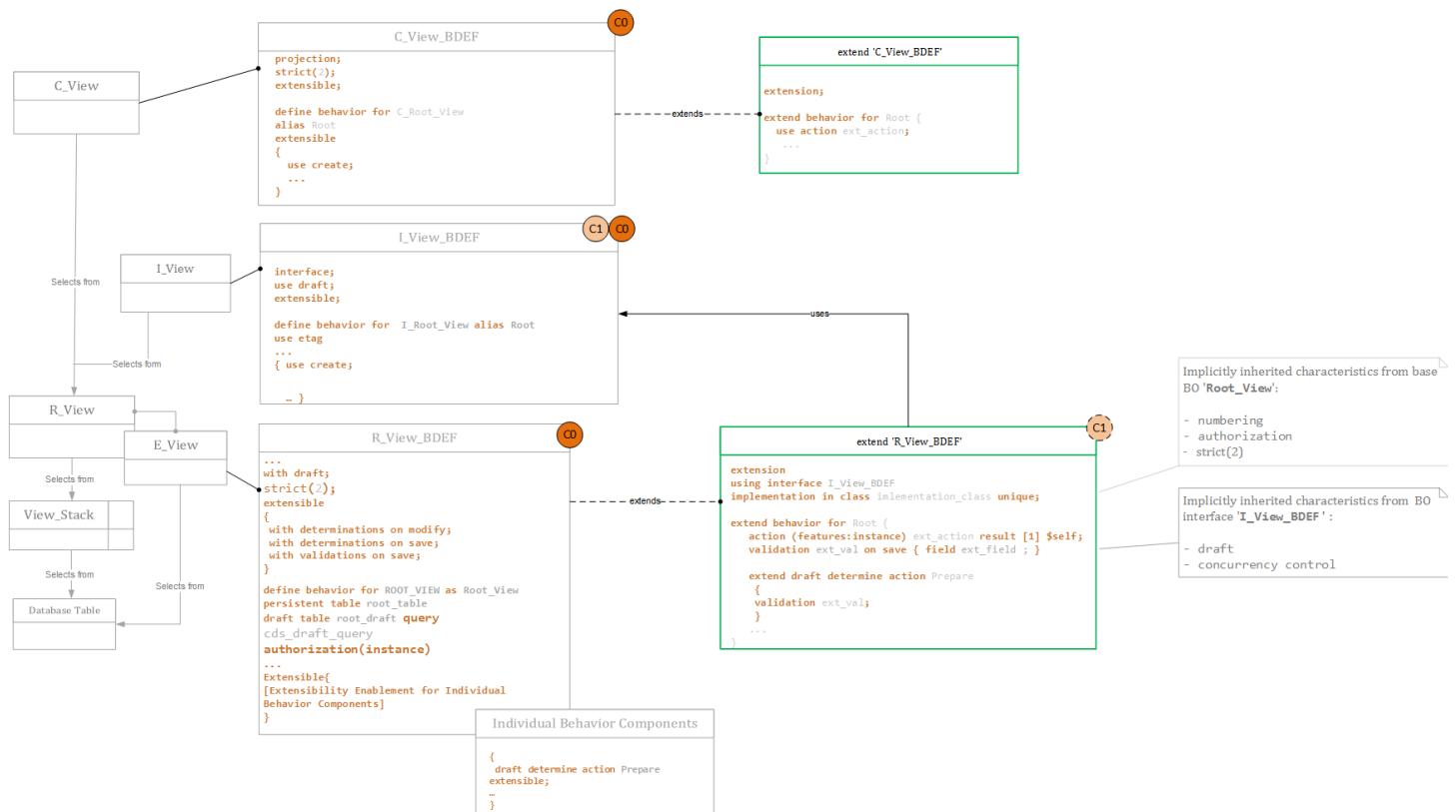
A prerequisite for behavior extensions is the extensibility-enablement for the required behavior. You can only define validations and determinations and an additional save if they're explicitly defined as extensibility-enabled in the base BO. For more details about the required extensibility-enablement for behavior extensions, see [Extensibility-Enablement for Behavior Extensions](#).

Relation between Behavior Extensions and BO Interfaces

You can extend the behavior of a business object with extension behavior on the base and projection levels of a RAP BO. On base level, you can add behavior that requires an implementation like validations or determinations or actions and functions. On projection level, you can only add behavior extensions that don't require an implementation.

Behavior extensions are stored in a Behavior Definitions Extension, which must always use a BO interface in its header. For more information about the development object and its definition, see [CDS BDL - BDEF Extension \(ABAP- Keyword Documentation\)](#).

A BDEF extension can only reference entities exposed in the BO interface - if the BO interface includes only a subset of BO entities of the underlying data model, only the exposed entities can be enriched with extension behavior. The BO interface also decides which other capabilities are exposed to extensions, like, for example, draft capabilities. Even if the underlying data model itself is draft-enabled, but the BO interface doesn't explicitly expose the draft behavior, behavior extensions can only access active instances in their implementation and can't use the `%is_draft` indicator.



For this example, the class definition for `implementation_class` is generated as follows:

Sample Code

```
CLASS implementation_class DEFINITION INHERITING FROM cl_abap_behavior_handler.
```

```

METHODS get_instance_features FOR INSTANCE FEATURES
  IMPORTING keys REQUEST requested_features FOR Root RESULT result.

METHODS ext_val FOR VALIDATE ON SAVE
  IMPORTING keys FOR Root~ext_val.

METHODS get_instance_authorizations FOR INSTANCE AUTHORIZATION
  IMPORTING keys REQUEST requested_authorizations FOR Root RESULT result.

METHODS ext_action FOR MODIFY
  IMPORTING keys FOR ACTION Root~ext_action RESULT result.

ENDCLASS.

```

Behavior Extensions on Base Level

The BDEF Extension on base level contains the actual new behavior you want to add to the data BO. You have all of the behavior options available that are enabled on the original BO. The example extension defines an extension action `ext_action` with instance feature control and a `validation on save` for the extension field `ext_field` added in [CDS Data Model Extensions](#). The BDEF extension inherits its basic characteristics, like numbering and authorization, from the original BO. Draft capabilities, etag handling, and strict version are inherited from the used BO interface.

If the original BO is defined with `authorization(global, instance)`, the behavior pool for the BDEF extension is generated with implementation exits `FOR INSTANCE AUTHORIZATION` and `FOR GLOBAL AUTHORIZATION`. In the example, only instance authorization is defined, so only an implementation exit `FOR INSTANCE AUTHORIZATION` is generated. You can overrule the authorization control definition in the authorization master of the base BO by defining authorization for the action in the BDEF extension. The derived types of the authorization implementation exit are reduced to elements from your BDEF extension - it's not possible to define additional authorization for elements from the base BO. The BDEF extension inherits the draft capabilities exposed in the BO interface `I_View_BDEF` with `use draft`; and the properties of `strict(2)`.

In the example, the determine action `PREPARE` is extended with the additional validation `ext_val` to ensure the correct handling of state messages from this validation.

i Note

In order to extend field mapping information, use the keywords `extend mapping for` if the extension is stored in a different software component than the original business object.

Behavior Extensions on Projection Level

The BDEF extension of the projection level exposes the extension behavior defined on base BO level. In this example, the BDEF extension exposes the action `ext_action` to expose it in the OData service.

Numbering in Behavior Extensions

The numbering behavior for key fields is inherited from the original BO entity and can't be changed in a behavior extension. For more information about numbering, see [Numbering](#).

Authorization in Behavior Extensions

i Note

You can't extend prechecks, but you can implement your own prechecks in the implementation.

Generally, authorizations are inherited from the base RAP BO, and the respective implementation is called during runtime for global authorizations and the standard operations. Except for actions, it's not possible to redefine authorization implementations, that are defined in an original BO, in the implementation of an extension.

i Note

If you extend the BO behavior with extension actions, you must always define authorization control for these actions. You can use the same authorization control as defined in the authorization master of the base BO or you can define other authorization and replace the authorization control of the authorization master for the action. This enables consistency of the authorization for your extension actions.

You can define additional authorizations for extension elements in the behavior pool of the behavior definition extension, for example for extension actions or functions you have added in the behavior definition extension.

For more information about authorizations, see [Authorization Control](#).

Concurrency Control in Behavior Extensions

The concurrency control handling for BDEF extensions is determined by the BO interface. If the original BO uses etag handling and the BO interface exposes the etag handling with use `etag`, the etag handling defined in the original BO is also applied to behavior in the BDEF extension. For more information, see [Optimistic Concurrency Control](#). If a BO interface exposes draft handling, the total Etag is always applied.

If the etag handling isn't exposed by the BO interface, pessimistic concurrency control is applied as a default. For more information, see [Pessimistic Concurrency Control \(Locking\)](#).

Draft-Specific Extension Behavior

Among the draft actions, only the determine action `PREPARE` is extensible. You can add determinations and validations that you've defined in the BDEF extension to determine action `PREPARE` in the extension. This is recommended to guarantee correct message handling.

Generally, determine actions can only be extended or defined in an extension if the BO interface is draft-enabled. For more information, see [Draft](#).

About Behavior and Operations in Behavior Extensions

Standard Operations in Extensions

The standard operations of a BO entity are defined as part of the original BO. If an original BO doesn't define the `Delete` operation as part of its behavior, you can't add this operation in a BDEF extension. For more information about standard operations, see [Standard Operations](#).

Actions and Functions in Extensions

Actions and functions follow the same rules as in non-extension use cases. For more information, see [Nonstandard Operations](#).

Validations and Determinations in Behavior Extensions

You can only define validations and determinations with their trigger conditions if the original BO is enabled for them, otherwise this results in a syntax error in the BDEF extension.

An extension validation or determination can use either extension fields or original fields as part of their trigger conditions, but within the implementation, the fields are treated differently at runtime. For more details, see [in local mode in Extensions](#).

For the required enablement, see [Extensibility-Enablement for Behavior Extensions](#).

i Note

If you define an extension field as **mandatory** and the base BO isn't extension-enabled for **validations on save**, it's not possible to implement a check in the extension to reinforce the **mandatory** attribute, and it's only used as consumer hint on the user interface.

Static and Dynamic Feature Control in Behavior Extensions

In the behavior definition extension, you can only define additional static and dynamic feature control for extension elements. It's not possible to redefine feature control for elements of the original BO. In extensions for projection views, you can only define static feature control for extension fields. Dynamic feature control cannot be defined.

For more information about feature control, see [Feature Control](#).

Save Options in Behavior Extensions

The available save options follow the same rules as in non-extension use cases. You can define **additional save** and **unmanaged save** in extensions. **Additional save** is only available for implementation type managed.

For more information, see [Save Options](#).

Messages and EML-Specific Extension Behavior

Messages in Extensions

Messages of extensions are handled by the BO provider framework of the base BO and the same guidelines apply regarding the differentiations between state and transition messages.

i Note

It's recommended to use different **%state_area** strings in your extension that differ from the strings defined in the original BO. If you use the same **%state_area**, all state messages belonging to the respective **state_area** are invalidated at the same time.

For more information about messages, refer to [Messages](#).

in local mode in Extensions

From the consumer's perspective, you can't distinguish if you're consuming part of an extension or part of the original base BO. The implicit framework behavior differs, however, depending on whether an element belongs to an extension or an original BO.

If you use **in local mode** in the behavior pool of your extension, the behavior differs. Generally, **in local mode** is used to circumvent authorization and feature control checks. This works as expected for all elements belonging to an extension. If you execute an EML modify in the implementation of your extension on an element of the original BO with the addition **in local mode**, the authorization and feature control checks of the original BO are executed anyway and can't be bypassed in the implementation even if **in local mode** is specified.

Exposing Extension Behavior on the UI

If you want to include behavior-like actions not only in the OData Service, but also on the UI, you must use the respective UI annotations to make the actions or other behavior visible on the UI.

For more information about UI annotations, see [CDS Annotations](#).

Node Extensions

With node extensibility, you can add an additional BO entity to an existing RAP BO. The new node contains its own data model that is integrated as an extension entity into the existing data model. The behavior for an extension entity is defined in a BDEF extension. Then the **Service Definition** is extended to include the new node in the service exposure.

i Note

- Extensions for determinations, validations, additional save and cleanup are only supported for the implementation type managed.

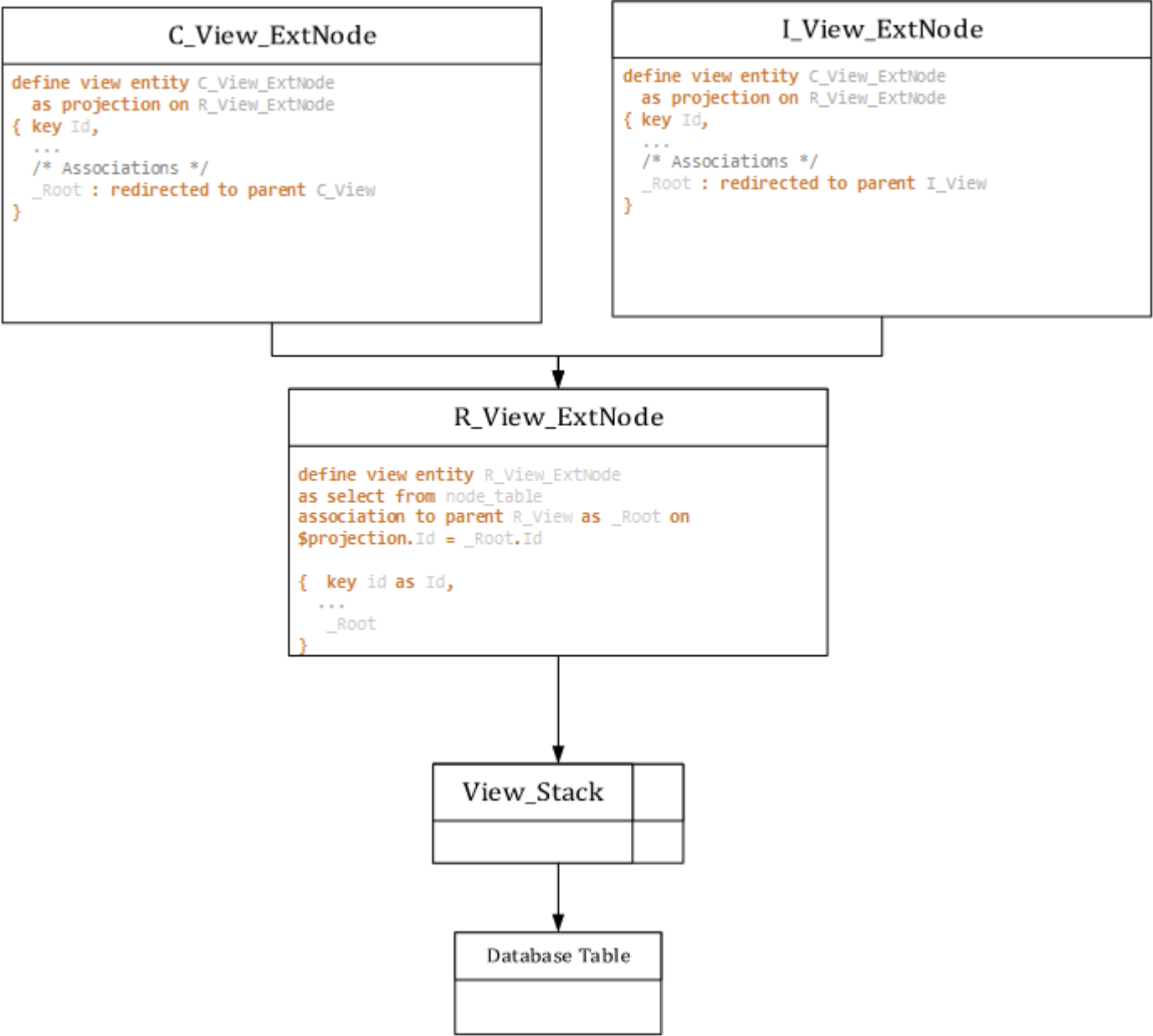
Node Extensibility Architecture

Data Model of Extension Node

A prerequisite is that you have defined an additional node with its own data model that you want to include in an existing data model. The original BO must be enabled for node extensibility as described in [Extensibility-Enablement for Node Extensibility](#).

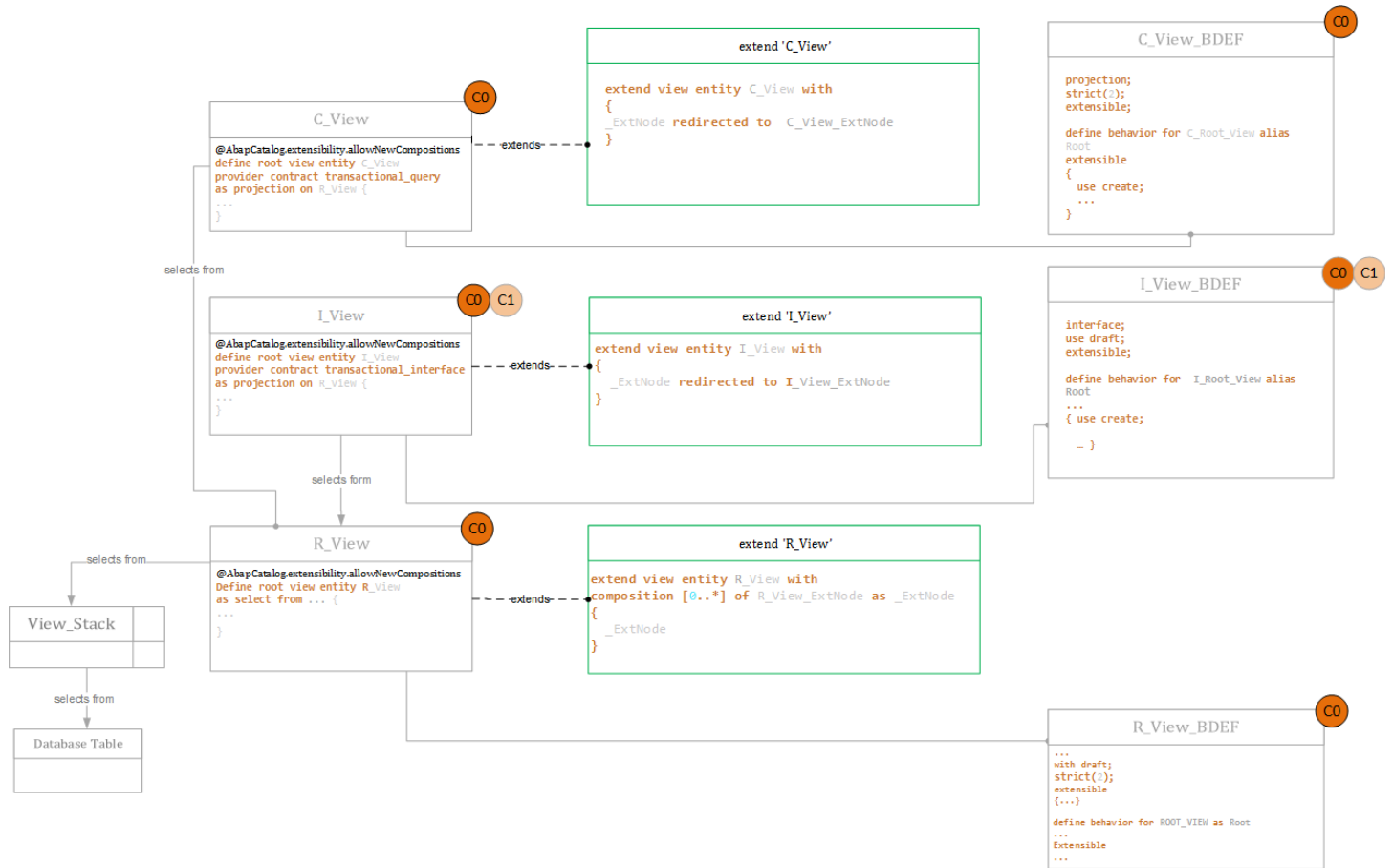
The node used in this example for illustration purposes consists essentially of the view `R_View_ExtNode` with the key field `ID` and a `to-parent` association to the root view of the BO data model that the extension node is to be added to. This association establishes the parent-child relationship required to add behavior in a behavior extension. The projection view `C_View_ExtNode` projects the `R_View_ExtNode` CDS view and redirects the association to `_Root` to the corresponding projection `C_View`. The CDS view entity `I_View_ExtNode` projects the `R_View_ExtNode` CDS View and redirects to the `I_View` CDS view.

The following graphic shows the CDS view stack for this example:



Including the Extension Node in the Original BO

The extension node must be included in the existing data model. Like data model extensions, the extension node must be included throughout the stack. The extension include view **R_View** is extended with the obligatory composition relationship that defines **_ExtNode** as composition of the root view entity. The corresponding to-parent relationship was modeled as part of **R_View_ExtNode** with the association **association to parent **R_View** as **_Root****. This reciprocal relationship defines the extension node as a dependent BO entity. Then the association is propagated throughout the CDS View stack with the respective redirects. In this aspect, the extension node is modeled like a regular BO entity:

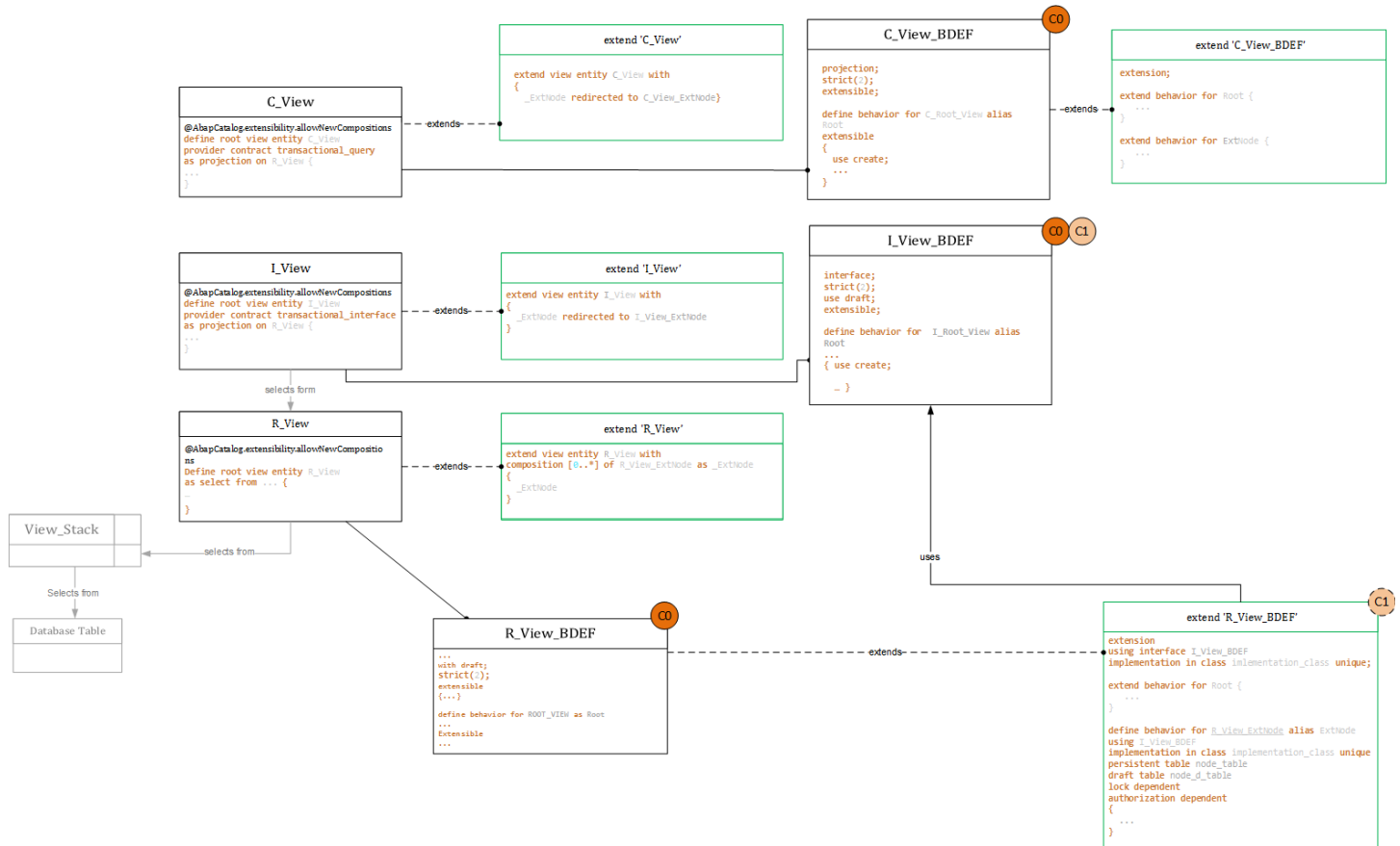


For an example, see [Adding the Extension Node to the Data Model](#).

Defining Behavior for the Extension Node

The behavior for the extension node is defined in an extension BDEF. Extension nodes are always added a to-parent relationship - so authorization and lock behavior is always modeled as dependent and the behavior is derived from the root node. It's recommended to use an Etag master on the extension node. You can also specify the numbering required for your use case. In this case, the ExtNode is defined with early numbering. This child node provides the update and delete operation for the BO entity. The create operation is enabled via a create by association on the root node with association `_ExtNode{ create; with draft; }`. The ExtNode also contains the reverse association to `_Root`.

On the projection layer, the BDEF extension to `C_View_BDEF` exposes the behavior defined in the base layer with the keyword `use`.



For an example, see [Extending the Behavior for the Node Extension](#) and [Adding the Extension Node to the Data Model](#).

Ancestor Associations

If an extension node is added to the node hierarchy, you want to make sure that associations from this node to the respective authorization and lock master are still working even if this master is redefined in the original business object. In order to define the associations from the extension node to the corresponding master flexibly, you need to define the dependencies in the header section anonymously and additionally provide a so called ancestor association to every node above in the node hierarchy except for the parent node. During runtime, the ancestor association with the required master is used respectively by the framework.

❖ Example

BO Hierarchy: In this example the node hierarchy is as follows: Root - ChildNode - GrandChild (toParent) - ExtensionNode (extension)

Then the behavior extension for ExtensionNode is defined as follows:

☰ Sample Code

```
define behavior for ExtensionNode alias ExtNode using BO_Interface
persistent table perst_table
draft table table_d
etag master LastChangedAtTask
lock dependent
authorization dependent
...
{
  update;
  delete;
```

```

    field ( readonly ) ID;
    ...

    association _GrandChild { with draft; }
    ancestor association _ChildNode { with draft; }
    ancestor association _Root { with draft; }

}

```

For more information about the syntax, see [CDS BDL - ancestor association \(ABAP Keyword Documentation\)](#).

Including the Extension Node in the OData Service

When the extension node has been modeled as part of the data model and behavior, it also needs to be included as part of the OData service. To include the extension node, you have to create a **Service Definition Extension** for the original service definition.

For an example, see [Including the Extension Node in the Service Exposure](#).

Develop RAP Extensions

This section offers how-to information and implementation examples for extending a RAP BO.

Reference Scenario

We work with the example from the ABAP Flight Reference Scenario that was used in chapter [Develop Extensibility-Enablement](#) to demonstrate the process of enabling a RAP BO for extensibility.

In this guide, the business object `/DMO/R_AGENCYTP`, which has been extensibility-enabled, is now extended with regard to different extensibility use cases. The contributing artifacts for each extensibility use case can be found in the following sub packages of the package `/DMO/FLIGHT_REUSE_AGENCY`:

- Data model extensibility: `/DMO/FLIGHT_REUSE_AGENCY_SLOGN`
- Behavior extensibility: `/DMO/FLIGHT_REUSE_AGENCY_CNTRY`
- Node extensibility: `/DMO/FLIGHT_REUSE_AGENCY_REV`

Extensibility Enablement Scenarios

The following guide is structured by the different extensibility use cases and guides you through the respective extension processes.

[Develop Data Model Extensions](#)

In the following section, you will develop a data model extension which adds a new field for an agency slogan to our reference business object `/DMO/R_AgencyTP`.

[Develop Behavior Extensions](#)

In the following section, you will develop behavior extensions which add new behavior to the reference business object `/DMO/R_AgencyTP`.

[Develop Node Extensions](#)

In the following section, you will extend the reference business object /DMO/R_AgencyTP with an extension node. This extension node is thereby added as a sub node to the existing /DMO/Agency node in the original business object.

Develop Data Model Extensions

In the following section, you will develop a data model extension which adds a new field for an agency slogan to our reference business object /DMO/R_AgencyTP.

Required Extension Steps

Data model extensions affect the whole stack of a business object. Hence you will develop extension artifacts for the database layer as well as for every involved layer of the CDS data model. The data model extension artifacts will make use of the extension points you developed as part of the data model extensibility enablement process.

Prerequisite

You have enabled your business object for data model extensions as described in chapter [Enabling Data Model Extensions](#).

[Extending the Database Tables](#)

This chapter guides you through the necessary steps to extend the database tables of our reference business object with a new field.

[Extending the CDS Views](#)

This chapter guides you through the necessary steps to extend the CDS data model of our reference business object with the field you added to the database tables in the previous step.

Extending the Database Tables

This chapter guides you through the necessary steps to extend the database tables of our reference business object with a new field.

Required Extension Steps

Adding a field to the database layer requires a structure extend which you need to create for the existing extension include structure /DMO/S_EXT_INCL_AGENCY. Since the extension include structure is referenced in the active table and in the draft table, the new field specified in the structure extend extends both tables.

Prerequisite

You have enabled your database tables for extensions as described in chapter [Enabling the Database Tables](#).

Procedure

Create a structure called /DMO/ZZ_S_EXT_AGENCY_SLOGAN and define it as a structure extend for the structure /DMO/S_EXT_INCL_AGENCY. Specify the new field as shown below:

```
@EndUserText.label : 'Extend structure'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
extend type /dmo/s_ext_incl_agency with /dmo/zz_s_ext_agency_slogan {
```

```

/dmo/zsloganzag : /dmo/zz_slogan;

}

```

i Note

The names of extension elements must start with the namespace prefix used by the related package. Since the namespace `/DMO/` is used to ship the whole agency reference scenario, the prefix `/DMO/` is also used for extension elements in this example. In a productive environment, the namespace prefix of extensions elements differs from the namespace of the original business object. This is indicated by the addition `ZZ` after the `/DMO/` namespace.

Result

You have extended the database tables with the new field. You can now add the field to the CDS data model as described in [Extending the CDS Views](#).

Extending the CDS Views

This chapter guides you through the necessary steps to extend the CDS data model of our reference business object with the field you added to the database tables in the previous step.

Prerequisites

- You have enabled your CDS data model for extensibility as described in chapter [Enabling the CDS Views](#).
- You have added the field `/dmo/zsloganzag` to the active database table and to the draft database table as described in chapter [Extending the Database Tables](#).

Required Extension Steps

In order to make the new field available in your CDS data model, you first need to create a CDS view extend for the extension include view `/DMO/E_Agency` and add the field to this view. You have access to the new field in the database table via the data source alias defined in the extension include view.

Next, create a CDS view extend for the CDS view `/DMO/R_AGENCYTP` and use the association to the extension include view to add the new field to the CDS view `/DMO/R_AGENCYTP`.

In the last step, make the new field available also in the CDS projection view `/DMO/C_AGENCYTP`, in the interface view `/DMO/I_AGENCYTP` and in the draft query view `/DMO/R_AGENCYDRAFT`. Access the field by using the data source alias defined in the extended CDS view respectively.

Since the new field has the same name on database and CDS level, no mapping extension is required.

Procedure

1. Create a CDS extension view called `/DMO/ZZ_X_SLOGAN_E_AGENCY` for the extension include view `/DMO/E_AGENCY` and add the new field in the field list. Since you have already extended the database table `/DMO/AGENCY` with the new field, you can use the data source alias `Agency` defined in the extension include view in a path expression to access the field.

```

extend view entity /DMO/E_Agency with
{

```

```
Agency./dmo/zsloganzag as /DMO/ZZSloganZAG
}
```

i Note

Aliasing that effects more than a change from lowercase to uppercase or vice versa should not be done on the base layer. Aliasing is possible on the interface layer or on the projection layer of the business object.

i Note

The names of extension elements must start with the namespace prefix used by the related package. Since the namespace `/DMO/` is used to ship the whole agency reference scenario, the prefix `/DMO/` is also used for extension elements in this example. In a productive environment, the namespace prefix of extensions elements differs from the namespace of the original business object. This is indicated by the addition `ZZ` after the `/DMO/` namespace.

2. Create a CDS extension view called `/DMO/ZZ_X_SLOGAN_R_AGENCY` for the CDS view `/DMO/R_AGENCYTP` and add the new field in the field list. In order to access the field, use the association `_Extension` to the extension include view `/DMO/E_AGENCY` in a path expression.

```
extend view entity /DMO/R_AgencyTP with
{
  _Extension./DMO/ZZSloganZAG
}
```

3. Create a CDS extension view called `/DMO/ZZ_X_SLOGAN_C_AGENCY` for the CDS projection view `/DMO/C_AGENCYTP` and add the new field in the field list. In order to access the field, use the data source alias `Agency` defined in the CDS projection view in a path expression. The field is available in the CDS view `/DMO/R_AGENCYTP` as you have extended it in the previous step. Add UI annotations as well as search annotations for the default search element and the fuzziness threshold of the field.

```
extend view entity /DMO/C_AgencyTP with
{
  @UI: {
    listItem: [ { position: 25, importance: #HIGH },
                 { position: 10, type: #FOR_ACTION, dataAction: '/DMO/createFromTemplate', l
    fieldGroup: [ { position: 25, qualifier: 'General_FG' } ],
    identification: [ { position: 10, type: #FOR_ACTION, dataAction: '/DMO/createFromTemplate
  }
  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.8
  Agency./DMO/ZZSloganZAG
}
```

4. Create a CDS extension view called `/DMO/ZZ_X_SLOGAN_I_AGENCY` for the CDS interface projection view `/DMO/I_AGENCYTP` and add the new field in the field list. In order to access the field, use the data source alias `Agency` defined in the interface projection view in a path expression. The field is available in the CDS view `/DMO/R_AGENCYTP` as you have extended it in a previous step.

```
extend view entity /DMO/I_AgencyTP with
{
  Agency./DMO/ZZSloganZAG
}
```

5. Create a CDS extension view called `/DMO/ZZ_X_SLOGAN_R_AGENCYDRAFT` for the draft query view `/DMO/R_AGENCYDRAFT` and add the new field in the field list. Since you have already extended the database table `/DMO/AGENCY_D` with the new field, you can use the data source alias `Agency` defined in the draft query view in a path expression to access the field.

```
extend view entity /DMO/R_AgencyDraft with
{
```

```
Agency./dmo/zzsloganzag as /DM0/ZZSloganZAG
}
```

Develop Behavior Extensions

In the following section, you will develop behavior extensions which add new behavior to the reference business object /DM0/R_AgencyTP.

Extension Behavior

The extensions added in this section involve the following behavior:

- Determination /DM0/zzDetermineCountryCode: Determines the country code of an agency instance based on an entered dialling code.
- Determination /DM0/zzDetermineDiallingCode: Determines the dialling code of an agency instance based on an entered country code.
- Validation /DM0/zzValidateDiallingCode: Checks whether entered dialling codes have the valid format and whether they exist in a reference table.
- Action /DM0/zzChangeAddress: This is a factory action. For each instance that is provided to the method, the action creates one agency instance with the location related values of the input instance.

The naming pattern "zz . . ." was chosen to mimic a different package or namespace, since these artefacts would not be located in the same package or namespace in a real business scenario.

Required Extension Steps

In order to extend the business object with this new behavior, you need to develop a behavior definition extension artifact, where the new behavior is defined as well as a behavior implementation artifact in which the defined behavior is implemented.

Prerequisites

Extending the business object with the above mentioned behavior requires the business object to be extensibility enabled accordingly. The required extensibility enablement depends on the type of the behavior to be added respectively:

- Determinations and validations: For adding determinations and validations, the business object needs to be enabled for standard behavior as described in [Enabling Standard Behavior Extensions](#)
- Actions: For adding actions, the business object needs to be enabled for non-standard behavior as described in [Enabling Non-Standard Behavior and Field-Related Behavior](#).

[Defining the Extension Behavior](#)

In this chapter, you create a behavior definition extension and define the behavior to be added via the extension.

[Implementing the Extension Behavior](#)

In this chapter, you implement the extension behavior you have defined in the previous step.

[Exposing the Action Extension for UI Consumption](#)

In this chapter, you define CDS annotations to expose the action implemented in the previous step on the UI.

Defining the Extension Behavior

In this chapter, you create a behavior definition extension and define the behavior to be added via the extension.

Procedure

1. In the context menu of the behavior definition `/DMO/R_AGENCYTP`, choose the option `New Behavior Extension`.
2. In the BDEF extension creation dialogue, enter the name of the package where the BDEF extension is to be stored as well as the name `/DMO/ZZ_X_COUNTRY_R_AGENCYTP`, a description and the BO interface `/DMO/I_AGENCYTP`.

The resulting BDEF extension looks like this and is ready for defining the extension behavior as described in the following section.

```
extension using interface /dmo/i_agencytp
implementation in class /dmo/zz_bp_x_cntry_r_agencytp unique;
```

3. Below the generated code in the behavior definition extension, add the keywords `extend behavior for /DMO/Agency { }`. The extension behavior can now be defined within the curly brackets.

```
extension using interface /dmo/i_agencytp
implementation in class /dmo/zz_bp_x_cntry_r_agencytp unique;

extend behavior for /DMO/Agency
{ }
```

4. Within the curly brackets, define the validation `/DMO/zzValidateDiallingCode` with the trigger time on `save` and the field trigger `PhoneNumber`.

```
validation /DMO/zzValidateDiallingCode on save { field PhoneNumber; }
```

5. Extend the draft determine action `Prepare` with the defined validation `/DMO/zzValidateDiallingCode` to ensure the adequate handling of state messages.

```
extend draft determine action Prepare
{
  validation /DMO/zzValidateDiallingCode;
}
```

6. Define the determination `/DMO/zzDetermineCountryCode` with the trigger time on `modify` and the field trigger `PhoneNumber`.

```
determination /DMO/zzDetermineCountryCode on modify { field PhoneNumber; }
```

7. Define the determination `/DMO/zzDetermineDiallingCode` with the trigger time on `modify` and the field trigger `CountryCode`.

```
determination /DMO/zzDetermineDiallingCode on modify { field CountryCode; }
```

8. Define the factory action `/DMO/zzChangeAddress` with the result cardinality `[1]`. Use `( authorization: update )` for the action.

```
factory action /DMO/zzChangeAddress [1];
```

The resulting behavior definition extension looks like this:

```

extension using interface /dmo/i_agencytp
implementation in class /dmo/zz_bp_x_cntry_r_agencytp unique;

extend behavior for /DMO/Agency
{

    validation /DMO/zzValidateDiallingCode on save { field PhoneNumber; }

    determination /DMO/zzDetermineCountryCode on modify { field PhoneNumber; }

    determination /DMO/zzDetermineDiallingCode on modify { field CountryCode; }

    extend draft determine action Prepare
    {
        validation /DMO/zzValidateDiallingCode;
    }

    factory action ( authorization : update ) /DMO/zzChangeAddress [1];

}

```

i Note

The names of extension elements must start with the namespace prefix used by the related package. Since the namespace `/DMO/` is used to ship the whole agency reference scenario, the prefix `/DMO/` is also used for extension elements in this example. In a productive environment, the namespace prefix of extensions elements differs from the namespace of the original business object.

9. Use the quickfix offered for the line `implementation in class /dmo/zz_bp_x_cntry_r_agencytp unique;` to create the behavior implementation class for the behavior definition extension.
10. In the context menu of the behavior definition `/DMO/C_AGENCYTP`, choose the option `New Behavior Extension`.
11. In the BDEF extension creation dialogue, enter the name of the package where the BDEF extension is to be stored as well as the name `/DMO/ZZ_X_COUNTRY_C_AGENCYTP` and a description.

In the generated BDEF, remove the keywords in the line `after extension for projection`. The resulting BDEF extension looks like this and is ready for defining the extension behavior as described in the following section.

```

extension for projection;

extend behavior for /DMO/Agency {
}

```

12. Extend the projection layer of the original business object with the action `/DMO/zzChangeAddress` from the base BDEF extension.

```

extension for projection;

extend behavior for /DMO/Agency
{
    use action /DMO/zzChangeAddress;
}

```

Result

You have defined the extension behavior in the behavior definition extension. You can now implement the behavior in the created implementation class, see [Implementing the Extension Behavior](#).

Implementing the Extension Behavior

In this chapter, you implement the extension behavior you have defined in the previous step.

The implementation of behavior defined in a behavior extension follows the same approach as the implementation of behavior defined for an original business object. There is no special class syntax required related to extensibility. However, there are differences when it comes to accessing the original business object from extensions via EML.

i Note

An interface is the consolidated consumption point of a business object, which needs to be used by every consumer. This also applies to extenders who access the original business object via EML in their extensions. This is why EML statements in behavior extension implementations must always address the original business object via its interface.

If the keywords `IN LOCAL MODE` are used in an EML statement of a behavior extension, only the feature control and authorization implementations of this extension are bypassed. Feature control and authorization implementations of the original business object are not bypassed with such a statement.

The following sections describe the required steps to implement the extension behavior.

Prerequisite

You have defined the extension behavior and created the behavior implementation class for the extension as described in chapter [Defining the Extension Behavior](#).

Defining required Class Components

As a preparational step for the implementation of the validation and the determinations, you need to define a state area for the messages reported by the validation. Furthermore, the validation and the determinations require a lookup table for the dialling and country codes to work with. Define these components as follows:

1. In the public section of the behavior implementation class, define the constant `validate_dialling_code` with the type `string` and the value `VALIDATE_DIALLING_CODE`. You will use this constant as the state area for messages reported by the validation `validateDiallingCode`.

```
CLASS lhc_Agency DEFINITION INHERITING FROM cl_abap_behavior_handler.
```

```
  PUBLIC SECTION.
```

```
  CONSTANTS:
```

```
    validate_dialling_code    TYPE string VALUE 'VALIDATE_DIALLING_CODE' ##NO_TEXT.
```

2. The validation and the determinations to be developed will use a lookup table for the dialling codes and country codes. The content of the lookup table is initialized in the class constructor. Define the necessary types and the lookup table in the public section and populate the table with sample values in the method `class_constructor`.

```
CLASS lhc_Agency DEFINITION INHERITING FROM cl_abap_behavior_handler.
```

```
PUBLIC SECTION.
```

```
CONSTANTS:
```

```
    validate_dialling_code    TYPE string VALUE 'VALIDATE_DIALLING_CODE' ##NO_TEXT.
```

```
TYPES: BEGIN OF t_countries,
        number TYPE /dmo/phone_number,
        code   TYPE land1,
      END OF t_countries.
```

```
CLASS-DATA: countries TYPE STANDARD TABLE OF t_countries WITH KEY number.
```

```
CLASS-METHODS: class_constructor.
```

```
PRIVATE SECTION.
```

```
METHODS validateDiallingCode FOR VALIDATE ON SAVE
    IMPORTING keys FOR /DMO/Agency~/DMO/zzValidateDiallingCode.
METHODS determineCountryCode FOR DETERMINE ON MODIFY
    IMPORTING keys FOR /DMO/Agency~/DMO/zzDetermineCountryCode.
METHODS determineDiallingCode FOR DETERMINE ON MODIFY
    IMPORTING keys FOR /DMO/Agency~/DMO/zzDetermineDiallingCode.
METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
    IMPORTING REQUEST requested_authorizations FOR /DMO/Agency RESULT result.
METHODS changeAddress FOR MODIFY
    IMPORTING keys FOR ACTION /DMO/Agency~/DMO/zzChangeAddress.
```

```
ENDCLASS.
```

```
CLASS lhc_Agency IMPLEMENTATION.
```

```
METHOD validateDiallingCode.
ENDMETHOD.
```

```
METHOD determineCountryCode.
ENDMETHOD.
```

```
METHOD determineDiallingCode.
ENDMETHOD.
```

```
METHOD get_global_authorizations.
ENDMETHOD.
```

```
METHOD changeAddress.
ENDMETHOD.
```

```
METHOD class_constructor.
```

```

countries = VALUE #( ( number = '+1'    code = 'US' )
                      ( number = '+49'  code = 'DE' )
                      ( number = '+39'  code = 'IT' )
                      ( number = '+43'  code = 'AT' )
                      ( number = '+44'  code = 'GB' )
                      ( number = '+81'  code = 'JP' )
                      ( number = '+33'  code = 'FR' )
                      ( number = '+358' code = 'FI' )
                      ( number = '+385' code = 'HR' ) ).

```

```
ENDMETHOD.
```

```
ENDCLASS.
```

Implementing the Validation `validateDiallingCode`

1. Read the fields `PhoneNumber` and `CountryCode` of all instances to be validated along with their keys and store the result in an internal table.
2. Loop over all instances to be validated. For every instance, do the following:
 - o Invalidate the state messages for the state area `validate_dialling_code` to avoid that messages with the same state area are continuously added for the agency instance.
 - o If the field `PhoneNumber` is initial, continue with the next instance. Instances with an empty phone number don't need to be validated.
 - o If the first two characters of the phone number are `00`, then replace them with the `+` sign.
 - o If the first character of the phone number is not `+`, then append an appropriate message to the `reported` structure for this instance using the state area `validate_dialling_code`. Mark the field `PhoneNumber` in the `element` component.
 - o If there is no entry in the lookup table `countries` where the phone number matches the first two, three and four phone number characters of the looped instance and where the country code matches the one of the looped instance, then append an appropriate message to the `reported` structure for this instance using the state area `validate_dialling_code`. Mark the field `PhoneNumber` in the `element` component.

```
METHOD validateDiallingCode.
```

```

READ ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
  ENTITY /DMO/Agency
  FIELDS ( PhoneNumber CountryCode ) WITH CORRESPONDING #( keys )
  RESULT DATA(agencies).

LOOP AT agencies INTO DATA(agency).
  APPEND VALUE #( %tky          = agency-%tky
                  %state_area = validate_dialling_code ) TO reported-/DMO/Agency.

  IF agency-PhoneNumber IS INITIAL.
    CONTINUE.
  ENDIF.

  IF agency-PhoneNumber(2) = '00'.
    REPLACE FIRST OCCURRENCE OF '00' IN agency-Phonenumber WITH '+'.
  ENDIF.

```

```
IF agency-PhoneNumber(1) <> '+'.
```

```
    APPEND VALUE #( %tky          = agency-%tky
                    %state_area = validate_dialling_code
                    %msg        = NEW /dmo/zz_cx_agency_country( textid      = /dmo/zz_cx_agency_country
                                                                phonenumber = agency-PhoneNumber )
                    %element-PhoneNumber = if_abap_behv=>mk-on )
    T0 reported-/DMO/Agency.
```

```
ELSEIF NOT line_exists( countries[ number = agency-phonenummer(2) code = agency-CountryCode ] )
AND NOT line_exists( countries[ number = agency-phonenummer(3) code = agency-CountryCode ] )
AND NOT line_exists( countries[ number = agency-phonenummer(4) code = agency-CountryCode ] ).
```

```
    APPEND VALUE #( %tky          = agency-%tky
                    %state_area = validate_dialling_code
                    %msg        = NEW /dmo/zz_cx_agency_country( textid      = /dmo/zz_cx_agency_country
                                                                phonenumber = agency-PhoneNumber )
                    %element-PhoneNumber = if_abap_behv=>mk-on )
    T0 reported-/DMO/Agency.
```

```
ENDIF.
```

```
ENDLOOP.
```

```
ENDMETHOD.
```

Implementing the Determination **determineCountryCode**

1. Read the fields **PhoneNumber** and **CountryCode** of all instances to be processed along with their keys and store the result in the internal table **agencies**.
2. Delete all instances from the internal table **agencies** that already have a country code. This prevents the unnecessary execution and an infinite loop between the determinations **determineCountryCode** and **determineDiallingCode**.
3. Declare a table called **agencies_to_update** with the derived type for EML updates on the interface of the original BO.
4. Add the content of the **%tky** component of all instances from the internal table **agencies** to the table **agencies_to_update** whose phone number starts with one of the numbers from the lookup table **countries**. Add the corresponding country code from the lookup table.
5. Perform the EML update using the internal table **agencies_to_update**.

```
METHOD determineCountryCode.
```

```
    READ ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
      ENTITY /DMO/Agency
      FIELDS ( PhoneNumber CountryCode ) WITH CORRESPONDING #( keys )
      RESULT DATA(agencies).
```

```
    DELETE agencies WHERE CountryCode IS NOT INITIAL.
    DATA: agencies_to_update TYPE TABLE FOR UPDATE /dmo/i_agencytp.
```

```
    LOOP AT countries INTO DATA(country).
      DATA(country_with_00) = country-number.
      REPLACE FIRST OCCURRENCE OF '+' IN country_with_00 WITH '00'.
      LOOP AT agencies INTO DATA(agency)
        WHERE PhoneNumber CP country-number && '*'
        OR   PhoneNumber CP country_with_00 && '*'.
        APPEND VALUE #( %tky          = agency-%tky
                        countrycode = country-code ) TO agencies_to_update.
      ENDLOOP.
    ENDLOOP.
```

```
    MODIFY ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
      ENTITY /DMO/Agency
      UPDATE FIELDS ( countrycode ) WITH agencies_to_update
      REPORTED DATA(reported_modify).
```

```
ENDMETHOD.
```

Implementing the Determination `determineDiallingCode`

1. Read the fields `PhoneNumber` and `CountryCode` of all instances to be processed along with their keys and store the result in the internal table `agencies`.
2. Delete all instances from the internal table `agencies` that already have a phone number. This prevents the unnecessary execution and an infinite loop between the determinations `determineDiallingCode` and `determineCountryCode`.
3. Declare a table called `agencies_to_update` with the derived type for EML updates on the interface of the original BO.
4. Add the content of the `%tky` component of all instances from the internal table `agencies` to the table `agencies_to_update` whose country code matches one of the country codes from the lookup table `countries`. Add the corresponding phone number from the lookup table.
5. Perform the EML update using the internal table `agencies_to_update`.

`METHOD determineDiallingCode.`

```
READ ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
  ENTITY /DMO/Agency
    FIELDS ( PhoneNumber CountryCode ) WITH CORRESPONDING #( keys )
  RESULT DATA(agencies).
```

```
DELETE agencies WHERE PhoneNumber IS NOT INITIAL.
```

```
DATA: agencies_to_update TYPE TABLE FOR UPDATE /dmo/i_agencytp.
```

```
LOOP AT agencies INTO DATA(agency).
```

```
  DATA(PhoneNumber) = VALUE #( countries[ code = agency-countrycode ]-number OPTIONAL ) .
```

```
  IF PhoneNumber IS NOT INITIAL.
```

```
    APPEND VALUE #( %tky = agency-%tky
                    phonenumber = PhoneNumber ) TO agencies_to_update.
```

```
  ENDIF.
```

```
ENDLOOP.
```

```
MODIFY ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
```

```
  ENTITY /DMO/Agency
```

```
    UPDATE FIELDS ( PhoneNumber ) WITH agencies_to_update
```

```
    REPORTED DATA(reported_modify).
```

`ENDMETHOD.`

Implementing the Action `changeAddress`

1. Read the values of the fields `CountryCode`, `PostalCode`, `City` and `Street` of all provided instances.
2. Declare an internal table called `agencies_to_create` with the derived type for create operations on the BO interface `/DMO/I_AgencyTP`.
3. Append all read agency instances to the table `agencies_to_create`. For each instance in `agencies_to_create`, assign the `%cid` from the read result and set the draft indicator to `if_abap_behv=>mk-on` respectively.
4. Finally, create the instances using the prepared internal table `agencies_to_create`. Forward the mapped response of the EML CREATE statement to the mapped response of the factory action.

`METHOD changeAddress.`

```
READ ENTITIES OF /DMO/I_AgencyTP IN LOCAL MODE
  ENTITY /DMO/Agency
    FIELDS ( CountryCode PostalCode City Street ) WITH CORRESPONDING #( keys )
  RESULT DATA(agencies).
```

```
DATA: agencies_to_create TYPE TABLE FOR CREATE /DMO/I_AgencyTP.
```

```
LOOP AT keys INTO DATA(key).
```

```

READ TABLE agencies INTO DATA(agency) WITH KEY id COMPONENTS %tky = key-%tky.
IF sy-subrc EQ 0.
  APPEND CORRESPONDING #( agency EXCEPT agencyid ) TO agencies_to_create ASSIGNING FIEL
    <agency_to_create>-%cid = key-%cid.
    <agency_to_create>-%is_draft = if_abap_behv=>mk-on. "The action always creates draft
ELSE.
  APPEND CORRESPONDING #( key MAPPING %fail = DEFAULT VALUE #( cause = if_abap_behv=>ca
ENDIF.
ENDLOOP.

MODIFY ENTITIES OF /DMO/I_AgencyTP IN LOCAL MODE
ENTITY /DMO/Agency
CREATE FIELDS ( CountryCode PostalCode City Street ) WITH agencies_to_create
MAPPED mapped.

ENDMETHOD.

```

Result

You have implemented the behavior of the extension. You can now expose the action for UI consumption as described in chapter [Exposing the Action Extension for UI Consumption](#).

Exposing the Action Extension for UI Consumption

In this chapter, you define CDS annotations to expose the action implemented in the previous step on the UI.

Required Steps

In order to expose action extensions on the UI, any field in the CDS view extend of the projection layer is required, which the UI annotations for the action extensions can refer to. In the following example, the field `/DMO/ZZSloganZAG` added in chapter [Develop Data Model Extensions](#) is used as reference destination for the action annotations.

Procedure

In the CDS view extend `/DMO/ZZ_X_SLOGAN_C_AGENCY`, add UI annotations for the action `/DMO/createFromTemplate` to the field `/DMO/ZZSloganZAG` in order to display the action on the UI list report page and on the UI object page.

```

extend view entity /DMO/C_AgencyTP with
{
  @UI: {
    lineItem: [ { position: 25, importance: #HIGH },
    fieldGroup: [ { position: 25, qualifier: 'General_FG' } ],
    identification: [ { position: 10, type: #FOR_ACTION, dataAction: '/DMO/zzChangeAddress', label:
    }
    @Search.defaultSearchElement: true
    @Search.fuzzinessThreshold: 0.8
    Agency./DMO/ZZSloganZAG
  }
}

```

Develop Node Extensions

In the following section, you will extend the reference business object `/DMO/R_AgencyTP` with an extension node. This extension node is thereby added as a sub node to the existing `/DMO/Agency` node in the original business object.

Extension Node

The extension node `/DMO/ZZ_Review` is to be added to the original business object and shall enable the end user to create and evaluate reviews for each agency instance. A review includes a rating between one and five stars as well as a textual description. Readers of a review can rate it as helpful or not helpful. The number of helpful reviews is then displayed for each review instance.

Required Extension Steps

At first, you need to define the data model for the extension node that you want to include in the existing data model. The created CDS view `/DMO/ZZ_R_Agency_ReviewTP` must hereby contain a to-parent association to the CDS view `/DMO/R_AgencyTP`. This association needs to be propagated to the projection views `/DMO/ZZ_C_Agency_ReviewTP` and `/DMO/ZZ_I_Agency_ReviewTP` and redirected to the corresponding layers.

Next, you need to extend the data model of the original BO with a composition that defines the node `/DMO/ZZ_R_AGENCY_REVIEWTP` as a sub node of the `/DMO/R_AgencyTP` node. This compositional relationship needs to be then propagated throughout the stack and redirected to the corresponding layers.

In the next step, you need to create a behavior definition extension for the original BDEF `/DMO/R_AgencyTP`. This BDEF extension defines the behavior of the `/DMO/ZZ_Review` node and extends the behavior of the original BO with a create-by-association operation using the defined composition. This extension is needed to enable creating Review instances for a given Agency instance. The added create-by-association needs to be exposed in a projection BDEF extension. Furthermore, you need to extend the draft determine action `prepare` of the original BO with the validation `/DMO/ratingInRange` which is part of the node logic, in order to ensure an adequate handling of state messages.

Afterwards, you need to implement the behavior defined in the behavior definition extension.

Lastly, you need to extend the service definition `/DMO/UI_AGENCY` with the CDS view `/DMO/ZZ_C_Agency_ReviewTP` to include the node into the exposure for service consumption.

Prerequisite

You have enabled the business object for node extensibility as described in [Enabling Node Extensions](#).

[Defining the Data Model for the Extension Node](#)

In this chapter, you define the database table and the CDS data model for the extension node.

[Adding the Extension Node to the Data Model](#)

In this chapter, you extend the data model of the original business object with a composition that defines the `/DMO/ZZ_Review` node as a subnode of the `/DMO/Agency` node.

[Extending the Behavior for the Node Extension](#)

In this chapter, you define the behavior for the extension node and integrate it into the behavior of the original business object.

[Implementing the Behavior for the Node Extension](#)

In this chapter, you implement the behavior defined for the node in the previous chapter.

[Including the Extension Node in the Service Exposure](#)

In this step, you create a service definition extension for the review node and add it to the service of the original BO.

Defining the Data Model for the Extension Node

In this chapter, you define the database table and the CDS data model for the extension node.

Creating the Database Table

Create the active database table with the fields required for persisting review data:

```
@EndUserText.label : 'Agency Review: Review. Active Instances'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/zz_agm_reva {

    key client          : abap.clnt not null;
    key agency_id       : /dmo/agency_id not null;
    key review_id       : /dmo/zz_review_id not null;
    rating              : /dmo/zz_rating;
    free_text_comment   : /dmo/zz_free_text_comment;
    helpful_count       : /dmo/zz_helpful_count;
    helpful_total       : /dmo/zz_helpful_total;
    @AbapCatalog.anonymizedWhenDelivered : true
    reviewer            : abp_creation_user;
    local_created_at    : abp_creation_tstmpl;
    local_last_changed_at : abp_locinst_lastchange_tstmpl;

}
```

The draft database table can be generated once the BDEF extension for the node has been defined.

Defining the CDS Data Model

1. Create a CDS view called `/DMO/ZZ_I_Agency_Review` which selects from the database table `/dmo/zz_agm_reva`.

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Agency Review'

define view entity /DMO/ZZ_I_Agency_Review
as select from /dmo/zz_agm_reva as Review
{
    key Review.agency_id          as AgencyId,
    key Review.review_id         as ReviewId,
    Review.rating                 as Rating,
    Review.free_text_comment      as FreeTextComment,
    Review.helpful_count          as HelpfulCount,
    Review.helpful_total          as HelpfulTotal,
    @Semantics.user.createdBy: true
    Review.reviewer               as Reviewer,
    @Semantics.systemDateTime.createdAt: true
    Review.local_created_at       as LocalCreatedAt,
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    Review.local_last_changed_at as LocalLastChangedAt
}
```

2. Create a CDS view called `/DMO/ZZ_R_Agency_ReviewTP` which selects from the created CDS view `/DMO/ZZ_I_Agency_Review`. Add an association to the parent entity `/DMO/R_AgencyTP` and expose this association in the field list. You will add the corresponding composition to the Agency node in chapter [Adding the Extension Node to the Data Model](#).

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Agency Review'
define view entity /DMO/ZZ_R_Agency_ReviewTP
as select from /DMO/ZZ_I_Agency_Review
association to parent /DMO/R_AgencyTP as _Agency on $projection.AgencyId = _Agency.AgencyID
{
    key AgencyId,
    key ReviewId,
    Rating,
    FreeTextComment,
    HelpfulCount,
    HelpfulTotal,
    Reviewer,
}
```

```

        LocalCreatedAt,
        LocalLastChangedAt,
        _Agency
    }

```

3. Create the interface projection CDS view called `/DM0/ZZ_I_Agency_ReviewTP` which projects the CDS view `/DM0/ZZ_R_Agency_ReviewTP`. Redirect the association `_Agency` to the interface layer of the `/DM0/Agency` node.

```

@Metadata.ignorePropagatedAnnotations: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Agency Review'
define view entity /DM0/ZZ_I_Agency_ReviewTP
    as projection on /DM0/ZZ_R_Agency_ReviewTP
{
    key AgencyId,
    key ReviewId,
    Rating,
    FreeTextComment,
    HelpfulCount,
    HelpfulTotal,
    @Semantics.user.createdBy: true
    Reviewer,
    @Semantics.systemDateTime.createdAt: true
    LocalCreatedAt,
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    LocalLastChangedAt,
    /* Associations */
    _Agency : redirected to parent /DM0/I_AgencyTP
}

```

4. Create the CDS projection view called `/DM0/ZZ_C_Agency_ReviewTP` which projects the CDS view `/DM0/ZZ_R_Agency_ReviewTP`. Redirect the association `_Agency` to the projection layer of the `/DM0/Agency` node.

```

@EndUserText.label: 'Agency Review'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true

@ObjectModel.semanticKey: ['Reviewer']

define view entity /DM0/ZZ_C_Agency_ReviewTP
    as projection on /DM0/ZZ_R_Agency_ReviewTP
{
    key AgencyId,
    key ReviewId,
    Rating,
    FreeTextComment,
    HelpfulCount,
    HelpfulTotal,
    Reviewer,
    LocalCreatedAt,
    LocalLastChangedAt,
    /* Associations */
    _Agency : redirected to parent /DM0/C_AgencyTP
}

```

5. Create a metadata extension for the CDS projection view `/DM0/ZZ_C_Agency_ReviewTP` for the display on the UI.

i Expand the following code sample to view the source code of the metadata extension `/DM0/ZZ_C_Agency_ReviewTP`.

Sample Code

```

@Metadata.layer: #CUSTOMER

@UI: { headerInfo: {
    typeName: 'Review',

```

```

        typeNamePlural: 'Reviews',
        title: { type: #STANDARD, value: 'ReviewId' }
    },
    presentationVariant: [{
        sortOrder: [{ by: 'LocalCreatedAt', direction: #DESC }],
        visualizations: [{type: #AS_LINEITEM}]
    }]
}

annotate view /DM0/ZZ_C_Agency_ReviewTP with
{

    @UI.facet: [
        {
            id:          'Review',
            purpose:      #STANDARD,
            type:         #IDENTIFICATION_REFERENCE,
            label:        'Review',
            position:     10
        },
        {
            id:          'HelpfulID',
            purpose:      #HEADER,
            type:         #DATAPOINT_REFERENCE,
            label:        'Helpful',
            position:     10,
            targetQualifier: 'HelpfulCount'
        }
    ]

    @UI.hidden: true
    AgencyId;
    @UI.hidden: true
    ReviewId;

    @UI:{
        dataPoint: {
            title: 'Rating'
            ,visualization: #RATING
            ,targetValue: 5
        },
        lineItem:
        [
            {
                position: 10,
                label: 'Rating',
                importance: #HIGH
                ,valueQualifier: 'Rating'
                ,type: #AS_DATAPOINT
            }
        ],
        identification: [
            {
                position: 10,
                label: 'Rating',
                importance: #HIGH
                ,type: #AS_DATAPOINT
            }
        ]
    }

    Rating;

    @UI.lineItem: [{ position: 20 }]
    @UI.identification: [{ position: 20 }]
    @UI.multiLineText: true
    FreeTextComment;

    @UI:{
        dataPoint: {
            title: 'Helpful',
            targetValueElement: 'HelpfulTotal'
            ,visualization: #PROGRESS

```

```

    },
    lineItem: [
        {
            position: 50,
            label: 'Helpful',
            importance: #HIGH
            ,type: #AS_DATAPOINT
            ,valueQualifier: 'Progress'
        }
    ]
}
HelpfulCount;

@UI.hidden: true
HelpfulTotal;

@UI.lineItem: [
    { type: #FOR_ACTION, dataAction: '/DM0/reviewWasHelpful', label: 'Helpful' },
    { type: #FOR_ACTION, dataAction: '/DM0/reviewWasNotHelpful', label: 'Not Helpful' },
    { position: 3 }
]
@UI.identification: [
    { type: #FOR_ACTION, dataAction: '/DM0/reviewWasHelpful', label: 'Helpful' },
    { type: #FOR_ACTION, dataAction: '/DM0/reviewWasNotHelpful', label: 'Not Helpful' },
    { position: 3 }
]
Reviewer;

@UI.hidden: true
LocalCreatedAt;
@UI.hidden: true
LocalLastChangedAt;
}

```

Result

You have defined the data model for the extension node. You can now add it to the data model of the original business object, see [Adding the Extension Node to the Data Model](#).

Adding the Extension Node to the Data Model

In this chapter, you extend the data model of the original business object with a composition that defines the `/DM0/ZZ_Review` node as a subnode of the `/DM0/Agency` node.

Procedure

1. Create a CDS extension view called `/DM0/ZZ_X_REVIEW_R_AGENCY` for the CDS view `/DM0/R_AgencyTP`. Extend the CDS view `/DM0/R_AgencyTP` with a compositional association to the entity `/DM0/ZZ_R_Agency_ReviewTP` and define the alias `/DM0/ZZ_ReviewZAG` for the composition. Expose this alias in the field list of the view.

i Note

If you use a namespace encompassed with slashes as shown in this example, no preceding underscore must be used for the association alias.

```

extend view entity /DM0/R_AgencyTP with
composition [0..*] of /DM0/ZZ_R_Agency_ReviewTP as /DM0/ZZ_ReviewZAG
{
    /DM0/ZZ_ReviewZAG
}

```

i Note

The names of extension elements must start with the namespace prefix used by the related package. Since the namespace `/DMO/` is used to ship the whole agency reference scenario, the prefix `/DMO/` is also used for extension elements in this example. In a productive environment, the namespace prefix of extensions elements differs from the namespace of the original business object. This is indicated by the addition `ZZ` after the `/DMO/` namespace.

2. Create a CDS extension view called `/DMO/ZZ_X_REVIEW_I_AGENCY` for the CDS view `/DMO/I_AgencyTP`. Extend the CDS view `/DMO/I_AgencyTP` with the composition added to the CDS view `/DMO/R_AgencyTP` in the previous step and redirect it to the CDS view `/DMO/ZZ_I_Agency_ReviewTP`. Since you have already extended the CDS view `/DMO/R_AgencyTP` with the composition, you can use the data source alias `Agency` defined in the CDS view `/DMO/I_AgencyTP` in a path expression to access the composition via its alias `/DMO/ZZ_ReviewZAG`.

```
extend view entity /DMO/I_AgencyTP with
{
  Agency./DMO/ZZ_ReviewZAG : redirected to composition child /DMO/ZZ_I_Agency_ReviewTP
}
```

3. Create a CDS extension view called `/DMO/ZZ_X_REVIEW_C_AGENCY` for the CDS view `/DMO/C_AgencyTP`. Extend the CDS view `/DMO/C_AgencyTP` with the composition added to the CDS view `/DMO/R_AGENCYTP` in step one and redirect it to the CDS view `/DMO/ZZ_C_Agency_ReviewTP`. Since you have already extended the CDS view `/DMO/R_AGENCYTP` with the composition, you can use the data source alias `Agency` defined in the CDS view `/DMO/C_AgencyTP` in a path expression to access the composition via its alias `/DMO/ZZ_ReviewZAG`.

Define UI annotations that enable the display of corresponding review instances on the UI object page of agency instances.

```
extend view entity /DMO/C_AgencyTP with
{
  @UI.facet: [
    {
      id:          'Review',
      purpose:     #STANDARD,
      type:        #LINEITEM_REFERENCE,
      label:       'Review',
      position:    20,
      targetElement: '/DMO/ZZ_ReviewZAG'
    }
  ]
  Agency./DMO/ZZ_ReviewZAG : redirected to composition child /DMO/ZZ_C_Agency_ReviewTP
}
```

Result

You have added the extension node to the CDS data model. In the next step, you define its behavior and include it in the behavior of the original business object, see [Extending the Behavior for the Node Extension](#).

Extending the Behavior for the Node Extension

In this chapter, you define the behavior for the extension node and integrate it into the behavior of the original business object.

Procedure

1. In the context menu of the behavior definition `/DMO/R_AGENCYTP`, choose the option **New Behavior Extension**.
2. In the BDEF extension creation dialogue, enter the name of the package where the BDEF extension is to be stored as well as the name `/DMO/ZZ_X_REVIEW_R_AGENCYTP`, a description and the BO interface `/DMO/I_AGENCYTP`.

The resulting BDEF extension looks like this and is ready for defining the extension behavior as described in the following section.

```
extension using interface /dmo/i_agencytp
implementation in class /dmo/zz_bp_x_review_r_agencytp unique;
```

3. Below the generated code in the behavior definition extension, add the keywords `extend behavior for /DMO/Agency { }`. The extension behavior can now be defined within the curly brackets.

```
extension using interface /dmo/i_agencytp
implementation in class /dmo/zz_bp_x_review_r_agencytp unique;

extend behavior for /DMO/Agency
{

}
```

4. In order to ensure adequate message handling, you need to add validations of the extension node to the draft determine action `Prepare` of the original business object.

i Note

The validation `/DMO/ratingInRange` has not been defined so far. You will catch up on that in the definition section of the extension node.

```
extend behavior for /DMO/Agency
{

    extend draft determine action Prepare{
        validation /DMO/ZZ_Review~/DMO/ratingInRange;
    }
}
```

5. You will define the update and the delete operation in the definition section of the extension node. However, in order to enable the creation of `Review` instances from a given `Agency` instance of the root node, you need to extend the original business object with a create-by-association operation using the defined composition `/DMO/ZZ_ReviewZAG`.

```
extend behavior for /DMO/Agency
{

    extend draft determine action Prepare{
        validation /DMO/ZZ_Review~/DMO/ratingInRange;
    }
    association /DMO/ZZ_ReviewZAG { create; with draft; }
}
```

6. Define the behavior for the extension node below the curly brackets of the `extend behavior` section.

Use the keywords `using /DMO/ZZ_I_AGENCY_REVIEWTP` behind the `define behavior` statement to establish a symmetrical relation between the node of the original BO and the node of the extension with respect to their interfaces.

Since extension nodes are always in a to-parent relationship, the lock and authorization behavior is modeled as dependent. An `etag master` is defined as recommended for extension nodes. Note that the extension node contains the reverse association `_Agency` to the root node. It is recommended that you specify the respective `Etag` field as read-only in the behavior definition, using the field characteristic `readOnly`.

Ancestor associations don't need to be defined in this case as the root node is reachable directly via the `_Agency` association.

```
extension using interface /dmo/i_agencytp
implementation in class /dmo/zz_bp_x_review_r_agencytp unique;
```

```

extend behavior for /DMO/Agency
{
    extend draft determine action Prepare{
        validation /DMO/ZZ_Review~/DMO/ratingInRange;
    }
    association /DMO/ZZ_ReviewZAG { create; with draft; }
}

define behavior for /DMO/ZZ_R_AGENCY_REVIEWTP alias /DMO/ZZ_Review using /DMO/ZZ_I_AGENCY_REV
persistent table /DMO/ZZ_AGN_REVA
draft table /DMO/ZZ_AGN_REVD
etag master LocalLastChangedAt
lock dependent
authorization dependent
late numbering
{
    update ( features : instance );
    delete ( features : instance );

    field ( readonly ) ReviewID, AgencyID, HelpfulCount, HelpfulTotal, Reviewer, LocalCreatedAt

    action ( authorization : update ) /DMO/reviewWasHelpful result [1] $self;
    action ( authorization : update ) /DMO/reviewWasNotHelpful result [1] $self;

    validation /DMO/ratingInRange on save { create; field Rating; }

    association _Agency { with draft; }

    mapping for /dmo/zz_agn_reva
    {
        AgencyId = agency_id;
        ReviewId = review_id;
        Rating = rating;
        FreeTextComment = free_text_comment;
        HelpfulCount = helpful_count;
        HelpfulTotal = helpful_total;
        Reviewer = reviewer;
        LocalCreatedAt = local_created_at;
        LocalLastChangedAt = local_last_changed_at;
    }
}

```

7. Use the quickfix offered for the line implementation in class zbp_cs_r_agency_ext_review unique;
8. Generate the draft table by using the quick fix offered for line draft table /DMO/ZZ_AGN_REVD.
9. In the context menu of the behavior definition /DMO/C_AGENCYTP, choose the option New Behavior Extension.
10. In the BDEF extension creation dialogue, enter the name of the package where the BDEF extension is to be stored as well as the name /DMO/ZZ_X_REVIEW_C_AGENCYTP and a description.

In the generated BDEF, remove the keywords in the line after extension for projection. The resulting BDEF extension looks like this and is ready for defining the extension behavior as described in the following section.

```

extension for projection;

extend behavior for /DMO/Agency {
}

```

11. Extend the projection layer of the original business object with the composition /DMO/ZZ_ReviewZAG from the base BDEF extension.

```

extension for projection;

extend behavior for /DMO/Agency
{

```

```

    use association /DMO/ZZ_ReviewZAG { create; with draft; }
}

```

12. Define the behavior for the extension node on the projection layer. Use the update and the delete operation, the actions /DMO/reviewWasHelpful and /DMO/reviewWasNotHelpful as well as the reverse association to the parent from the base BDEF extension.

```

extension for projection;

extend behavior for /DMO/Agency
{
    use association /DMO/ZZ_ReviewZAG { create; with draft; }
}

define behavior for /DMO/ZZ_C_Agency_ReviewTP alias /DMO/ZZ_Review
{
    use update;
    use delete;

    use action /DMO/reviewWasHelpful;
    use action /DMO/reviewWasNotHelpful;

    use association _Agency { with draft; }
}

```

Result

You have extended the behavior of the original business object for the node extension. You can now implement the defined behavior, see [Implementing the Behavior for the Node Extension](#).

Implementing the Behavior for the Node Extension

In this chapter, you implement the behavior defined for the node in the previous chapter.

The implementation of behavior defined in a behavior extension follows the same approach as the implementation of behavior defined for an original business object. There is no special class syntax required related to extensibility. However, there are differences when it comes to accessing the original business object from extensions via EML.

i Note

An interface is the consolidated consumption point of a business object, which needs to be used by every consumer. This also applies to extenders who access the original business object via EML in their extensions. This is why EML statements in behavior extension implementations must always address the original business object via its interface.

If the keywords `IN LOCAL MODE` are used in an EML statement of a behavior extension, only the feature control and authorization implementations of this extension are bypassed. Feature control and authorization implementations of the original business object are not bypassed with such a statement.

The following sections describe the required steps to implement the extension behavior.

Implementing the Method `get_instance_features`

Read all reviewers of review instances that need to be validated.

Enable the update and the delete operation for all reviewers with an existing user.

```

METHOD get_instance_features.
  READ ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
  ENTITY /dmo/zz_review
  FIELDS ( reviewer )
  WITH CORRESPONDING #( keys )
  RESULT DATA(reviews)
  FAILED failed.

  result = VALUE #(
    FOR review IN reviews
    LET enabled = COND #( WHEN review-reviewer = cl_abap_context_info=>get_user_technical_name(
      THEN if_abap_behv=>fc-o-enabled
      ELSE if_abap_behv=>fc-o-disabled ) IN
      (
        %tky = review-%tky
        %update = enabled
        %delete = enabled
      )
    ).
ENDMETHOD.

```

Implementing the Method reviewWasHelpful

Read all fields of review instances the action is executed on.

Increment the number of helpful votes as well as the number of total votes by 1 for every review instance in an internal table and append the instances to the result table of the action.

Update the number of helpful votes and the number of total votes for the respective instances using the updated internal table.

```

METHOD reviewwashelpful.
  READ ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
  ENTITY /dmo/zz_review
  ALL FIELDS WITH CORRESPONDING #( keys )
  RESULT DATA(reviews)
  FAILED failed.

  LOOP AT reviews ASSIGNING FIELD-SYMBOL(<review>).
    <review>-helpfulcount += 1.
    <review>-helpfultotal += 1.

    APPEND VALUE #(
      %tky   = <review>-%tky
      %param = CORRESPONDING #( <review> )
    ) TO result.
  ENDLOOP.

  MODIFY ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
  ENTITY /dmo/zz_review
  UPDATE FIELDS ( helpfulcount helpfultotal )
  WITH CORRESPONDING #( reviews ).
ENDMETHOD.

```

Implementing the Method reviewWasNotHelpful

Read all fields of review instances the action is executed on.

Increment the number of total votes by 1 for every review instance in an internal table and append the instances to the result table of the action.

Update the number of total votes for the respective instances using the updated internal table.

```

METHOD reviewwasnothelpful.
  READ ENTITIES OF /dmo/i_agencytp IN LOCAL MODE

```

```

ENTITY /dmo/zz_review
  ALL FIELDS WITH CORRESPONDING #( keys )
  RESULT DATA(reviews)
  FAILED failed.

LOOP AT reviews ASSIGNING FIELD-SYMBOL(<review>).
  <review>-helpfultotal += 1.

  APPEND VALUE #(
    %tky      = <review>-%tky
    %param    = CORRESPONDING #( <review> )
  ) TO result.
ENDLOOP.

MODIFY ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
  ENTITY /dmo/zz_review
    UPDATE FIELDS ( helpfultotal )
    WITH CORRESPONDING #( reviews ).
ENDMETHOD.

```

Implementing the Method `ratingInRange`

Read the ratings of all review instances to be validated as well as the keys of their corresponding agency instances.

For every read review instance, clear the state area. If the rating of a review instance is higher than 5 or lower than 1, append a message to the reported structure that identifies the affected review and the corresponding agency instance.

```

METHOD ratinginrange.
  READ ENTITIES OF /dmo/i_agencytp IN LOCAL MODE
    ENTITY /dmo/zz_review
      FIELDS ( rating )
      WITH CORRESPONDING #( keys )
      RESULT DATA(reviews)
    ENTITY /dmo/zz_review BY \_agency
      FROM CORRESPONDING #( keys )
      LINK DATA(agency_links).

  LOOP AT reviews INTO DATA(review).
    APPEND VALUE #(
      %tky      = review-%tky
      %state_area = rating_in_range
    ) TO reported-/dmo/zz_review.

    IF review-rating > 5 OR review-rating < 1.
      APPEND VALUE #(
        %tky      = review-%tky
        %state_area = rating_in_range
        %msg      = NEW /dmo/zz_cx_agency_review( /dmo/zz_cx_agency_review=>rating_inva
        %element-rating = if_abap_behv=>mk-on
        %path-/DMO/Agency-%tky = VALUE #( agency_links[ KEY draft source-%tky = review-%tky ] .
      ) TO reported-/dmo/zz_review.
      APPEND VALUE #( %tky = review-%tky ) TO failed-/dmo/zz_review.
    ENDIF.
  ENDLOOP.
ENDMETHOD.

```

Implementing the Method `adjust_numbers`

Perform late numbering only, if the mapped response for the review node is not empty - in other words, if there are review instances that require keys.

Store all distinct agency IDs contained in the mapped response for the review in an internal table called `agencies`. Terminate the execution if there are no agency IDs in this internal table.

Create an internal table called `max_reviews` that contains the highest review ID from the table `/dmo/zz_agn_reva` for each agency ID of the `agencies` table.

For every distinct agency ID in the mapped response of the review node, store the highest review ID in the variable `max_review_id`. For every review with the current agency ID, assign a new review ID. The new review ID is calculated by incrementing the variable `max_review_id` by 1.

```
METHOD adjust_numbers.
  TYPES: t_mapped_review TYPE STRUCTURE FOR MAPPED LATE /dmo/zz_i_agency_reviewtp.
  DATA: agencies TYPE STANDARD TABLE OF /dmo/agency_id WITH KEY table_line.
  FIELD-SYMBOLS: <mapped_review> TYPE t_mapped_review.

  CHECK mapped-/dmo/zz_review IS NOT INITIAL.

  LOOP AT mapped-/dmo/zz_review ASSIGNING <mapped_review> GROUP BY <mapped_review>-%tmp-agencyid.
    APPEND <mapped_review>-%tmp-agencyid TO agencies.
  ENDLLOOP.

  ASSERT agencies IS NOT INITIAL.

  SELECT
    FROM /dmo/zz_agn_reva AS db
    JOIN @agencies AS itab
      ON db~agency_id = itab~table_line
  FIELDS DISTINCT
    db~agency_id,
    MAX( db~review_id ) AS max_review_id
  GROUP BY db~agency_id
  INTO TABLE @DATA(max_reviews).

  LOOP AT mapped-/dmo/zz_review INTO DATA(mapped_review_group) GROUP BY mapped_review_group-%tmp-
    DATA(max_review_id) = VALUE #( max_reviews[ agency_id = mapped_review_group-%tmp-agencyid ]-r
  LOOP AT mapped-/dmo/zz_review ASSIGNING <mapped_review> WHERE %tmp-agencyid = mapped_review-<
    <mapped_review>-agencyid = <mapped_review>-%tmp-agencyid.
    max_review_id += 1.
    <mapped_review>-reviewid = max_review_id.
  ENDLLOOP.
ENDLOOP.
ENDMETHOD.
```

Result

You have implemented the behavior for the extension node. You can now extend the service definition of the original business object with the extension node to provide it for service consumption, see [Including the Extension Node in the Service Exposure](#).

Including the Extension Node in the Service Exposure

In this step, you create a service definition extension for the review node and add it to the service of the original BO.

Procedure

1. In the context menu of the service definition `/DMO/UI_AGENCY`, choose the option **New Service Definition**.
2. In the service definition creation dialogue, enter the name of the package where the service definition extension is to be stored as well as the name `/DMO/ZZ_X_REVIEW_UI_AGENCY`, a description and the type **Extension**.

The resulting service definition extension looks like this and is ready for defining the extension as described in the following section.

```
extend service /DM0/UI_AGENCY with {
}
```

3. Expose the CDS projection view /DM0/ZZ_C_Agency_ReviewTP with the alias /DM0/ZZ_Review.

```
extend service /DM0/UI_AGENCY with {
  expose /DM0/ZZ_C_Agency_ReviewTP as /DM0/ZZ_Review;
}
```

Result

You have added the extension node to the service of the original business object. The node can now be consumed using the service /DM0/UI_AGENCY.

Testing RAP Business Objects

This section provides detailed step-by-step descriptions on how to develop automated tests for the different artifacts of RAP business objects and for related OData services.

Motivation

In times of ever shortening development cycles, continuous delivery and cloud deployments, manual regression tests of business applications are no reasonable choice for efficient software quality assurance. Regression tests are time and cost consuming and bear the risk of slips as they are performed for every new software release manually. Automated tests, however, guarantee an ever-constant test execution with a significantly lower consumption of resources.

Fiori applications built with the ABAP RESTful Application Programming Model require automated tests tailored for the different artefact types on multiple test levels.

Scope

The current version of this guide focuses on unit tests and on integration tests. Use the links below to navigate to the particular sections of this guide.

Test Design

The test classes you will develop in this section serve as examples on how automated tests can be developed. The chosen test design, however, is not supposed to serve as a blueprint. How the test classes are structured, how many test methods are created respectively and how the test code is allocated among these test methods, depends on the individual logic to be tested and on the desired degree of test granularity.

Content of this Guide

[Used Frameworks and Tools](#)

[Test Concepts](#)

[Unit Tests](#)

[Integration Tests](#)

Dependency Management with RAP BO Test Double Framework

The RAP BO Test Double Framework enables the isolation of RAP Business Objects as dependencies, thus helping to focus on the logic to be tested.

Used Frameworks and Tools

ABAP Unit

ABAP Unit is the framework for developing automated tests in ABAP. It provides various components and syntax elements for ABAP test classes.

For more information, see [ABAP Unit \(ABAP Keyword Documentation\)](#).

ABAP Unit in ABAP Development Tools

ABAP Development Tools provides support for the ABAP Unit framework and enables you to write, launch and evaluate ABAP Unit test classes.

Dependency Management

This test guide uses multiple test double frameworks that enable the isolation of dependencies from your code under test:

- SQL Test Double Framework

This framework enables you to manage database dependencies by creating test doubles for the database tables. These test doubles are then accessed during the test instead of the real database tables.

- RAP BO Test Double Framework

This framework is used to isolate dependencies on RAP business objects that result from EML statements. Such a dependency can be, for example, an (un)managed read/modify operation and/or a determination that is triggered by such an operation. The RAP BO Test Double Framework isolates such dependencies by replacing the actual provider implementations with mocked implementations. The RAP BO Test Double Framework provides these mocked implementations and calls them when respective EML statements are executed. The RAP BO Test Double Framework is available in two variants:

- Variant with transactional buffer double: This variant offers generic mock implementations for the operations create, read, update and delete as well as a doubled transactional buffer.
- Variant with mocked EML responses: With this variant, the framework provides a set of APIs that enable the definition of EML output responses for configurable input structures.

The current version of the RAP test guide uses the RAP BO Test Double Framework in the variant with transactional buffer double support.

An example for a custom BO double implementation is `ltd_custom_test_double` which is part of the class `/DMO/BP_C_SUPPLEMENT`.

- RAP Event Test Double Framework

This framework is used to test whether RAP Business Events implemented in RAP behavior pools are raised as expected.

The following table shows which test double framework is used in which chapter of this test guide respectively:

Chapter	SQL Test Double Framework	RAP BO Test Double Framework	RAP Event Test Double Framework
Unit Tests for a Behavior Implementation - Managed scenario	X	-	-
Unit Tests for a Behavior Implementation - Draft scenario	X	-	-
Developing a Test Class for the Supplement Reference Scenario	-	X	-
Developing Test Classes for the Agency Reference Scenario	-	-	X
EML Integration Tests	X	-	-
OData Integration Tests	-	-	-

Test Concepts

This chapter contains information about the different test concepts applied when testing RAP business objects.

Unit Tests

Unit tests focus on the smallest software components in the overall system architecture. Every unit is isolated from other units as much as possible in order to prevent the impact from external factors. Thus, the reason for failed unit tests can be directly traced back to the component, which has been tested.

Integration Tests

Whereas a unit test focuses on an individual software component, an integration test validates the integration of multiple software components contributing to the same application scenario. Even though every single software component of such a group might have been validated successfully by a unit test, the application scenario might not work as expected. For example, when the result determined by a software component is processed incorrectly by another software component. Integration tests check the correct interaction between software components and ensure the correct operability of their interfaces.

Basic Test Class Structure

There are essential test class elements that can be applied to any RAP test class. These elements form a basic test class structure which is explained in this chapter. The explained test class structure can serve as a template for RAP test classes.

Test Class Definition

The definition section of the test class contain basic meta information, reference variables and declarations for the fixture and test methods.

Assigning Test Properties

Depending on the complexity of your tests, choose a suitable duration time for your test class. Smaller test classes usually have an execution time of less than 5 seconds. In this case, you can use the `DURATION SHORT`. Depending on the respective run mode and runtime monitor, exceeding the chosen execution time can affect the results of your test.

By using test doubles, you prevent persistent data from being changed during your tests. If no persistent data or system settings are changed, you can use the `RISK LEVEL HARMLESS`. If the chosen risk level is higher than allowed in your system, your tests are not executed.

```
CLASS ltc_class_name DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.
```

Specifying the Test Relation

A test relation links your test class to the object to be tested. After specifying a test relation for a behavior implementation, a CDS view or a service, all of their related tests can be executed directly from their context menu in the Project Explorer of ADT. A test relation is specified above the class definition statement. For behavior implementations, the name of the corresponding behavior definition follows after the keyword `BDEF :`. For service bindings, the prefix `SRVB` is used.

```
"! @testing [BDEF:BDEF_name] | [CDS_view_name] | [SRVB:SRVB_name]
CLASS ltc_class_name DEFINITION FINAL FOR TESTING
```

Declaring Reference Variables

For unit tests of a behavior implementation, instantiate the corresponding class. For this purpose, declare a reference variable for this class in the private section. In the example below, the name of the tested behavior implementation class is `lhc_name`.

The CDS and SQL test double framework can be used to create test doubles that will be used in your tests. In order to use one of these frameworks, you need to declare reference variables for their interfaces as well.

For OData tests using the Local Client Proxy, declare a reference variable for its interface.

```
PRIVATE SECTION.
CLASS-DATA: class_under_test      TYPE REF TO lhc_name,
            cds_test_environment TYPE REF TO if_cds_test_environment,
            sql_test_environment TYPE REF TO if_osql_test_environment,
            go_client_proxy       TYPE REF TO /iwbp/if_cp_client_proxy.
```

Declaring Fixture and Test Methods

In addition to the declaration of your test methods, declare fixture methods. Fixture methods are used to set up and tear down the test environment which produces unique test behavior for every executed test. The declared test methods may have the same name as the methods to be tested.

```
CLASS-METHODS:
  class_setup,
  class_teardown.

METHODS:
  setup,
  teardown,
  method_name FOR TESTING.
```

Test Class Implementation

This section contains the implementation of the fixture and test methods declared above.

Implementing the Method `class_setup`

For unit tests of a behavior implementation, create an object of the corresponding class using the reference variable declared above. The addition `FOR TESTING` enables instantiating handler classes of behavior implementations.

For OData tests, create an instance of the Local Client Proxy and assign it to a reference variable which you will use in your test method to create the desired OData request. For a concrete example class which uses the Local Client Proxy interface to create Proxy instances for different systems, see [OData Tests: Refining the Test Class Structure](#).

Now, create a test double for the required database tables with the CDS test double framework or the SQL test double framework. Choose the CDS test double framework, if the data access is performed via the CDS entities in your code under test. Choose the SQL test double framework, if your test involves direct database access using OpenSQL.

To create a test double using the CDS test double framework, you can use the `create` method and provide the name of the CDS view selecting the database table that is to be doubled. As a result, the test double is called for data access during the test instead of the real database table. Assign the reference to the created test environment object to the interface reference variable declared above. For creating multiple test doubles, the CDS test double framework offers the method `create_for_multiple_cds`. If the test double is to be accessed not only via the CDS view, but also via Open SQL calls, additionally use the method `enable_double_redirection`. The SQL test double framework does not need to be used additionally for this case. The double redirection is automatically enabled if the method `create_for_multiple_cds` is used.

To create a test double using the SQL test double framework, you can use the `create` method and provide the name of the database tables that are to be doubled. As a result, the test doubles are called for data access during the test instead of the real database tables. Assign the reference to the created test environment object to the interface reference variable declared above.

If the data for your test doubles don't depend on the particular test cases of your test methods, you can insert the test data into your test doubles already in the class setup method. To do so, declare a table, type it with the created test double and insert the desired test data into the table. Next, insert the test data into your double by means of the populated table.

```
METHOD class_setup.
  CREATE OBJECT class_under_test FOR TESTING. "For unit tests of behavior implementations
  go_client_proxy = [Call of creation methods for Local Client Proxy]
  cds_test_environment = cl_cds_test_environment=>create( i_for_entity = 'CDS_view1' ).
  cds_test_environment = cl_cds_test_environment=>create_for_multiple_cds(
    VALUE #(
      ( i_for_entity = 'CDS_view1' )
      ( i_for_entity = 'CDS_view2' )
      ( i_for_entity = 'CDS_view3' )
    )
  ). "For multiple test doubles
  cds_test_environment->enable_double_redirection( ). "For Open SQL access to the test double
  sql_test_environment = cl_osql_test_environment=>create( i_dependency_list = VALUE #(
    ( 'database1' )
    ( 'database2' )
    ( 'database3' )
  ) ).

  " Insert test data into CDS test double
  DATA cds_mock_data TYPE STANDARD TABLE OF test_double.
  cds_mock_data = VALUE #( ( instance_data ) ).
  cds_test_environment->insert_test_data( cds_mock_data ).

  " Insert test data into SQL test double
  DATA sql_mock_data TYPE STANDARD TABLE OF test_double.
  sql_mock_data = VALUE #( ( instance_data ) ).
  sql_test_environment->insert_test_data( sql_mock_data ).
ENDMETHOD.
```

Implementing the Method `setup`

To make sure that your test methods are not influenced by data of other test methods, clear the contents of the test doubles before every test method execution if required.

```
METHOD setup.
  cds_test_environment->clear_doubles( ).
  sql_test_environment->clear_doubles( ).
ENDMETHOD.
```

Implementing the Method **teardown**

To make sure that the transactional buffer is reset for the involved entities after every test method execution, use the EML statement `ROLLBACK ENTITIES` in the teardown method.

```
METHOD teardown.
  ROLLBACK ENTITIES.
ENDMETHOD.
```

Implementing the Method **class_teardown**

At the end of the test, clear the test environments and the created test doubles.

```
METHOD class_teardown.
  cds_test_environment->destroy( ).
  sql_test_environment->destroy( ).
ENDMETHOD.
```

Implementing the Test Methods for Unit Tests

Irrespective of the object to be tested, implementing test methods for unit tests usually involves a number of fundamental steps.

At first, insert the test data into the test doubles using the respective test double framework. To do so, declare a table, type it with the created test double and insert the desired test data into the table. Next, insert the test data into your double by means of the populated table.

Next, the object under test is called. If you want to test a behavior implementation, call the method to be tested and provide the data required respectively, usually the keys of the instances in the test double. If you want to test a CDS view, perform a `SELECT` operation on the tested fields of the CDS view and save the results in an internal table.

Finally use assertion methods of the class `cl_abap_unit_assert` to check the returned values against the expected ones. For example, you can check whether a value returned by the test method equals an expected value. The assert checks are usually preceded by some preparatory tasks like reading the result of the operation performed during the test or populating internal tables used to compare with the result. Which of these tasks is relevant, depends on the respective RAP artefact and the logic to be tested.

i *Expand the following code sample to view the source code of the method `method_name`.*

Sample Code

```
METHOD method_name.

  " Insert test data into CDS test double
  DATA cds_mock_data TYPE STANDARD TABLE OF test_double.
  cds_mock_data = VALUE #( ( instance_data ) ).
  cds_test_environment->insert_test_data( cds_mock_data ).

  " Insert test data into SQL test double
  DATA sql_mock_data TYPE STANDARD TABLE OF test_double.
  sql_mock_data = VALUE #( ( instance_data ) ).
```

```

sql_test_environment->insert_test_data( sql_mock_data ).

" Run test for behavior implementation method
class_under_test->method_under_test(
  keys = CORRESPONDING #( mock_data ) ).

" Run test for CDS view
SELECT * FROM cds_view INTO TABLE @DATA(act_results).

" Preparatory tasks for assert checks
...

" Perform assert checks
cl_abap_unit_assert=>assert_equals( msg = 'message when assert fails' act = returned_value exp

ENDMETHOD.

```

Implementing the Test Methods for EML Integration Tests

Every EML integration test involves a number of fundamental steps. At first, perform the desired EML call and save the response parameters in respective variables. You can then use these variables to check whether the EML call has been executed as expected.

In cases where instance data needs to be read to validate the test result, you can now perform an EML READ call on the business object. Save the result in respective variables.

Finally use assertion methods of the class `cl_abap_unit_assert` to check the actual values against the expected ones. For example, you can check whether a value returned in the read result equals an expected value.

The EML statements used in the following code block are just examples and need to be chosen according to the given test case.

i Expand the following code sample to view the source code of the method `method_name`.

Sample Code

```

METHOD method_name.

" Create a Travel
MODIFY ENTITIES OF /dmo/i_travel_m
  ENTITY travel
    CREATE FIELDS ( ) WITH
    VALUE #( ( ) )
  MAPPED DATA(mapped_create)
  FAILED DATA(failed_create)
  REPORTED DATA(reported_create).

" Perform assert checks to validate intermediate result
cl_abap_unit_assert=>assert_equals( msg = 'message when assert fails' act = returned_value exp

" Read result
READ ENTITIES OF /dmo/i_travel_m
  ENTITY travel
    FIELDS ( )
    WITH VALUE #( ( ) )
  RESULT DATA(result)
  FAILED DATA(failed_read).

" Perform assert checks to validate test result
cl_abap_unit_assert=>assert_equals( msg = 'message when assert fails' act = returned_value exp
ENDMETHOD.

```

Implementing the Test Methods for OData Integration Tests

Every OData integration test involves a number of fundamental steps. At first, create the desired OData request and assign a reference to it to a variable.

Execute the created request and assign a reference to its response to another variable.

Check whether there is a response to the executed request.

Get the business data of the response and perform the desired assert checks on them.

i Expand the following code sample to view the source code of the method `method_name`.

☰ Sample Code

```
METHOD method_name.

"Create OData request
DATA(lo_request) = go_client_proxy->create_resource_for_entity_set( 'Booking' )->create_request_

"Execute OData request
DATA(lo_response) = lo_request->execute( ).

"Check request response
cl_abap_unit_assert=>assert_not_initial( lo_response ).

"Get the business data from the response
DATA ls_response_data TYPE STANDARD TABLE OF [CDS entity].
lo_response->get_business_data( IMPORTING et_business_data = ls_response_data ).

"Compare the business data to the expected result
cl_abap_unit_assert=>assert_equals( exp = [expected values] act = ls_response_data ).
ENDMETHOD.
```

Unit Tests

This section shows how the described concepts of unit tests apply in RAP. It contains step-by-step guides explaining how to develop unit tests for behavior implementations.

For unit tests of CDS entities, see [Testing CDS Entities](#) (ABAP Data Models guide).

Developing Unit Tests for a Behavior Implementation

In this chapter, you are guided through all the steps necessary to develop unit tests for behavior implementations.

The unit tests described in this section test methods from the managed and from the draft flight reference scenario. The tests cover typical RAP methods such as actions, determinations and validations, but don't test draft characteristics in particular.

Unit tests test the smallest functional units of an application. When accessing a business object from a consumer's perspective, e.g. via EML, a series of methods can be called by the framework. For instance, an EML MODIFY CREATE statement does not only trigger the create method, but also all determinations, validations as well as feature control and authorization implementations affected by the create method. On the other hand, methods like a validation cannot be called directly from a consumer's perspective as they are always triggered by another method, e.g. a create method. In order to limit the scope of a test on one single method, your unit tests will not act from a consumer's perspective, but they will instantiate the class under test and then call the methods to be tested directly. This enables you to prevent the impact from other methods and to focus on the methods that you actually want to test.

Developing a Test Class for the Managed Flight Reference Scenario

The test class you develop in this chapter tests methods for the travel entity in the managed flight reference scenario (package /DMO/FLIGHT_MANAGED).

Tested Methods

The validation `validateTravelStatus` checks if the value of the `overall_status` field is valid. In case of an invalid value, the key of the failed travel instance is appended to the `failed` structure and a corresponding message is stored in the `reported` structure.

The method `get_features` enables or disables the actions `rejectTravel` and `acceptTravel` as well as the association `_Booking` depending on the value of the `overall_status` field.

The validation `validate_customer` checks if the value of the `customer_id` field is valid. If the value is initial or not contained in the database table `/dmo/customer`, the key of the failed travel instance is appended to the `failed` structure and a corresponding message is stored in the `reported` structure.

The action `set_status_accepted` sets the value of the field `overall_status` to **Accepted**.

Contents

[Managed Scenario: Creating the Test Class](#)

[Managed Scenario: Providing Access to Tested Methods](#)

[Managed Scenario: Refining the Test Class Structure](#)

[Managed Scenario: Implementing the Test Methods](#)

Managed Scenario: Creating the Test Class

In this step, you will create the test class, which serves as the basis for the test methods to be developed.

To create the test class, proceed as follows:

1. Launch **ABAP Development Tools**.
2. In your ABAP project (or ABAP cloud project), select the behavior implementation `/DMO/BP_TRAVEL_M`.
3. In the tab **Test Classes**, click the button **Create Test Class**, type the word "test" and use code completion to insert the test class template.
4. Name the class as `ltc_managed` and remove the declaration and implementation of the dummy method `first_test`.

i Expand the following code sample to view the source code of the test class `ltc_managed`.

Sample Code

```
CLASS ltc_managed DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.
```

```

PRIVATE SECTION.

ENDCLASS.

CLASS ltc_managed IMPLEMENTATION.

ENDCLASS.

```

Managed Scenario: Providing Access to Tested Methods

The tested methods are private methods. Since you will want to call the methods in order to test them, you need to set the test class `ltc_managed` as a friend of the local handler class `lhc_travel` in the **Local Types** tab of the class `/DMO/BP_TRAVEL_M`. Since the test class is not known to the local handler class at this point, you need to specify the test class before using the addition **DEFINITION DEFERRED FOR TESTING**.

```

CLASS ltc_managed DEFINITION DEFERRED FOR TESTING.
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler
FRIENDS ltc_managed.

```

Managed Scenario: Refining the Test Class Structure

Now that you have created the basic test class structure, you can refine this structure by defining basic test properties, the test relation, variables, and fixture methods that provide the desired test configuration. The defined test class forms the basis for the test methods that you will implement in the upcoming chapters.

This topic covers the specific structure of the test class for the behavior implementation of the managed flight reference scenario. For general information on the structure of RAP test classes, see [Basic Test Class Structure](#).

Test Class Definition

The definition section of the test class contains basic meta information, reference variables and declarations for the fixture and test methods.

Assigning Test Properties

Since the tested methods have a relatively small size, the test code is not expected to have a longer execution time than 5 seconds. Hence, you can use the **DURATION SHORT**. Depending on the respective run mode and runtime monitor, exceeding the chosen execution time can affect the results of your test.

As you won't change persistent data or system settings with your unit tests, the **RISK LEVEL HARMLESS** can be used. If the chosen risk level is higher than allowed in your system, your tests are not executed.

```

CLASS ltc_managed DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.

```

Specifying the Test Relation

You will develop the unit tests for the business object `/DMO/I_Travel_M`. Specifying a test relation to this business object enables you to run your unit tests directly via the context menu of the business object's behavior definition.

```

"! @testing BDEF:/DMO/I_Travel_M
CLASS ltc_managed DEFINITION FINAL FOR TESTING
...

```

Declaring Reference Variables

In order to call the methods to be tested, you need to instantiate their class before. For this purpose, declare a reference variable for the class `lhc_travel` in the private section.

Since you will use the interfaces of the CDS Test Double Framework and of the SQL Test Double Framework to mock your test data, you need to declare reference variables for these interfaces as well.

```

PRIVATE SECTION.
CLASS-DATA: class_under_test      TYPE REF TO lhc_travel,
            cds_test_environment TYPE REF TO if_cds_test_environment,
            sql_test_environment TYPE REF TO if_osql_test_environment.

```

Declaring Fixture and Test Methods

In addition to the declaration of your test methods, declare fixture methods. Fixture methods are used to set up and tear down the test environment which produces unique test behavior for every executed test. The declared test methods may have the same name as the methods to be tested. For some of the methods, you will develop multiple test methods to ensure test stability and ease of maintenance.

i *Expand the following code sample to view the source code.*

Sample Code

```

CLASS-METHODS:
"! Instantiate class under test and setup test double frameworks
class_setup,

"! Destroy test environments and test doubles
class_teardown.

METHODS:
"! Reset test doubles
setup,

"! Reset transactional buffer
teardown,

"! Check instance with overall status 'Open'
validate_travel_status_open    FOR TESTING,

"! Check instance with overall status 'Accepted'
validate_travel_status_acpt    FOR TESTING,

"! Check instance with overall status 'Rejected'
validate_travel_status_rej     FOR TESTING,

"! Check instance with invalid overall status
validate_travel_status_invalid FOR TESTING,

"! Check returned features for instance with overall status 'Open'
get_features_open             FOR TESTING,

"! Check returned features for instance with overall status 'Accepted'
get_features_accepted         FOR TESTING,

"! Check returned features for instance with overall status 'Rejected'
get_features_rejected         FOR TESTING,

```

```
"! Check returned features for instance with invalid overall status
get_features_invalidstatus FOR TESTING,
```

```
"! Check returned features for instance with invalid key
get_features_invalidkey FOR TESTING,
```

```
"! Check instance with valid customer ID
validate_customer_valid FOR TESTING,
```

```
"! Check instance with invalid customer ID
validate_customer_invalid FOR TESTING,
```

```
"! Check instance with initial customer ID
validate_customer_initial FOR TESTING,
```

```
"! Check set_status_accepted action
set_status_accepted FOR TESTING.
```

For information on the design of the particular tests, see:

- [Testing the Validation Method validate_travel_status](#)
- [Testing the Feature Control Method get_features](#)
- [Testing the Validation Method validate_customer](#)
- [Testing the Action Method set_status_accepted](#)

Test Class Implementation

This section covers the implementation of the fixture methods that specify the test configuration used by the test methods.

Implementing the Method `class_setup`

In order to call a method of the tested class `lhc_travel`, you need to instantiate this class before. The addition `FOR TESTING` enables instantiating handler classes of behavior implementations.

The tested methods contain EML statements that access instance data via the CDS entity `Travel`. In order to make these EML statements access the data of a test double database instead of the real database table, you now need to create a test double using the CDS Test Double Framework. Specify the CDS view `/DMO/I_TRAVEL_M` to ensure that the database table `/dmo/travel_m` is doubled.

The tested method `validate_customer` contains Open SQL statements for direct database access. In order to make these SQL statements access the data of a test double instead of the real database, you also need to create a test double for the database table `/DMO/CUSTOMER` using the SQL Test Double Framework.

```
METHOD class_setup.
  CREATE OBJECT class_under_test FOR TESTING.
  cds_test_environment = cl_cds_test_environment=>create( i_for_entity = '/DMO/I_TRAVEL_M' ).
  sql_test_environment = cl_osql_test_environment=>create( i_dependency_list = VALUE #( ( '/DMO/C'
ENDMETHOD.
```

Implementing the Method `setup`

To make sure that your test methods are not influenced by data of other test methods, clear the contents of the test doubles before every test method execution.

```

METHOD setup.
    cds_test_environment->clear_doubles( ).
    sql_test_environment->clear_doubles( ).
ENDMETHOD.

```

Implementing the Method **teardown**

To make sure that the transactional buffer is reset for the involved entities after every test method execution, use the EML statement `ROLLBACK ENTITIES` in the teardown method.

```

METHOD teardown.
    ROLLBACK ENTITIES.
ENDMETHOD.

```

Implementing the Method **class_teardown**

At the end of the test, clear the test environments and the created test doubles.

```

METHOD class_teardown.
    cds_test_environment->destroy( ).
    sql_test_environment->destroy( ).
ENDMETHOD.

```

Resulting Test Class

After following the steps described above, your test class looks like this and is ready for the implementation of the test methods:

i *Expand the following code sample to view the source code of the test class `lhc_managed`.*

Sample Code

```

"! @testing BDEF:/DMO/I_Travel_M
CLASS lhc_managed DEFINITION FINAL FOR TESTING
    DURATION SHORT
    RISK LEVEL HARMLESS.
    PRIVATE SECTION.

        CLASS-DATA: class_under_test      TYPE REF TO lhc_travel,
                    cds_test_environment TYPE REF TO if_cds_test_environment,
                    sql_test_environment TYPE REF TO if_osql_test_environment.

    CLASS-METHODS:
        "! Instantiate class under test and setup test double frameworks
        class_setup,

        "! Destroy test environments and test doubles
        class_teardown.

    METHODS:
        "! Reset test doubles
        setup,

        "! Reset transactional buffer
        teardown,

        "! Check instance with overall status 'Open'
        validate_travel_status_open    FOR TESTING,

        "! Check instance with overall status 'Accepted'
        validate_travel_status_acpt    FOR TESTING,

        "! Check instance with overall status 'Rejected'
        validate_travel_status_rej     FOR TESTING,

```

```
"! Check instance with invalid overall status
validate_travel_status_invalid FOR TESTING,
```

```
"! Check returned features for instance with overall status 'Open'
get_features_open          FOR TESTING,
```

```
"! Check returned features for instance with overall status 'Accepted'
get_features_accepted      FOR TESTING,
```

```
"! Check returned features for instance with overall status 'Rejected'
get_features_rejected      FOR TESTING,
```

```
"! Check returned features for instance with invalid overall status
get_features_invalidstatus FOR TESTING,
```

```
"! Check returned features for instance with invalid key
get_features_invalidkey    FOR TESTING,
```

```
"! Check instance with valid customer ID
validate_customer_valid    FOR TESTING,
```

```
"! Check instance with invalid customer ID
validate_customer_invalid  FOR TESTING,
```

```
"! Check instance with initial customer ID
validate_customer_initial  FOR TESTING,
```

```
"! Check set_status_accepted action
set_status_accepted FOR TESTING.
```

```
ENDCLASS.
```

```
CLASS ltc_managed IMPLEMENTATION.
```

```
METHOD class_setup.
```

```
    CREATE OBJECT class_under_test FOR TESTING.
```

```
    cds_test_environment = cl_cds_test_environment=>create( i_for_entity = '/DMO/I_TRAVEL_M' ).
```

```
    sql_test_environment = cl_osql_test_environment=>create( i_dependency_list = VALUE #( ( '/DM
```

```
ENDMETHOD.
```

```
METHOD setup.
```

```
    cds_test_environment->clear_doubles( ).
```

```
    sql_test_environment->clear_doubles( ).
```

```
ENDMETHOD.
```

```
METHOD teardown.
```

```
    ROLLBACK ENTITIES.
```

```
ENDMETHOD.
```

```
METHOD class_teardown.
```

```
    cds_test_environment->destroy( ).
```

```
    sql_test_environment->destroy( ).
```

```
ENDMETHOD.
```

```
METHOD validate_travel_status_open.
```

```
ENDMETHOD.
```

```
METHOD validate_travel_status_acpt.
```

```
ENDMETHOD.
```

```
METHOD validate_travel_status_rej.
```

```
ENDMETHOD.
```

```
METHOD validate_travel_status_invalid.
```

```
ENDMETHOD.
```

```

METHOD get_features_open.
ENDMETHOD.

METHOD get_features_accepted.
ENDMETHOD.

METHOD get_features_rejected.
ENDMETHOD.

METHOD get_features_invalidstatus.
ENDMETHOD.

METHOD get_features_invalidkey.
ENDMETHOD.

METHOD validate_customer_valid.
ENDMETHOD.

METHOD validate_customer_invalid.
ENDMETHOD.

METHOD validate_customer_initial.
ENDMETHOD.

METHOD set_status_accepted.
ENDMETHOD.

ENDCLASS.

```

Managed Scenario: Implementing the Test Methods

This section shows how you can implement the test methods for methods of the managed flight reference scenario.

Contents

[Testing the Validation Method validate_travel_status](#)

[Testing the Feature Control Method get_features](#)

[Testing the Validation Method validate_customer](#)

[Testing the Action Method set_status_accepted](#)

Testing the Validation Method validate_travel_status

This topic shows a possible unit test for the validation method `validate_travel_status`.

The documentation in this topic only refers to the respective test methods. For information on the other parts of the test class, see [Managed Scenario: Refining the Test Class Structure](#).

Test Design

The test contains four test methods, which reflect different cases that are covered by the tested method. The first three methods refer to cases where a travel instance contains a valid overall status value - **Accepted**, **Rejected**, and **Open**. The fourth test method covers the case where the travel instance contains an overall status value different from the ones mentioned above. Such a value is considered as invalid by the validation.

Breaking down the test code into separate test methods facilitates the test maintenance. For instance, if the implementation for invalid overall statuses changes, then only the respective test method needs to be adjusted, all other test methods can be left as they are.

Test Method `validate_travel_status_open`

The test method first adds a travel instance with the value **O** (meaning "open") for the overall status to the test double.

Since the method under test expects structures for the `failed` and `reported` responses as changing parameters, two structures are declared using the respective types derived from the travel CDS entity and its behavior definition.

Now, the method to be tested is called. The travel ID of the instance, which has been inserted into the test double is provided as an exporting parameter, the declared `failed` and `reported` structures as changing parameters.

Finally, the `failed` and `reported` structures returned by the tested method are evaluated using assert checks. Since the value for the overall status field is accepted by the validation, the `failed` and `reported` structures are expected to be empty.

i Expand the following code sample to view the source code of the method `validate_travel_status_open`.

Sample Code

```
METHOD validate_travel_status_open.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '52' overall_status = 'O' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required structures
DATA failed TYPE RESPONSE FOR FAILED LATE /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED LATE /DMO/I_TRAVEL_M.

" Call method to be tested
class_under_test->validate_travel_status(
  EXPORTING
    keys      = CORRESPONDING #( travel_mock_data )
  CHANGING
    failed    = failed
    reported  = reported
).

" Check for content in failed and reported
cl_abap_unit_assert=>assert_initial( msg = 'failed' act = failed ).
cl_abap_unit_assert=>assert_initial( msg = 'reported' act = reported ).

ENDMETHOD.
```

Test Method `validate_travel_status_acpt`

This test method follows the same approach as the test method for the overall status **open**, but this time using the overall status **accepted**, which is valid as well.

i Expand the following code sample to view the source code of the method `validate_travel_status_acpt`.

Sample Code

```
METHOD validate_travel_status_acpt.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '52' overall_status = 'A' ) ).
```

```

cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required structures
DATA failed TYPE RESPONSE FOR FAILED LATE /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED LATE /DMO/I_TRAVEL_M.

" Call method to be tested
class_under_test->validate_travel_status(
  EXPORTING
    keys      = CORRESPONDING #( travel_mock_data )
  CHANGING
    failed    = failed
    reported  = reported
).

" Check for content in failed and reported
cl_abap_unit_assert=>assert_initial( msg = 'failed' act = failed ).
cl_abap_unit_assert=>assert_initial( msg = 'reported' act = reported ).

ENDMETHOD.

```

Test Method `validate_travel_status_rej`

This test method follows the same approach as the test method for the overall status **open**, but this time using the overall status **rejected**, which is valid as well.

i Expand the following code sample to view the source code of the method `validate_travel_status_rej`.

Sample Code

```

METHOD validate_travel_status_rej.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '52' overall_status = 'X' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required structures
DATA failed TYPE RESPONSE FOR FAILED LATE /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED LATE /DMO/I_TRAVEL_M.

" Call method to be tested
class_under_test->validate_travel_status(
  EXPORTING
    keys      = CORRESPONDING #( travel_mock_data )
  CHANGING
    failed    = failed
    reported  = reported
).

" Check for content in failed and reported
cl_abap_unit_assert=>assert_initial( msg = 'failed' act = failed ).
cl_abap_unit_assert=>assert_initial( msg = 'reported' act = reported ).
ENDMETHOD.

```

Test Method `validate_travel_status_invalid`

This test method follows the same approach as the test method for the expected values, but this time, the overall status is set to a value different from the ones expected by the validation.

The method asserts that the **failed** and **reported** structures each contain exactly one row in their travel tables. This prevents a dump, which would occur during the subsequent assert check when trying to access a row in a table that is not available. The subsequent check asserts that the failed and reported travel ID equals to the one provided for the CDS test double. For the

reported structure, the method additionally asserts that the correct element is flagged in the `%element` component and that it contains a valid message reference.

i Expand the following code sample to view the source code of the method `validate_travel_status_invalid`.

Sample Code

```
METHOD validate_travel_status_invalid.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '52' overall_status = 'T' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required structures
DATA failed TYPE RESPONSE FOR FAILED LATE /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED LATE /DMO/I_TRAVEL_M.

" Call method to be tested
class_under_test->validate_travel_status(
  EXPORTING
    keys      = CORRESPONDING #( travel_mock_data )
  CHANGING
    failed    = failed
    reported  = reported
).

" Check number of returned instances in failed-travel
cl_abap_unit_assert=>assert_equals( act = lines( failed-travel ) exp = 1 msg = 'lines in fai

" Check travel id in failed-travel
cl_abap_unit_assert=>assert_equals( act = failed-travel[ 1 ]-travel_id exp = '52' msg = 'tra

" Check number of returned instances in reported-travel
cl_abap_unit_assert=>assert_equals( act = lines( reported-travel ) exp = 1 msg = 'lines in r

" Check travel id in reported-travel
cl_abap_unit_assert=>assert_equals( act = reported-travel[ 1 ]-travel_id exp = '52' msg = '

" Check marked field in reported-travel
cl_abap_unit_assert=>assert_equals( act = reported-travel[ 1 ]-%element-overall_status exp

" Check message reference in reported-travel
cl_abap_unit_assert=>assert_bound( act = reported-travel[ 1 ]-%msg msg = 'message reference

ENDMETHOD.
```

Testing the Feature Control Method `get_features`

This topic shows a possible unit test for the feature control method `get_features`.

The documentation in this topic only refers to the respective test method. For information on the other parts of the test class, see [Managed Scenario: Refining the Test Class Structure](#).

Test Design

The test contains five test methods, which reflect different cases that are covered by the tested method. There is one test method for each overall status value, which is considered to be valid by the validation `validate_travel_status`. The fourth test method covers the case for an invalid overall status value. The fifth one covers the case where the test method processes a travel instance with an invalid travel ID.

Breaking down the test code into separate test methods facilitates the test maintenance. For instance, if a dedicated handling of the overall status **Open** is implemented in the tested method, then only the respective test method needs to be adjusted, all other test methods can be left as they are.

Test Method `get_features_open`

The test method first adds a travel instance with the value **O** (meaning "Open") to the test double.

Since the method under test expects structures for the `result`, `failed` and `reported` responses as changing parameters, three structures are declared using the respective types derived from the travel CDS entity and its behavior definition.

Now, the method to be tested is called. The travel ID of the instance, which has been inserted into the test double as well as the features to be checked are provided as exporting parameters. The declared `result`, `failed` and `reported` structures are provided as changing parameters.

The last part of the code checks whether the result has been populated with the expected feature control information. To be able to compare the expected result to the actual result, the internal table `expected` is declared by referencing the data type of the `result` table. The declared table is then populated with the expected content - a travel instance containing the mocked travel ID and the expected operation control constant `enabled` for the actions `acceptTravel` and `rejectTravel` as well as for the association `_Booking`.

i *Expand the following code sample to view the source code of the test method `get_features_open`.*

Sample Code

```
METHOD get_features_open.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '43' overall_status = 'O' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required table and structures
DATA result TYPE TABLE FOR INSTANCE FEATURES RESULT /DMO/I_TRAVEL_M\\travel.
DATA failed TYPE RESPONSE FOR FAILED EARLY /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED EARLY /DMO/I_TRAVEL_M.

" Call the method to be tested
class_under_test->get_features(
  EXPORTING
    keys                = CORRESPONDING #( travel_mock_data )
    requested_features = VALUE #( %action-accepttravel = if_abap_behv=>mk-on
                                   %action-rejectTravel = if_abap_behv=>mk-on
                                   %assoc-_Booking       = if_abap_behv=>mk-on )

  CHANGING
    result      = result
    failed      = failed
    reported    = reported
  ).

" Check result
DATA expected LIKE result.
expected = VALUE #( ( travel_id                = '43'
                     %action-accepttravel    = if_abap_behv=>fc-o-enabled
                     %action-rejectTravel    = if_abap_behv=>fc-o-enabled
                     %assoc-_Booking         = if_abap_behv=>fc-o-enabled ) ).

cl_abap_unit_assert=>assert_equals( msg = 'result' exp = expected act = result ).

ENDMETHOD.
```

Test Method `get_features_accepted`

This test method follows the same approach as the test method for the overall status **open**, but this time using the overall status **accepted**. In this case, the action `accepttravel` is expected to be disabled whereas the action `rejecttravel` and the association `_Booking` are expected to be enabled.

i Expand the following code sample to view the source code of the test method `get_features_accepted`.

Sample Code

```
METHOD get_features_accepted.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '43' overall_status = 'A' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required table and structures
DATA result TYPE TABLE FOR INSTANCE FEATURES RESULT /DMO/I_TRAVEL_M\\travel.
DATA failed TYPE RESPONSE FOR FAILED EARLY /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED EARLY /DMO/I_TRAVEL_M.

" Call the method to be tested
class_under_test->get_features(
  EXPORTING
    keys                = CORRESPONDING #( travel_mock_data )
    requested_features = VALUE #( %action-accepttravel = if_abap_behv=>mk-on
                                  %action-rejectTravel = if_abap_behv=>mk-on
                                  %assoc-_Booking      = if_abap_behv=>mk-on )

  CHANGING
    result      = result
    failed      = failed
    reported    = reported
).

" Check result
DATA expected LIKE result.
expected = VALUE #( ( travel_id          = '43'
                     %action-accepttravel = if_abap_behv=>fc-o-disabled
                     %action-rejectTravel = if_abap_behv=>fc-o-enabled
                     %assoc-_Booking     = if_abap_behv=>fc-o-enabled ) ).

cl_abap_unit_assert=>assert_equals( msg = 'result' exp = expected act = result ).
ENDMETHOD.
```

Test Method `get_features_rejected`

This test method follows the same approach as the test methods for the previously covered overall statuses, but this time using the overall status **rejected**. In this case, the action `accepttravel` is expected to be enabled whereas the action `rejecttravel` and the association `_Booking` are expected to be disabled.

i Expand the following code sample to view the source code of the test method `get_features_rejected`.

Sample Code

```
METHOD get_features_rejected.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '43' overall_status = 'X' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required table and structures
```

```
DATA result TYPE TABLE FOR INSTANCE FEATURES RESULT /DMO/I_TRAVEL_M\\travel.
DATA failed TYPE RESPONSE FOR FAILED EARLY /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED EARLY /DMO/I_TRAVEL_M.
```

```
" Call method to be tested
class_under_test->get_features(
  EXPORTING
    keys                = CORRESPONDING #( travel_mock_data )
    requested_features = VALUE #( %action-accepttravel = if_abap_behv=>mk-on
                                   %action-rejectTravel = if_abap_behv=>mk-on
                                   %assoc-Booking       = if_abap_behv=>mk-on )

  CHANGING
    result      = result
    failed      = failed
    reported    = reported
).

" Check result
DATA expected LIKE result.
expected = VALUE #( ( travel_id      = '43'
                     %action-accepttravel = if_abap_behv=>fc-o-enabled
                     %action-rejectTravel = if_abap_behv=>fc-o-disabled
                     %assoc-Booking      = if_abap_behv=>fc-o-disabled ) ).

cl_abap_unit_assert=>assert_equals( msg = 'result' exp = expected act = result ).

ENDMETHOD.
```

Test Method `get_features_invalidstatus`

This test method follows the same approach as the test methods for the previously covered overall statuses, but this time using an invalid overall status value. In this case, the actions and the association are expected to be enabled.

i Expand the following code sample to view the source code of the test method `get_features_invalidstatus`.

Sample Code

```
METHOD get_features_invalidstatus.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '43' overall_status = 'T' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required table and structures
DATA result TYPE TABLE FOR INSTANCE FEATURES RESULT /DMO/I_TRAVEL_M\\travel.
DATA failed TYPE RESPONSE FOR FAILED EARLY /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED EARLY /DMO/I_TRAVEL_M.

" Call the method to be tested
class_under_test->get_features(
  EXPORTING
    keys                = CORRESPONDING #( travel_mock_data )
    requested_features = VALUE #( %action-accepttravel = if_abap_behv=>mk-on
                                   %action-rejectTravel = if_abap_behv=>mk-on
                                   %assoc-Booking       = if_abap_behv=>mk-on )

  CHANGING
    result      = result
    failed      = failed
    reported    = reported
).

" Check result
DATA expected LIKE result.
expected = VALUE #( ( travel_id      = '43'
                     %action-accepttravel = if_abap_behv=>fc-o-enabled
```

```

        %action-rejectTravel = if_abap_behv=>fc-o-enabled
        %assoc-Booking      = if_abap_behv=>fc-o-enabled ) ).

    cl_abap_unit_assert=>assert_equals( msg = 'result' exp = expected act = result ).

ENDMETHOD.

```

Test Method `get_features_invalidkey`

This test method calls the tested method with a travel ID which is not available in the test double. Hence the tested method is expected to return one failed travel instance with the travel ID provided during the method call and the fail cause `NOT_FOUND`. The first assert check prevents a dump, which would occur during the following assert checks when trying to access a row in a table that is not available.

i *Expand the following code sample to view the source code of the test method `get_features_invalidkey`.*

Sample Code

```

METHOD get_features_invalidkey.

    " Declare required table and structures
    DATA result TYPE TABLE FOR INSTANCE FEATURES RESULT /DMO/I_TRAVEL_M\\travel.
    DATA failed TYPE RESPONSE FOR FAILED EARLY /DMO/I_TRAVEL_M.
    DATA reported TYPE RESPONSE FOR REPORTED EARLY /DMO/I_TRAVEL_M.

    " Call the method to be tested with invalid/not existing key
    class_under_test->get_features(
        EXPORTING
            keys                = VALUE #( ( travel_id = '43' ) )
            requested_features = VALUE #( %action-accepttravel = if_abap_behv=>mk-on
                                          %action-rejectTravel = if_abap_behv=>mk-on
                                          %assoc-Booking      = if_abap_behv=>mk-on )

        CHANGING
            result      = result
            failed      = failed
            reported    = reported
    ).

    " Check number of returned instances in failed-travel
    cl_abap_unit_assert=>assert_equals( msg = 'lines in failed-travel' act = lines( failed-travel

    " Check travel id in failed-travel
    cl_abap_unit_assert=>assert_equals( msg = 'travel id in failed-travel' act = failed-travel[

    " Check fail-cause in failed-travel
    cl_abap_unit_assert=>assert_equals( msg = 'fail-cause in failed-travel' act = failed-travel[
ENDMETHOD.

```

Testing the Validation Method `validate_customer`

This topic shows a possible unit test for the validation method `validate_customer`.

The documentation in this topic only refers to the respective test method. For information on the other parts of the test class, see [Managed Scenario: Refining the Test Class Structure](#).

Test Design

The test class contains three test methods, which reflect different cases that are covered by the tested method. The first test method covers the case for a valid customer ID, the second one for invalid customer ID whereas the third one covers the case,

where no customer ID has been provided for the travel instance.

Breaking down the test code into separate test methods facilitates the test maintenance. For instance, if a dedicated handling for travel instances with an initial customer ID is implemented, then only the respective test method needs to be adjusted, all other test methods can be left as they are.

Test Method `validate_customer_valid`

The test method first adds a travel instance with the customer ID 1 to the CDS test double.

Next, the method adds a customer dataset with the customer ID 1 to the SQL test double.

Since the method under test expects structures for the `failed` and `reported` responses as changing parameters, three structures are declared using the respective types derived from the travel CDS entity and its behavior definition.

Now, the method to be tested is called. The travel ID of the instance, which has been inserted into the test double is provided as an exporting parameter, the declared `failed` and `reported` structures as changing parameters.

Finally, the `failed` and `reported` structures returned by the tested method are evaluated using assert checks. Since the customer ID of the validated travel instance is contained in the doubled customer table, the `failed` and `reported` structures are expected to be empty.

i *Expand the following code sample to view the source code of the test method `validate_customer_valid`.*

Sample Code

```
METHOD validate_customer_valid.

" Fill in test data for entity travel
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '43' customer_id = '1' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Fill in test data for customer table
DATA customer_mock_data TYPE STANDARD TABLE OF /DMO/CUSTOMER.
customer_mock_data = VALUE #( ( customer_id = '1' ) ).
sql_test_environment->insert_test_data( customer_mock_data ).

" Declare required structures
DATA failed TYPE RESPONSE FOR FAILED LATE /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED LATE /DMO/I_TRAVEL_M.

" Call the method to be tested
class_under_test->validate_customer(
  EXPORTING
    keys                = CORRESPONDING #( travel_mock_data )
  CHANGING
    failed              = failed
    reported            = reported
).

" Check for content in failed and reported
cl_abap_unit_assert=>assert_initial( msg = 'failed' act = failed ).
cl_abap_unit_assert=>assert_initial( msg = 'reported' act = reported ).

ENDMETHOD.
```

Test Method `validate_customer_invalid`

This test method covers the case where the customer ID of the travel instance is not contained in the customer table, hence the travel instance is considered as invalid. To realize this case, the method adds a travel instance containing a customer ID to the CDS

test double, but no test data to the SQL test double.

The method asserts that the `failed` and `reported` tables for the travel entity each contain exactly one row. This prevents a dump, which would occur during the subsequent assert check when trying to access a row in the table that is not available. The subsequent check asserts that the failed and reported travel ID equals to the one provided for the CDS test double. For the reported structure, the method additionally asserts that the correct element is flagged in the `%element` component and that it contains a reference to an existing message object.

i *Expand the following code sample to view the source code of the test method `validate_customer_invalid`.*

Sample Code

```
METHOD validate_customer_invalid.

" Fill in test data for entity travel
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '43' customer_id = '1' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required structures
DATA failed TYPE RESPONSE FOR FAILED LATE /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED LATE /DMO/I_TRAVEL_M.

" Call the method to be tested
class_under_test->validate_customer(
  EXPORTING
    keys          = CORRESPONDING #( travel_mock_data )
  CHANGING
    failed        = failed
    reported      = reported
).

" Check number of returned instances in failed-travel
cl_abap_unit_assert=>assert_equals( msg = 'lines in failed-travel' act = lines( failed-travel

" Check travel id in failed-travel
cl_abap_unit_assert=>assert_equals( msg = 'travel id in failed-travel' act = failed-travel[

" Check number of returned instances in reported-travel
cl_abap_unit_assert=>assert_equals( msg = 'lines in reported-travel' act = lines( reported-t

" Check travel id in reported-travel
cl_abap_unit_assert=>assert_equals( msg = 'travel id in reported-travel' act = reported-trav

" Check marked field in reported-travel
cl_abap_unit_assert=>assert_equals( msg = 'field customer id in reported-travel' act = repor

" Check message reference in reported-travel
cl_abap_unit_assert=>assert_bound( msg = 'message reference in reported-travel' act = repor

ENDMETHOD.
```

Test Method `validate_customer_initial`

This test method covers the case where no customer ID is provided for the travel instance, hence the travel instance is considered as invalid by the validation. To realize this case, the method adds a travel instance containing no customer ID to the CDS test double. Test data for the SQL test double don't need to be inserted.

The method asserts that the `failed` and `reported` tables for the travel entity each contain exactly one row. This prevents a dump, which would occur during the subsequent assert check when trying to access a row in the table that is not available. The subsequent check asserts that the failed and reported travel ID equals to the one provided for the CDS test double. For the reported structure, the method additionally asserts that the correct element is flagged in the `%element` component and that it contains a reference to an existing message object.

i Expand the following code sample to view the source code of the test method `validate_customer_initial`.

≡ Sample Code

```
METHOD validate_customer_initial.

" Fill in test data for entity travel
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '43' customer_id = '' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required structures
DATA failed TYPE RESPONSE FOR FAILED LATE /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED LATE /DMO/I_TRAVEL_M.

" Call the method to be tested
class_under_test->validate_customer(
  EXPORTING
    keys          = CORRESPONDING #( travel_mock_data )
  CHANGING
    failed        = failed
    reported      = reported
).

" Check number of returned instances in failed-travel
cl_abap_unit_assert=>assert_equals( msg = 'lines in failed-travel' act = lines( failed-travel

" Check travel id in failed-travel
cl_abap_unit_assert=>assert_equals( msg = 'travel id in failed-travel' act = failed-travel[

" Check number of returned instances in reported-travel
cl_abap_unit_assert=>assert_equals( msg = 'lines in reported-travel' act = lines( reported-t

" Check travel id in reported-travel
cl_abap_unit_assert=>assert_equals( msg = 'travel id in reported-travel' act = reported-trav

" Check marked field in reported-travel
cl_abap_unit_assert=>assert_equals( msg = 'field customer id in reported-travel' act = repor

" Check message reference in reported-travel
cl_abap_unit_assert=>assert_bound( msg = 'message reference in reported-travel' act = repor

ENDMETHOD.
```

Testing the Action Method `set_status_accepted`

This topic shows a possible unit test for the action method `set_status_accepted`.

The documentation in this topic only refers to the respective test method. For information on the other parts of the test class, see [Managed Scenario: Refining the Test Class Structure](#).

Test Design

The method to be tested can be divided into two parts: The first part performs an EML modify operation on the respective instances. The second part reads the modified instances and assigns the read result to the result parameter of the action. In contrast to the methods discussed earlier, the method `set_status_accepted` does not distinguish between different cases, e.g. existing overall statuses. Both parts of the method are run during every method execution, hence the whole method can be covered by a single test method.

Test Method `set_status_accepted`

The test method first adds four travel instances with different overall statuses to the test double, two with a valid status, one with an invalid status and one with an initial status.

Since the tested method expects an internal table for the `result` of the action as well as structures for the `mapped`, `failed` and `reported` responses as changing parameters, four structures are declared using the respective types derived from the travel CDS entity and its behavior definition.

Now, the method to be tested is called. The travel IDs of the instances, which have been inserted into the test double are provided as exporting parameters, the declared table and structures as changing parameters.

To determine whether errors occurred during the `modify` operation that updates the overall status field, the method asserts that the `failed` and `reported` structures are empty. Since the `mapped` structure is not populated in the action implementation, an analogous assert check is specified.

The following part of the code checks whether the action result has been populated correctly. According to the behavior definition, the action is expected to return one travel instance for each instance the action has been executed on in the result. To be able to compare the expected action result to the actual action result, two internal tables are required. The table `exp` is declared by referencing the data type of the `result` table and is then populated with the expected content - four travel instances containing the mocked travel IDs and the overall status `A`. The travel IDs need to be specified twice: Once for the field `travel_id` in order to identify the source instance and once for the field `%param-travel_id` to identify the target instance. The table `act` is also declared with the data type of the action result. To ensure that the `act` table only gets values for fields that are also contained in the `exp` table, the assignment of the action result content is done by means of the `corresponding` operator using mapping rules for the required fields. Finally, the method asserts that the contents of the tables `exp` and `act` are equal.

The last part of the method checks whether the instances themselves have been updated correctly. To do so, the method reads the travel IDs and the overall statuses of the mocked travel instances from the transactional buffer. The result of the read operation is then assigned to the `act` table, thus overriding the content of the action result. Finally, the method asserts that the contents of the tables `exp` and `act` are equal.

i Expand the following code sample to view the source code of the method `set_status_accepted`.

Sample Code

```
METHOD set_status_accepted.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/TRAVEL_M.
travel_mock_data = VALUE #( ( travel_id = '42' overall_status = 'A' )
                             ( travel_id = '43' overall_status = 'B' )
                             ( travel_id = '44' overall_status = 'X' )
                             ( travel_id = '45' overall_status = '' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required table and structures
DATA result TYPE TABLE FOR ACTION RESULT /DMO/I_TRAVEL_M\\travel~accepttravel.
DATA mapped TYPE RESPONSE FOR MAPPED EARLY /DMO/I_TRAVEL_M.
DATA failed TYPE RESPONSE FOR FAILED EARLY /DMO/I_TRAVEL_M.
DATA reported TYPE RESPONSE FOR REPORTED EARLY /DMO/I_TRAVEL_M.

" Call the method to be tested
class_under_test->set_status_accepted(
  EXPORTING
    keys      = CORRESPONDING #( travel_mock_data )
  CHANGING
    result    = result
    mapped    = mapped
    failed    = failed
    reported  = reported
).

" Check for content in mapped, failed and reported
cl_abap_unit_assert=>assert_initial( msg = 'mapped' act = mapped ).
```

```

cl_abap_unit_assert=>assert_initial( msg = 'failed' act = failed ).
cl_abap_unit_assert=>assert_initial( msg = 'reported' act = reported ).

" Check action result for fields of interest: travel_id, %param-travel_id, %param-overall_st
DATA exp LIKE result.
exp = VALUE #( ( travel_id = 42 %param-travel_id = '42' %param-overall_status = 'A' )
               ( travel_id = 43 %param-travel_id = '43' %param-overall_status = 'A' )
               ( travel_id = 44 %param-travel_id = '44' %param-overall_status = 'A' )
               ( travel_id = 45 %param-travel_id = '45' %param-overall_status = 'A' ) ).

DATA act LIKE result.
act = CORRESPONDING #( result MAPPING travel_id = travel_id
                      ( %param = %param MAPPING travel_id = travel_id
                      overall_status = overall_status
                      EXCEPT * )
                      EXCEPT * ).
cl_abap_unit_assert=>assert_equals( msg = 'action result' exp = exp act = act ).

" Additionally check modified instances
READ ENTITY /DMO/I_TRAVEL_M
  FIELDS ( travel_id overall_status ) WITH CORRESPONDING #( travel_mock_data )
  RESULT DATA(read_result).

act = VALUE #( FOR t IN read_result ( travel_id = t-travel_id
                                     %param-travel_id = t-travel_id
                                     %param-overall_status = t-overall_status ) ).
cl_abap_unit_assert=>assert_equals( msg = 'read result' exp = exp act = act ).

ENDMETHOD.

```

Developing a Test Class for the Draft Flight Reference Scenario

The test class you develop in this chapter tests a method for the travel entity in the draft flight reference scenario (package /DMO/FLIGHT_DRAFT).

Tested Method

The determination `setTravelNumber` sets a travel ID for newly created travel instances. The new travel ID is calculated based on the highest travel ID available.

Contents

[Draft Scenario: Creating the Test Class](#)

[Draft Scenario: Implementing the Test Methods](#)

Draft Scenario: Creating the Test Class

In this step, you will create the test class, which serves as the basis for the test methods to be developed.

To create the test class, proceed as follows:

1. Launch **ABAP Development Tools**.
2. In your ABAP project (or ABAP cloud project), select the behavior implementation /DMO/BP_TRAVEL_D.
3. In the tab **Test Classes**, click the button **Create Test Class**, type the word "test" and use code completion to insert the test class template.

4. Name the class as `lhc_draft` and remove the declaration and implementation of the dummy method `first_test`.
5. Activate your changes.
6. Add the created test class as a friend of the local handler class `lhc_travel` in order to enable the access to its private methods. Proceed as described in [Managed Scenario: Providing Access to Tested Methods](#).
7. Refine the test class structure as described in [Managed Scenario: Refining the Test Class Structure](#). In addition to the instructions linked above, consider the following aspects for your test class:

- o Test Methods

Declare the test methods `setTravelNumber_idempotence`, `setTravelNumber_newTravelID` and `settravelnumber_mixed` and insert their signatures in the implementation part of the class using the offered quick fix. For information on the test design, see [Draft Scenario: Implementing the Test Methods](#).

- o Constants for UUIDs

Declare four constants for travel UUIDs to be used as test data.

- o CDS Test Double Framework

You need the CDS Test Double Framework to make the EML calls of the tested method access the doubled database dependency via the CDS view `/DMO/I_TRAVEL_D`. Provide this CDS view name as the importing parameter for the `create` method of the class `cl_cds_test_environment`.

- o Double Redirection for Open SQL calls

The tested method contains an Open SQL call that accesses the database table `/dmo/a_travel_d` directly. To make this Open SQL call access the test double instead of the real database table, call the method `enable_double_redirection` in the class setup after the creation of the CDS test environment and the test double.

After following the steps described above, your test class looks like this and is ready for the implementation of the test method:

i Expand the following code sample to view the source code of the test class `lhc_draft`.

Sample Code

```

"! @testing BDEF:/DMO/I_TRAVEL_D
CLASS lhc_draft DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.
  PRIVATE SECTION.

    CLASS-DATA: class_under_test      TYPE REF TO lhc_travel,
                cds_test_environment TYPE REF TO if_cds_test_environment.

    CONSTANTS: uuid1 TYPE sysuuid_x16 VALUE '66657221A8E4645C17002DF03754A1',
                uuid2 TYPE sysuuid_x16 VALUE '66657221A8E4645C17002DF03754A2',
                uuid3 TYPE sysuuid_x16 VALUE '66657221A8E4645C17002DF03754A3',
                uuid4 TYPE sysuuid_x16 VALUE '66657221A8E4645C17002DF03754A4'.

    CLASS-METHODS:
      "! Instantiate class under test and set up CDS Test Double Framework
      class_setup,

      "! Destroy test environment and test double
      class_teardown.

    METHODS:
      "! Reset test double
      setup,

```

```

"! Reset transactional buffer
teardown,

"! Check instance with already existing travel id
setTravelNumber_idempotence FOR TESTING,

"! Check instances with new travel ids
settravelnumber_newtravelids FOR TESTING,

"! Check instances with and without travel ids
settravelnumber_mixed FOR TESTING.

ENDCLASS.

CLASS ltc_draft IMPLEMENTATION.

METHOD class_setup.
  CREATE OBJECT class_under_test FOR TESTING.
  cds_test_environment = cl_cds_test_environment=>create( i_for_entity = '/DMO/I_TRAVEL_D' ).
  cds_test_environment->enable_double_redirection( ).
ENDMETHOD.

METHOD setup.
  cds_test_environment->clear_doubles( ).
ENDMETHOD.

METHOD teardown.
  ROLLBACK ENTITIES.
ENDMETHOD.

METHOD class_teardown.
  cds_test_environment->destroy( ).
ENDMETHOD.

METHOD setTravelNumber_idempotence.
ENDMETHOD.

METHOD settravelnumber_newtravelids.
ENDMETHOD.

METHOD settravelnumber_mixed.
ENDMETHOD.

ENDCLASS.

```

Draft Scenario: Implementing the Test Methods

This topic shows a possible unit test for the determination method `setTravelNumber`.

The documentation in this topic only refers to the respective test method. For information on the other parts of the test class, see [Draft Scenario: Creating the Test Class](#).

Test Design

The method to be tested can be divided into two parts: The first part reads the travel instances with the imported keys and removes those travel instances from the read result table `travels` that already have a travel ID. This ensures idempotence for the determination. If the table `travels` is empty after this step, the method is terminated. If the table is not empty, the second part of the method calculates new travel IDs and assigns them to the travel instances.

The first test method checks the idempotence by providing a travel instance with an already existing travel ID. The second test method checks whether new travel IDs are calculated correctly for instances without travel ID. The third test method checks a combination of the first two test cases.

Test Method `settravelnumber_idempotence`

The test method first adds a travel instance with a travel ID to the CDS test double.

Since the method under test expects the structure for the `reported` response as a changing parameter, the structure `reported` is declared using the respective type derived from the travel CDS entity and its behavior definition.

Now, the method to be tested is called. The travel UUID of the instance, which has been inserted into the test double is provided as an exporting parameter, the declared `reported` structure as changing parameter.

The `reported` structure may contain messages in case of a failed update operation, so the method asserts that the `reported` structure is initial.

The method now checks whether the instance still has its travel ID. To do so, it first reads the travel UUID and the travel ID from the transactional buffer. Next, the method performs the assert checks: The first one asserts that the read result table contains one row. This assert check ensures the correct number of read instances and is used to prevent a dump, which would occur during the next assert check in case that the table `read_result` is empty. The second check asserts that the travel ID has not changed.

i *Expand the following code sample to view the source code of the method `settravelnumber_idempotence`.*

Sample Code

```
METHOD settravelnumber_idempotence.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DMO/A_Travel_D.
travel_mock_data = VALUE #( ( travel_uuid = uuid1 travel_id = '1' ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required structure
DATA reported TYPE RESPONSE FOR REPORTED LATE /DMO/I_Travel_D.

" Call the method to be tested
class_under_test->setTravelNumber(
  EXPORTING
    keys           = VALUE #( ( TravelUUID = travel_mock_data[ 1 ]-travel_uuid ) )
    CHANGING
    reported       = reported
  ).

" Check for content in reported
cl_abap_unit_assert=>assert_initial( msg = 'reported' act = reported ).

" Read instance data
READ ENTITY /DMO/I_Travel_D
  FIELDS ( TravelID ) WITH VALUE #( ( TravelUUID = travel_mock_data[ 1 ]-travel_uuid ) )
  RESULT DATA(read_result).

" Check number of returned instances in read_result
cl_abap_unit_assert=>assert_equals( msg = 'lines in read result' exp = 1 act = lines( read_r

" Check idempotence
cl_abap_unit_assert=>assert_equals( msg = 'idempotence' exp = '1' act = read_result[ 1 ]-Tra

ENDMETHOD.
```

Test Method `settravelnumber_newtravelids`

This test method covers the case where the determination is executed for travel instances that do not contain travel IDs yet. Since the test double contains no other travel instances with a travel ID, the method asserts that the calculated travel IDs for the instance are 1 and 2.

i Expand the following code sample to view the source code of the method `settravelnumber_newtravelids`.

Sample Code

```
METHOD settravelnumber_newtravelids.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DM0/A_Travel_D.
travel_mock_data = VALUE #( ( travel_uuid = uuid1 )
                             ( travel_uuid = uuid2 ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required structure
DATA reported TYPE RESPONSE FOR REPORTED LATE  /DM0/I_Travel_D.

" Call the method to be tested
class_under_test->setTravelNumber(
  EXPORTING
    keys          = VALUE #( FOR key IN travel_mock_data ( TravelUUID = key-travel_uui
    CHANGING
    reported      = reported
  ).

" Check for content in reported
cl_abap_unit_assert=>assert_initial( msg = 'reported' act = reported ).

" Read and sort instance data
READ ENTITY /DM0/I_Travel_D
  FIELDS ( TravelID ) WITH VALUE #( FOR key IN travel_mock_data ( TravelUUID = key-travel_u
  RESULT DATA(read_result).

SORT read_result BY travelid.

" Check number of returned instances in read_result
cl_abap_unit_assert=>assert_equals( msg = 'lines in read result' exp = 2 act = lines( read_r

" Check new travel ids
cl_abap_unit_assert=>assert_equals( msg = 'new travel id' exp = '1' act = read_result[ 1 ]-T
cl_abap_unit_assert=>assert_equals( msg = 'new travel id' exp = '2' act = read_result[ 2 ]-T

ENDMETHOD.
```

Test Method `settravelnumber_mixed`

The last test method checks the combination of the previous test cases: two travel instances that need a travel ID and one travel instance that already has a travel ID. To make sure, that the expected travel IDs belong to the correct travel instances, the method additionally checks the travel UUIDs in the read result.

i Expand the following code sample to view the source code of the method `settravelnumber_newtravel_mixed`.

Sample Code

```
METHOD settravelnumber_newtravel_mixed.

" Fill in test data
DATA travel_mock_data TYPE STANDARD TABLE OF /DM0/A_Travel_D.
travel_mock_data = VALUE #( ( travel_uuid = uuid1 )
                             ( travel_uuid = uuid2 travel_id = '1' )
                             ( travel_uuid = uuid3 ) ).
cds_test_environment->insert_test_data( travel_mock_data ).

" Declare required structure
```

```
DATA reported TYPE RESPONSE FOR REPORTED LATE /DMO/I_Travel_D.
```

```
" Call the method to be tested
class_under_test->setTravelNumber(
  EXPORTING
    keys          = VALUE #( FOR key IN travel_mock_data ( TravelUUID = key-travel_uui
    CHANGING
    reported      = reported
  ).

" Check for content in reported
cl_abap_unit_assert=>assert_initial( msg = 'reported' act = reported ).

" Read and sort instance data
READ ENTITY /DMO/I_Travel_D
  FIELDS ( TravelID ) WITH VALUE #( FOR key IN travel_mock_data ( TravelUUID = key-travel_u
  RESULT DATA(read_result).

SORT read_result BY travelid.

" Check number of returned instances in read result
cl_abap_unit_assert=>assert_equals( msg = 'lines in read result' exp = 3 act = lines( read_r

" Check travel uuid of first instance
cl_abap_unit_assert=>assert_equals( msg = 'travel uuid' act = read_result[ 1 ]-TravelUUID ex

" Check idempotence
cl_abap_unit_assert=>assert_equals( msg = 'idempotence' act = read_result[ 1 ]-TravelID exp

" Check travel uuid of second instance
cl_abap_unit_assert=>assert_equals( msg = 'travel uuid' act = read_result[ 2 ]-TravelUUID ex

" Check new travel id
cl_abap_unit_assert=>assert_equals( msg = 'new travel id' act = read_result[ 2 ]-TravelID ex

" Check travel uuid of third instance
cl_abap_unit_assert=>assert_equals( msg = 'travel uuid' act = read_result[ 3 ]-TravelUUID ex

" Check new travel id
cl_abap_unit_assert=>assert_equals( msg = 'new travel id' act = read_result[ 3 ]-TravelID ex

ENDMETHOD.
```

Developing a Test Class for the Supplement Reference Scenario

The test class you develop in this chapter tests a method from the supplement reference scenario (package /DMO/FLIGHT_REUSE_SUPPLEMENT). The code under test augments requests on the projection layer by adding create-by-association and update requests in order to consider the denormalized `SupplementDescription` field in `MODIFY` transactions. Your tests will check whether the original requests are augmented correctly and whether they arrive as expected on the base layer of the business object. The used RAP BO Test Double Framework enables you to isolate dependencies from EML statements and to check the test results on a transactional buffer double.

For further information on the RAP BO Test Double Framework, see [Used Frameworks and Tools](#).

For further information on the used reference scenario, see [Editing Language-Dependent Fields](#).

Contents

[Supplement Scenario: Creating the Test Class](#)

Supplement Scenario: Creating the Test Class

In this step, you will create the test class, which serves as the basis for the test methods to be developed.

To create the test class, proceed as follows:

1. Launch **ABAP Development Tools**.
2. In your ABAP project (or ABAP cloud project), select the behavior implementation /DMO/BP_C_SUPPLEMENT.
3. In the tab **Test Classes**, click the button **Create Test Class**, type the word "test" and use code completion to insert the test class template.
4. Name the class as `ltcl_tdf_buffer` and remove the declaration and implementation of the dummy method `first_test`.

i Expand the following code sample to view the source code of the test class `ltcl_tdf_buffer`.

Sample Code

```
CLASS ltcl_tdf_buffer DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.

  PRIVATE SECTION.

ENDCLASS.

CLASS ltcl_tdf_buffer IMPLEMENTATION.

ENDCLASS.
```

Supplement Scenario: Refining the Test Class Structure

Now that you have created the basic test class structure, you can refine this structure by defining basic test properties, the test relation, variables and fixture methods that provide the desired test configuration. The defined test class forms the basis for the test methods that you will implement in the upcoming chapter.

This topic covers the specific test class structure for unit tests using the RAP BO test double framework. For general information on the structure of RAP test classes, see [Basic Test Class Structure](#).

Test Class Definition

The definition section of the test class contains basic meta information, reference variables and declarations for the used methods.

Assigning Test Properties

Since the tested method has a relatively small size, the test code is not expected to have a longer execution time than 5 seconds. Hence, you can use the DURATION SHORT. Depending on the respective run mode and runtime monitor, exceeding the chosen execution time can affect the results of your test.

As you won't change persistent data or system settings with your unit tests, the RISK LEVEL HARMLESS can be used. If the chosen risk level is higher than allowed in your system, your tests are not executed.

```
CLASS ltcl_tdf_buffer DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.
```

Specifying the Test Relation

You will develop the unit tests for the business object projection /DMO/C_SUPPLEMENT. Specifying a test relation to this behavior definition enables you to run your unit tests directly via the context menu.

```
"! @testing BDEF:/DMO/C_SUPPLEMENT
CLASS ltcl_tdf_buffer DEFINITION FINAL FOR TESTING
...
```

Declaring Class Components

Firstly, define a constant for the name of the business object you want to isolate using the RAP BO test double framework.

Since you will be using the RAP BO Test Double Framework to isolate dependencies resulting from EML statements, you need to declare reference variables for the respective interfaces:

Define a static variable to be used in the fixture methods and type it with the interface `if_botd_txbufdbl_bo_test_env`. The framework's implementation of this interface is used to create the environment which enables the access to and the maintenance of the test double for the RAP business object. Next, define an instance variable for the setup and teardown method and type it with the interface `if_botd_txbufdbl_test_double`. This variable will hold the reference to the RAP BO test double itself.

Additionally, define structures for mapped, failed and reported responses on the projection layer and on the base layer using the respective derived types.

```
PRIVATE SECTION.
  CONSTANTS:
    cv_i_bdef_name TYPE abp_root_entity_name VALUE '/DMO/I_Supplement'

  CLASS-DATA:
    environment TYPE REF TO if_botd_txbufdbl_bo_test_env.

  DATA:
    double_supplement TYPE REF TO if_botd_txbufdbl_test_double.

  DATA:
    mapped      TYPE RESPONSE FOR MAPPED   EARLY /dmo/c_supplement,
    failed      TYPE RESPONSE FOR FAILED   EARLY /dmo/c_supplement,
    reported    TYPE RESPONSE FOR REPORTED EARLY /dmo/c_supplement,
    i_mapped    TYPE RESPONSE FOR MAPPED   EARLY /dmo/i_supplement,
    i_failed    TYPE RESPONSE FOR FAILED   EARLY /dmo/i_supplement,
    i_reported  TYPE RESPONSE FOR REPORTED EARLY /dmo/i_supplement.
```

i Note

In this example, you use the transactional buffer double variant of the RAP BO Test Double Framework. This variant provides a generic mock implementation of READ and MODIFY operations as well as a doubled transactional buffer for these operations to work with.

If you want to configure a specific EML output for your provided input, you need to use the framework variant with mocked EML responses. .

Declaring Fixture and Test Methods

In addition to the declaration of your test methods, declare fixture methods. Fixture methods are used to set up and tear down the test environment which produces unique test behavior for every executed test.

i *Expand the following code sample to view the source code.*

Sample Code

```

CLASS-METHODS:
    class_setup,
    class_teardown.

METHODS:
    setup,
    teardown.

METHODS:

    supplement_create          FOR TESTING RAISING cx_static_check,
    supplement_create_text_update FOR TESTING RAISING cx_static_check,
    supplement_update_text_create FOR TESTING RAISING cx_static_check,
    supplement_update_text_update FOR TESTING RAISING cx_static_check,
    text_create                FOR TESTING RAISING cx_static_check,
    text_update                FOR TESTING RAISING cx_static_check,
    update_with_invalid_keys   FOR TESTING RAISING cx_static_check,
    update_invalid_keys_descr  FOR TESTING RAISING cx_static_check.

```

Test Class Implementation

This section covers the implementation of the fixture methods that specify the test configuration used by the test methods.

Implementing the Method `class_setup`

Prepare the configuration for the RAP BO Test Double Framework environment: Provide the root entity name of the business object to be doubled and enable the draft handling for this test double. Create the framework environment using this configuration.

```

METHOD class_setup.
    DATA(env_config) = cl_botd_txbufdbl_bo_test_env=>prepare_environment_config(
        )->set_bdef_dependencies( VALUE #( ( cv_i_bdef_name ) )
        )->handle_draft( VALUE #( ( cv_i_bdef_name ) ) ).

    environment = cl_botd_txbufdbl_bo_test_env=>create( env_config ).
ENDMETHOD.

```

Implementing the Method `setup`

Clear the configuration of all doubles created by the RAP BO Test Double Framework environment.

Get the test double for your business object from the framework environment and assign the reference to it to the reference variable `double_supplement`.

In your test, you will need to create supplement instances in order to trigger the augmentation method to be tested. In a natural business object environment, the primary key for these instances would be provided by early numbering. Since the early numbering logic, however, is part of the behavior which is isolated by the RAP BO Test Double Framework, you need to provide the primary key for the supplement instance yourself. However, you cannot provide this supplement ID using the MODIFY CREATE statement, because static feature control information such as the readonly property of the field supplement ID is not isolated by the RAP BO Test Double Framework. In order to still provide the supplement IDs for your instances, you need to implement a fields handler which takes care of the supplement ID assignment for instances to be created on the transactional buffer double. Name the class for this handler `ltd_fields_handler` and set it as an additional behavior for your business object test double. You will implement the handler class in the next section.

```
METHOD setup.
  environment->clear_doubles( ).
  double_supplement = environment->get_test_double( cv_i_bdef_name ).
  double_supplement->configure_additional_behavior( )->set_fields_handler( fields_handler = NEW
ENDMETHOD.
```

Defining the fields handler class `ltd_fields_handler`

You need to be able to instantiate the handler class in another class, hence define the class option `CREATE PUBLIC`.

Handler classes that provide additional behavior for the fields of the RAP BO test double need to implement the interface `if_botd_bufdbl_fields_handler`. Specify this interface in the public section.

Declare an instance variable with the type `/dmo/supplement_id` and the value 0.

```
CLASS ltd_fields_handler DEFINITION CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES: if_botd_bufdbl_fields_handler.
  PROTECTED SECTION.
  PRIVATE SECTION.
    DATA:
      max_supplement_id TYPE /dmo/supplement_id VALUE 0.

ENDCLASS.
```

Implementing the fields handler class `ltd_fields_handler`

Implement the method `set_readonly_fields` of the interface `if_botd_bufdbl_fields_handler` as follows:

For create operations on the entity `/DMO/I_SUPPLEMENT`, assign incoming instances to a field symbol typed with the derived type for create operations on this entity.

For each agency instance in the table referred to by the field symbol, increment the number of supplement IDs by 1 and assign this number to the field `supplementid` of the instance.

```
CLASS ltd_fields_handler IMPLEMENTATION.

  METHOD if_botd_bufdbl_fields_handler~set_readonly_fields.
    CASE entity_name.
      WHEN '/DMO/I_SUPPLEMENT'.
        CASE operation.
          WHEN if_abap_behv=>op-m-create.
            TYPES: ty_create_instances TYPE TABLE FOR CREATE /dmo/i_supplement.
            FIELD-SYMBOLS: <create_instances> TYPE ty_create_instances.
            ASSIGN instances TO <create_instances>.
            LOOP AT <create_instances> ASSIGNING FIELD-SYMBOL(<instance>).
              max_supplement_id += 1.
              <instance>-supplementid = max_supplement_id.
            ENDLOOP.
          .
        .
      .
    .
  .
ENDMETHOD.
```

```

        ENDCASE.
    ENDCASE.
    ENDMETHOD.

ENDCLASS.

```

Implementing the Method **teardown**

Clear the data of your BO test double from the transactional buffer double.

```

METHOD teardown.
    double_supplement->clear_double( ).
ENDMETHOD.

```

Implementing the Method **class_teardown**

Tear down the RAP BO Test Double Framework environment.

```

METHOD class_teardown.
    environment->destroy( ).
ENDMETHOD.

```

Resulting Test Class

After following the steps described above, your test class looks like this and is ready for the implementation of the test methods:

i *Expand the following code sample to view the source code of the test class **ltd_tdf_buffer**.*

Sample Code

```

CLASS ltd_fields_handler DEFINITION CREATE PUBLIC.

    PUBLIC SECTION.
        INTERFACES: if_botd_bufdbl_fields_handler.
    PROTECTED SECTION.
    PRIVATE SECTION.
        DATA:
            max_supplement_id TYPE /dmo/supplement_id VALUE 0.

ENDCLASS.

CLASS ltd_fields_handler IMPLEMENTATION.

    METHOD if_botd_bufdbl_fields_handler~set_readonly_fields.
        CASE entity_name.
            WHEN '/DMO/I_SUPPLEMENT'.
                CASE operation.
                    WHEN if_abap_behv=>op-m-create.
                        TYPES: ty_create_instances TYPE TABLE FOR CREATE /dmo/i_supplement.
                        FIELD-SYMBOLS: <create_instances> TYPE ty_create_instances.
                        ASSIGN instances TO <create_instances>.
                        LOOP AT <create_instances> ASSIGNING FIELD-SYMBOL(<instance>).
                            max_supplement_id += 1.
                            <instance>-supplementid = max_supplement_id.
                        ENDLOOP.
                    ENDCASE.
                ENDCASE.
            ENDCASE.
        ENDMETHOD.

    ENDMETHOD.

ENDCLASS.

```

```
"! @testing BDEF:/DMO/C_SUPPLEMENT
```

```
CLASS ltcl_tdf_buffer DEFINITION FINAL FOR TESTING
```

```
DURATION SHORT
```

```
RISK LEVEL HARMLESS.
```

```
PRIVATE SECTION.
```

```
CONSTANTS:
```

```
    cv_i_bdef_name TYPE abp_root_entity_name VALUE '/DMO/I_Supplement'
```

```
CLASS-DATA:
```

```
    environment TYPE REF TO if_botd_txbufdbl_bo_test_env.
```

```
DATA:
```

```
    double_supplement TYPE REF TO if_botd_txbufdbl_test_double.
```

```
DATA:
```

```
    mapped      TYPE RESPONSE FOR MAPPED    EARLY /dmo/c_supplement,
```

```
    failed      TYPE RESPONSE FOR FAILED    EARLY /dmo/c_supplement,
```

```
    reported    TYPE RESPONSE FOR REPORTED  EARLY /dmo/c_supplement,
```

```
    i_mapped    TYPE RESPONSE FOR MAPPED    EARLY /dmo/i_supplement,
```

```
    i_failed    TYPE RESPONSE FOR FAILED    EARLY /dmo/i_supplement,
```

```
    i_reported  TYPE RESPONSE FOR REPORTED  EARLY /dmo/i_supplement.
```

```
CLASS-METHODS:
```

```
    class_setup,
```

```
    class_teardown.
```

```
METHODS:
```

```
    setup,
```

```
    teardown.
```

```
METHODS:
```

```
    supplement_create          FOR TESTING RAISING cx_static_check,
```

```
    supplement_create_text_update FOR TESTING RAISING cx_static_check,
```

```
    supplement_update_text_create FOR TESTING RAISING cx_static_check,
```

```
    supplement_update_text_update FOR TESTING RAISING cx_static_check,
```

```
    text_create                FOR TESTING RAISING cx_static_check,
```

```
    text_update                FOR TESTING RAISING cx_static_check,
```

```
    update_with_invalid_keys    FOR TESTING RAISING cx_static_check,
```

```
    update_invalid_keys_descr   FOR TESTING RAISING cx_static_check.
```

```
ENDCLASS.
```

```
CLASS ltcl_tdf_buffer IMPLEMENTATION.
```

```
METHOD class_setup.
```

```
    DATA(env_config) = cl_botd_txbufdbl_bo_test_env=>prepare_environment_config(
        )->set_bdef_dependencies( VALUE #( ( cv_i_bdef_name ) )
        )->handle_draft( VALUE #( ( cv_i_bdef_name ) ) ).
```

```
    environment = cl_botd_txbufdbl_bo_test_env=>create( env_config ).
```

```
ENDMETHOD.
```

```
METHOD class_teardown.
```

```
    environment->destroy( ).
```

```
ENDMETHOD.
```

```
METHOD setup.
```

```
    environment->clear_doubles( ).
```

```
    double_supplement = environment->get_test_double( cv_i_bdef_name ).
```

```
    double_supplement->configure_additional_behavior( )->set_fields_handler( fields_handler = N
ENDMETHOD.
```

```
METHOD teardown.
```

```

    double_supplement->clear_double( ).
ENDMETHOD.

METHOD supplement_create.
ENDMETHOD.

METHOD supplement_create_text_update.
ENDMETHOD.

METHOD supplement_update_text_create.
ENDMETHOD.

METHOD supplement_update_text_update.
ENDMETHOD.

METHOD text_create.
ENDMETHOD.

METHOD text_update.
ENDMETHOD.

METHOD update_with_invalid_keys.
ENDMETHOD.

METHOD update_invalid_keys_descr.
ENDMETHOD.
ENDCLASS.

```

Supplement Scenario: Implementing the Test Methods

The unit tests described in this chapter cover the different request scenarios the augmentation method needs to handle. The goal of every test is to check whether the augmentation result arrives as expected on the base layer. In order to get the consolidated result from the original request and the augmentation in your test, the augmentation needs to be triggered via EML respectively.

Since the RAP BO Test Double Framework is used, the tested augmentation method does not refer to the original MODIFY implementations by the provider, but to the generic MODIFY operations provided by this framework. Neither the original provider implementations for create and update nor their dependencies are executed.

Test Method supplement_create

This test method covers the case where a create operation on the supplement entity with a field setting for the supplementdescription field is requested. This should lead to a created supplement and supplement text instance on the base layer.

In the first step, the method triggers an EML create request on the projection layer using test data and checks whether it was executed by asserting against its response structures.

In the second step, the method reads and checks the resulting instance data of both the supplement instance and of the supplement text instance on the base layer. Since the augmentation method is expected to add the supplement text instance using a create by association operation, the data of this instance are read via read by association.

```

METHOD supplement_create.
  " Test
  DATA:
    c_create TYPE STRUCTURE FOR CREATE /dmo/c_supplement\\supplement,
    i_create TYPE STRUCTURE FOR CREATE /dmo/i_supplement\\supplement.

    c_create = VALUE #(

```

```

        %cid                = 'TEST_CID'
        %is_draft           = if_abap_behv=>mk-on
        supplementcategory  = 'XX'
        supplementdescription = 'Test'
        price               = '42'
        currencycode        = 'EUR'
    ).

MODIFY ENTITIES OF /dmo/c_supplement
    ENTITY supplement
        CREATE AUTO FILL CID
        FIELDS ( supplementcategory supplementdescription price currencycode )
        WITH VALUE #( ( c_create ) )
    MAPPED    mapped
    FAILED    failed
    REPORTED  reported.

" Verification
cl_abap_unit_assert=>assert_initial( failed ).
cl_abap_unit_assert=>assert_initial( reported ).

cl_abap_unit_assert=>assert_not_initial( mapped-supplement ).

READ ENTITIES OF /dmo/i_supplement
    ENTITY supplement
        FIELDS ( supplementcategory price currencycode )
        WITH CORRESPONDING #( mapped-supplement )
        RESULT DATA(supplements)
    ENTITY supplement
        BY \_supplementtext
        FIELDS ( description )
        WITH CORRESPONDING #( mapped-supplement )
        RESULT DATA(supplement_texts)
    FAILED    i_failed
    REPORTED  i_reported.

cl_abap_unit_assert=>assert_initial( i_failed ).
cl_abap_unit_assert=>assert_initial( i_reported ).

" Supplement
cl_abap_unit_assert=>assert_not_initial( supplements ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( supplements ) ).

i_create = CORRESPONDING #( c_create ).
DATA: created_supplement_without_id LIKE LINE OF supplements.
created_supplement_without_id = CORRESPONDING #( supplements[ 1 ] EXCEPT supplementid ).
cl_abap_unit_assert=>assert_equals( exp = i_create-%data
                                   act = created_supplement_without_id-%data ).
cl_abap_unit_assert=>assert_equals( exp = i_create-%is_draft
                                   act = created_supplement_without_id-%is_draft ).

" Supplement Text
cl_abap_unit_assert=>assert_not_initial( supplement_texts ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( supplement_texts ) ).
DATA(act_i_cba) = supplement_texts[ 1 ].
cl_abap_unit_assert=>assert_equals( exp = sy-langu
                                   act = act_i_cba-languagecode ).
cl_abap_unit_assert=>assert_equals( exp = c_create-supplementdescription
                                   act = act_i_cba-description ).
ENDMETHOD.

```

Test Method supplement_create_text_update

This test method covers the case where a create operation on the supplement entity with a field setting for the supplementdescription field is requested and in the same modify request, an update on the supplement description field is performed. This should lead to a created supplement and supplement text instance with a respectively updated description field on the base layer.

In the first step, the method triggers an EML MODIFY request including a create and an update operation on the projection layer. Both operations involve the supplementdescription field. The method ensures that both operations have been performed by asserting against the response structures.

In the second step, the method reads and checks the resulting instance data of both the supplement instance and of the supplement text instance on the base layer. Since the augmentation method is expected to add the supplement text instance using a create by association operation, the data of this instance are read via read by association.

```

METHOD supplement_create_text_update.
  " Preparation
  DATA:
    c_create          TYPE STRUCTURE FOR CREATE /dmo/c_supplement\\supplement,
    c_update          TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
    c_supplement_act  TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
    c_supplement_exp  TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
    i_supplement_act  TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement,
    i_supplement_exp  TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement,
    i_supplement_text_act TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplementtext.

    c_create = VALUE #(
      %cid          = 'TESTCID'
      %is_draft     = if_abap_behv=>mk-on
      supplementcategory = 'XX'
      supplementdescription = 'Test Create'
      price         = '42'
      currencycode  = 'EUR'
    ).

    c_update = VALUE #(
      %cid_ref      = c_create-%cid
      %is_draft     = c_create-%is_draft
      supplementdescription = 'Test Update'
      price         = '123'
      currencycode  = 'USD'
    ).

  " Test
  MODIFY ENTITIES OF /dmo/c_supplement
    ENTITY supplement
      CREATE
        FIELDS ( supplementcategory supplementdescription price currencycode )
        WITH VALUE #( ( c_create ) )
      UPDATE
        FIELDS ( supplementdescription price currencycode )
        WITH VALUE #( ( c_update ) )
    MAPPED mapped
    FAILED failed
    REPORTED reported.

  " Verification
  cl_abap_unit_assert=>assert_initial( failed ).
  cl_abap_unit_assert=>assert_initial( reported ).

  cl_abap_unit_assert=>assert_not_initial( mapped-supplement ).

  READ ENTITIES OF /dmo/i_supplement
    ENTITY supplement
      FIELDS ( supplementcategory price currencycode )
      WITH CORRESPONDING #( mapped-supplement )
      RESULT DATA(i_supplements)
    ENTITY supplement
      BY \_supplementtext
      FIELDS ( description )
      WITH CORRESPONDING #( mapped-supplement )
      RESULT DATA(i_supplement_texts)
    FAILED i_failed
    REPORTED i_reported.

  cl_abap_unit_assert=>assert_initial( i_failed ).

```

```

cl_abap_unit_assert=>assert_initial( i_reported ).

" Supplement
cl_abap_unit_assert=>assert_not_initial( i_supplements ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( i_supplements ) ).

i_supplement_act = CORRESPONDING #( i_supplements[ 1 ] EXCEPT %key ).
i_supplement_exp = CORRESPONDING #( c_update EXCEPT %cid_ref %key supplementcategory ).
i_supplement_exp-supplementcategory = c_create-supplementcategory.

cl_abap_unit_assert=>assert_equals( exp = i_supplement_exp
                                   act = i_supplement_act ).

" Supplement Text
cl_abap_unit_assert=>assert_not_initial( i_supplement_texts ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( i_supplement_texts ) ).

i_supplement_text_act = CORRESPONDING #( i_supplement_texts[ 1 ] EXCEPT supplementid ).

cl_abap_unit_assert=>assert_equals( exp = sy-langu
                                   act = i_supplement_text_act-languagecode ).
cl_abap_unit_assert=>assert_equals( exp = c_update-supplementdescription
                                   act = i_supplement_text_act-description ).

ENDMETHOD.

```

Test Method supplement_update_text_create

This test covers the case where an update on a supplement instance is requested. The instance to be updated does not contain a value for the field supplement description. This value is set only by the update request.

In the first step, the method prepares a supplement instance by performing a MODIFY create request. The create request is performed on the base layer where the supplement description field is not part of the supplement entity.

The actual test is performed by a MODIFY update request on the projection layer which sets values for the supplement description field and other fields of the supplement entity. Since the augmentation method is expected to add the supplement text instance using a create by association operation, the data of this instance are read via read by association and are checked afterwards respectively.

```

METHOD supplement_update_text_create.
  " Preparation
  DATA:
    i_create TYPE STRUCTURE FOR CREATE /dmo/i_supplement\\supplement.

  i_create = VALUE #(
    %is_draft      = if_abap_behv=>mk-on
    supplementcategory = 'XX'
    price          = '10'
    currencycode    = 'USD'
  ).
  MODIFY ENTITIES OF /dmo/i_supplement
    ENTITY supplement
    CREATE
      AUTO FILL CID
      FIELDS ( supplementcategory price currencycode )
      WITH VALUE #( ( i_create ) )
    MAPPED i_mapped
    FAILED i_failed
    REPORTED i_reported.

  cl_abap_unit_assert=>assert_initial( i_failed ).
  cl_abap_unit_assert=>assert_initial( i_reported ).
  cl_abap_unit_assert=>assert_initial( i_mapped-supplementtext ).

```

```

cl_abap_unit_assert=>assert_not_initial( i_mapped-supplement ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( i_mapped-supplement ) ).

DATA(supplementid_to_test) = VALUE #( i_mapped-supplement[ 1 ]-supplementid ).

" Test
DATA:
  c_update TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
  i_update TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement.

c_update = VALUE #(
  %is_draft          = if_abap_behv=>mk-on
  supplementid       = supplementid_to_test
  supplementdescription = 'Test'
  price              = '42'
  currencycode       = 'EUR'
  ).

MODIFY ENTITIES OF /dmo/c_supplement
  ENTITY supplement
  UPDATE
    FIELDS ( supplementdescription price currencycode )
    WITH VALUE #( ( c_update ) )
  MAPPED    mapped
  FAILED    failed
  REPORTED  reported.

" Verification
cl_abap_unit_assert=>assert_initial( failed ).
cl_abap_unit_assert=>assert_initial( reported ).
cl_abap_unit_assert=>assert_initial( mapped ).

READ ENTITIES OF /dmo/i_supplement
  ENTITY supplement
  FIELDS ( supplementcategory price currencycode )
  WITH CORRESPONDING #( i_mapped-supplement )
  RESULT DATA(supplements)
  ENTITY supplement
  BY \_supplementtext
  FIELDS ( description )
  WITH CORRESPONDING #( i_mapped-supplement )
  RESULT DATA(supplement_texts)
  FAILED    i_failed
  REPORTED  i_reported.

cl_abap_unit_assert=>assert_initial( i_failed ).
cl_abap_unit_assert=>assert_initial( i_reported ).

" Supplement
cl_abap_unit_assert=>assert_not_initial( supplements ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( supplements ) ).

i_update = CORRESPONDING #( c_update ).
i_update-supplementcategory = i_create-supplementcategory.
cl_abap_unit_assert=>assert_equals( exp = i_update-%data
                                   act = supplements[ 1 ]-%data ).
cl_abap_unit_assert=>assert_equals( exp = i_update-%is_draft
                                   act = supplements[ 1 ]-%is_draft ).

" Supplement Text
cl_abap_unit_assert=>assert_not_initial( supplement_texts ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( supplement_texts ) ).
DATA(act_i_cba) = supplement_texts[ 1 ].
cl_abap_unit_assert=>assert_equals( exp = sy-langu
                                   act = act_i_cba-languagecode ).
cl_abap_unit_assert=>assert_equals( exp = c_update-supplementdescription
                                   act = act_i_cba-description ).

ENDMETHOD.

```

Test Method supplement_update_text_update

This test covers the case where an update on a supplement instance is requested. The instance to be updated already contains a value for the field supplement description. This value is then updated by the update request.

In the first step, the method prepares a supplement instance and a supplement text instance by performing a MODIFY create and a create by association request on the base layer. The latter instance contains a value for the description field.

The actual test is performed by a MODIFY update request on the projection layer which sets a value for the supplement description field. The data of the supplement and the supplement text instance are then read and checked respectively.

```
METHOD supplement_update_text_update.
" Preparation
DATA:
  i_supplement_create      TYPE STRUCTURE FOR CREATE /dmo/i_supplement\\supplement,
  i_supplement_text_create TYPE STRUCTURE FOR CREATE /dmo/i_supplement\\supplement\\supplementt

i_supplement_create = VALUE #(
  %cid          = 'TEST_CID'
  %is_draft     = if_abap_behv=>mk-on
  supplementcategory = 'XX'
  price         = '10'
  currencycode  = 'USD'
).
i_supplement_text_create = VALUE #(
  %cid_ref = i_supplement_create-%cid
  %is_draft = if_abap_behv=>mk-on
  %target   = VALUE #(
    (
      %is_draft      = if_abap_behv=>mk-on
      languagecode = sy-langu
      description    = 'Before Description'
    )
  )
).
MODIFY ENTITIES OF /dmo/i_supplement
ENTITY supplement
  CREATE
    FIELDS ( supplementcategory price currencycode )
    WITH VALUE #( ( i_supplement_create ) )
  CREATE BY \_supplementtext
  AUTO FILL CID
  FIELDS ( description )
  WITH VALUE #( ( i_supplement_text_create ) )
MAPPED    i_mapped
FAILED    i_failed
REPORTED  i_reported.

cl_abap_unit_assert=>assert_initial( i_failed ).
cl_abap_unit_assert=>assert_initial( i_reported ).

cl_abap_unit_assert=>assert_not_initial( i_mapped-supplement ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( i_mapped-supplement ) ).
cl_abap_unit_assert=>assert_not_initial( i_mapped-supplementtext ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( i_mapped-supplementtext ) ).

DATA(supplementid_to_test) = VALUE #( i_mapped-supplement[ 1 ]-supplementid ).

" Test
DATA:
  c_update TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
  i_update TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement.

c_update = VALUE #(
  %is_draft      = if_abap_behv=>mk-on
  supplementid   = supplementid_to_test
  supplementdescription = 'Updated Description'
  price         = '42'
```

```

        currencycode = 'EUR'
    ).

MODIFY ENTITIES OF /dmo/c_supplement
ENTITY supplement
UPDATE
    FIELDS ( supplementdescription price currencycode )
    WITH VALUE #( ( c_update ) )
MAPPED mapped
FAILED failed
REPORTED reported.

cl_abap_unit_assert=>assert_initial( failed ).
cl_abap_unit_assert=>assert_initial( reported ).
cl_abap_unit_assert=>assert_initial( mapped ).

READ ENTITIES OF /dmo/i_supplement
ENTITY supplement
    FIELDS ( supplementcategory price currencycode )
    WITH CORRESPONDING #( i_mapped-supplement )
    RESULT DATA(supplements)
ENTITY supplement
    BY \_supplementtext
    FIELDS ( description )
    WITH CORRESPONDING #( i_mapped-supplement )
    RESULT DATA(supplement_texts)
FAILED i_failed
REPORTED i_reported.

cl_abap_unit_assert=>assert_initial( i_failed ).
cl_abap_unit_assert=>assert_initial( i_reported ).

" Supplement
cl_abap_unit_assert=>assert_not_initial( supplements ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                act = lines( supplements ) ).

DATA:
    i_supplement_exp TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement,
    i_supplement_act TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement.

i_supplement_exp = CORRESPONDING #( c_update EXCEPT supplementid ).
i_supplement_exp-supplementcategory = i_supplement_create-supplementcategory.
i_supplement_act = CORRESPONDING #( supplements[ 1 ] EXCEPT supplementid ).
cl_abap_unit_assert=>assert_equals( exp = i_supplement_exp
                                act = i_supplement_act ).

" Supplement Text
cl_abap_unit_assert=>assert_not_initial( supplement_texts ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                act = lines( supplement_texts ) ).
cl_abap_unit_assert=>assert_not_initial( supplement_texts[ 1 ] ).
DATA(act_i_update) = supplement_texts[ 1 ].
cl_abap_unit_assert=>assert_equals( exp = sy-langu
                                act = act_i_update-languagecode ).
cl_abap_unit_assert=>assert_equals( exp = c_update-supplementdescription
                                act = act_i_update-description ).

ENDMETHOD.

```

Test Method text_create

This method covers a similar case as the test [Test Method supplement_update_text_create](#). In this test method, however, the update on the projection layer only refers to the supplement description field.

```

METHOD text_create.
    " Preparation

```

```

DATA:
  i_supplement_create      TYPE STRUCTURE FOR CREATE /dmo/i_supplement\\supplement.

i_supplement_create = VALUE #(
  %cid                     = 'TEST_CID'
  %is_draft                = if_abap_behv=>mk-on
  supplementcategory       = 'XX'
  price                    = '10'
  currencycode             = 'USD'
).

MODIFY ENTITIES OF /dmo/i_supplement
  ENTITY supplement
    CREATE
      FIELDS ( supplementcategory price currencycode )
      WITH VALUE #( ( i_supplement_create ) )
  MAPPED    i_mapped
  FAILED    i_failed
  REPORTED  i_reported.

cl_abap_unit_assert=>assert_initial( i_failed ).
cl_abap_unit_assert=>assert_initial( i_reported ).

cl_abap_unit_assert=>assert_not_initial( i_mapped-supplement ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( i_mapped-supplement ) ).
cl_abap_unit_assert=>assert_initial( i_mapped-supplementtext ).

DATA(supplementid_to_test) = VALUE #( i_mapped-supplement[ 1 ]-supplementid ).

" Test
DATA:
  c_update TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
  i_update TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement.

c_update = VALUE #(
  %is_draft                = if_abap_behv=>mk-on
  supplementid             = supplementid_to_test
  supplementdescription     = 'Test'
).

MODIFY ENTITIES OF /dmo/c_supplement
  ENTITY supplement
    UPDATE
      FIELDS ( supplementdescription )
      WITH VALUE #( ( c_update ) )
  MAPPED    mapped
  FAILED    failed
  REPORTED  reported.

READ ENTITIES OF /dmo/i_supplement
  ENTITY supplement
    FIELDS ( supplementcategory price currencycode )
    WITH CORRESPONDING #( i_mapped-supplement )
    RESULT DATA(supplements)
  ENTITY supplement
    BY \_supplementtext
    FIELDS ( description )
    WITH CORRESPONDING #( i_mapped-supplement )
    RESULT DATA(supplement_texts)
  FAILED    i_failed
  REPORTED  i_reported.

cl_abap_unit_assert=>assert_initial( i_failed ).
cl_abap_unit_assert=>assert_initial( i_reported ).

" Supplement
cl_abap_unit_assert=>assert_not_initial( supplements ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( supplements ) ).

DATA:
  i_supplement_exp TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement,

```

```

i_supplement_act TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement.

i_supplement_exp = CORRESPONDING #( i_supplement_create EXCEPT supplementid ).
i_supplement_act = CORRESPONDING #( supplements[ 1 ] EXCEPT supplementid ).
cl_abap_unit_assert=>assert_equals( exp = i_supplement_exp
                                     act = i_supplement_act ).

" Supplement Text
cl_abap_unit_assert=>assert_not_initial( supplement_texts ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( supplement_texts ) ).
cl_abap_unit_assert=>assert_not_initial( supplement_texts[ 1 ] ).
DATA(act_i_update) = supplement_texts[ 1 ].
cl_abap_unit_assert=>assert_equals( exp = sy-langu
                                   act = act_i_update-languagecode ).
cl_abap_unit_assert=>assert_equals( exp = c_update-supplementdescription
                                   act = act_i_update-description ).
ENDMETHOD.

```

Test Method text_update

This method covers a similar case as the test [Test Method supplement_update text_update](#). In this test method, however, the update on the projection layer only refers to the supplement description field.

```

METHOD text_update.
  " Preparation
  DATA:
    i_supplement_create      TYPE STRUCTURE FOR CREATE /dmo/i_supplement\\supplement,
    i_supplement_text_create TYPE STRUCTURE FOR CREATE /dmo/i_supplement\\supplement\\supplementid

  i_supplement_create = VALUE #(
    %cid      = 'TEST_CID'
    %is_draft = if_abap_behv=>mk-on
    supplementcategory = 'XX'
    price      = '10'
    currencycode = 'USD'
  ).
  i_supplement_text_create = VALUE #(
    %cid_ref = i_supplement_create-%cid
    %is_draft = if_abap_behv=>mk-on
    %target   = VALUE #(
      (
        %is_draft      = if_abap_behv=>mk-on
        languagecode = sy-langu
        description   = 'Before Description'
      )
    )
  ).
  MODIFY ENTITIES OF /dmo/i_supplement
  ENTITY supplement
  CREATE
    FIELDS ( supplementcategory price currencycode )
    WITH VALUE #( ( i_supplement_create ) )
  CREATE BY \_supplementtext
  AUTO FILL CID
  FIELDS ( description )
  WITH VALUE #( ( i_supplement_text_create ) )
  MAPPED i_mapped
  FAILED i_failed
  REPORTED i_reported.

  cl_abap_unit_assert=>assert_initial( i_failed ).
  cl_abap_unit_assert=>assert_initial( i_reported ).

  cl_abap_unit_assert=>assert_not_initial( i_mapped-supplement ).
  cl_abap_unit_assert=>assert_equals( exp = 1
                                     act = lines( i_mapped-supplement ) ).
  cl_abap_unit_assert=>assert_not_initial( i_mapped-supplementtext ).
  cl_abap_unit_assert=>assert_equals( exp = 1

```

```

                                act = lines( i_mapped-supplementtext ) ).

DATA(supplementid_to_test) = VALUE #( i_mapped-supplement[ 1 ]-supplementid ).

" Test
DATA:
  c_update TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
  i_update TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement.

c_update = VALUE #(
                                %is_draft          = if_abap_behv=>mk-on
                                supplementid        = supplementid_to_test
                                supplementdescription = 'Test'
                                ).

MODIFY ENTITIES OF /dmo/c_supplement
  ENTITY supplement
    UPDATE
      FIELDS ( supplementdescription )
      WITH VALUE #( ( c_update ) )
  MAPPED      mapped
  FAILED      failed
  REPORTED    reported.

" Verification
cl_abap_unit_assert=>assert_initial( failed ).
cl_abap_unit_assert=>assert_initial( reported ).
cl_abap_unit_assert=>assert_initial( mapped ).

READ ENTITIES OF /dmo/i_supplement
  ENTITY supplement
    FIELDS ( supplementcategory price currencycode )
    WITH CORRESPONDING #( i_mapped-supplement )
    RESULT DATA(supplements)
  ENTITY supplement
    BY \_supplementtext
    FIELDS ( description )
    WITH CORRESPONDING #( i_mapped-supplement )
    RESULT DATA(supplement_texts)
  FAILED      i_failed
  REPORTED    i_reported.

cl_abap_unit_assert=>assert_initial( i_failed ).
cl_abap_unit_assert=>assert_initial( i_reported ).

" Supplement Text
cl_abap_unit_assert=>assert_not_initial( supplement_texts ).
cl_abap_unit_assert=>assert_equals( exp = 1
                                   act = lines( supplement_texts ) ).
DATA(act_i_cba) = supplement_texts[ 1 ].
cl_abap_unit_assert=>assert_equals( exp = sy-langu
                                   act = act_i_cba-languagecode ).
cl_abap_unit_assert=>assert_equals( exp = c_update-supplementdescription
                                   act = act_i_cba-description ).

ENDMETHOD.

```

Test Method update_with_invalid_keys

This test method covers the case where an update on two instances with an initial and an invalid key value is requested. This should lead to two failed entries with the fail cause NOT_FOUND.

```

METHOD update_with_invalid_keys.
  TYPES:
    t_c_failed_supplement LIKE LINE OF failed-supplement,
    t_c_failed_supplement_tky TYPE t_c_failed_supplement-%tky.

```

```

DATA:
  c_update_no_key      TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
  c_update_invalid_key TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
  i_update             TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement.

c_update_no_key      = VALUE #( %is_draft = if_abap_behv=>mk-on  supplementid = ''      ).
c_update_invalid_key = VALUE #( %is_draft = if_abap_behv=>mk-on  supplementid = 'TEST' ).

MODIFY ENTITIES OF /dmo/c_supplement
  ENTITY supplement
    UPDATE
      FIELDS ( supplementdescription price currencycode )
      WITH VALUE #( ( c_update_no_key ) ( c_update_invalid_key ) )
  MAPPED      mapped
  FAILED      failed
  REPORTED    reported.

cl_abap_unit_assert=>assert_not_initial( failed ).
cl_abap_unit_assert=>assert_equals(
  exp = 2
  act = lines( failed-supplement )
).

cl_abap_unit_assert=>assert_not_initial( VALUE #( failed-supplement[ KEY draft  %tky = CORRESPONDING ] ) ).
cl_abap_unit_assert=>assert_equals(
  exp = if_abap_behv=>cause-not_found
  act = failed-supplement[ KEY draft  %tky = c_update_no_key-%tky      ]-%fail-cause
).

cl_abap_unit_assert=>assert_not_initial( VALUE #( failed-supplement[ KEY draft  %tky = CORRESPONDING ] ) ).
cl_abap_unit_assert=>assert_equals(
  exp = if_abap_behv=>cause-not_found
  act = failed-supplement[ KEY draft  %tky = c_update_invalid_key-%tky ]-%fail-cause
).
ENDMETHOD.

```

Test Method update_invalid_keys_descr

This method covers a similar case as the test [Test Method update with invalid keys](#).

In this test method, however, invalid key values are provided for both instances. Additionally, a value for the supplement description is provided and the %CONTROL flag is activated for one instance whereas it is deactivated for the other instance. This should lead to two failed entries with the fail cause NOT_FOUND.

```

METHOD update_invalid_keys_descr.
  TYPES:
    t_c_failed_supplement LIKE LINE OF failed-supplement,
    t_c_failed_supplement_tky TYPE t_c_failed_supplement-%tky.

  DATA:
    c_update_no_flag      TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
    c_update_invalid_key TYPE STRUCTURE FOR UPDATE /dmo/c_supplement\\supplement,
    i_update              TYPE STRUCTURE FOR UPDATE /dmo/i_supplement\\supplement.

c_update_no_flag      = VALUE #( %is_draft = if_abap_behv=>mk-on  supplementid = 'TEST1'  supplementdescription = 'TEST1' ).
c_update_invalid_key = VALUE #( %is_draft = if_abap_behv=>mk-on  supplementid = 'TEST2'  supplementdescription = 'TEST2' ).

MODIFY ENTITIES OF /dmo/c_supplement
  ENTITY supplement
    UPDATE
      FIELDS ( supplementdescription price currencycode )
      WITH VALUE #( ( c_update_no_flag ) ( c_update_invalid_key ) )
  MAPPED      mapped
  FAILED      failed
  REPORTED    reported.

cl_abap_unit_assert=>assert_not_initial( failed ).

```

```

cl_abap_unit_assert=>assert_equals(
  exp = 2
  act = lines( failed-supplement )
).

cl_abap_unit_assert=>assert_not_initial( VALUE #( failed-supplement[ KEY draft %tky = CORRESP(
cl_abap_unit_assert=>assert_equals(
  exp = if_abap_behv=>cause-not_found
  act = failed-supplement[ KEY draft %tky = c_update_no_flag-%tky ]-%fail-cause
).

cl_abap_unit_assert=>assert_not_initial( VALUE #( failed-supplement[ KEY draft %tky = CORRESP(
cl_abap_unit_assert=>assert_equals(
  exp = if_abap_behv=>cause-not_found
  act = failed-supplement[ KEY draft %tky = c_update_invalid_key-%tky ]-%fail-cause
).
ENDMETHOD.

```

Developing Test Classes for the Agency Reference Scenario

The test class you develop in this chapter tests a method which has been added to the agency reference scenario as part of an extension node (package /DMO/FLIGHT_REUSE_AGENCY_REV). In this test, you will use the Event Test Double Framework in order to test whether the event contained in the tested method is raised as expected.

Tested Method

The method `save_modified` provides the additional save implementation for the node extension. It raises the RAP business event /DMO/AGENCYREVIEWCREATED against the interface entity /DMO/I_AGENCYTP when one or more instances of the extension node /DMO/ZZ_REVIEW are created. The payload of the event contains information about the created review as well as the agency ID and the email address of the corresponding agency instance.

Contents

- [Agency Scenario: Creating the Test Class](#)
- [Agency Scenario: Providing Access to Tested Method](#)
- [Agency Scenario: Refining the Test Class Structure](#)
- [Agency Scenario: Implementing the Test Methods](#)

Agency Scenario: Creating the Test Class

In this step, you will create the test class, which serves as the basis for the test method to be developed.

To create the test class, proceed as follows:

1. Launch **ABAP Development Tools**.
2. In your ABAP project (or ABAP cloud project), select the behavior implementation /DMO/ZZ_BP_X_REVIEW_R_AGENCYTP.
3. In the tab **Test Classes**, click the button **Create Test Class**, type the word "test" and use code completion to insert the test class template.

4. Name the class as `ltcl_sc_r_agency` and remove the declaration and implementation of the dummy method `first_test`.

i Expand the following code sample to view the source code of the test class `ltcl_sc_r_agency`.

Sample Code

```
CLASS ltcl_sc_r_agency DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.

  PRIVATE SECTION.

ENDCLASS.

CLASS ltcl_sc_r_agency IMPLEMENTATION.

ENDCLASS.
```

Agency Scenario: Providing Access to Tested Method

The tested method is a protected method. Since you will want to call the method in order to test it, you need to set the test class `ltcl_sc_r_agency` as a friend of the local saver class `lsc_r_agency` in the **Local Types** tab of the class `/DMO/ZZ_BP_X_REVIEW_R_AGENCYTP`. As the test class is not known to the local saver class at this point, you need to specify the test class before using the addition `DEFINITION DEFERRED FOR TESTING`.

```
CLASS ltcl_sc_r_agency DEFINITION DEFERRED FOR TESTING.
CLASS lsc_r_agency DEFINITION INHERITING FROM cl_abap_behavior_saver FRIENDS ltcl_sc_r_agency.
```

Agency Scenario: Refining the Test Class Structure

Now that you have created the basic test class structure, you can refine this structure by defining basic test properties, the test relation, variables and fixture methods that provide the desired test configuration. The defined test class forms the basis for the test methods that you will implement in the upcoming chapter.

Test Class Definition

The definition section of the test class contains basic meta information, reference variables and declarations for the used methods.

Assigning Test Properties

Since the tested method has a relatively small size, the test code is not expected to have a longer execution time than 5 seconds. Hence, you can use the `DURATION SHORT`. Depending on the respective run mode and runtime monitor, exceeding the chosen execution time can affect the results of your test.

As you won't change persistent data or system settings with your unit tests, the `RISK LEVEL HARMLESS` can be used. If the chosen risk level is higher than allowed in your system, your tests are not executed.

```
CLASS ltcl_sc_r_agency DEFINITION FINAL FOR TESTING
  DURATION SHORT
```

RISK LEVEL HARMLESS.

Specifying the Test Relation

You will develop the unit tests for the node extension defined in the BDEF /DMO/ZZ_X_REVIEW_R_AGENCYTP. Specifying a test relation to this BDEF enables you to run your unit tests directly via the context menu of the node extension's behavior definition.

```
"! @testing BDEF:/DMO/ZZ_X_REVIEW_R_AGENCYTP
CLASS ltcl_sc_r_agency DEFINITION FINAL FOR TESTING
...
```

Declaring Constants and Class Attributes

Define constants for the name of the RAP Business Event and for the entity the event is expected to be raised for.

Define a reference variable for your code under test in the private section.

Since you will be using the CDS Test Double Framework and the Event Test Double Framework, you need to declare reference variables for their respective interfaces as well.

```
PRIVATE SECTION.
  CONSTANTS agency_event TYPE c length 30 VALUE '/DMO/AGENCYREVIEWCREATED'.
  CONSTANTS agency_entity TYPE abp_entity_name VALUE '/DMO/I_AGENCYTP'.
  CLASS-DATA:
    class_under_test      TYPE REF TO lsc_r_agency,
    cds_test_environment   TYPE REF TO if_cds_test_environment,
    event_test_environment TYPE REF TO if_rap_event_test_environment.
```

Declaring Fixture and Test Methods

In addition to the declaration of your test methods `save_md_no_review_created`, `save_md_one_review_created` and `save_md_multi_reviews_created`, declare fixture methods. Fixture methods are used to set up and tear down the test environment which produces unique test behavior for every executed test.

i *Expand the following code sample to view the source code.*

Sample Code

```
CLASS-METHODS:
  class_setup,
  class_teardown.

METHODS:
  setup,
  teardown.

METHODS:
  save_md_no_review_created FOR TESTING,
  save_md_one_review_created FOR TESTING,
  save_md_multi_reviews_created FOR TESTING.
```

For information on the implementation of the test methods and the chosen test approach, see [Agency Scenario: Implementing the Test Methods](#).

Test Class Implementation

This section covers the implementation of the fixture methods that specify the test configuration used by the test methods.

Implementing the Method `class_setup`

First, instantiate the behavior implementation class to be tested using the reference variable `class_under_test`.

Next, create the CDS Test Double Framework environment in order to isolate database dependencies: Provide the name of the entity which is addressed by the EML Read statement in the code under test and ensure that the database tables, which are located below this entity in the dependency tree, are doubled.

Finally, prepare the configuration for the RAP Event Test Double Framework environment: Provide the name of the event as well as the name of the entity, which the event is expected to be raised against, as importing parameters for the `create` method. Since the event to be tested is raised in the behavior implementation of a node extension, you need to specify the entity name of the RAP BO interface.

```
METHOD class_setup.
  CREATE OBJECT class_under_test FOR TESTING.
  sql_test_environment = cl_osql_test_environment=>create( i_dependency_list = VALUE #( ( '/DMO/'
  cds_test_environment = cl_cds_test_environment=>create( i_for_entity          = agency_en
                                                         i_select_base_dependencies = abap_true
  event_test_environment = cl_rap_event_test_environment=>create( VALUE #( ( entity_name = agency
ENDMETHOD.
```

Implementing the Method `setup`

Clear the CDS and event test doubles before every test method execution.

```
METHOD setup.
  cds_test_environment->clear_doubles( ).
  event_test_environment->clear( ).
ENDMETHOD.
```

Implementing the Method `teardown`

Clear the transactional buffer.

```
METHOD teardown.
  ROLLBACK ENTITIES.
ENDMETHOD.
```

Implementing the Method `class_teardown`

Tear down the environments and test doubles of the CDS and event test double framework.

```
METHOD class_teardown.
  cds_test_environment->destroy( ).
  event_test_environment->destroy( ).
ENDMETHOD.
```

Resulting Test Class

After following the steps described above, your test class looks like this and is ready for the implementation of the test methods:

i Expand the following code sample to view the source code of the test class `ltcl_sc_r_agency`.

⌵ **Sample Code**

```
"! @testing BDEF:/DM0/ZZ X REVIEW_R AGENCYTP
CLASS ltcl_sc_r_agency DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.
```

```
PRIVATE SECTION.
```

```
  CLASS-DATA:
```

```
    CONSTANTS agency_event TYPE c length 30 VALUE '/DM0/AGENCYREVIEWCREATED'.
    CONSTANTS agency_entity TYPE abp_entity_name VALUE '/DM0/I_AGENCYTP'.
    class_under_test TYPE REF TO lsc_r_agency,
    cds_test_environment TYPE REF TO if_cds_test_environment,
    event_test_environment TYPE REF TO if_rap_event_test_environment.
```

```
  CLASS-METHODS:
```

```
    class_setup,
    class_teardown.
```

```
  METHODS:
```

```
    setup,
    teardown.
```

```
  METHODS:
```

```
    save_md_no_review_created FOR TESTING,
    save_md_one_review_created FOR TESTING,
    save_md_multi_reviews_created FOR TESTING.
```

```
ENDCLASS.
```

```
CLASS ltcl_sc_r_agency IMPLEMENTATION.
```

```
  METHOD class_setup.
```

```
    CREATE OBJECT class_under_test FOR TESTING.
```

```
    sql_test_environment = cl_osql_test_environment=>create( i_dependency_list = VALUE #( ( '/DM
    cds_test_environment = cl_cds_test_environment=>create( i_for_entity           = agency_
                                                              i_select_base_dependencies = abap_tr
```

```
    event_test_environment = cl_rap_event_test_environment=>create( VALUE #( ( entity_name = age
ENDMETHOD.
```

```
  METHOD setup.
```

```
    cds_test_environment->clear_doubles( ).
    event_test_environment->clear( ).
```

```
ENDMETHOD.
```

```
  METHOD teardown.
```

```
    ROLLBACK ENTITIES.
```

```
ENDMETHOD.
```

```
  METHOD class_teardown.
```

```
    cds_test_environment->destroy( ).
    event_test_environment->destroy( ).
```

```
ENDMETHOD.
```

```
  METHOD save_md_no_review_created.
```

```
ENDMETHOD.
```

```
  METHOD save_md_one_review_created.
```

```
ENDMETHOD.
```

```
  METHOD save_md_multi_reviews_created.
```

```
ENDMETHOD.
```

```
ENDCLASS.
```

Agency Scenario: Implementing the Test Methods

The unit tests developed in this section test whether the RAP Business Event /DM0/AgencyReviewCreated is raised as expected in the method `save_modified`. This method represents the additional save implementation of the extension node

Test Design

The event /DMO/AGENCYREVIEWCREATED is raised once one or more review instances are created. The chosen test methods cover the cases where no, one and multiple review instances are handed over to the additional save exit.

Implementing the Test Method **save_md_no_review_created**

First, the method declares a structure for the late reported response using the respective derived type for the entity /DMO/I_AGENCYTP.

Next, the method executes the code under test using the declared reported structure as the changing parameter. No data are provided for the importing parameters `create`, `update` and `delete`.

Finally, the method asserts that the event /DMO/AGENCYREVIEWCREATED has not been raised against the entity /DMO/I_AGENCYTP.

```
METHOD save_md_no_review_created.
```

```

" Prepare
DATA reported TYPE RESPONSE FOR REPORTED LATE /dmo/i_agencytp.

" Execute
class_under_test->save_modified( EXPORTING create   = VALUE #( )
                                update    = VALUE #( )
                                delete    = VALUE #( )
                                CHANGING  reported = reported ).

" Verify
event_test_environment->get_event( entity_name = agency_entity event_name = agency_event )->ver

ENDMETHOD.
```

Implementing the Test Method **save_md_one_review_created**

The tested method requires the data of an agency instance in the mocked database table in order to populate the field `email address` of the event payload when raising the event. Hence, the test method populates a respectively typed internal table with an agency ID and an email address. Then, it inserts these test data into the database test double using the CDS test double framework.

Next, the method declares a structure for importing created review instances into the additional save exit using the respective derived type for the entity /DMO/I_AGENCYTP. Then, the method populates its tabular component /DMO/ZZ_REVIEW with the data of a review instance. The review instance has the same agency ID as the agency instance which has been inserted into the database test double beforehand.

In the next step, the method declares a structure for the late reported response using the respective derived type for the entity /DMO/I_AGENCYTP.

Now, the method executes the code under test using the declared `create` structure as the importing parameter and the `reported` structure as the changing parameter.

Next, the method asserts that the event /DMO/AGENCYREVIEWCREATED has been raised against the entity /DMO/I_AGENCYTP once.

Finally, the method checks whether the payload of the event is as expected.

```
METHOD save_md_one_review_created.
```

```
" Prepare
```

```
DATA agency_mock_data TYPE STANDARD TABLE OF /dmo/agency.
```

```
agency_mock_data = VALUE #( ( agency_id = '1' email_address = 'test@test.example' ) ).  
cds_test_environment->insert_test_data( agency_mock_data ).
```

```
DATA create TYPE REQUEST FOR CHANGE /dmo/i_agencytp.
```

```
create-/dmo/zz_review = VALUE #( ( agencyid      = '1'  
                                reviewid       = '1'  
                                rating          = '1'  
                                freetextcomment = 'Bad' ) ).
```

```
DATA reported TYPE RESPONSE FOR REPORTED LATE /dmo/i_agencytp.
```

```
" Execute
```

```
class_under_test->save_modified( EXPORTING create = create  
                                update   = VALUE #( )  
                                delete   = VALUE #( )  
                                CHANGING reported = reported ).
```

```
" Verify
```

```
event_test_environment->get_event( entity_name = agency_entity event_name = agency_event )->ver
```

```
DATA event_payload_act TYPE TABLE FOR EVENT /dmo/i_agencytp~/dmo/agencyreviewcreated.
```

```
event_payload_act = event_test_environment->get_event( entity_name = agency_entity event_name
```

```
DATA event_payload_exp TYPE TABLE FOR EVENT /dmo/i_agencytp~/dmo/agencyreviewcreated.
```

```
event_payload_exp = VALUE #( ( agencyid      = '1'  
                                reviewid     = '1'  
                                rating        = '1'  
                                freetextcomment = 'Bad'  
                                emailaddress  = 'test@test.example' ) ).
```

```
cl_abap_unit_assert=>assert_equals( act = event_payload_act exp = event_payload_exp ).
```

```
ENDMETHOD.
```

Implementing the Test Method `save_md_multi_reviews_created`

The implementation of this test method follows a similar principle as the test method `save_md_one_review_created`. In this test method, however, three review instances that refer to different agency instances are imported into the additional save method. Despite the fact that multiple review instances are created for multiple agencies, the event `/DMO/AGENCYREVIEWCREATED` is expected to be raised only once. The data of the different review instances are reflected in the payload of the event.

```
METHOD save_md_multi_reviews_created.
```

```
" Prepare
```

```
DATA agency_mock_data TYPE STANDARD TABLE OF /dmo/agency.
```

```
agency_mock_data = VALUE #( ( agency_id = '1' email_address = 'test@test.example' )  
                            ( agency_id = '2' email_address = 'test2@test.example' ) ).  
cds_test_environment->insert_test_data( agency_mock_data ).
```

```
DATA create TYPE REQUEST FOR CHANGE /dmo/i_agencytp.
```

```
create-/dmo/zz_review = VALUE #( ( agencyid      = '1'  
                                reviewid       = '1'  
                                rating          = '1'  
                                freetextcomment = 'Bad' )  
                                ( agencyid      = '1'  
                                reviewid       = '2'  
                                rating          = '4'  
                                freetextcomment = 'Good' )  
                                ( agencyid      = '2'  
                                reviewid       = '1'  
                                rating          = '2'  
                                freetextcomment = 'Medium' ) ).
```

```
DATA reported TYPE RESPONSE FOR REPORTED LATE /dmo/i_agencytp.
```

```

" Execute
class_under_test->save_modified( EXPORTING create    = create
                                update      = VALUE #( )
                                delete      = VALUE #( )
                                CHANGING    reported = reported ).

" Verify
event_test_environment->get_event( entity_name = agency_entity event_name = agency_event )->verify(

DATA event_payload_act TYPE TABLE FOR EVENT /dmo/i_agencytp~/dmo/agencyreviewcreated.
event_payload_act = event_test_environment->get_event( entity_name = agency_entity event_name = agency_event )->verify(

DATA event_payload_exp TYPE TABLE FOR EVENT /dmo/i_agencytp~/dmo/agencyreviewcreated.
event_payload_exp = VALUE #( ( agencyid      = '1'
                              reviewid     = '1'
                              rating       = '1'
                              freetextcomment = 'Bad'
                              emailaddress  = 'test@test.example' )
                             ( agencyid      = '1'
                              reviewid     = '2'
                              rating       = '4'
                              freetextcomment = 'Good'
                              emailaddress  = 'test@test.example' )
                             ( agencyid      = '2'
                              reviewid     = '1'
                              rating       = '2'
                              freetextcomment = 'Medium'
                              emailaddress  = 'test2@test.example' ) ).

cl_abap_unit_assert=>assert_equals( act = event_payload_act exp = event_payload_exp ).

ENDMETHOD.

```

Integration Tests

The following section shows how the described concepts of integration tests apply in RAP. It contains step-by-step guides explaining how to develop integration tests for a RAP business object.

Whereas unit tests operate within a business object to validate each functional unit of a particular RAP artefact, integration tests work outside of business objects and validate consumption use cases via EML and OData, where multiple functional units are involved. Reading or creating instances from outside for example make use of the whole RAP application and involve dependent operations, such as determinations and validations. Integration tests validate the consumption result and show whether the interaction between the involved functional units works as expected.

EML Integration Tests

The test class you will develop in this chapter contains a selection of EML integration tests, that test the interaction between the create operation and various dependencies. The business object that the test class refers to is the managed flight reference scenario (package /DMO/FLIGHT_MANAGED). The name of the test class containing the result of this section is called /DMO/TC_TRAVEL_M. You can use it as a reference to build your own test class as described in the following chapter.

EML Tests: Creating the Test Class

In this step, you will create the test class, which serves as the basis for the test methods to be developed.

To create the test class, proceed as follows:

1. Launch [ABAP Development Tools for Eclipse](#).

2. In your ABAP Project (or ABAP cloud project), create an ABAP class with the name `/DMO/TC_TRAVEL_M`.
3. In the tab **Global Class**, add the keywords `FOR TESTING` below the keywords `CLASS /DMO/TC_TRAVEL_M DEFINITION`.

i Expand the following code sample to view the source code of the test class `/DMO/TC_TRAVEL_M`.

Sample Code

```
CLASS /DMO/TC_TRAVEL_M DEFINITION
  FOR TESTING
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS /DMO/TC_TRAVEL_M IMPLEMENTATION.
ENDCLASS.
```

EML Tests: Refining the Test Class Structure

Now that you have created the basic test class structure, you can refine this structure by defining basic test properties, the test relation, variables and fixture methods that provide the desired test configuration. The defined test class forms the basis for the test methods that you will implement in the upcoming chapters.

This topic covers the specific integration test class structure for the managed flight reference scenario. For general information on the structure of RAP test classes, see [Basic Test Class Structure](#).

Test Class Definition

The definition section of the test class contains basic meta information, reference variables and declarations for the fixture and test methods.

Assigning Test Properties

Since the execution of the test methods is not expected to last longer than 5 seconds, you can use the `DURATION SHORT`. Depending on the respective run mode and runtime monitor, exceeding the chosen execution time can affect the results of your test.

As you won't change persistent data or system settings with your unit tests, the `RISK LEVEL HARMLESS` can be used. If the chosen risk level is higher than allowed in your system, your tests won't be executed.

```
CLASS /DMO/TC_TRAVEL_M DEFINITION
  FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS
```

Specifying the Test Relation

You will develop the integration tests for the business object `/DMO/I_TRAVEL_M`. Specifying a test relation to this business object enables you to run your integration tests directly via the context menu of the business object's behavior definition.

```
"! @testing BDEF:/DMO/I_TRAVEL_M
CLASS /DMO/TC_TRAVEL_M DEFINITION
```

Declaring Class Components

Since you will be using the interfaces of the CDS Test Double Framework and of the SQL Test Double Framework to mock your test data, you need to declare reference variables for these interfaces in the `CLASS-DATA` section.

Additionally, declare variables for the begin and end date using the respective types from the flight reference scenario. You will use these variables to prepare the flight data for the travel instances to be created.

Declare two tables for agency and customer data that will be used to store the respective mock data. Choose the suiting type from the flight reference scenario.

Lastly, declare constants for the content IDs used for each entity level.

PRIVATE SECTION.

```
CLASS-DATA:
  cds_test_environment TYPE REF TO if_cds_test_environment,
  sql_test_environment TYPE REF TO if_osql_test_environment,

  begin_date          TYPE /dmo/begin_date,
  end_date            TYPE /dmo/end_date,

  agency_mock_data     TYPE STANDARD TABLE OF /dmo/agency,
  customer_mock_data   TYPE STANDARD TABLE OF /dmo/customer.

  CONSTANTS cid        TYPE abp_behv_cid    VALUE 'ROOT1'.
  CONSTANTS cid_node   TYPE abp_behv_cid    VALUE 'NODE1'.
  CONSTANTS cid_subnode TYPE abp_behv_cid    VALUE 'SUBNODE1'.
```

Declaring Fixture and Test Methods

Declare the required test and fixture methods. Fixture methods are used to set up and tear down the test environment which produces unique test behavior for every executed test.

```
CLASS-METHODS:
  class_setup,
  class_teardown.

METHODS:
  setup,
  teardown,

  create_validation_negative FOR TESTING RAISING cx_static_check,
  create_validation_positive FOR TESTING RAISING cx_static_check,
  cba_fc_negative           FOR TESTING RAISING cx_static_check,
  cba_fc_positive          FOR TESTING RAISING cx_static_check,
  deep_create_determination FOR TESTING RAISING cx_static_check,
  create_action_fc_negative FOR TESTING RAISING cx_static_check,
  create_action_fc_positive FOR TESTING RAISING cx_static_check,
  create_additional_save    FOR TESTING RAISING cx_static_check.
```

Test Class Implementation

This section covers the implementation of the fixture methods that specify the test configuration used by the test methods.

Implementing the Method `class_setup`

Populate the tables `agency_mock_data` and `customer_mock_data` with sample content.

Your test methods will contain EML statements that access instance data via the CDS entities `Travel`, `Booking` and `Booking Supplement`. In order to make these EML statements access the data of test double databases instead of the real database tables, you now need to create test doubles using the CDS Test Double Framework. Specify the CDS views `/DMO/I_TRAVEL_M`, `/DMO/I_BOOKING_M` and `/DMO/I_BOOKSUPPL_M` to ensure that the underlying database tables are doubled.

The tested business object contains validations and an additional save implementation with OpenSQL statements that access the database tables `/DMO/AGENCY`, `/DMO/CUSTOMER` and `/DMO/LOG_TRAVEL`. In order to make these OpenSQL statements access the test doubles instead of the real databases, you also need to create test doubles for these database tables using the SQL Test Double Framework.

Lastly, insert the agency and customer mock data into the test doubles. For the mocked database table `/DMO/LOG_TRAVEL`, no test data are required.

```
METHOD class_setup.
    agency_mock_data      = VALUE #( ( agency_id   = '987654' name       = 'Miles and More'   ) ).
    customer_mock_data    = VALUE #( ( customer_id = '987653' last_name = 'Buchholm' ) ).

    cds_test_environment = cl_cds_test_environment=>create_for_multiple_cds(
        VALUE #(
            ( i_for_entity = '/DMO/I_TRAVEL_M'           )
            ( i_for_entity = '/DMO/I_BOOKING_M'          )
            ( i_for_entity = '/DMO/I_BOOKSUPPL_M'        )
        )
    ).

    sql_test_environment = cl_osql_test_environment=>create( i_dependency_list = VALUE #(
        ( '/DMO/AGENCY' )
        ( '/DMO/CUSTOMER' )
        ( '/DMO/LOG_TRAVEL' )
    ) ) .

    sql_test_environment->insert_test_data( agency_mock_data ).
    sql_test_environment->insert_test_data( customer_mock_data ).
ENDMETHOD.
```

Implementing the Method `setup`

To make sure that your test methods won't be influenced by data of other test methods, clear the contents of the CDS test doubles before every test method execution.

```
METHOD setup.
    cds_test_environment->clear_doubles( ).
ENDMETHOD.
```

Implementing the Method `teardown`

To make sure that the transactional buffer is reset for the involved CDS entities after every test method execution, use the EML statement `ROLLBACK ENTITIES` in the `teardown` method.

```
METHOD teardown.
    ROLLBACK ENTITIES.
ENDMETHOD.
```

Implementing the Method `class_teardown`

At the end of the test, clear the test environments and the created test doubles.

```
METHOD class_teardown.
  cds_test_environment->destroy( ).
  sql_test_environment->destroy( ).
ENDMETHOD.
```

Resulting Test Class

After following the steps described above, your test class should look like this and is ready for the implementation of the test methods:

i *Expand the following code sample to view the source code of the test class /DMO/TC_TRAVEL_M.*

Sample Code

```
"! @testing BDEF:/DMO/I_TRAVEL_M
CLASS /dmo/tc_travel_m DEFINITION
  FOR TESTING
    RISK LEVEL HARMLESS
    DURATION SHORT
    PUBLIC
    FINAL
    CREATE PUBLIC .

  PUBLIC SECTION.
  PROTECTED SECTION.
  PRIVATE SECTION.

  CLASS-DATA:
    cds_test_environment TYPE REF TO if_cds_test_environment,
    sql_test_environment TYPE REF TO if_osql_test_environment,

    begin_date           TYPE /dmo/begin_date,
    end_date             TYPE /dmo/end_date,

    agency_mock_data     TYPE STANDARD TABLE OF /dmo/agency,
    customer_mock_data   TYPE STANDARD TABLE OF /dmo/customer.

  CONSTANTS cid          TYPE abp_behv_cid    VALUE 'ROOT1'.
  CONSTANTS cid_node     TYPE abp_behv_cid    VALUE 'NODE1'.
  CONSTANTS cid_subnode  TYPE abp_behv_cid    VALUE 'SUBNODE1'.

  CLASS-METHODS:
    class_setup,
    class_teardown.

  METHODS:
    setup,
    teardown,

    create_validation_negative FOR TESTING RAISING cx_static_check,
    create_validation_positive FOR TESTING RAISING cx_static_check,
    deep_create_determination FOR TESTING RAISING cx_static_check,
    create_action_fc_negative  FOR TESTING RAISING cx_static_check,
    create_action_fc_positive  FOR TESTING RAISING cx_static_check,
    cba_fc_negative            FOR TESTING RAISING cx_static_check,
    cba_fc_positive            FOR TESTING RAISING cx_static_check,
    create_additional_save     FOR TESTING RAISING cx_static_check.

ENDCLASS.

CLASS /dmo/tc_travel_m IMPLEMENTATION.

  METHOD class_setup.
    agency mock data      = VALUE #( ( agency id    = '987654' name      = 'Miles and More' ) ).
```

```

customer_mock_data = VALUE #( ( customer_id = '987653' last_name = 'Buchholm' ) ).

cds_test_environment = cl_cds_test_environment=>create_for_multiple_cds(
    VALUE #(
        ( i_for_entity = '/DMO/I_TRAVEL_M' )
        ( i_for_entity = '/DMO/I_BOOKING_M' )
        ( i_for_entity = '/DMO/I_BOOKSUPPL_M' )
    )
).

sql_test_environment = cl_osql_test_environment=>create( i_dependency_list = VALUE #(
    ( '/DMO/AGENCY' )
    ( '/DMO/CUSTOMER' )
    ( '/DMO/LOG_TRAVEL' )
) ) .

sql_test_environment->insert_test_data( agency_mock_data ).
sql_test_environment->insert_test_data( customer_mock_data ).
ENDMETHOD.

METHOD setup.
    cds_test_environment->clear_doubles( ).
ENDMETHOD.

METHOD teardown.
    ROLLBACK ENTITIES.
ENDMETHOD.

METHOD class_teardown.
    cds_test_environment->destroy( ).
    sql_test_environment->destroy( ).
ENDMETHOD.

METHOD create_validation_negative.
ENDMETHOD.

METHOD create_validation_positive.
ENDMETHOD.

METHOD deep_create_determination.
ENDMETHOD.

METHOD create_action_fc_negative.
ENDMETHOD.

METHOD create_action_fc_positive.
ENDMETHOD.

METHOD cba_fc_negative.
ENDMETHOD.

METHOD cba_fc_positive.
ENDMETHOD.

METHOD create_additional_save.
ENDMETHOD.

```

ENDCLASS.

EML Tests: Implementing the Test Methods

The test class you develop in this chapter tests the business object from the managed flight reference scenario (package /DMO/FLIGHT_MANAGED).

Test Methods

The test method `create_validation_negative` triggers the creation of a travel instance with invalid flight data and checks whether the validation `validateDates` returns the expected results. The test method `create_validation_positive` tests the case where the specified flight data are correct.

The test method `deep_create_determination` triggers a create operation and two create-by-association operations on the booking and supplement entity. The test method tests whether the triggered determination `calculateTotalPrice` returns the correct total price.

The test method `create_action_fc_negative` creates a travel instance and then triggers the action `acceptTravel`. Since the overall status of the created travel instance already is **accepted**, the execution of the action is expected to be prevented by the instance feature control implementation. The test method checks the result of the instance feature control implementation. The test method `create_action_fc_positive` tests the case where the execution of the action is allowed.

The test method `cba_fc_negative` creates a travel instance and then triggers a create-by-association operation to create a booking instance. Since the overall status of the created travel instance is **rejected**, the execution of the create-by-association operation is prevented by the instance feature control implementation. The test method `cba_fc_positive` tests the case where the execution of the create-by-association operation is allowed.

The test method `create_additional_save` creates a travel instance and then tests whether the used operation and the changed fields have been logged correctly by the triggered additional save implementation.

Contents

[Test Scenario Create + Validation](#)

[Test Scenario Deep Create + Determination](#)

[Test Scenario Create + Feature Control + Action](#)

[Test Scenario Create + Feature Control + Create-by-Association](#)

[Test Scenario Create + Additional Save](#)

Test Scenario Create + Validation

This integration test checks whether the validation `validateDates` prevents the persistence of a created travel instance containing invalid flight data.

Test Method `create_validation_negative`

This test method tests the case where a consumer tries to create the travel instance with an invalid flight date. Hence, in the given example, a begin date which lies in the past is assigned to the variable `begin_date`. The variable `end_date` gets a valid flight date.

The method now performs an EML modify create call on the business object using the prepared variables. Since the method only focuses on the validation `validateDates`, valid agency and customer data are provided.

The create operation is expected to run correctly as the validation `validateDates` is only executed as part of the save sequence, which will be triggered in a subsequent EML statement. Hence, the method asserts that the `mapped` response from the EML create statement contains the expected content and travel ID and that the `failed` and `reported` responses are empty.

Using the `COMMIT ENTITIES` statement, the test method now triggers the save sequence during which the validation `validateDates` is executed.

Since the provided start date lies in the past, the method asserts that the `failed` response of the `COMMIT ENTITIES` operation contains the travel ID of the failed instance. Additionally, the method asserts that the system field `sy-subrc` contains the return value 4, indicating that the `check_before_save` has not been carried out successfully.

According to the logic of the validation implementation, both the fields for the start and the end date are marked in the `reported` response, if a wrong start date is provided. The test methods specifies respective assert statements to check this. Additionally, the method asserts the correct message information is contained in the `reported` response.

i Expand the following code sample to view the source code of the method `create_validation_negative`.

Sample Code

```
METHOD create_validation_negative.

begin_date = cl_abap_context_info=>get_system_date( ) - 10.
end_date   = cl_abap_context_info=>get_system_date( ) + 30.

" Create a Travel with invalid begin_date
MODIFY ENTITIES OF /dmo/i_travel_m
ENTITY travel
  CREATE FIELDS ( agency_id customer_id begin_date end_date description booking_fee curren
  VALUE #( (      %cid = cid
               agency_id      = agency_mock_data[ 1 ]-agency_id
               customer_id     = customer_mock_data[ 1 ]-customer_id
               begin_date      = begin_date
               end_date        = end_date
               description      = 'TestTravel 1'
               booking_fee     = '10.5'
               currency_code    = 'EUR'
               overall_status  = '0' ) )
MAPPED DATA(mapped_create)
FAILED DATA(failed_create)
REPORTED DATA(reported_create).

" Travel was created successfully
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( mapped_create-travel ) ).
DATA(key_of_new_travel) = mapped_create-travel[ 1 ].
cl_abap_unit_assert=>assert_equals( exp = cid act = key_of_new_travel-%cid ).
cl_abap_unit_assert=>assert_not_initial( act = key_of_new_travel-travel_id ).

cl_abap_unit_assert=>assert_initial( act = failed_create ).
cl_abap_unit_assert=>assert_initial( act = reported_create ).

" Trigger Validations
COMMIT ENTITIES
RESPONSE OF /DMO/I_Travel_M
FAILED DATA(failed_commit)
REPORTED DATA(reported_commit).

" Commit failed as begin_date is invalid ( must be after system date )
```

```

cl_abap_unit_assert=>assert_subrc( exp = 4 ).

cl_abap_unit_assert=>assert_equals( act = lines( failed_commit-travel ) exp = 1 ).
cl_abap_unit_assert=>assert_not_initial( act = failed_commit-travel[ 1 ]-travel_id ).

cl_abap_unit_assert=>assert_equals( act = lines( reported_commit-travel ) exp = 1 ).
cl_abap_unit_assert=>assert_not_initial( act = reported_commit-travel[ 1 ]-travel_id ).
cl_abap_unit_assert=>assert_equals( act = reported_commit-travel[ 1 ]-%element-begin_date exp
cl_abap_unit_assert=>assert_equals( act = reported_commit-travel[ 1 ]-%element-end_date exp
cl_abap_unit_assert=>assert_equals( exp = 004 act = reported_commit-travel[ 1 ]-%msg->if_t10
cl_abap_unit_assert=>assert_equals( exp = '/DMO/CM_FLIGHT' act = reported_commit-travel[ 1 ]

```

ENDMETHOD.

Test Method create_validation_positive

The second test method tests the case where a consumer creates a travel instance with valid flight data. In contrast to the negative case shown above, the failed and reported responses of the COMMIT ENTITIES statement are now expected to be empty. The system field sy-subrc is expected to contain the value 0, indicating that the commit has been successful.

i Expand the following code sample to view the source code of the method create_validation_positive.

Sample Code

METHOD create_validation_positive.

```

begin_date = cl_abap_context_info=>get_system_date( ) + 10.
end_date   = cl_abap_context_info=>get_system_date( ) + 30.

" Create a Travel with valid begin_date
MODIFY ENTITIES OF /dmo/i_travel_m
ENTITY travel
  CREATE FIELDS ( agency_id customer_id begin_date end_date description booking_fee curren
    VALUE #( ( %cid = cid
               agency_id      = agency_mock_data[ 1 ]-agency_id
               customer_id     = customer_mock_data[ 1 ]-customer_id
               begin_date      = begin_date
               end_date        = end_date
               description      = 'TestTravel 1'
               booking_fee     = '10.5'
               currency_code   = 'EUR'
               overall_status  = '0' ) )
MAPPED DATA(mapped_create)
FAILED DATA(failed_create)
REPORTED DATA(reported_create).

" Travel was created successfully
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( mapped_create-travel ) ).
DATA(key_of_new_travel) = mapped_create-travel[ 1 ].
cl_abap_unit_assert=>assert_equals( exp = cid act = key_of_new_travel-%cid ).
cl_abap_unit_assert=>assert_not_initial( act = key_of_new_travel-travel_id ).

cl_abap_unit_assert=>assert_initial( act = failed_create ).
cl_abap_unit_assert=>assert_initial( act = reported_create ).

" Trigger Validations
COMMIT ENTITIES
RESPONSE OF /DMO/I_Travel_M
FAILED DATA(failed_commit)
REPORTED DATA(reported_commit).

" Commit was successful
cl_abap_unit_assert=>assert_subrc( exp = 0 ).
cl_abap_unit_assert=>assert_initial( act = failed_commit ).
cl_abap_unit_assert=>assert_initial( act = reported_commit ).

```

ENDMETHOD.

Test Scenario Deep Create + Determination

This integration test checks whether the determination `calculateTotalPrice` returns the correct total price for a created travel instance containing a booking and a supplement instance.

First, the test method populates variables for the begin and end date.

Next, the method performs an EML modify call which creates a travel and corresponding booking and supplement instances. To ensure that the create and create-by-association operations are performed correctly, valid data are provided for the flight dates as well as for the agency and customer IDs.

Now the method performs an EML read call to get the calculation result of the determination `calculateTotalPrice`. In this EML read statement, the travel instance is identified via the travel ID taken from the mapped response of the preceding EML create statement.

The method finally asserts that the read calculation result equals the expected value.

i *Expand the following code sample to view the source code of the method `deep_create_determination`.*

Sample Code

```
METHOD deep_create_determination.
```

```
begin_date = cl_abap_context_info=>get_system_date( ) + 10.
end_date   = cl_abap_context_info=>get_system_date( ) + 30.
```

```
" Create a Travel, Booking and Supplement
```

```
MODIFY ENTITIES OF /dmo/i_travel_m
```

```
ENTITY travel
```

```
CREATE FIELDS (      agency_id customer_id begin_date end_date description booking_fee cur
VALUE #( (          %cid = cid
                  agency_id      = agency_mock_data[ 1 ]-agency_id
                  customer_id     = customer_mock_data[ 1 ]-customer_id
                  begin_date      = begin_date
                  end_date        = end_date
                  description      = 'TestTravel 1'
                  booking_fee     = '10.5'
                  currency_code   = 'EUR'
                  overall_status  = '0' ) )
```

```
CREATE BY \_booking FIELDS( booking_date customer_id carrier_id connection_id flight_da
                           booking_status )
```

```
WITH VALUE #( ( %cid_ref = cid
                %target = VALUE #( ( %cid              = cid_node
                                   booking_date         = begin_date
                                   customer_id           = customer_mock_data[ 1 ]-customer_
                                   carrier_id            = '1'
                                   connection_id         = '1'
                                   flight_date           = begin_date
                                   flight_price          = '200'
                                   currency_code         = 'EUR'
                                   booking_status        = 'N' ) )
```

```
) )
```

```
ENTITY booking
```

```
CREATE BY \_booksupplement FIELDS ( supplement_id price currency_code ) WITH
```

```
VALUE #( ( %cid_ref = cid_node
          %target = VALUE #( ( %cid              = cid_subnode
                              supplement_id      = '100'
                              price              = '1000'
                              currency_code      = 'EUR' ) )
```

```
) )
```

```

MAPPED DATA(mapped_cba)
FAILED DATA(failed_cba)
REPORTED DATA(reported_cba).

```

```

" Read determination result
READ ENTITIES OF /dmo/i_travel_m
ENTITY travel
  FIELDS ( total_price )
  WITH VALUE #( ( travel_id = mapped_cba-travel[ 1 ]-%tky ) )
  RESULT DATA(result)
  FAILED DATA(failed_read).

" Determination calculation was successful
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( result ) ).
cl_abap_unit_assert=>assert_equals( act = result[ 1 ]-total_price exp = '1210.50' ).

ENDMETHOD.

```

Test Scenario Create + Feature Control + Action

This integration test checks whether the instance feature control implementation `get_features` prevents the execution of the action `acceptTravel`, which is performed on a created travel instance.

Test Method `create_action_fc_negative`

First, the test method populates variables for the begin and end date.

The method now performs an EML modify create call on the business object using the prepared date variables as well as prepared agency and customer IDs. Since the method tests the case where the execution of the action `acceptTravel` is prevented, the overall status **A** is provided.

The create operation is expected to run correctly. Hence, the method asserts that the `mapped` response from the EML create statement contains the expected content and travel ID and that the `failed` and `reported` responses are empty.

Now, the method triggers the action `acceptTravel` on the created travel instance. In this EML modify statement, the travel instance is identified via the travel ID taken from the `mapped` response of the preceding EML create statement.

According to the logic of the `get_features` method, the action `acceptTravel` is disabled for travel instances with the overall status **A**. Hence, the method asserts the `failed` response of the action contains a travel ID and that the action `acceptTravel` is marked respectively.

No messages are expected to be returned by the action, hence the method asserts that the `reported` response is empty.

i Expand the following code sample to view the source code of the method `create_action_fc_negative`.

Sample Code

```

METHOD create_action_fc_negative.

  begin_date = cl_abap_context_info=>get_system_date( ) + 10.
  end_date   = cl_abap_context_info=>get_system_date( ) + 30.

  " Create a travel
  MODIFY ENTITIES OF /dmo/i_travel_m
  ENTITY travel
    CREATE FIELDS ( agency_id customer_id begin_date end_date description booking_fee curr
    VALUE #( ( %cid = cid
              agency_id = agency_mock_data[ 1 ]-agency_id
              customer_id = customer_mock_data[ 1 ]-customer_id

```

```

begin_date      = begin_date
end_date        = end_date
description     = 'TestTravel 1'
booking_fee     = '10.5'
currency_code   = 'EUR'
overall_status  = 'A' ) )
MAPPED DATA(mapped_create)
FAILED DATA(failed_create)
REPORTED DATA(reported_create).

" Travel was created successfully
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( mapped_create-travel ) ).
DATA(key_of_new_travel) = mapped_create-travel[ 1 ].
cl_abap_unit_assert=>assert_equals( exp = cid act = key_of_new_travel-%cid ).
cl_abap_unit_assert=>assert_not_initial( act = key_of_new_travel-travel_id ).

cl_abap_unit_assert=>assert_initial( failed_create ).
cl_abap_unit_assert=>assert_initial( reported_create ).

" Execute action
MODIFY ENTITIES OF /dmo/i_travel_m
ENTITY travel
EXECUTE acceptTravel FROM VALUE #( ( travel_id = mapped_create-travel[ 1 ]-%tky ) )
FAILED DATA(failed_action)
REPORTED DATA(reported_action).

" Action failed as overall status is already accepted
cl_abap_unit_assert=>assert_equals( act = lines( failed_action-travel ) exp = 1 ).
cl_abap_unit_assert=>assert_not_initial( act = failed_action-travel[ 1 ]-travel_id ).
cl_abap_unit_assert=>assert_equals( exp = if_abap_behv=>mk-on act = failed_action-travel[ 1 ]

cl_abap_unit_assert=>assert_initial( reported_action ).

ENDMETHOD.

```

Test Method create_action_fc_positive

The second test method tests the case where a consumer creates a travel instance with the overall status X. In this case, the action can be executed successfully.

i Expand the following code sample to view the source code of the method **create_action_fc_positive**.

Sample Code

```

METHOD create_action_fc_positive.

begin_date = cl_abap_context_info=>get_system_date( ) + 10.
end_date   = cl_abap_context_info=>get_system_date( ) + 30.

" Create a Travel
MODIFY ENTITIES OF /dmo/i_travel_m
ENTITY travel
CREATE FIELDS (      agency_id customer_id begin_date end_date description booking_fee curr
VALUE #( (          %cid = cid
              agency_id      = agency_mock_data[ 1 ]-agency_id
              customer_id     = customer_mock_data[ 1 ]-customer_id
              begin_date      = begin_date
              end_date         = end_date
              description      = 'TestTravel 1'
              booking_fee      = '10.5'
              currency_code    = 'EUR'
              overall_status   = 'X' ) ) )
MAPPED DATA(mapped_create)
FAILED DATA(failed_create)
REPORTED DATA(reported_create).

" Travel was created successfully
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( mapped_create-travel ) ).
DATA(key_of_new_travel) = mapped_create-travel[ 1 ].

```

```

cl_abap_unit_assert=>assert_equals( exp = cid act = key_of_new_travel-%cid ).
cl_abap_unit_assert=>assert_not_initial( act = key_of_new_travel-travel_id ).

cl_abap_unit_assert=>assert_initial( failed_create ).
cl_abap_unit_assert=>assert_initial( reported_create ).

" Execute action
MODIFY ENTITIES OF /dmo/i_travel_m
  ENTITY travel
    EXECUTE acceptTravel FROM VALUE #( ( travel_id = mapped_create-travel[ 1 ]-%tky ) )
  FAILED DATA(failed_action)
  REPORTED DATA(reported_action).

" Read action result
READ ENTITIES OF /dmo/i_travel_m
  ENTITY travel
    FIELDS ( overall_status )
    WITH VALUE #( ( travel_id = mapped_create-travel[ 1 ]-%tky ) )
  RESULT DATA(result)
  FAILED DATA(failed_read).

" Read was executed successfully
cl_abap_unit_assert=>assert_initial( failed_read ).

" Action was executed successfully
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( result ) ).
cl_abap_unit_assert=>assert_equals( act = result[ 1 ]-overall_status exp = 'A' ).

ENDMETHOD.

```

Test Scenario Create + Feature Control + Create-by-Association

This integration test checks whether the instance feature control implementation `get_features` prevents the execution of a create-by-association operation which is performed on a created instance.

Test Method `cba_fc_negative`

First, the test method populates variables for the begin and end date.

Next, the method performs an EML modify create call which creates a travel instance. To ensure that the create operation is performed correctly, valid data are provided for the flight dates as well as for the agency and customer IDs. Since the method tests the case where the execution of the subsequent create-by-association operation is prevented, the overall status X is provided.

The create operation is expected to run correctly. Hence, the method asserts that the mapped response from the EML create statement contains the expected content and travel ID and that the failed and reported responses are empty.

Now, the method triggers a create-by-association operation.

According to the logic of the `get_features` method, the association `_booking` is disabled for travel instances with the overall status X. Hence, the method asserts that the mapped response of the create-by-association operation is empty. Additionally, the method asserts that the failed response contains a travel ID and that the association `_booking` is marked respectively. No messages are expected to be returned, hence the method asserts that the reported response is empty.

i Expand the following code sample to view the source code of the method `cba_fc_negative`.

Sample Code

```

METHOD cba_fc_negative.

  begin_date = cl_abap_context_info=>get_system_date( ) + 10.

```

```

end_date    = cl_abap_context_info=>get_system_date( ) + 30.

" Create a Travel with overall status "Cancelled"
MODIFY ENTITIES OF /dmo/i_travel_m
ENTITY travel
  CREATE FIELDS (      agency_id customer_id begin_date end_date description booking_fee curre
  VALUE #( (          %cid = cid
                    agency_id      = agency_mock_data[ 1 ]-agency_id
                    customer_id     = customer_mock_data[ 1 ]-customer_id
                    begin_date      = begin_date
                    end_date        = end_date
                    description      = 'TestTravel 1'
                    booking_fee     = '10.5'
                    currency_code   = 'EUR'
                    overall_status  = 'X' ) )
MAPPED DATA(mapped_create)
FAILED DATA(failed_create)
REPORTED DATA(reported_create).

" Travel was created successfully
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( mapped_create-travel ) ).
DATA(key_of_new_travel) = mapped_create-travel[ 1 ].
cl_abap_unit_assert=>assert_equals( exp = cid act = key_of_new_travel-%cid ).
cl_abap_unit_assert=>assert_not_initial( act = key_of_new_travel-travel_id ).

cl_abap_unit_assert=>assert_initial( failed_create ).
cl_abap_unit_assert=>assert_initial( reported_create ).

" Create a Booking for the created Travel
MODIFY ENTITIES OF /dmo/i_travel_m
ENTITY travel
  CREATE BY \_booking FIELDS ( booking_date customer_id carrier_id connection_id flight_date
  VALUE #( (          travel_id = mapped_create-travel[ 1 ]-%tky
                    %target = VALUE #( ( %cid = cid_node
                                      booking_date   = begin_date
                                      customer_id    = customer_mock_data[ 1 ]-customer_
                                      carrier_id     = '1'
                                      connection_id  = '1'
                                      flight_date    = begin_date
                                      flight_price   = '200'
                                      currency_code  = 'EUR'
                                      booking_status = 'B' ) )
                    ) )
MAPPED DATA(mapped_cba)
FAILED DATA(failed_cba)
REPORTED DATA(reported_cba).

" Booking was not created as overall status of Travel is "Cancelled" (should be "Open" or "A
cl_abap_unit_assert=>assert_initial( act = mapped_cba-booking ).

cl_abap_unit_assert=>assert_equals( act = lines( failed_cba-travel ) exp = 1 ).
cl_abap_unit_assert=>assert_not_initial( act = failed_cba-travel[ 1 ]-travel_id ).
cl_abap_unit_assert=>assert_equals( exp = if_abap_behv=>mk-on act = failed_cba-travel[ 1 ]-%

cl_abap_unit_assert=>assert_initial( act = reported_cba ).

ENDMETHOD.

```

Test Method cba_fc_positive

The second test method tests the case where a consumer creates a travel instance with the overall status **A**. In contrast to the negative case shown above, the mapped response of the create-by-association operation is now expected to contain the respective IDs and the failed response is expected to be empty.

i Expand the following code sample to view the source code of the method **cba_fc_positive**.

≡, **Sample Code**

METHOD cba_fc_positive.

```
begin_date = cl_abap_context_info=>get_system_date( ) + 10.
end_date   = cl_abap_context_info=>get_system_date( ) + 30.
```

" Create a Travel with overall status "Accepted"

MODIFY ENTITIES OF /dmo/i_travel_m

ENTITY travel

```
CREATE FIELDS (      agency_id customer_id begin_date end_date description booking_fee curre
VALUE #( (          %cid = cid
                  agency_id      = agency_mock_data[ 1 ]-agency_id
                  customer_id     = customer_mock_data[ 1 ]-customer_id
                  begin_date      = begin_date
                  end_date        = end_date
                  description      = 'TestTravel 1'
                  booking_fee     = '10.5'
                  currency_code   = 'EUR'
                  overall_status  = 'A' ) )
```

MAPPED DATA(mapped_create)

FAILED DATA(failed_create)

REPORTED DATA(reported_create).

" Travel was created successfully

```
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( mapped_create-travel ) ).
```

```
DATA(key_of_new_travel) = mapped_create-travel[ 1 ].
```

```
cl_abap_unit_assert=>assert_equals( exp = cid act = key_of_new_travel-%cid ).
```

```
cl_abap_unit_assert=>assert_not_initial( act = key_of_new_travel-travel_id ).
```

```
cl_abap_unit_assert=>assert_initial( failed_create ).
```

```
cl_abap_unit_assert=>assert_initial( reported_create ).
```

" Create a Booking for the created Travel

MODIFY ENTITIES OF /dmo/i_travel_m

ENTITY travel

```
CREATE BY \_booking FIELDS ( booking_date customer_id carrier_id connection_id flight_date
VALUE #( (          travel_id = mapped_create-travel[ 1 ]-%tky
                  %target = VALUE #( ( %cid = cid_node
                                      booking_date   = begin_date
                                      customer_id    = customer_mock_data[ 1 ]-customer_
                                      carrier_id     = '1'
                                      connection_id  = '1'
                                      flight_date    = begin_date
                                      flight_price   = '200'
                                      currency_code  = 'EUR'
                                      booking_status = 'B' ) )
    ) )
```

MAPPED DATA(mapped_cba)

FAILED DATA(failed_cba)

REPORTED DATA(reported_cba).

" Booking was created successfully

```
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( mapped_cba-booking ) ).
```

```
DATA(key_of_new_booking) = mapped_cba-booking[ 1 ].
```

```
cl_abap_unit_assert=>assert_equals( exp = cid_node act = key_of_new_booking-%cid ).
```

```
cl_abap_unit_assert=>assert_not_initial( act = key_of_new_booking-booking_id ).
```

```
cl_abap_unit_assert=>assert_initial( act = failed_cba ).
```

```
cl_abap_unit_assert=>assert_initial( act = reported_cba ).
```

ENDMETHOD.

Test Scenario Create + Additional Save

This integration test checks whether the additional save implementation logs the used operation and the changed fields correctly for a performed modify create operation.

First, the test method populates variables for the begin and end date.

Next, the method performs an EML modify create call which creates a travel instance. To ensure that the create operation is performed correctly, valid data are provided for the flight dates as well as for the agency and customer IDs.

The create operation is expected to be performed without errors. Hence, the method asserts that the mapped response from the EML create statement contains the expected content and travel ID and that the failed and reported responses are empty.

Using the COMMIT ENTITIES statement, the test method now triggers the save sequence during which the additional save implementation is executed.

The save sequence is expected to be executed without errors. Hence, the method asserts that the failed and reported responses of the commit operation are empty. Moreover, the system field sy - subrc is expected to contain the value 0, indicating that the commit has been executed successfully.

Finally, the method asserts that the table /dmo/log_travel, which is used by the additional save implementation, contains the expected log entries for the used operation and the changed fields.

i Expand the following code sample to view the source code of the method `create_additional_save`.

Sample Code

```
METHOD create_additional_save.
```

```
begin_date = cl_abap_context_info=>get_system_date( ) + 10.
end_date   = cl_abap_context_info=>get_system_date( ) + 30.
```

```
" Create a Travel
```

```
MODIFY ENTITIES OF /dmo/i_travel_m
```

```
ENTITY travel
```

```
  CREATE FIELDS (    agency_id customer_id begin_date end_date description booking_fee curre
    VALUE #( (      %cid = cid
                  agency_id      = agency_mock_data[ 1 ]-agency_id
                  customer_id     = customer_mock_data[ 1 ]-customer_id
                  begin_date      = begin_date
                  end_date        = end_date
                  description      = 'TestTravel 1'
                  booking_fee     = '10.5'
                  currency_code   = 'EUR'
                  overall_status  = 'A' ) )
```

```
MAPPED DATA(mapped_create)
```

```
FAILED DATA(failed_create)
```

```
REPORTED DATA(reported_create).
```

```
" Travel was created successfully
```

```
cl_abap_unit_assert=>assert_equals( exp = 1 act = lines( mapped_create-travel ) ).
```

```
DATA(key_of_new_travel) = mapped_create-travel[ 1 ].
```

```
cl_abap_unit_assert=>assert_equals( exp = cid act = key_of_new_travel-%cid ).
```

```
cl_abap_unit_assert=>assert_not_initial( act = key_of_new_travel-travel_id ).
```

```
cl_abap_unit_assert=>assert_initial( failed_create ).
```

```
cl_abap_unit_assert=>assert_initial( reported_create ).
```

```
" Trigger additional save
```

```
COMMIT ENTITIES
```

```
RESPONSE OF /DMO/I_Travel_M
```

```
FAILED DATA(failed_commit)
```

```
REPORTED DATA(reported_commit).
```

```
cl_abap_unit_assert=>assert_subrc( exp = 0 ).
```

```
cl_abap_unit_assert=>assert_initial( act = failed_commit ).
```

```
cl_abap_unit_assert=>assert_initial( act = reported_commit ).
```

```
" Ensure that a create operation has been executed
```

```
SELECT changed_field_name FROM /dmo/log_travel WHERE changing_operation = 'CREATE' ORDER BY
```

```
cl_abap_unit_assert=>assert_equals( act = lines( log_fields ) exp = 2 ).
```

```
cl_abap_unit_assert=>assert_equals( act = log_fields[ 1 ] exp = 'booking_fee' ).
```

```
cl_abap_unit_assert=>assert_equals( act = log_fields[ 2 ] exp = 'overall_status' ).
```

ENDMETHOD.

OData Integration Tests

The test class you will develop in this chapter tests an OData service exposing a business object with a virtual element implementation which is described in chapter [Using Virtual Elements in CDS Projection Views](#). Use the linked chapter to implement a virtual element in your existing business object. You can then use the instructions of this chapter to develop OData integration tests for it.

i Note

The test class described in this chapter is not part of the /DMO/FLIGHT package.

The described test class uses the OData Client Proxy to execute OData calls on a BO which is exposed as an OData service. Your test class will then evaluate the results of these OData calls in order to ensure, that the OData service works as a whole and that it fulfills typical UI use cases.

For more information on the OData Client Proxy, see [OData Client Proxy](#).

The test class uses the Local Client Proxy since its OData calls run in the same ABAP session as the OData service itself. This enables the usage of the CDS Test Double Framework and other test double frameworks to mock dependencies.

For information on OPA (One Page Acceptance) tests of SAP Fiori Elements for OData V4 Applications, see <https://developers.sap.com/group.fiori-elements-mockserver-opa.html>.

OData Tests: Creating the Test Class

In this step, you will create the test class, which serves as the basis for the test method to be developed.

ADT provides the option to generate an OData test class template containing the basic structure for different calls executed by the OData Client Proxy. This option is available in the context menu of a selected entity set in a service binding. In the following guide, however, you will start from scratch and develop the OData test class individually.

To create the test class, proceed as follows:

1. Launch **ABAP Development Tools**.
2. In your ABAP Project (or ABAP cloud project), create an ABAP class with the name /DMO/TC_BOOKING_ODATA.
3. In the tab **Global Class**, add the keywords FOR TESTING below the keywords CLASS /DMO/TC_BOOKING_ODATA DEFINITION.

i Expand the following code sample to view the source code of the test class /DMO/TC_BOOKING_ODATA.

≡ Sample Code

```
CLASS /DMO/TC_BOOKING_ODATA DEFINITION
  FOR TESTING
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
```

```

PROTECTED SECTION.
PRIVATE SECTION.
ENDCLASS.

```

```

CLASS /DMO/TC_BOOKING_ODATA IMPLEMENTATION.
ENDCLASS.

```

OData Tests: Refining the Test Class Structure

Now that you have created the basic test class structure, you can refine this structure by defining basic test properties, the test relation, variables and fixture methods that provide the desired test configuration. The defined test class forms the basis for the test methods that you will implement in the upcoming chapter.

This topic covers the specific integration test class structure for OData tests. For general information on the structure of RAP test classes, see [Basic Test Class Structure](#).

Test Class Definition

The definition section of the test class contains basic meta information, reference variables and declarations for the used methods.

Assigning Test Properties

Since the execution of the test methods is not expected to last longer than 5 seconds, you can use the `DURATION SHORT`. Depending on the respective run mode and runtime monitor, exceeding the chosen execution time can affect the results of your test.

As you won't change persistent data or system settings with your unit tests, the `RISK LEVEL HARMLESS` can be used. If the chosen risk level is higher than allowed in your system, your tests won't be executed.

```

CLASS /DMO/TC_BOOKING_ODATA DEFINITION
  FOR TESTING
    DURATION SHORT
    RISK LEVEL HARMLESS

```

Specifying the Test Relation

You will develop the integration tests for the service binding `/DMO/UI_BOOKING_02` which exposes the business object as an OData V2 service. Specifying a test relation enables you to run your integration tests directly via the context menu of the service binding.

```

"!@testing SRVB:/DMO/UI_BOOKING_02
CLASS /DMO/TC_BOOKING_ODATA DEFINITION

```

Declaring Class Components

Since you will be using the Local Client Proxy, filter APIs for the OData response and the CDS Test Double Framework to mock the database dependencies, you need to declare reference variables for the respective interfaces in the CLASS-DATA section. Furthermore, a structure type is required which will be later used to identify an entity instance for an OData request.

The filter criteria for the OData response will be built using a ranges table. Hence you need to type a ranges table with the field the filter will be applied for.

Lastly, declare a table and a variable used to prepare the booking instance data for the test double.

PRIVATE SECTION.

```

CLASS-DATA: go_client_proxy      TYPE REF TO /iwbsp/if_cp_client_proxy,
            lo_filter_factory    TYPE REF TO /iwbsp/if_cp_filter_factory,
            lo_filter_node       TYPE REF TO /iwbsp/if_cp_filter_node,
            cds_test_environment TYPE REF TO if_cds_test_environment,
            lt_range_currencycode TYPE RANGE OF /dmo/currency_code,
            date                 TYPE /dmo/begin_date,
            booking_mock_data    TYPE TABLE OF /dmo/booking,
            booksupplement_mock_data TYPE STANDARD TABLE OF /dmo/book_suppl.

```

TYPES:

```

BEGIN OF ts_key,
    travelid TYPE /dmo/travel_id,
    bookingid TYPE /dmo/booking_id,
END OF ts_key.

```

Declaring required Methods

Declare the required test and fixture methods. Fixture methods are used to set up and tear down the test environment which produces unique test behavior for every executed test. The method `create_local_client_proxy` is used to create an instance of the Local Client Proxy in the method `class_setup`.

CLASS-METHODS :

```

class_setup RAISING cx_static_check,
class_teardown,

create_local_client_proxy
IMPORTING
    service_key      TYPE /iwbsp/if_cp_client_proxy=>ty_s_service_key_v2
RETURNING
    VALUE(client_proxy) TYPE REF TO /iwbsp/if_cp_client_proxy .

```

METHODS:

```

setup,
teardown,
virtual_element FOR TESTING RAISING cx_static_check,
rba_with_filter FOR TESTING RAISING cx_static_check.

```

Test Class Implementation

This section covers the implementation of the fixture methods that specify the test configuration used by the test methods as well as the method `create_local_client_proxy` used to create the local client proxy instance.

Implementing the Method `class_setup`

Call the method `create_local_client_proxy` with the service binding `/DMO/UI_BOOKING_02` and the service ID `0001` as parameters and assign the result to the reference variable `go_client_proxy` declared above.

In order to make the OData requests access the data of test doubles instead of the real database tables, you now need to create test doubles using the CDS Test Double Framework. Specify the CDS views `/DMO/C_Booking_VE` and `/DMO/C_BOOKINGSUPPLEMENT_U` and enable the parameter `i_select_base_dependencies` for both of the views. Since the database tables to be doubled are not reachable directly via the specified CDS views, this parameter is required to enable hierarchy testing. It ensures that test doubles are created for all leaf nodes of the underlying structure.

Next, populate the variable `date` with a date that is in the future using the method `get_system_date`. This method is also used in the tested virtual element implementation to calculate the difference between the flight date and the respective current date.

Lastly, populate the tables `booking_mock_data` and `booksupplement_mock_data` with instance data to be inserted into the test doubles. Use the content of the variable `date` for the field `flight_date`.

METHOD `class_setup`.

```

go_client_proxy = create_local_client_proxy( VALUE #(
                                                    service_id      = '/DMO/UI_BOOKING_02'
                                                    service_version = '0001' ) ).

CDS_TEST_ENVIRONMENT = cl_cds_test_environment=>create_for_multiple_cds( i_for_entities = VALUE
( i_for_entity = '/DMO/C_Booking_VE'          i_select_base_dependencies = abap_true )
( i_for_entity = '/DMO/C_BOOKINGSUPPLEMENT_U' i_select_base_dependencies = abap_true )
) ) .

date = cl_abap_context_info=>get_system_date( ) + 10.

booking_mock_data      = VALUE #( ( travel_id = '1' booking_id = '1' flight_date = date )
                                   ( travel_id = '1' booking_id = '2' flight_date = date )
                                   ( travel_id = '2' booking_id = '3' flight_date = date )
                                   ( travel_id = '1' booking_id = '4' flight_date = date )
                                   ( travel_id = '3' booking_id = '5' flight_date = date )
                                   ) .

booksupplement_mock_data = VALUE #( ( travel_id = '1' booking_id = '1' booking_supplement_id =
                                   ( travel_id = '1' booking_id = '1' booking_supplement_id =
                                   ( travel_id = '1' booking_id = '1' booking_supplement_id =
                                   ( travel_id = '1' booking_id = '1' booking_supplement_id =
                                   ( travel_id = '1' booking_id = '1' booking_supplement_id =
                                   ( travel_id = '1' booking_id = '1' booking_supplement_id =
                                   )
ENDMETHOD.

```

Implementing the Method `setup`

To make sure that your test methods won't be influenced by data of other test methods, clear the contents of the CDS test doubles before every test method execution.

```

METHOD setup.
    cds_test_environment->clear_doubles( ).
ENDMETHOD.

```

Implementing the Method `teardown`

To make sure that the transactional buffer is reset for the involved CDS entities after every test method execution, use the EML statement `ROLLBACK ENTITIES` in the `teardown` method.

```

METHOD teardown.
    ROLLBACK ENTITIES.
ENDMETHOD.

```

Implementing the Method `class_teardown`

At the end of the test, clear the test environments and the created test doubles.

```

METHOD class_teardown.
    cds_test_environment->destroy( ).
ENDMETHOD.

```

Implementing the Method `create_local_client_proxy`

This method creates a V2 local client proxy instance used to execute OData V2 calls on the OData service. Depending on the respective system, the method returns the cloud version or an on premise version of the local client proxy instance.

Since both classes might not be available in a system, use dynamic calls to avoid syntax errors.

If none of the two called methods returns a local client proxy instance, stop the test execution with an assertion error.

METHOD create_local_client_proxy.

```
TRY.
  " the Cloud version
  DATA(class1) = 'CL_WEB_ODATA_CLIENT_FACTORY'.
  CALL METHOD (class1)=>create_v2_local_proxy
    EXPORTING
      is_service_key = service_key
    RECEIVING
      ro_client_proxy = client_proxy.
  CATCH cx_root.
ENDTRY.

IF client_proxy IS NOT BOUND.
  TRY.
    " the onPrem version
    DATA(class2) = '/IWBE/CL_CP_CLIENT_PROXY_FACT'.
    CALL METHOD (class2)=>create_v2_local_proxy
      EXPORTING
        is_service_key = service_key
      RECEIVING
        ro_client_proxy = client_proxy.
    CATCH cx_root.
  ENDTRY.
ENDIF.
```

```
cl_abap_unit_assert=>assert_bound( msg = 'cannot get client proxy factory or service binding not found' ).
```

ENDMETHOD.

Resulting Test Class

After following the steps described above, your test class should look like this and is ready for the implementation of the test methods:

i Expand the following code sample to view the source code of the test class /DMO/TC_BOOKING_ODATA.

Sample Code

```
"!@testing SRVB:/DMO/UI_BOOKING_02
CLASS /DMO/TC_BOOKING_ODATA DEFINITION
  FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
  PROTECTED SECTION.
  PRIVATE SECTION.

  CLASS-DATA: go_client_proxy          TYPE REF TO /iwbe/if_cp_client_proxy,
              lo_filter_factory        TYPE REF TO /iwbe/if_cp_filter_factory,
              lo_filter_node           TYPE REF TO /iwbe/if_cp_filter_node,
              cds_test_environment     TYPE REF TO if_cds_test_environment,
              lt_range_currencycode    TYPE RANGE OF /dmo/currency_code,
              date                     TYPE /dmo/begin_date,
              booking_mock_data        TYPE TABLE OF /dmo/booking,
```

booksupplement_mock_data TYPE STANDARD TABLE OF /dmo/book_suppl.

TYPES:

```
BEGIN OF ts_key,
  travelid TYPE /dmo/travel_id,
  bookingid TYPE /dmo/booking_id,
END OF ts_key.
```

CLASS-METHODS :

```
class_setup RAISING cx_static_check,
class_teardown,
```

```
create_local_client_proxy
```

```
IMPORTING
```

```
  service_key TYPE /iwbeb/if_cp_client_proxy=>ty_s_service_key_v2
```

```
RETURNING
```

```
  VALUE(client_proxy) TYPE REF TO /iwbeb/if_cp_client_proxy .
```

METHODS:

```
  setup,
  teardown,
  virtual_element FOR TESTING RAISING cx_static_check,
  rba_with_filter FOR TESTING RAISING cx_static_check.
```

ENDCLASS.

CLASS /DMO/TC_BOOKING_ODATA IMPLEMENTATION.

METHOD class_setup.

```
go_client_proxy = create_local_client_proxy( VALUE #(
                                                    service_id      = '/DMO/UI_BOOKING_02'
                                                    service_version = '0001' ) ).
```

```
CDS_TEST_ENVIRONMENT = cl_cds_test_environment=>create_for_multiple_cds( i_for_entities = VA
  ( i_for_entity = '/DMO/C_Booking_VE' i_select_base_dependencies = abap_true )
  ( i_for_entity = '/DMO/C_BOOKINGSUPPLEMENT_U' i_select_base_dependencies = abap_true )
) ) .
```

```
date = cl_abap_context_info=>get_system_date( ) + 10.
```

```
booking_mock_data = VALUE #( ( travel_id = '1' booking_id = '1' flight_date = date )
                              ( travel_id = '1' booking_id = '2' flight_date = date )
                              ( travel_id = '2' booking_id = '3' flight_date = date )
                              ( travel_id = '1' booking_id = '4' flight_date = date )
                              ( travel_id = '3' booking_id = '5' flight_date = date )
                              ) .
```

```
booksupplement_mock_data = VALUE #( ( travel_id = '1' booking_id = '1' booking_supplement_id
                                      ( travel_id = '1' booking_id = '1' booking_supplement_id
                                      ( travel_id = '1' booking_id = '1' booking_supplement_id
                                      ( travel_id = '1' booking_id = '1' booking_supplement_id
                                      ( travel_id = '1' booking_id = '1' booking_supplement_id
                                      ( travel_id = '1' booking_id = '1' booking_supplement_id
                                      )
                                      )
                                      )
                                      )
                                      )
                                      ) .
```

ENDMETHOD.

METHOD setup.

```
  cds_test_environment->clear_doubles( ).
```

ENDMETHOD.

METHOD teardown.

```
  ROLLBACK ENTITIES.
```

ENDMETHOD.

METHOD class_teardown.

```
  cds_test_environment->destroy( ).
```

ENDMETHOD.

METHOD create_local_client_proxy.

TRY.

" the Cloud version

DATA(class1) = 'CL_WEB_ODATA_CLIENT_FACTORY'.

CALL METHOD (class1)=>create_v2_local_proxy

EXPORTING

is_service_key = service_key

RECEIVING

ro_client_proxy = client_proxy.

CATCH cx_root.

ENDTRY.

IF client_proxy IS NOT BOUND.

TRY.

" the onPrem version

DATA(class2) = '/IWBEP/CL_CP_CLIENT_PROXY_FACT'.

CALL METHOD (class2)=>create_v2_local_proxy

EXPORTING

is_service_key = service_key

RECEIVING

ro_client_proxy = client_proxy.

CATCH cx_root.

ENDTRY.

ENDIF.

cl_abap_unit_assert=>assert_bound(msg = 'cannot get client proxy factory or service binding

ENDMETHOD.

METHOD virtual_element.

ENDMETHOD.

METHOD rba_with_filter.

ENDMETHOD.

ENDCLASS.

OData Tests: Implementing the Test Methods

The test methods you develop in this chapter execute OData read requests on the OData service.

Test Methods

The test method `virtual_element` reads instance data of the entity `Booking` and checks whether the implementation of the virtual element `DaysToFlight` works as expected. Virtual elements cannot be tested via EML and are therefore a significant test case for OData integration tests.

The test method `rba_with_filter` reads the data of all `BookingSupplement` instances of a given `Booking` instance and uses filter conditions to check for specific datasets. The filter conditions used in this test cannot be formulated with EML, as EML read calls can only identify entity instances by their keys.

Testing a Virtual Element

This test method performs an OData read request and checks whether the value for the virtual element `DaysToFlight` is calculated as expected.

In the first step, the booking instance data is inserted into the test double.

Next, a read request for the entity `Booking` is created using the Local Client Proxy interface.

Now the OData read request is executed. A reference to its response is assigned to the reference variable `lo_response`.

Before the business data are received from the response, the method checks whether there actually is a response to the OData request.

Now, the booking instance data are read from the response and saved to an internal table typed with the projection view `/DMO/C_Booking_VE`.

Finally, the method checks the number of returned instances and the virtual element value of one of the booking instances.

i Expand the following code sample to view the source code of the method `virtual_element`.

≡ Sample Code

```
METHOD virtual_element.

    cds_test_environment->insert_test_data( booking_mock_data ).

    DATA(lo_request) = go_client_proxy->create_resource_for_entity_set( 'Booking' )->create_requ

    DATA(lo_response) = lo_request->execute( ).

    cl_abap_unit_assert=>assert_not_initial( lo_response ).

    DATA ls_response_data TYPE STANDARD TABLE OF /DMO/C_Booking_VE.
    lo_response->get_business_data( IMPORTING et_business_data = ls_response_data ).

    cl_abap_unit_assert=>assert_equals( exp = '5' act = lines( ls_response_data ) ).
    cl_abap_unit_assert=>assert_equals( exp = '10' act = ls_response_data[ 1 ]-DaysToFlight ).

ENDMETHOD.
```

Testing an OData Read Request with Filter Options

This test method performs an OData read request from the entity `Booking` on the child entity `Booking Supplement` using the compositional relationship between these entities. The read request is formulated with filter conditions that limit the amount of resulting entries. The test checks whether the expected number of result entries and whether the correct values are returned respectively.

In the first step, the test data is inserted into the booking and booking supplement test doubles.

Next, the entity set resource on which the request will be executed is prepared. At first, a resource object for the entity `Booking` is created. Then, the booking instance whose child instances will be read is identified. Finally, the association to be used for the read-by-association request is specified.

Now the read request for the prepared entity set resource is created.

In the next step, a filter condition is applied that makes the OData request read only the booking supplement instances with the currency code `EUR`. In order to achieve this, the ranges table `lt_range_currencycode` is specified. It is used by the method

`create_by_range` to create a filter tree. The created filter tree is then applied on the OData request.

To limit the number of resulting entries further, the request is set to only include the first two result entries.

The OData read request is executed. A reference to its response is assigned to the reference variable `lo_response`. Before the business data are received from the response, the method checks whether there actually is a response to the OData request.

Next, the booking supplement instance data are read from the response and saved to an internal table typed with the projection view `/DMO/C_BookingSupplement_U`. This internal table is sorted by the `BookingSupplementID`.

Finally, the method checks the number of returned instances as well as their price values.

i Expand the following code sample to view the source code of the method `rba_with_filter`.

Sample Code

METHOD `rba_with_filter`.

```
cds_test_environment->insert_test_data( booking_mock_data ).
cds_test_environment->insert_test_data( booksupplement_mock_data ).

DATA(lo_booking) = go_client_proxy->create_resource_for_entity_set( 'Booking' )->navigate_wi
                    )->navigate_to_many( 'TO_BOOKSUPPLEMENT' ).

DATA(lo_request) = lo_booking->create_request_for_read( ).

lo_filter_factory = lo_request->create_filter_factory( ).
lt_range_currencycode = VALUE #( ( sign = 'I' option = 'EQ' low = 'EUR' ) ).

lo_filter_node = lo_filter_factory->create_by_range( iv_property_path      = 'CURRENCYCODE'
                                                    it_range                = lt_range_curre

lo_request->set_filter( lo_filter_node ).

lo_request->set_top( iv_top = 2 ).

DATA(lo_response) = lo_request->execute( ).

cl_abap_unit_assert=>assert_not_initial( lo_response ).

DATA ls_response_data TYPE STANDARD TABLE OF /DMO/C_BookingSupplement_U.
lo_response->get_business_data( IMPORTING et_business_data = ls_response_data ).

SORT ls_response_data BY BookingSupplementID.

cl_abap_unit_assert=>assert_equals( exp = '2' act = lines( ls_response_data ) ).
cl_abap_unit_assert=>assert_equals( exp = '5' act = ls_response_data[ 1 ]-Price ).
cl_abap_unit_assert=>assert_equals( exp = '8' act = ls_response_data[ 2 ]-Price ).

ENDMETHOD.
```

Consuming RAP Business Objects and Events

This section shows different ways of consuming RAP business objects and RAP business events.

- Business Object Consumption

Business objects can be consumed directly via their business object interface. This chapter outlines the ABAP based and the CDS based consumption of business object interfaces.

- OData Service Consumption

Business objects can be exposed as OData services. This chapter shows different ways to consume these OData services.

- Business Event Consumption

This chapter outlines different ways to consume business events raised by business objects.

Lifecycle Stability

When consuming a released RAP business object interface or RAP-based OData service, you can rely on its technical and lifecycle stability. The consumed API will only undergo compatible changes according to its defined release contract. This stability encompasses two key aspects:

- **Technical Stability:** The API structure and behavior remain consistent across system updates and patches.
- **Implementation Contract Compliance:** The API adheres to the defined RAP business object contract (see [RAP Business Object Contract](#)). As a consumer, you can rely on this contract while ensuring your implementation handles all results permitted within its scope.

When implementing against released RAP APIs, follow these guidelines:

- **Handle all potential outcomes:** An update operation may fail with various causes (DISABLED with HTTP 422, UNSPECIFIED with HTTP 400) and accompanying T100 messages. Your code must handle all possible failure scenarios defined in the contract.
- **Avoid dependencies on specific messages:** Don't implement logic based on concrete T100 message IDs. Message content may change while remaining within the contract boundaries. Design your error handling to be resilient to such changes.
- **Expect contract-compliant variations:** While the overall behavior remains stable, the specific details of responses may vary within the contract's defined parameters.

Business Object Consumption

RAP business objects can be consumed via RAP BO interfaces. This section gives an overview on possible consumption scenarios.

ABAP Consumption	CDS Consumption for Business Service Exposure	
Entity Manipulation Language e.g. in Behavior implementation	CDS projection view	CDS projection BDEF
Business Object (Interface)		

For more information on RAP BO interfaces, see [Business Object Interface](#).

ABAP Consumption

RAP BO interfaces can be consumed in ABAP using the Entity Manipulation Language (EML). EML can be used, for instance, in a BAdI implementation or in a behavior implementation of a business object.

In order to consume a RAP BO with EML, address it via its business object interface in the EML statement.

```
MODIFY ENTITIES OF My_Interface
                        ENTITY BO_Entity
```

For more information on EML, see [Entity Manipulation Language \(EML\)](#).

CDS Consumption

A RAP BO interface can be consumed by CDS projections that can then be exposed as business services. A CDS projection is made up of a CDS projection view with the provider contract `transactional_query` and a CDS behavior definition projection. These CDS artifacts project the data model and the behavior from the RAP BO interface.

For information on how to expose a CDS projection as a business service, see [Business Service](#).

Different types of OData exposure are described in [OData Exposure Comparison](#).

Exposed OData services can be consumed as described in [OData Service Consumption](#).

For OData UI services, the artifacts for the consuming projection layer and for the business service exposure can be generated using a wizard in ADT. It is accessible via the context menu of the RAP BO interface. If the generated service is supposed to include a DCL, the DCL needs to be created manually.

Released RAP BO interfaces may include value help definitions that are annotated with `@Consumption.valueHelpDefinition.useAsTemplate`. Value helps that are annotated like this are not included in the metadata exposure, but represent suggestions for the usage in service projections that are built on top of these interfaces. If you want to use the suggested value helps, you can copy them into your projections and remove the annotation `@Consumption.valueHelpDefinition.useAsTemplate` there. This way, you include the value helps in the metadata exposure of your business service. The wizard for custom OData UI services uses all suggested value help definitions in generated projections.

Release Contract

For business object consumption, use business objects that are released with the C0 contract. This ensures stability at dedicated extension points to allow for a certain extensibility.

See: [Release Contracts \(ABAP Keyword Documentation\)](#)

OData Service Consumption

Business objects can be exposed as OData services. These OData services can be consumed either locally or remotely.

Local Consumption

OData UI consumption

A business object exposed as an OData UI service is consumable, for instance, by an SAP Fiori Elements app. Every front-end configuration, which is manifested in the back-end development object (for example, UI annotations), is exposed within the metadata of the service. That means, a Fiori UI reads the information in the metadata and creates the matching UI for the service. These UI settings can be enhanced and overwritten in the Business Application Studio.

A UI service can be previewed with the Fiori Elements preview in the service binding tool. The preview mocks a real UI app and has the same look and feel as a Fiori Elements app. It is therefore a powerful tool to test the UI of your OData service already in the backend. However, it does not substitute the development in the SAP Business Application Studio .

For further information about the Fiori Elements preview in the service binding, see [Previewing the Resulting UI Service](#).

For more information about SAPUI5 to get more information about creating a deployable SAP Fiori app, see [Developing Apps with SAP Fiori Elements](#).

Local Client Proxy

The local variant of the OData Client proxy enables the programmatic consumption of OData services on the same application server. This enables, for instance, integration tests of RAP business objects. Since the tested OData service is processed in the same ABAP session by the local client proxy, test doubles can be used. An example for OData integration tests of an exposed RAP business object is available in chapter [OData Integration Tests](#).

Remote Consumption

OData services on a remote application server can be consumed using the remote variant of the OData client proxy. An example for a business object which remotely consumes another business object exposed as an OData WebAPI service by means of the remote OData client proxy and the Service Consumption Model is available in chapter [Developing a UI Service with Access to a Remote Service](#).

For general information on OData service consumption, see <https://help.sap.com/docs/btp/sap-business-technology-platform/odata-services>.

Release Contract

For OData service consumption, use business objects that are released with a C1 contract for UI services, or a C2 contract for web API services. This ensures you have a stable, reliable public interface or API that won't break your consuming applications during system upgrades or maintenance cycles.

See: [Release Contracts \(ABAP Keyword Documentation\)](#)

Business Event Consumption

RAP business events raised by RAP business objects can be consumed either locally or remotely.

i Note

Events can be released for a release contract. For more information, see [Released APIs](#).

i Note

The payload of a managed event is defined using a CDS view entity. By default, the RAP framework accesses this view entity with PRIVILEGED access rights. This means that no authorization checks are performed when reading the payload data.

Local Consumption

Business events can be consumed in the same system as they are raised. This local consumption is implemented with the help of an event handler class. The event handler class is set up as a class pool, similar to a behavior pool.

The event to be consumed needs to be exposed in a C1 released RAP BO interface.

i Note

- For local consumption of business events, remote communication is not necessary. Therefore neither a service binding nor a connection to an event exchange infrastructure is needed.
- Local events may fail to execute due to temporary issues like locks on draft instances. An automatic retry feature can address these problems, with the framework attempting to restart the local event execution up to three times. To enable this, you can create custom exceptions that inherit from `CX_RAP_EVENT_HDLR_ERROR_RETRY`. Raise these exceptions in the event handler method, specifying a delay for the retry attempt, to react to any issues that prevent successful execution of the local event.

Creating an Event Handler Class

1. Create a class that will serve as the event handler class. For the introductory syntax of the global class, use the keyword `FOR EVENTS OF`, followed by the name of the behavior definition that contains the event to be consumed.

```
CLASS cl_local_event_consumption DEFINITION

PUBLIC

ABSTRACT

FINAL

FOR EVENTS OF root_entity.
```

2. In the definition section of the local class contained in the **Local Types**, include `cl_abap_behavior_event_handler` as superclass to inherit from.

```
CLASS lcl_local_event_consumption DEFINITION
  INHERITING FROM cl_abap_behavior_event_handler.
```

3. Define a method for each event to be handled. In the method signature, specify the importing parameter, the entity, as well as the event to be consumed.

```
PRIVATE SECTION.

  METHODS consume_event_1 FOR ENTITY EVENT importing_parameter_1 FOR my_entity~event_1.
  METHODS consume_event_2 FOR ENTITY EVENT importing_parameter_2 FOR my_entity~event_2.
...ENDCLASS.

CLASS lcl_local_event_consumption IMPLEMENTATION.
METHOD consume_event_1.
  CHECK importing_parameter_1 IS NOT INITIAL.
... ENDMETHOD.

METHOD consume_event_2.
  CHECK importing_parameter_2 IS NOT INITIAL.
... ENDMETHOD.
ENDCLASS.
```

i Note

Within an event handler class, an event can only be handled by one method. However, method handling across multiple handler classes is possible.

Complete Example of an Event Handler Class

In this example, an e-mail is sent every time a BO is created.

i *Expand the following code sample to view the source code.*

Global class:

Sample Code

```
CLASS cl_local_event_consumption DEFINITION
  PUBLIC
  ABSTRACT
  FINAL
  FOR EVENTS OF root_entity.

  PUBLIC SECTION.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS cl_local_event_consumption IMPLEMENTATION.
ENDCLASS.
```

i *Expand the following code sample to view the source code.*

Local Types:

Sample Code

```
CLASS lcl_abap_behv_event_handler DEFINITION INHERITING FROM cl_abap_behavior_event_handler.

  PRIVATE SECTION.

    METHODS consume_created FOR ENTITY EVENT created_instances FOR root_entity~created.

ENDCLASS.

CLASS lcl_abap_behv_event_handler IMPLEMENTATION.

  METHOD consume_created.

    LOOP AT created_instances INTO DATA(created_instance).

      " Example for processing of table content
      some_mail_api=>send_mail(
        recipient      = created_instance-recipient
        message_body = |The instance { created_instance-id } was created|
      ).

    
```

ENDLOOP .

ENDMETHOD .

ENDCLASS .

Remote Consumption

Business events raised in a remote system can be consumed from the [SAP Event Mesh](#) by subscribing to the respective messaging queues. Prerequisite to using the [SAP Event Mesh](#) is a configured communication arrangement for inbound and outbound communication for your respective product.

For more information about the [SAP Event Mesh](#), see [What is SAP Event Mesh?](#).

An event consumption model can consume the remote events published by a RAP BO. For more information, see [Creating an Event Consumption Model](#).

The event to be consumed needs to be exposed in a C2 released event version.

Using Reuse Services in RAP

Technical reuse services allow you to significantly increase developer efficiency by using released APIs to integrating standard processes or standard functionality in your application.

About Reuse Services

Reuse services offer standard functionality and allow you to significantly increase the developer efficiency. These services aren't application or business object-specific, but offer general capabilities that are required by multiple applications and business areas.

In some cases it is even required to have a single service being used across all applications and business objects being part of a business process. For RAP, a variety of reuse services is available and ready-to-run, ranging from applications jobs of logging, forms and emails up to change documents or the workflow. For details about the offered reuse services, see [Released Components and Objects](#).

Using Reuse Services in RAP

How reuse services can be integrated in RAP, depends on the respective reuse service:

- **Using a RAP BO Interface:** A BO interface can be called to trigger the respective reuse, like for example the workflow service.
- **Using class-based APIs:** By calling the class-based APIs, you can use the respective reuse service like for example number ranges or factory calendar.

[Integrating SAP Workflow Management](#)

The workflow capability offers modern process automation capabilities.

[Integrating Change Documents](#)

Integrating SAP Workflow Management

The workflow capability offers modern process automation capabilities.

About SAP Workflow Management

i Note

Workflow services always require an integration scenario. For details, see [Implementing a Workflow](#).

With SAP Workflow Management, you design workflows based on Business Process Model and Notation (BPMN). The workflow definition defines the individual consecutive process steps that are processed when a workflow instance is called by a RAP instance.

For more information about the workflow service, see [Workflow Introduction](#) and [Workflow Definition versus Workflow Instance](#).

Workflows and Numbering

When a workflow instance at runtime must be triggered depends on the numbering defined for a RAP BO. The actions required to trigger a workflow instance are defined as save actions and can thus only be called during the early or late save phase of a RAP BO:

- **Early Numbering:** It's recommended to call trigger the workflow instance in a determination on save with the required methods.
- **Late Numbering:** It's recommended to trigger the workflow in a determination on save. If required, you can set the payload with the `setPayload` method even later in the `adjust_numbers` method.

To see at which point in time the respective implementation is called at runtime, see [Save Sequence Runtime](#) and for details about the specifics for implementing workflows in a late numbering scenario, see [API for the Workflow Capability](#).

For more information about numbering, see [Numbering](#).

Mass-Enablement

To ensure that triggering the workflow is mass-enabled, the key `CpWfHandle` of interface `I_CPWF_INST` needs to be specified externally. A UUID with 32 characters is expected as input parameter. If the action `registerWorkflow` is called repeatedly with the same `CpWfHandle` and an identical payload, this results in a short dump.

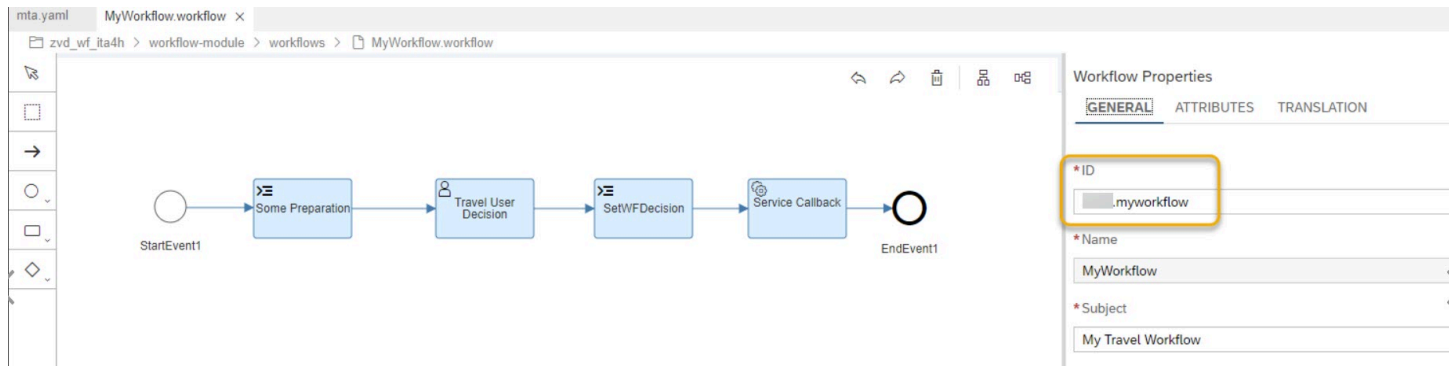
It's recommended to use the class `cl_system_uuid=>create_uuid_c32_static( )` to create the required UUIDs.

Callback Class

The service callback class implements the process that is executed after the workflow has been finished. This class must implement the interface `if_swf_cpwf_callback`. This class is required when a workflow is registered.

Workflow Example

In this simplified example, the workflow consists four consecutive process steps and implements a simple approval workflow for travels based on a predefined condition:



The Workflow ID is required to trigger a workflow instance from your implementation in the RAP behavior pool.

The following code snippet shows how the user input from the second step contained in the variable `lastUserTask1.decision` is set in the third step:

i Expand the following code sample to view the source code

Sample Code

```
var lastUserTask1 = $.usertask1.last;
$.context.Wfdecision = "undefined";

if (lastUserTask1.decision === "myaccept") {
    $.context.Wfdecision = "WF-accepted";
}

if (lastUserTask1.decision === "myreject") {
    $.context.Wfdecision = "WF-rejected";
}
```

Process Overview

BO Interface I_CPWF_INST

In the RAP behavior pool, workflows can be integrated via the released BO interface `I_CPWF_INST`. The initial call consists of the following steps:

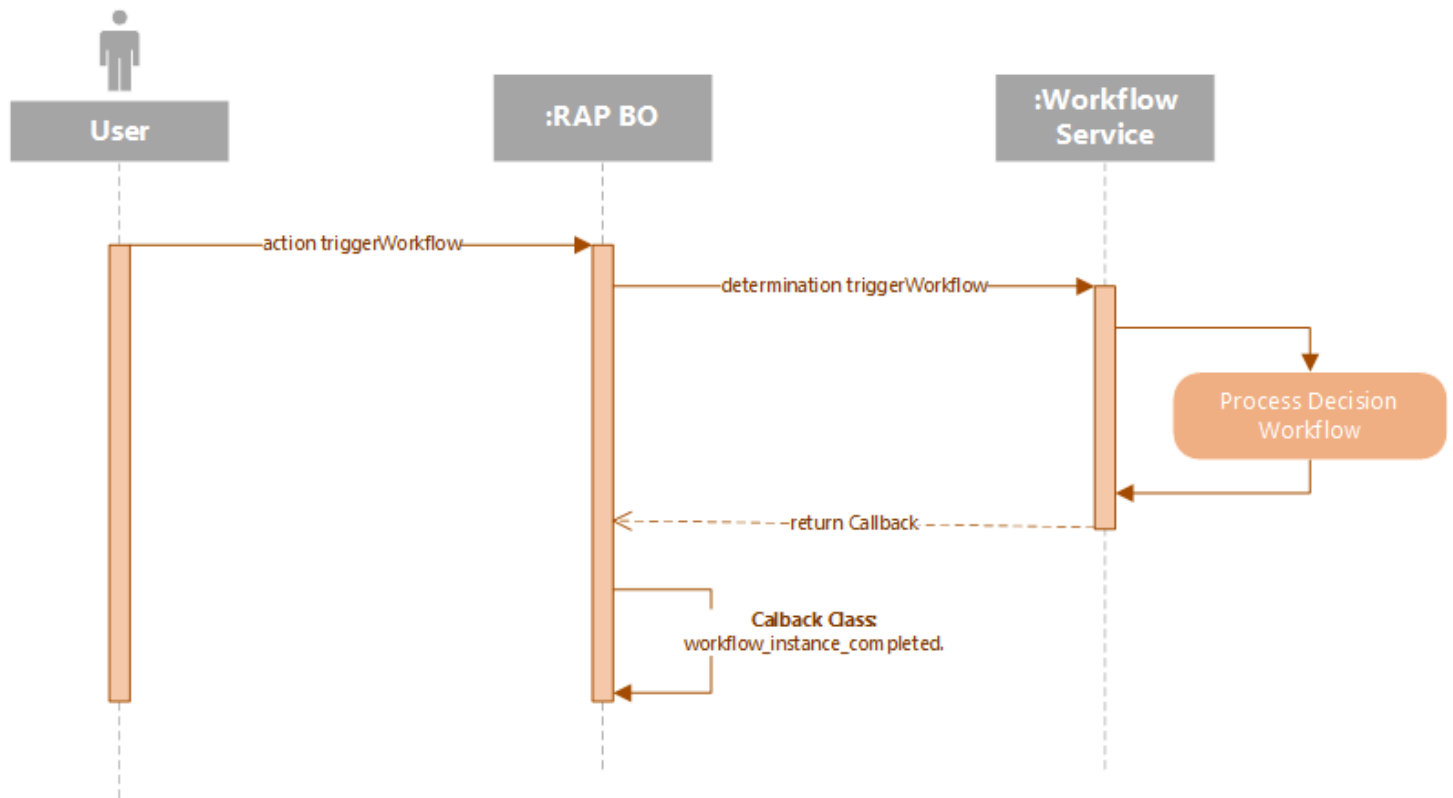
- **registerWorkflow:** Checks the required configuration set up and if successful registers a new workflow instance.
- **setPayload:** provides the input data for the workflow instance in json format.

Both process steps must be called either in the early save phase or late save phase of your RAP BO runtime depending on the numbering pattern defined for the respective RAP BO.

For more information about the BO Interface `I_CPWF_INST`, see [API for the Workflow Capability](#) or see the respective KTD documentation in ADT.

Implemented Example Process

This example uses a simplified workflow to show how to trigger a workflow and how to receive the service callback. The BO is implemented with `early numbering`, so both determinations are implemented as `determination on save` as part of the `FINALIZE` in the save sequence. The workflow is triggered by the action `TriggerDecisionWorkflow` which sets the fields required for the workflow. This triggers the `determination on save` and starts the actual workflow.



1. [Creating the Data Model for Workflow Example](#)

This section outlines the data model used for the workflow example.

2. [Implementing a Workflow](#)

When a workflow is triggered in a RAP BO, the predefined decision workflow is triggered via the Workflow Service. The example is triggered with an action and executed with a determination on save.

3. [Implementing the Service Callback Class](#)

The service callback class implements the process that is executed after the decision workflow has been finished.

Creating the Data Model for Workflow Example

This section outlines the data model used for the workflow example.

Root View: /DM0/R_Travel_W:

The root view uses numbering via a UUID in combination with a semantic key. The field **WFRecipient** contains the email address of the employee receiving the email once the decision workflow has been triggered. For more information about the contained standard fields, see [ABAP Flight Reference Scenario](#).

Sample Code

```

@EndUserText.label: 'Travel View Entity for Workflow'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define root view entity /DM0/R_Travel_W
as select from /DM0/A_TRAVEL_W
...

{
...
    //Field triggers the Workflow
  
```

```
@Semantics.eMail.type: 'HOME'
workflow_recipient    as WfRecipient,
```

```
...
```

```
}
```

Structures **ty_wf_travel** and **ty_wf_json**

Both structures are used in the implementation in the `determination` on `save` that triggers the workflow. For more information, see [Implementing a Workflow](#).

Structure **ty_wf_travel**

This structure contains fields of the travel instance and fields relevant for the workflow response. The field `wfrecipient` contains the email address and the field `wfdecision` records the workflow decision.

```
...
BEGIN OF ty_wf_travel,
  TravelUUID    TYPE sysuuid_x16,
  TravelID      TYPE /dmo/tra0vl_d_w,
  AgencyID      TYPE /dmo/age0ncy_d_w,
  CustomerID    TYPE /dmo/cstmr_d_w,
  BeginDate     TYPE /dmo/bgn_dt_w,
  TotalPrice    TYPE /dmo/ttl_prc_w,
  CurrencyCode  TYPE /dmo/crrncy_cd_w,
  EndDate       TYPE /dmo/end_dt_w,
  Description    TYPE /dmo/dscrptn_w,
  OverallStatus TYPE /dmo/ovrll_stts_w,
  wfrecipient   TYPE c LENGTH 128,
  wfdecision    TYPE c LENGTH 128,
END OF ty_wf_travel,
...
```

Structure **ty_wf_json**

This structure consists of the unique UUID required for each travel instance and the travel %data in json format. This structure is used as payload for triggering the workflow.

```
...
BEGIN OF ty_wf_json,
  wfhandle TYPE sysuuid_x16,
  json     TYPE string,
END OF ty_wf_json.
...
```

Parent topic: [Integrating SAP Workflow Management](#)

Next: [Implementing a Workflow](#)

Implementing a Workflow

When a workflow is triggered in a RAP BO, the predefined decision workflow is triggered via the **Workflow Service**. The example is triggered with an action and executed with a **determination on save**.

Prerequisites

- You have modeled a workflow as described in [Modelling a Workflow](#)
- You have created an integration scenario for the workflow service as described in [Workflow Integration](#)

Defining the Behavior Definition for /DMO/R_Travel_W

1. Create a behavior definition based on /DMO/R_Travel_W.

The behavior definition requires the following behavior elements to trigger a workflow:

- **field (readonly) WFRipient**: : Changes to this field trigger the **determination on save**. This field is defined as **readonly**, because the value is filled in via the action **triggerDecisionWorkflow**.
- **action triggerDecisionWorkflow**: Sets the field **WFRipient** and triggers the determination on save.
- **determination triggerWorkflow**: Executes the workflow registration and defines the payload for the workflow service.

The numbering scenario is the decisive factor to decide how the workflow needs to be integrated in your RAP BO. For more information, see [Workflows and Numbering](#). In this case, implementing it in a **determination on save** is the only option.

i *Expand the following code sample to view the source code*

Sample Code

```
managed implementation in class /dmo/bp_trvl_w unique;
strict(2);
with draft;

define behavior for /DMO/R_TRVL_W alias Travel
implementation in class /dmo/bp_trvl_w unique
persistent table /dmo/a_travel_w
draft table /dmo/d_travel_w
...
{
    ...
    field ( readonly ) WFRipient;

    ...
    action ( features : instance ) triggerDecisionWorkflow parameter /dmo/a_trvl_triggerwf_w

    determination triggerWorkflow on save { field WFRipient; }
    ...

    mapping for /dmo/a_travel_w corresponding
    {...
    }
}
```

Implementing Action triggerDecisionWorkflow

1. Implement the triggerDecisionWorkflow action

This action updates the workflow relevant fields in the EML MODIFY statement. The update to WFRecipient triggers the workflow registration in the determination on save. The result returns all travel instances with all fields.

Sample Code

```
METHOD TriggerDecisionWorkflow.

MODIFY ENTITIES OF /dmo/r_trvl_d_w IN LOCAL MODE
ENTITY Travel
  UPDATE FIELDS ( WFRecipient WFDecision WFTimestamp WFHandle )
  WITH VALUE #( FOR key IN keys ( %tky      = key-%tky
                                   WFRecipient = key-%param-WFRecipient
                                   WFDecision   = ''
                                   WFTimestamp  = 0
                                   WFHandle     = '' ) ).

READ ENTITIES OF /dmo/r_trvl_d_w IN LOCAL MODE
ENTITY Travel
  ALL FIELDS WITH
  CORRESPONDING #( keys )
  RESULT DATA(travels).

result = VALUE #( FOR travel IN travels ( %tky   = travel-%tky
                                           %param = travel ) ).

ENDMETHOD.
```

Implementing Determination triggerWorkflow

When the determination is triggered, it registers the workflow and defines its payload. The following steps the determination is implemented.

Here you can find the complete code snippet:

 *Expand the following code sample to view the source code*

Sample Code

```
METHOD triggerWorkflow.

READ ENTITIES OF /dmo/r_trvl_d_w IN LOCAL MODE
ENTITY Travel
  ALL FIELDS WITH VALUE #( FOR key IN keys ( %tky = key-%tky ) )
  RESULT DATA(travels).

" Skip all Travels that have no recipient or for which a workflow has already been triggered
DELETE travels WHERE WFRecipient IS INITIAL OR WFHandle IS NOT INITIAL.
CHECK travels[] IS NOT INITIAL.

" We need to specify our own WFHandle, as otherwise the mass-registration via the RAP facade w
LOOP AT travels ASSIGNING FIELD-SYMBOL(<travel>).
TRY.
  <travel>-WFHandle = cl_system_uuid=>create_uuid_c32_static( ).
```

```

    CATCH cx_uuid_error.
      ASSERT 1 = 2.
    ENDTRY.
  ENDLLOOP.

MODIFY ENTITIES OF i_cpwf_inst
  ENTITY CPWFInstance
  EXECUTE registerWorkflow
  FROM VALUE #( FOR travel IN travels
    ( %key-CpWfHandle      = travel-WFHandle
      %param-RetentionTime = '2'
      %param-CpWfDefId     = 'z123.myworkflow'
      %param-CallbackClass = '/DMO/CL_WF_CALLBACK'
      %param-Consumer      = 'DEFAULT' ) )
  MAPPED DATA(mapped)
  FAILED DATA(failed).

" Check that n Travels have resulted in n workflow instances (both are not yet linked)
ASSERT lines( travels ) = lines( mapped-cpwfinstance ).
ASSERT failed-cpwfinstance IS INITIAL.

" Persist the Workflow Handle in the Travel - to indicate that a workitem has been triggered
MODIFY ENTITIES OF /dmo/r_trvl_d_w IN LOCAL MODE
  ENTITY Travel
  UPDATE FIELDS ( WFHandle )
  WITH VALUE #( FOR travel IN travels ( %tky      = travel-%tky
                                         WFHandle = travel-WFHandle ) ).

" Set the workflow context for the new workflow instances
DATA: wf_travel TYPE /dmo/bp_trvl_d_w=>ty_wf_travel,
      wf_json   TYPE STANDARD TABLE OF /dmo/bp_trvl_d_w=>ty_wf_json.

TRY.
  DATA(cpwf_api) = cl_swf_cpwf_api_factory_a4c=>get_api_instance( ).
  DATA(cpwf_json) = cpwf_api->get_json_converter( ).
  CATCH cx_swf_cpwf_api INTO DATA(lx_cpwf).
  ASSERT 1 = 2.
ENDTRY.

LOOP AT travels ASSIGNING <travel>.

  wf_travel = CORRESPONDING #( <travel>-%data ).

  cpwf_json->serialize(
    EXPORTING
      data    = wf_travel
    RECEIVING
      result = DATA(json) ).

  APPEND VALUE #( wfhandle = <travel>-WFHandle
                  json      = json ) TO wf_json.
ENDLOOP.

```

```

MODIFY ENTITIES OF i_cpwf_inst
  ENTITY CPWFInstance
  EXECUTE setPayload
  FROM VALUE #( FOR wf_item IN wf_json
    ( %key-CpWfHandle = wf_item-wfhandle
      %param-context = wf_item-json
    ) )
  MAPPED DATA(mapped2)
  FAILED DATA(failed2).

ASSERT lines( travels ) = lines( mapped2-cpwfinstance ).
ASSERT failed2-cpwfinstance IS INITIAL.

ENDMETHOD.

```

1. Read Required Travels

The READ statement reads all fields for a travel instance. Then all instances without recipient in the `WFRecipient` field are deleted from `travels`, as well as all instances for which the workflow had already been triggered previously.

```

READ ENTITIES OF /dmo/r_trvl_d_w IN LOCAL MODE
  ENTITY Travel
  ALL FIELDS WITH VALUE #( FOR key IN keys ( %tky = key-%tky ) )
  RESULT DATA(travels).

" Skip all travels that have no recipient or for which a workflow has already been triggered
DELETE travels WHERE WFRecipient IS INITIAL OR WFHandle IS NOT INITIAL.
CHECK travels[] IS NOT INITIAL.
...

```

2. Define `WFHandle` for Primary Key in BO Interface

The class `cl_system_uuid` with method `create_uuid_c32_static( )` is called to define a unique UUID for `WFHandle` for each travel instance as primary key for interface `i_cpwf_inst`.

```

...
" Specify WFHandle, as otherwise the mass-registration via the RAP BO interface would not work
LOOP AT travels ASSIGNING FIELD-SYMBOL(<travel>).
  TRY.
    <travel>-WFHandle = cl_system_uuid=>create_uuid_c32_static( ).
  CATCH cx_uuid_error.
    ASSERT 1 = 2.
  ENDTRY.
ENDLOOP.
...

```

3. Register Workflow

The action `registerWorkflow` of BO interface `CPWFInstance` is executed to register the workflow with the following input parameters:

- **%key-CpWfHandle:** UUID to uniquely identify each travel instance. UUID was generated in the previous step.
- **%param-CpWfDefId:** Workflow ID of the designed workflow.

- **%param-CallbackClass**: Specifies the callback class which implements interface `if_swf_cpwf_callback`.

Other input parameters are available depending on your use case. For more information, see [API for the Workflow Capability](#) or refer to the respective KTD documentation.

The asserts check that all travel instances have been transferred to the workflow instances during the operation.

```
...
MODIFY ENTITIES OF i_cpwf_inst
  ENTITY CPWFInstance
  EXECUTE registerWorkflow
  FROM VALUE #( FOR travel IN travels
    ( %key-CpWfHandle      = travel-WFHandle
      %param-RetentionTime = '2'
      %param-CpWfDefId     = 'myworkflow'
      %param-CallbackClass = '/DMO/CL_WF_CALLBACK'
      %param-Consumer      = 'DEFAULT' ) )
  MAPPED DATA(mapped)
  FAILED DATA(failed).

" Check that n Travels have resulted in n workflow instances (both are not yet linked)
ASSERT lines( travels ) = lines( mapped-cpwfinstance ).
ASSERT failed-cpwfinstance IS INITIAL.
...
```

4. Call Helper class for JSON Conversion

Create two variables based on the structures `ty_wf_travel` and `ty_wf_json`. For details about the structures, see [Structures ty_wf_travel and ty_wf_json](#). The class `cl_swf_cpwf_api_factory_a4c` is used as a helper class to get access to the json converter required in the next step.

Code Syntax

```
...
" Set the workflow context for the new workflow instances
DATA: wf_travel TYPE /dmo/bp_trvl_d_w=>ty_wf_travel,
      wf_json   TYPE STANDARD TABLE OF /dmo/bp_trvl_d_w=>ty_wf_json.

TRY.
  DATA(cpwf_api) = cl_swf_cpwf_api_factory_a4c=>get_api_instance( ).
  DATA(cpwf_json) = cpwf_api->get_json_converter( ).
  CATCH cx_swf_cpwf_api INTO DATA(lx_cpwf).
  ASSERT 1 = 2.
ENDTRY.
...
```

5. Update Status for Started Workflow

The field `WFHandle` is updated to indicate that the workflow has been triggered for a specific travel instance.

Code Syntax

```
...
" Persist the Workflow Handle in the Travel - to indicate that a workitem has been triggered
MODIFY ENTITIES OF /dmo/r_trvl_d_w IN LOCAL MODE
  ENTITY Travel
    UPDATE FIELDS ( WFHandle )
    WITH VALUE #( FOR travel IN travels ( %tky      = travel-%tky
```

```
WFHandle = travel-WFHandle ) ).
```

```
...
```

6. Set Payload

wf_travel contains a subset of the travel structure. The structure is converted to json format. Together with the UUID WFHandle it's appended to the wf_json table of type TYPE STANDARD TABLE OF /dmo/bp_trvl_d_w=>ty_wf_json.

Then the setPayload action transmits the payload in JSON format with the corresponding UUID.

```
...
LOOP AT travels ASSIGNING <travel>.

  wf_travel = CORRESPONDING #( <travel>-%data ).

  cpwf_json->serialize(
    EXPORTING
      data    = wf_travel
    RECEIVING
      result  = DATA(json) ).

  APPEND VALUE #( wfhandle = <travel>-WFHandle
                  json      = json ) TO wf_json.
ENDLOOP.

MODIFY ENTITIES OF i_cpwf_inst
  ENTITY CPWFInstance
  EXECUTE setPayload
  FROM VALUE #( FOR wf_item IN wf_json
                ( %key-CpWfHandle = wf_item-wfhandle
                  %param-context  = wf_item-json
                ) )
  MAPPED DATA(mapped2)
  FAILED DATA(failed2).

ASSERT lines( travels ) = lines( mapped2-cpwfinstance ).
ASSERT failed2-cpwfinstance IS INITIAL.
```

Parent topic: [Integrating SAP Workflow Management](#)

Previous: [Creating the Data Model for Workflow Example](#)

Next: [Implementing the Service Callback Class](#)

Implementing the Service Callback Class

The service callback class implements the process that is executed after the decision workflow has been finished.

In this example the callback applies the accept / reject decision of the workflow to the acceptTravel / rejectTravel actions of the related travel instance.

1. Create the class /dmo/cl_wf_callback.
2. Implement the interface if_swf_cpwf_callback:

```
CLASS /dmo/cl_wf_callback DEFINITION
  PUBLIC
  ...
  PUBLIC SECTION.
```

```

INTERFACES if_swf_cpwf_callback .

PROTECTED SECTION.
PRIVATE SECTION.
ENDCLASS.

CLASS /dmo/cl_wf_callback IMPLEMENTATION.

METHOD if_swf_cpwf_callback~workflow_instance_completed.
...
ENDMETHOD.

ENDCLASS.

```

3. Implement the workflow_instance_completed method.

- a. Define the parameters wf_travel and wf_timestampl. Wf_travel contains all fields required to receive the callback from the workflow and wf_timestampl contains the timestamp of the point in time that the callback was received.

```

METHOD if_swf_cpwf_callback~workflow_instance_completed.

DATA: wf_travel TYPE /dmo/bp_trvl_d_w=>ty_wf_travel,
      wf_timestampl TYPE timestampl.
...
ENDMETHOD.

```

4. To receive the callback data, a helper classes is required:

- o **cl_swf_cpwf_api_factory_a4c**: With this class, you can retrieve an instance of the Workflow API which allows access to workflows For further details, refer to the system documentation.

With the method get_context_data_from_json, you receive the data from the workflow response:

```

...
TRY.
DATA(cpwf_api) = cl_swf_cpwf_api_factory_a4c=>get_api_instance( ).
DATA(json) = cpwf_api->get_workflow_context( iv_cpwf_handle = iv_cpwf_handle ).

cpwf_api->get_context_data_from_json(
    EXPORTING
        iv_context = json
    IMPORTING
        ev_data = wf_travel
    ).

CATCH cx_swf_cpwf_api INTO DATA(lx_cpwf).
RAISE EXCEPTION TYPE /dmo/cx_wf_callback
    EXPORTING
        previous = lx_cpwf.

ENDTRY.
...

```

5. Fill the timestamp to save the point in time the response was received with the GET TIME STAMP method:

```
...
GET TIME STAMP FIELD wf_timestampl.
...
```

6. Update the travel fields relevant for the workflow for the active travel instance. This includes the WFDecision to save the user decision, WFTimestamp to save the time information.

```
...
MODIFY ENTITIES OF /dmo/r_trvl_d_w
ENTITY Travel
  UPDATE FIELDS ( WFDecision WFTimestamp WFHandle )
  WITH VALUE #( ( TravelUUID      = wf_travel-TravelUUID
                  %is_draft       = if_abap_behv=>mk-off
                  WFDecision      = wf_travel-WFDecision
                  WFTimestamp     = wf_timestampl
                  WFHandle        = '' ) )
  FAILED DATA(failed)
  REPORTED DATA(reported).
```

7. If the modify fails, handle the exception handling based on the fail-cause.

For more information about fail-causes, see [Reponse Parameter](#).

```
...
IF failed-travel IS NOT INITIAL.

CASE failed-travel[ 1 ]-%fail-cause.
  WHEN if_abap_behv=>cause-not_found.
    RAISE EXCEPTION TYPE /dmo/cx_wf_callback
    EXPORTING
      textid      = /dmo/cx_wf_callback=>travel_not_found
      travel_id   = wf_travel-travelid.

  WHEN if_abap_behv=>cause-locked.
    RAISE EXCEPTION TYPE /dmo/cx_wf_callback
    EXPORTING
      textid      = /dmo/cx_wf_callback=>travel_locked
      travel_id   = wf_travel-travelid.

  WHEN OTHERS.
    RAISE EXCEPTION TYPE /dmo/cx_wf_callback.

ENDCASE.
ENDIF.
...
```

8. Call either acceptTravel or rejectTravel actions based on the user decision saved in parameter WFDecision. If WFDecision contains WF - accepted, acceptTravel is called to update the status. If it contains WF - rejected, the action rejectTravel is called. If the rejectTravel fails an exception is raised. Commit the final decision to the database with COMMIT ENTITIES.

```
...

" Call accept/reject Action based on WF outcome
IF wf_travel-WFDecision EQ 'WF-accepted'.
  MODIFY ENTITIES OF /dmo/r_trvl_d_w
  ENTITY Travel
    EXECUTE acceptTravel
  FROM VALUE #( ( TravelUUID      = wf_travel-TravelUUID
                  %is_draft       = if_abap_behv=>mk-off ) )
```

```

        FAILED DATA(failed_accept)
        REPORTED DATA(reported_accept).

    IF failed_accept-travel IS NOT INITIAL.
        RAISE EXCEPTION TYPE /dmo/cx_wf_callback
        EXPORTING
            textid      = /dmo/cx_wf_callback=>travel_set_status_failed
            travel_id = wf_travel-travelid.
    ENDIF.

    ELSEIF wf_travel-WFDecision EQ 'WF-rejected'.
        MODIFY ENTITIES OF /dmo/r_trvl_d_w
        ENTITY Travel
        EXECUTE rejectTravel
        FROM VALUE #( ( TravelUUID      = wf_travel-TravelUUID
                        %is_draft        = if_abap_behv=>mk-off ) )
        FAILED DATA(failed_reject)
        REPORTED DATA(reported_reject).

    IF failed_reject-travel IS NOT INITIAL.
        RAISE EXCEPTION TYPE /dmo/cx_wf_callback
        EXPORTING
            textid      = /dmo/cx_wf_callback=>travel_set_status_failed
            travel_id = wf_travel-travelid.
    ENDIF.
ENDIF.

COMMIT ENTITIES.
...

```

9. Raise an exception, if the callback fails.

```

...
    IF sy-subrc NE 0.
        RAISE EXCEPTION TYPE /dmo/cx_wf_callback.
    ENDIF.
...

```

Result

You have implement the callback class to persist the accept or reject travel information received via the workflow.

i *Expand the following code sample to view the source code*

Sample Code

```

CLASS /dmo/cl_wf_callback DEFINITION
    PUBLIC
    FINAL
    CREATE PUBLIC .

    PUBLIC SECTION.

    INTERFACES if_swf_cpwf_callback .

```

```

PROTECTED SECTION.
PRIVATE SECTION.
ENDCLASS.

```

```

CLASS /dmo/cl_wf_callback IMPLEMENTATION.

```

```

METHOD if_swf_cpwf_callback~workflow_instance_completed.

```

```

DATA: wf_travel      TYPE /dmo/bp_trvl_d_w=>ty_wf_travel,
      wf_timestampl  TYPE timestampl.

```

```

TRY.

```

```

  DATA(cpwf_api) = cl_swf_cpwf_api_factory_a4c=>get_api_instance( ).
  DATA(json) = cpwf_api->get_workflow_context( iv_cpwf_handle = iv_cpwf_handle ).

```

```

  cpwf_api->get_context_data_from_json(
    EXPORTING
      iv_context = json
    IMPORTING
      ev_data = wf_travel
  ).

```

```

  CATCH cx_swf_cpwf_api INTO DATA(lx_cpwf).
  RAISE EXCEPTION TYPE /dmo/cx_wf_callback
    EXPORTING
      previous = lx_cpwf.

```

```

ENDTRY.

```

```

GET TIME STAMP FIELD wf_timestampl.

```

```

MODIFY ENTITIES OF /dmo/r_trvl_d_w

```

```

  ENTITY Travel
    UPDATE FIELDS ( WfDecision WfTimestamp WfHandle )
    WITH VALUE #( ( TravelUUID      = wf_travel-TravelUUID
                  %is_draft       = if_abap_behv=>mk-off
                  WfDecision      = wf_travel-WfDecision
                  WfTimestamp     = wf_timestampl
                  WfHandle        = '' ) )

```

```

  FAILED DATA(failed)
  REPORTED DATA(reported).

```

```

IF failed-travel IS NOT INITIAL.

```

```

  CASE failed-travel[ 1 ]-%fail-cause.
    WHEN if_abap_behv=>cause-not_found.
      RAISE EXCEPTION TYPE /dmo/cx_wf_callback
        EXPORTING
          textid      = /dmo/cx_wf_callback=>travel_not_found

```

```

        travel_id = wf_travel-travelid.

WHEN if_abap_behv=>cause-locked.
    RAISE EXCEPTION TYPE /dmo/cx_wf_callback
    EXPORTING
        textid      = /dmo/cx_wf_callback=>travel_locked
        travel_id = wf_travel-travelid.

WHEN OTHERS.
    RAISE EXCEPTION TYPE /dmo/cx_wf_callback.

ENDCASE.
ENDIF.

" Call accept/reject Action based on WF outcome
IF wf_travel-WFDecision EQ 'WF-accepted'.
    MODIFY ENTITIES OF /dmo/r_trvl_d_w
    ENTITY Travel
    EXECUTE acceptTravel
    FROM VALUE #( ( TravelUUID      = wf_travel-TravelUUID
                    %is_draft       = if_abap_behv=>mk-off ) )
    FAILED DATA(failed_accept)
    REPORTED DATA(reported_accept).

    IF failed_accept-travel IS NOT INITIAL.
        RAISE EXCEPTION TYPE /dmo/cx_wf_callback
        EXPORTING
            textid      = /dmo/cx_wf_callback=>travel_set_status_failed
            travel_id = wf_travel-travelid.
    ENDIF.

ELSEIF wf_travel-WFDecision EQ 'WF-rejected'.
    MODIFY ENTITIES OF /dmo/r_trvl_d_w
    ENTITY Travel
    EXECUTE rejectTravel
    FROM VALUE #( ( TravelUUID      = wf_travel-TravelUUID
                    %is_draft       = if_abap_behv=>mk-off ) )
    FAILED DATA(failed_reject)
    REPORTED DATA(reported_reject).

    IF failed_reject-travel IS NOT INITIAL.
        RAISE EXCEPTION TYPE /dmo/cx_wf_callback
        EXPORTING
            textid      = /dmo/cx_wf_callback=>travel_set_status_failed
            travel_id = wf_travel-travelid.
    ENDIF.
ENDIF.

COMMIT ENTITIES.

IF sy-subrc NE 0.
    RAISE EXCEPTION TYPE /dmo/cx_wf_callback.

```

ENDIF .

ENDMETHOD .

ENDCLASS .

Parent topic: [Integrating SAP Workflow Management](#)

Previous: [Implementing a Workflow](#)

Integrating Change Documents

About the Change Document Solution

You can document changes made to business objects, such as the time, the content, and the way changes are made, by logging these changes in a change document.

For more information about the change document solution, see [Change Document Solution](#).

Change Document Integration in RAP

In managed RAP BOs, change documents are used to log changes to persisted RAP BO data. The change document solution is integrated with RAP and can be enabled using a few declarative statements in the behavior definition, without the need to write executable code.

A RAP BO entity can be enabled for change documents in the entity behavior definition. Further logging options are defined in the entity behavior body. A change document object is required for logging changes made to the database tables assigned to it. In addition, the fields whose values should be logged must be enabled for logging. This can be done in one of the following ways:

- If a field of a view entity is typed with a simple type, the annotation `@AbapCatalog.typeSpec.changeDocumentRelevant` must be set to `true` in the respective simple type.
- If a field of a view entity is typed with a DDIC data element, the flag for change documents must be set in the respective data element.

i Note

In ABAP Development Tools for Eclipse, in the editor for data elements, under **Additional Properties**, there is a flag **Change Document Logging**.

Change documents are enabled using the following syntax elements in the behavior definition:

```
...
define behavior for CDSEntity [alias AliasedEntityName]
implementation in class ABAP_CLASS_NAME [unique]
...
changedocuments master (ChangeDocumentObject) |
changedocuments dependent [by _AssocToMaster]
...
{
```

```

changeddocuments(Options);
}

```

For more information about the syntax of RAP change documents, refer to [RAP - ChangeDocuments \(ABAP Keyword Documentation\)](#) and [RAP - ChangeDocuments, Options \(ABAP Keyword Documentation\)](#).

Data changes are logged for the active persistence, not for each step of the change in the transactional buffer. For example, if an instance is created and updated in the same LUW, the change document records this as a create operation with the updated values.

Instead of using the built-in solution for change documents, change documents can also be integrated into RAP by calling functions of the Change Document API within an additional save implementation. This custom implementation can be used when the generic integration does not cover the use case and non-standard handling is required.

By default, changes to the persisted BO data are logged at the CDS entity level and not at the level of the corresponding persistent database tables. This means that the metadata of the fields of the CDS entities are evaluated, for example, details about typing, reference annotations to a currency or unit key, and the change document settings on field level. The optional addition `for persistent tables` can be used to change the default behavior. When used, the field information of the fields of the RAP persistent table is evaluated, as well as the table names.

Change Documents Master Entity

Change documents master entities have their own change document object for logging changes to persisted data.

Change Documents Dependent Entity

Change documents dependent entities use the change document object of their master entity for change logging. The change documents master entity is identified via the association to the change documents master entity. This association can either be explicitly specified in the behavior definition, or it can be implicitly derived using the ancestor association.

i Note

The fields that are used in the **ON** condition of the association to the change documents master entity should be declared as **readOnly** in the behavior definition.

Change Document Object

A change document object is a repository object that logs the changes made to specific database tables assigned to it. It can be generated by a Quick Assist in ABAP development tools for Eclipse.

Change Document Options

Traditionally, change document objects contain options for the generation of change documents. These are defined in the form-based editor of the change document object. In RAP, these options are irrelevant, the only information of the change document object that is evaluated is its name, which serves as a key field in the change documents results. Further options are specified in the entity behavior body of a change-documents-enabled entity. With these options, you can define logging details for the standard operations create, update, and delete. For more information, see [RAP - ChangeDocuments, Options \(ABAP Keyword Documentation\)](#).

Enabling Change Documents Integration

The following simplified example shows how to enable change documents for a managed RAP business object.

Simple Type with Active Change Logging

This simple type defines that changes to fields defined with reference to this simple type are logged when change documents are enabled. It has the annotation `@AbapCatalog.typeSpec.changeDocumentRelevant` set to `true`.

```
@EndUserText.label: 'Travel Status'
@AbapCatalog.typeSpec.changeDocumentRelevant: true
define type /DMO/BT_STATUS: abap.char(1);
```

View Entity: /DMO/R_Travel

The view entity casts the field `status` to the simple type `/DMO/BT_STATUS` to ensure that changes to this field are logged in the change document object.

Sample Code

```
@EndUserText.label: 'Travel View Entity for Change Documents'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define root view entity /DMO/R_Travel
as select from /DMO/A_TRAVEL
...

{
  ...
  //Field with flag for change documents
  cast(status as /DMO/BT_STATUS) as Status,
  ...
}
```

Defining the Behavior Definition for /DMO/R_Travel

Create a behavior definition based on `/DMO/R_Travel`. The behavior definition requires the following behavior elements to enable change documents integration:

- The root entity is defined as change documents master entity.
- The change document object `/DMO/TRAVEL_CHD` is used for logging changes.
- The logging options are defined in the entity behavior body.

Sample Code

```
managed implementation in class /dmo/bp_trvl unique;
strict(2);

define behavior for /DMO/R_Travel alias Travel
implementation in class /dmo/bp_trvl unique
persistent table /dmo/a_travel
changedocuments master ( /DMO/TRAVEL_CHD )
...
```

```
{
...
changedocuments ( create:key, update: data, delete:none );
...
}
```

Evaluating Change Document Data

You can display the content of a change document object on the SAP Fiori UI of your RAP application, or evaluate it in ABAP.

Displaying Change Documents on the SAP Fiori UI

To display change documents data on an SAP Fiori UI, you need to associate your change documents master entity to the change documents view entity `I_ChangeDocument_2`, expose the association in the consumption layer, and use it in a UI facet. This procedure enables the reading of change documents data from the change documents objects and displaying it on the SAP Fiori UI.

View Entity: /DMO/R_Travel

The view entity in the base layer defines and exposes an association to the change documents view entity `I_ChangeDocument_2`.

Sample Code

```
@EndUserText.label: 'Travel View Entity for Change Documents'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define root view entity /DMO/R_Travel
as select from /DMO/A_TRAVEL

    association of one to many I_ChangeDocument_2 as _ChangeDocs
        on _ChangeDocs.ChangeDocObjectClass = '/DMO/TRAVEL_CHD'
        and _ChangeDocs.ChangeDocObject      = $projection.TravelID

{
...
//exposing the association to the change documents view entity
_ChangeDocs

}
```

Projection View: /DMO/C_Travel

The projection view redirects the association to the change documents view to the projection view `C_ChangeDocument_2` in the consumption layer.

Sample Code

```
@EndUserText.label: 'Projection View for /DMO/R_Travel'
@AccessControl.privilegedAssociations: [ '_ChangeDocs' ]
```

```
define root view entity /DMO/C_Travel
provider contract transactional_query
as projection on /DMO/R_Travel

{
  ...
  //redirecting the association to the change documents view entity
  _ChangeDocs:redirected to C_ChangeDocument_2
}
```

i Note

The projection view C_ChangeDocument_2 must be included in the service definition.

Metadata Extension: /DMO/C_Travel

The following UI annotations are needed to configure the display of change documents data on the SAP Fiori UI. Under targetElement, the association to the change documents projection view is defined as a UI facet for displaying change documents data.

 Sample Code

```
@UI.facet: [
  ...
  { id:          'ChangeDocs',
    purpose:     #STANDARD,
    type:        #LINEITEM_REFERENCE,
    label:       'Change Documents',
    position:    30,
    targetElement: '_ChangeDocs'}
  ...
]
```

Result: Change Documents on the SAP Fiori UI

On the SAP Fiori UI, in the section [Change Documents](#), the change log is displayed. In this example, the [Change Indicator](#) column displays the kind of change performed, and the old value and the new value are shown.

Change Documents (1)						
Object ID	Change No.	Table Name	Change Indicator	Field	Old Value	New Value
37e1d849-3e71-1edf-b3a1-dbf5b2845f65	4448561		U	STATUS	A	B

Reading Change Document Data in ABAP

In ABAP, the data logged in the change document object can be evaluated using the ABAP class CL_CHDO_READ_TOOLS. The following code snippet shows an example:

Sample Code

```
cl_chdo_read_tools=>changedocument_read(
  EXPORTING
    i_objectclass    = '/DMO/TRAVEL_CHD'    "change document object name
    i_date_of_change = sy-datum
  IMPORTING
    et_cdredadd_tab  = FINAL(result) ).    "change document object details
  ...
```

For more information about reading from change document object definitions, see [Reading a Change Document Object Definition](#).

CDS Annotations

The following list summarizes SAP annotations of the Data Definition Language (DDL) of ABAP CDS that are relevant in the context of ABAP RESTful programming model and released for ABAP Cloud Platform.

SAP CDS annotations are evaluated by SAP frameworks and can be either **ABAP annotations** or **framework-specific annotations**.

About

A CDS annotation adds metadata to a CDS object that goes beyond the syntax features of ABAP SQL. It can be specified using the @ symbol in specific places of a CDS object. SAP delivers a predefined set of CDS annotations and only these can be used. User-defined annotations are currently not supported. There are two types of SAP-delivered CDS annotations: **ABAP annotations** and **framework-specific annotations**.

- **ABAP annotation:**

ABAP annotations define technical and semantic attributes of a CDS object. They are evaluated by the ABAP runtime framework when the object defined in the CDS source code is activated or when the object is used in the ABAP runtime environment. ABAP annotations can modify the behavior of ABAP SQL statements that access a CDS object. All available ABAP annotations are listed under [ABAP Annotations \(ABAP Keyword Documentation\)](#).

- **Framework-specific annotation:**

Framework-specific annotations define the technical and semantic properties of CDS objects that are required by specific software components, such as OData, UI, and Analytics. They are evaluated by the frameworks of the corresponding software components using suitable APIs, and are not relevant for activations and the ABAP runtime environment. All available framework-specific annotations are listed under [Framework-Specific Annotations \(ABAP Keyword Documentation\)](#).

ABAP CDS - ABAP Annotations

CDS annotations that are evaluated by **ABAP runtime**:

- AbapCatalog Annotations
- AccessControl Annotations
- ClientHandling Annotations
- EndUserText Annotations

- Environment Annotations
- MappingRole Annotations
- Metadata Annotations
- Semantics Annotations

See also: [ABAP CDS - View Annotations \(ABAP Keyword Documentation\)](#)

Annotation Documentation

The following resources are available for annotation documentation:

- Opening the tooltip information (`F2` key) for an annotation in ABAP development tools for Eclipse (ADT) shows some technical details and the annotation documentation for a CDS annotation.
- A complete list of all available SAP annotations, their technical properties and their annotation documentation is available in the ABAP Keyword Documentation under [Annotation Documentation \(ABAP Keyword Documentation\)](#).
- The recommended annotations for CDS view entities that are part of the standard template are explained in the topic [Recommended Annotations for View Entities](#).
- For scenario-specific recommendations for annotations, see the respective development guides:
 - [ABAP Analytics](#)
 - [ABAP RESTful Application Programming Model](#)

What's New

Descriptions of the changes of interest for developers made to the ABAP RESTful Application Programming Model for your usage type.

SAP BTP ABAP environment

Available as of	Feature	Description
2605	Support of CDS Table Entities as Draft Persistence in RAP	CDS table entities can now be used as a draft persistency within RAP, allowing access control (DCLs) directly on the draft table.
2602	RAP Recommendations	Previously, business users had to fill out input fields manually and choose the right value; recommendations standardize and ease this process: In RAP applications on an SAP Fiori recommendations provide input assistance by proposing field values for input fields based on input and interaction history. RAP recommendations can be implemented using deterministic rules or using generative AI. The generative AI predicts the most likely values for each input field based on the context of the data. By using the ABAP AI SDK powered by Intelligent Scenario Lifecycle Management, you can embed AI capabilities into the application, enriching traditional value help with context-aware suggestions.
2602	RAP Collaborative Draft	Collaborative draft now also works in cross-business object scenarios if both business objects are collaborative draft-enabled, and if the cross-business object scenario is fully draft scope-enabled.
2511	RAP Change Operations for Data Records	With the introduction of the new syntax addition <code>read entity MyEntity with change fields fields (...)) with ... result data(rdc)</code> , an application developer can now read and update data records in a more efficient way.

Available as of	Feature	Description
		now access the corresponding change operations of a data record processed within the t
2511	Fiori Analytical Table - Read-Only	You miss the ABAP List Viewer (ALV) in Fiori with simple analytical capabilities? The Fiori will be introduced to combine the transactional and light-weight analytical world. RAP ne further OData URL to ABAP SQL mapping with analytical capabilities. For advanced analy dimensional analytical reporting please check the analytical clients.
2511	RAP Global Side Effects	RAP side effects are an optional part of a RAP behavior definition that define interdepend BO properties that trigger a reload of the affected properties on the user interface. Globa those side effects that are defined without any source properties or source entities and c an end user with a dedicated key stroke.
2508	Privileged Access for Cross-BO Associations	In cross-BO scenarios, when association-related ABAP EML requests (such as read-by-as create-by-association) are performed on the association source and specify IN LOCAL association target's implementation of the operation is accessed in privileged mode (not bypassing the association target's authorization checks. This applies only if the associati specified with a privileged context.
2508	Derivation Functions	Derivation functions derive context-specific values that can be used to restrict data in CD output of a derivation function can be further influenced by the use of parameters and fil
2508	Background Processing Framework - Final Version	Background processing is very important for business applications. A lot of logic can be e asynchronously to improve performance. In scenarios where changes of the current LUW communicated remotely, the asynchronous execution is even mandatory to be compliant LUW (due to implicit database commits). We want to simplify and improve the background processing by covering all required qual "exactly once", "in order", testability and an integration into the ADT cross trace for better In the next step of bgPF the AIF integration and its monitoring capabilities are provided. In order" capabilities are supported.
2508	Fiori-based Multi-Dimensional Reporting - Cloud-compliant	You need to quickly build an analytical Fiori application, but simple list reporting doesn't s needs? Instead, you already built an analytical model with the new embedded analytics c developer extensibility and now want to unleash its full potential? In that case, we have th for you: Without writing a single line of additional code, you can simply expose your mode fledged analytical application with a modern Fiori-based UI and real multi-dimensional "s capabilities. ABAP Cloud-compliant creation of UIAD and App Variant (based on "MDA Generator")
2508	Outbound SQL via CDS External Entities : Static Access	External Entites allow to access external databases with the means of ABAP CDS & ABAP consumption External Entities directly in ASQL via the "dynamic access", we will now also called "static access": The "static access" will allow modelling of these external data sources via CDS External E External Schemata and consumption of those CDS External Entities via CDS View Entity: can create view entity stacks on top of external data sources allowing to for example reali analytics scenarios combining local and remote data sources within one star schema.
2508	Generation Wizard: Outbound SQL via CDS External Entities	The ADT Generation Wizard allows to develop Outbound SQL scenarios more efficiently b required CDS External Entities based on on an imported CSN defintion of the Remote SQ
2508	Fiori-based Multi-Report Applications	You need to quickly build an analytical Fiori application, that puts multiple analytical quer context in a single UI? All you need to do is to provide the queries, bundle them in an InA Service and you are go the brand new "Review Booklet Application" wizard to generate the application. Afterwar refine the configuration of your Review Booklet in an easy to use Fiori UI.

Available as of	Feature	Description
2508	CDS Hierarchy Generation Wizard	Defining hierarchies for the usage in a CDS view can become a complex task with the need for the CDS hierarchy definition itself, but also for a directory, nodes, text views and more. With the hierarchy generation wizard, their creation becomes easier, quicker and less error-prone. You can now derive potential hierarchy nodes already from the start (on the master data view) and guide the creation process.
2508	Collaborative Draft	You can now add collaborative capabilities to your business object. The collaborative draft allows multiple users to work concurrently on the same draft instance.
2508	RAP Factory Actions	RAP factory actions now return the http status 201 instead of 200 in case of a successful response.
2508	Table Entities as Active Persistence	The RAP managed BO runtime now supports CDS table entities as the active persistence scenarios without draft.
2508	Cross-BO Transactional Enablement	Cross-BO scenarios can now be transactionally enabled and make use of an improved metadata implementation with additional syntax checks and error handling.
2505	Improved CDS Buffering - Buffer Delegation	Optimize the ABAP memory consumption for scenarios which make use of CDS Entity Buffering. Provide support for "delegated buffers" to allow buffering on CDS view entity-level by reusing buffers on table-level at runtime.
2505	OData UI Service from Scratch	You can create all RAP-service related repository objects from scratch by using the OData UI Service from Scratch wizard.
2505	Null Value Handling	You can now find an overview of the differences in the behavior of null values and initial values in V2 and V4 with regard to sorting and filtering on the Fiori Elements UI as well as explicit filtering on OData Web API request.
2502	Nonstandard Operations for Associations	You can now define nonstandard operations for associations. These are link action, unlink action and inverse function.
2502	ABAP SQL Service - (read-only) Database Procedures	Have you already used the ABAP SQL Service / ODBC Driver for ABAP to expose ABAP-based data via SQL-APIs to external SQL consumers? We are now increasing the feature scope for scenarios. Besides exposing CDS view entities for read-only scenarios, it will be possible to expose "read-only" AMDP-based database procedures to be used by external ODBC clients. By doing so, we can both expose SQL and SQLScript-based business logic to external ODBC clients.
2502	CDS Updatable View Entities	With CDS Writeable View Entities, you can wrap existing DDIC Tables (or CDS Table Entities) with a native CDS signature that you can use with ABAP SQL (SELECT, INSERT, UPDATE, DELETE) in combination with DCL-based access control. CDS Writeable View Entities are projections of the underlying DDIC Table / CDS Table Entity. CDS Writeable View Entities will allow the following scenarios: • Enabling of "Cloudy-ready" READ/WRITE AMDPs, as they ensure "client safe" native SQLScript coding by preventing cross-client access • Exposure of a stable CDS signature for full SQL DML access but still allowing characteristics of the underlying persistence model • Enabling the usage of Dynamic View Caches based on released SAP content • Enabling of WRITE scenarios via the ABAP SQL Service / ABAP ODBC Driver

Available as of	Feature	Description
2502	Event-Driven Side Effects	You can now define an event to trigger a side effect in your RAP business object. Whenever an event is raised by the application server, the side effect is triggered and the defined target is executed. For more information, see Event-Driven Side Effects .
2502	Communication Management: Local APIs	The communication management will provide local APIs for the creation and update of communication arrangements, communication systems, communication users, and the upload of client communication data. A new API will be provided for the usage in Developer Extensibility.
2502	Navigation From SAP Fiori Launchpad Towards Event Broker UI	It is now possible to navigate from the SAP Fiori launchpad of SAP S/4HANA Public Cloud towards the UI of the connected SAP Cloud Application Event Hub.
2502	Generic Audit Filter	You can now define generic audit filter(s) and assign auditor users accordingly to maintain or time-based restrictions. The key benefits: • Simple: Using only few but central dimensions, audit filters can be maintained and applied easily to restrict data access. • Powerful: Also advanced scenarios with multiple dimensions and intervals (organizational data-based) are supported. • Efficient: Time restriction filter logic is directly applied at the data selection level where the filter has been assigned. • Modern: Using the state of the art design concepts and technology. • Modular: Implemented on application-level complementing Z1 requirements with external tax auditor. See Generic Audit Filter.
2502	Editable Treeview with Draft Capabilities	In RAP, a treeview visualizes data modeled by a CDS hierarchy, using an SAP Fiori Element. Previously, you could model hierarchical structures and expose them as read-only treeviews in the SAP Fiori UI. Now you can also expose hierarchical structures as editable treeviews with draft capabilities.
2408	ABAP Auxiliary Classes	You can use ABAP auxiliary classes to support the implementation of ABAP behavior pools. These classes provide reusable methods, facilitate concurrent development of behavior implementations, and offer utility methods for triggering RAP business events.
2408	Annotation: ObjectModel.supportedCapabilities	You can use this annotation to check whether or not the UI use case is supported. If supported, either the creation of UI projections or alternatively the direct consumption in an UI service is possible. Otherwise, warnings or errors occur indicating that the UI usage is not supported and may lead to unexpected errors. For more information, see the annotation documentation of the domain in KTD.
2408	Annotation: Semantics.valueRange	You can use this annotation to inform consumers about the value range of a CDS element. You can specify minimal and/or maximal values and indicate whether these are exclusive or inclusive. This annotation can be applied to all fully ordered data types, i.e. Characters, Strings, Number. For more information, see the annotation documentation of the Semantics domain in KTD.

Available as of	Feature	Description
2408	Annotation: UI.dataPoint.title	You can use this annotation to define the title of a data point. For more information, see the documentation of the UI domain in KTD.
2408	Using Dynamic Topics in Advanced Event Mesh	You can use dynamic topics for routing and filtering outbound events in the Advanced Event Mesh. Dynamic topics make use of the context attribute annotations to extend the corresponding event with further dynamic segments, derived from the corresponding annotated payload field. This enables detailed event filtering and routing, ensuring that events are delivered only to the interested components, improving efficiency and reducing unnecessary processing.
2408	Direct Push-Based Integration	With direct push-based integration, you can connect neighboring ABAP Systems with each other. In this scenario, the connected systems exchange data (like business events) by calling each other's RFC.
2408	Communication Targets for HTTP Outbound Communication	The usage of the new application destinations via outbound services of type communication is recommended when developing a new application with HTTP outbound communication. This allows developers to define and control which applications can use a particular application destination.
2408	Hello, ABAP Cloud! Page	Are you wondering where to find information about ABAP Cloud like documentation, release notes, roadmap or where to find examples with different levels of complexity? The ABAP Cloud landing page in the ABAP Development Tools for Eclipse shows all of this information and everything about ABAP Cloud in one place.
2405	Result Cardinality for Factory Actions	You can now define the result cardinality [1..*] for factory actions.
2405	Entity assignment of RAP Business Events	You can now assign RAP business events to entities that are not defined as root entity.
2405	Subentities as Authorization Master	You can now define entities as authorization master that are not defined as root entity.
2402	Generator for custom OData UI services based on RAP BO interfaces	You can now generate custom OData UI services based on RAP BO interfaces using a generator (ADT).
2311	Changed runtime behavior for determinations and validations	For performance reasons, determinations and validations that are assigned to a determination action (including prepare) are skipped during this determine action, if these determinations and validations have already been executed as part of a previous determine action, that has been executed in the same instance (draft or active) in the same transaction. Determinations and validations are skipped in the save sequence, if these determinations and validations have already been executed as part of a previous determine action that has been executed in the same instance in the same transaction. The performance optimizations outlined above are not applied in the following cases:

Available as of	Feature	Description
		• A modification performed after the first determine action triggers a determination. • A validation in the first determine action reports a failed key. • A determination/validation has been assigned using the addition always. Ensure that the determinations and validations of your business object follow the respective guidelines described in Determination and Validation Modelling. Neglecting these guidelines can lead to unexpected runtime behavior.
2311	CDS Simple Types	Simple types can be used for data typing on CDS level.
2311	Service Consumption Model	The Service Consumption Model now generates all data type information necessary for consuming remote services in one single ABAP proxy model class.
2311	RAP Default Values Function	You can now default fields and input parameters for operations in OData Services using Default Functions.
2311	RAP Treeview	You can now use Treeviews to define read-only hierarchical entity structures in OData V4.
2311	Large Objects	You can now define static feature control for large object fields in OData V2.
2311	Ancestor Associations Binding Fields	Ancestor associations binding fields now are filled automatically by the RAP framework.
2308	Local Consumption of Business Events	You can now consume business events in the same system as they are raised (local consumption).

Available as of	Feature	Description
2308	Strict Mode	It is now best practice to use strict(2) for all RAP BOs, except for: • Business Configuration (BC) BOs • BOs that consume a BC API or one of the following: ◦ CL_EXCHANGE_RATES ◦ CL_NUMBERRANGE_INTERVALS ◦ CL_UOM_DIM_MAINTENANCE ◦ CL_UOM_MAINTENANCE ◦ XCO I18N APIs ◦ XCO Transport APIs
2308	SAP Object Types	You can now use SAP Object Types to semantically relate and group development artifact
2308	Verifying the Business Event Payload	You can now verify the payload of RAP Business Events using the Event Test Double Fram
2308	Derived Events	You can now define a custom payload for already existing RAP Business Events.
2308	Background Processing Framework: Initial Version	Background processing is important for business applications. Much logic can be executed asynchronously to improve performance. In scenarios where changes of the current LUW communicated remotely, the asynchronous execution is even mandatory to be compliant LUW (because implicit database commits). We want to simplify and improve the background processing by covering all required quality "exactly once", "in order", testability and an integration into the ADT cross trace for better In the first version, only limited monitoring capabilities are supported (bgRFC monitoring
2308	Extensibility Enablement of Service Definitions	You can now enable service definitions for extensibility by defining respective provider co
2308	Transactional Enablement of WHERE Clauses in CDS Projection Views	WHERE clauses in CDS Projection Views can now be used transactionally.

Available as of	Feature	Description
2305	Authorization Control for Actions	You can now define authorization for actions that overrule the authorization definition in authorization master. In BDEF extension, authorization control must be defined for every action.
2305	Extension Layering	You can now define behavior definition extensions that extend the behavior which is defined by behavior definition extensions.
2305	Defining a Leading Entity in RAP BOs	You can now define a leading entity in the service definition. In the service binding this entity is marked.
2302	Side Effects	You can now use side effects in your RAP behavior definition to trigger reloads based on draft-enabled UI services.
2302	Integrating SAP Workflow Management	You can now integrate SAP Workflow Management in your RAP BO to use the workflow capabilities for process automation.
2211	Static and Dynamic Feature Control for Large Objects	You can now use dynamic and static feature control for large objects in OData V2.
2211	Define Own Authorization Context and Privileged Access	You can use the own authorization context to define the Identity and Access signature for own authorization context lists all authorization objects required during the RAP BO consumption runtime. In combination a RAP BO can offer privileged access for consumers to allow them to access certain authorization objects.
2211	Static Feature Controls for Fields in RAP BO Interfaces	You can define static feature control for fields in a RAP BO interface.
2211	abapGit Plugin	All RAP related development objects can be pushed and pulled with the abapGit plugin.
2211	Modeling Parameters for Non-Standard Operations	There is now documentation about how to model parameters and use their representation in RAP BOs.

Available as of	Feature	Description
2208	Large Objects	You can enable your RAP application for maintaining large objects (LOBs). By doing so, you users the option to incorporate external binary files or text files when editing entity instances.
2208	Business Events	You can now define and raise business events in a RAP BO or in a RAP BO behavior extension. Business events provide the opportunity of light-weight, decoupled process integration based on stable APIs and they are now a native part of the ABAP RESTful Application Programming Model. In the RAP Business Event Bindings Editor , you can create RAP Event Bindings which provide a mapping between the definition of RAP Events via behavior definition (BDEF) and the representation of Business Events.
2208	Extensibility-Enabling for RAP Business Objects	RAP extensibility is based on an opt-in approach to provide you with fine-granular control. Parts of your business object can be extended. You can easily enable your RAP BO and define use case extensions can be created. Using the respective CDS annotations, the BDEF syntax <code>Extend release contract</code> , you can decide if you want to enable your BO for data model extensions, behavior extensions or node extensions depending on your business requirements.
2208	Extending RAP Business Objects	RAP enables you to build upgrade-safe and lifecycle-stable extensions using well-defined anchors of the original business object. You can extend the data model and behavior of the BO and build data model extensions, behavior extensions or node extensions to meet your business requirements.
2208	Repeatable Actions and Functions	You can now define actions and functions that can be executed multiply on the same instance for the same request.
2208	More Flexible Usage of Parameters	In Analytics with CDS, parameters are restricted to single values. More general input is possible with <code>@AnalyticsDetails.variable</code> can be used to create any variable in the analytics model from a parameter.
2205	Business Object Interfaces	You can now define business object interfaces to introduce an additional abstraction layer between a BO and its business service projection layer. With RAP BO interfaces, you can technically define requirements for stable consumption from the behavior and data model of the underlying object. You can release RAP BO interfaces for C1 with visibility <code>Use in Cloud Development</code> to enable EML-consumption between different software components.
2205	Unmanaged/additional save with full data	It is now possible to use the option <code>WITH FULL DATA</code> for an unmanaged/additional save of a BO. With this option enabled, when an instance is modified, not only the changed fields but also the full data is provided into the <code>save_modified</code> method.
2202	Filter Tree API for Unmanaged Queries	You can now use the method <code>get_as_tree</code> of the interface <code>if_rap_query_filtering</code> to retrieve the filter conditions in an easily parsable expression tree.
2202	Business Object Interfaces	You can now use business object interfaces to introduce an additional abstraction layer between a base BO and its business service projection layer. With RAP BO interfaces, you can technically define requirements for stable consumption from the behavior and data model of the underlying

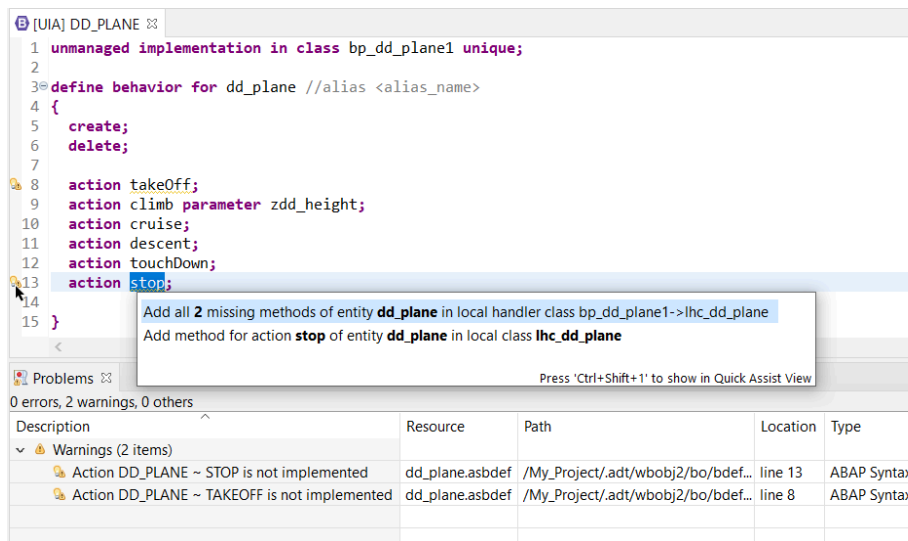
Available as of	Feature	Description
2202	Add Dimension to Query at Runtime (New)	With 2021 support package 1, some BI-tools like WebDynpro Grid offer the possibility to add dimensions from the cube view with the "free"-axis at runtime. With this, it isn't necessary to include dimensions into the CDS analytical query view that should be available at runtime. Only the most common ones can be added to the query view. With this the UI view becomes clearer and possibly slightly better. Since the cube may contain sensible dimensions that not all users are allowed to query has to be enabled for this feature with the header annotation <code>@Analytics.Settings.runtimeExtensibility.allowed: true</code> . Dimensions that should be exposed at runtime can be excluded with the annotation <code>@Analytics.Setting.excludedFromRuntimeExtensibility</code> on field level in the cube view.
2202	Changes in the Consumption or Provisioning of RAP BOs	You can find an overview of changes in the RAP framework in SAP Note 2943761  .
2111	Using Client-Independent Tables in Transactional Apps with Draft Capabilities	You can now use client-independent tables as data sources for managed transactional applications with draft capabilities.
2111	Multiple Message Targets in OData V4 RAP Services	OData V4 now supports multiple message targets. In the reported structure of RAP handover, you can now define multiple elements in the derived type component <code>%element</code> . These elements are supported by OData and are highlighted on a Fiori V4 UI.
2111	Using <code>UI.hidden</code> Annotation for Action Input Parameters	You can now use <code>UI.hidden</code> with action input parameters fields to prevent the input field from being displayed on the UI, while leaving the input parameter fields available for a WebAPI client.
2111	Read Access Logging (RAL)	You can now use Read Access Logging for your RAP services. With this feature you can monitor read accesses.
2111	Service Binding Errors in ATC Checks	ATC checks now also expose errors that are related to a RAP service. These errors are displayed in the ATC service binding. They can now also be retrieved by running ATC checks for the service.
2111	Late Numbering for Managed Implementation Type	Late numbering can now also be used for the managed implementation type.
2111	Consumption of released Business Objects	You can now consume released Business Objects via C1 released RAP BO Interfaces.

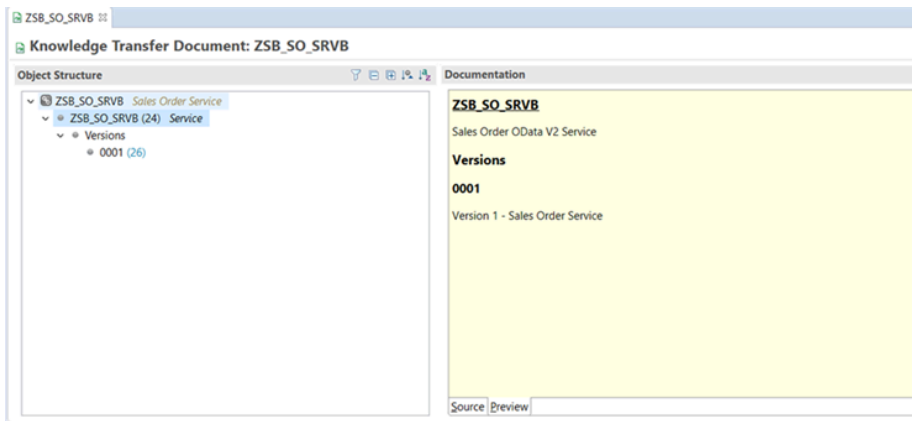
Available as of	Feature	Description
2111	Instance Factory Actions	You can now use instance factory actions to model actions that create new instances based on an instance. Instance factory actions can be useful if you want to copy specific values of an instance.
2111	Operations in Reported Response Parameter	When providing the reported response parameter in methods of the RAP behavior pool, you can now specify the operation the provided message belongs to. The derived type component <code>%op</code> provides the operation. The relevant one must be marked with <code>if_abap_behv=>mk-on</code> .
2111	Generate ABAP Repository Objects	With the Generate ABAP Repository Objects wizard, you can now generate RAP artifacts for a transactional application based on a database table.
2111	Semantic Versioning for OData V4 Service	In the service binding editor, you can now specify semantic versioning in the service versioning by adding minor and patch version.
2111	Changes in the Consumption or Provisioning of RAP BOs	You can find an overview of changes in the RAP framework in SAP Note 3107113  .
2108	Authorization and Feature Control Checks in Unmanaged Business Objects	The implementation for authorization and feature control in the corresponding RAP handler is now also called during every modify operation in business objects with unmanaged implementation.
2108	Delegating Authorization Checks	You can now use the syntax <code>authorization: update</code> in the behavior definition to delegate the authorization check for delete, create-by-association, or action operation to the authorization check of the update operation.
2108	Additional Implementation for Draft Actions	You can now add an additional implementation for all draft actions to extend the RAP inheritance cycle. The additional implementation is defined in the behavior definition with <code>with additional implementation</code> and implemented in the behavior pool.
2108	Runtime Changes for Cleanup Execution	The Cleanup is now called after a <code>ROLLBACK/COMMIT ENTITIES</code> statement also for an instance access including lock operation, precheck, instance authorization, and instance creation.
2108	Idempotency for OData V4 services	Idempotent behavior is now supported for OData V4 services. With this feature, it is guaranteed that synchronous messages are delivered exactly once.
2108	RAP Reuse Data Elements	You can now use reuse data elements for the administrative fields in RAP applications.

Available as of	Feature	Description
2108	Unit and Currency Types as Built-In Entity Sets	In OData V2 services, unit and currency types are automatically added to the service met entity sets. With this feature, unit and currency values are always displayed correctly and regarding decimals on a UI.
2108	Fail Causes for failed in RAP Handler Methods	The fail causes <code>disabled</code> , <code>readonly</code> and <code>dependency</code> can now be provided in the res parameter <code>failed</code> of RAP handler methods.
2108	Semantic Versioning for OData V2 Service	In the service binding editor, you can now specify semantic versioning in the service versi service by adding minor and patch version.
2108	Using Service Binding Editor for SQL Service	You can now expose a RAP business service as an SQL service. The binding type must be creating the service binding for the service.
2108	Save Actions for Managed BOs	You can now use actions that are executed only during the save sequence. On-save action the behavior definition with the following syntax: <code>[static] save action actionName [parameter parType] [result carc</code>
2108	Changes in the Consumption or Provisioning of RAP BOs	You can find an overview of changes in the RAP framework in SAP Note 3070134 
2105	Adding Actions and Functions in a Projection Behavior Definition	You can now add actions and functions to a projection behavior definition. You can also add for the new actions and functions.
2105	Using Strict Mode	You can now implement your RAP business object in <code>strict</code> mode to ensure that your E and implemented in accordance with the RAP best practices.
2105	RAP BO Contract	Rules on how to implement a RAP BO are now available in the RAP documentation.
2105	Developing a RAP BO that Supports Multi-Inline-Editing on a Fiori UI	You can now implement a RAP business object with a singleton mechanism to support m editing on a Fiori UI.

Available as of	Feature	Description
2105	Using <code>AUTO FILL CID</code> Option	In EML statements for static operations, you can now use the option <code>AUTO FILL CID</code> . This ensures that <code>%cid</code> is filled with a value automatically generated by the framework.
2105	Flag <code>always</code> for Determinations/Validations on Subentities	You can now use the <code>always</code> flag in determine actions also for determinations or validations on subentities.
2105	<code>COMMIT ENTITIES</code> in Simulation Mode	You can now use the <code>COMMIT ENTITIES</code> statement in simulation mode. This triggers the save of RAP BOs without actually saving data.
2105	Mapping for Action and Function Parameters	You can now map names of action and function parameters that are represented as abstract ABAP dictionary tables and structures.
2105	Suppress Fields in Transactional Handling	You can now remove fields that are only added to a RAP BO data model for technical reasons, for example in derived EML fields. The annotation <code>@Consumption.hidden:true</code> is a prerequisite for this. Syntax in CDS projection or abstract behavior definition: <pre>projection abstract; ... define behavior for BO_Entity { field (suppress) TechnicalField; ... }</pre>
2105	Flexible Association Modeling	It is now possible to define associations to CDS projection view entities in CDS view entity model relationships between consumption / projection layers. It is not supported to retrieve fields or other associations via such association.
2105	Field Control for Modify-Enabled Fields	Virtual or denormalized fields are <code>readonly</code> by default. You can modify-enable them in a behavior definition with the syntax <code>field (modify)</code> . Now, you can also add field characteristics to such fields.
2105	Early Unmanaged Numbering in Unmanaged Business Objects with Draft	You can now use early unmanaged numbering in unmanaged business objects with managed drafts.
2105	Harmonization of <code>Commit Work</code> and <code>Commit Entities</code>	<code>Commit Work</code> and <code>Commit Entities</code> have been harmonized, so that <code>Commit Work</code> and <code>Commit Entities</code> can be used interchangeably in the RAP save sequence.

Available as of	Feature	Description
2105	Possibly Incompatible Changes for ABAP RESTful Application Programming	You can find an overview of possibly incompatible changes for the ABAP RESTful Application Programming Model in SAP Note 3050328  .
2102	Implementing Unmanaged Early Numbering in Managed Business Objects	CREATE.
2102	Retrieving Permission Information from Business Objects	You can now use the EML statement <code>Get Permissions</code> to retrieve information about features and authorization permissions from business objects on global and instance level.
2102	Implementing Cleanup in Managed Business Objects	You can now implement the cleanup method in managed BOs if an additional or an unmanaged cleanup method is defined in the behavior definition. In this implementation you can handle legacy coding and cleanup of managed BOs. The cleanup for managed BOs must be defined in the behavior definition and implemented as a redefined method in the local saver class.
2102	Possibly Incompatible Changes for ABAP RESTful Application Programming	You can find an overview of possibly incompatible changes for the ABAP RESTful Application Programming in SAP Note 3017989  .
2011	Using Service Binding Editor for OData V4 Service	You can now expose a RAP business service as an OData V4 service. The binding type must be chosen while creating the service binding for the service.
2011	Using Service Binding Editor for InA Service	You can now expose CDS Analytical Queries via Information Access (InA) service using service binding editor. The binding type must be chosen while creating the service binding for the service. To expose an Analytical Query as an InA service, the CDS view must be added to the service definition. The service binding definition must be used to create the service binding.
2011	Setting Default Parameter Values in OData V4 UI Services	It is now possible to model a default value for action input parameters in OData V4 UI services. The default value is defined with the annotation <code>@Consumption.defaultValue: '<value>'</code> on an abstract CDS entity that defines the input parameter. This feature is only available for UI services with OData V4.
2011	Nested Determinations on modify	It is now possible to trigger a determination on modify by another determination on modify.
2011	Always Flag in Determine Actions	You can now use the flag <code>always</code> for determinations and validations that are assigned to an action. When the determine action is called, determinations and validations with this flag are triggered regardless of their trigger conditions.
2011	Global Feature Control	You can now define global feature control for feature control that is independent of a business object instance.

Available as of	Feature	Description																				
2011	Global Authorization Control	You can now define global authorization control to check authorization for incoming requests particular for static operations.																				
2011	Augmenting Incoming Requests	You can now define augmentation in your projection behavior definition to enhance incoming requests with custom implementation, for example with default values.																				
2011	Non-Locking Actions	You can now suppress the locking mechanism for action execution with the syntax <code>(lock = false)</code> in action definitions in the behavior definition.																				
2011	Possibly Incompatible Changes for ABAP RESTful Application Programming	2943761  .																				
2011	Quick Fixes For Creating Behavior Implementations	In case of missing implementation classes or missing methods, corresponding warnings displayed in the Behavior Definition Editor and in the Problems View. You can apply Quick Fixes to generate the entire implementation class or missing methods.  The screenshot shows the Behavior Definition Editor for entity <code>dd_plane</code>. The code defines a local handler class <code>bp_dd_plane1</code> with a <code>unique</code> implementation. It lists several actions: <code>create</code>, <code>delete</code>, <code>takeOff</code>, <code>climb</code> (with parameter <code>zdd_height</code>), <code>cruise</code>, <code>descent</code>, <code>touchDown</code>, and <code>stop</code>. A Quick Assist popup suggests adding missing methods for <code>dd_plane</code> in the local handler class <code>bp_dd_plane1</code> and for the <code>stop</code> action in the local class <code>lhc_dd_plane</code>. The Problems View below shows two warnings: <table><tr><th>Description</th><th>Resource</th><th>Path</th><th>Location</th><th>Type</th></tr><tr><td colspan="5">Warnings (2 items)</td></tr><tr><td>Action DD_PLANE ~ STOP is not implemented</td><td>dd_plane.asbdef</td><td>/My_Project/adt/wbobj2/bo/bdef...</td><td>line 13</td><td>ABAP Syntax</td></tr><tr><td>Action DD_PLANE ~ TAKEOFF is not implemented</td><td>dd_plane.asbdef</td><td>/My_Project/adt/wbobj2/bo/bdef...</td><td>line 8</td><td>ABAP Syntax</td></tr></table> Quick Fixes to Generate Missing Methods	Description	Resource	Path	Location	Type	Warnings (2 items)					Action DD_PLANE ~ STOP is not implemented	dd_plane.asbdef	/My_Project/adt/wbobj2/bo/bdef...	line 13	ABAP Syntax	Action DD_PLANE ~ TAKEOFF is not implemented	dd_plane.asbdef	/My_Project/adt/wbobj2/bo/bdef...	line 8	ABAP Syntax
Description	Resource	Path	Location	Type																		
Warnings (2 items)																						
Action DD_PLANE ~ STOP is not implemented	dd_plane.asbdef	/My_Project/adt/wbobj2/bo/bdef...	line 13	ABAP Syntax																		
Action DD_PLANE ~ TAKEOFF is not implemented	dd_plane.asbdef	/My_Project/adt/wbobj2/bo/bdef...	line 8	ABAP Syntax																		
2008	Draft Support for RAP Business Services	The ABAP RESTful Application Programming Model now provides draft support for your business services. Draft-enabled business objects ensure that transactional data does not get lost interrupted or continued from a different device. You draft-enable your business object by using the keyword <code>with draft</code> in the behavior definition. Draft capabilities can be included for both business object implementation types, <code>managed</code> and <code>unmanaged</code>.																				
2008	Draft Scenario on GitHub	You can now download a RAP managed business service with draft from GitHub. The development objects are now available as part of the ABAP Flight Reference Scenario that you can download into your ABAP system.																				
2008	Controlling Active Data Access in Draft Business Objects	You can now define a designated field, the <code>total ETag</code> field, that controls concurrent access to data in draft business objects. The RAP runtime framework checks its value on every transaction.																				

Available as of	Feature	Description
		draft to active data to prevent concurrent BO consumers from overwriting each other with information. The total ETag is defined in the behavior definition and is updated by the RAP runtime framework automatically if annotated with the annotation <code>@Semantics.systemDateTime.last true</code> in the corresponding CDS view.
2008	Working with Determinations and Validations	Determinations and validations are now processed by the determinations-validations-manager. You can now define determinations and validations with all three standard operation triggers <code> update delete</code>, and in combination with field triggers. In addition, the method definition in the behavior pool uses different syntax: <pre> METHODS method_name FOR VALIDATE ON SAVE IMPORTING keys FOR AliasedEntityName~ValidationName. METHODS method_name FOR DETERMINE ON MODIFY SAVE IMPORTING keys FOR AliasedEntityName~ValidationName. </pre>
2008	Executing Determinations and Validations on Request	New determine actions allow the business object consumer to execute determinations and validations without fulfilling the trigger conditions. You can now assign determinations and validation determine action that is called by a consumer's trigger, just like any other actions.
2008	Prechecking Modify Operations	You can now prevent illegal changes from reaching the application buffer by prechecking operations. The precheck method is called during runtime before the assigned modify operation removes all input from the modifying request for which the condition in the precheck is not met.
2008	Authorization Control	Consumer hints in the OData service metadata now include authorization control by a path description for the respective operation. That means, if you define authorization control for your business authorization-relevant entity sets receive a path description instead of a static information about the ability to be updated or deleted. In addition, the corresponding properties are included in the entity type.
2008	RAP Runtime Checks	Stricter RAP runtime checks are introduced to help you make your RAP implementation more friendly, and compatible to further RAP features. These runtime checks affect - creating and updating read only fields via EML, - executing modify calls in a late save phase, - consuming actions or functions with parameters exceeding the defined length.
2008	Documenting Service Binding	You can now provide documentation using knowledge transfer documents (KTD) for service bindings.  The screenshot shows the SAP Knowledge Transfer Document (KTD) interface for the service binding ZSB_SO_SRV. The left pane displays the 'Object Structure' with a tree view showing the service and its versions. The right pane shows the 'Documentation' for the selected version (0001), including the service name and a description.

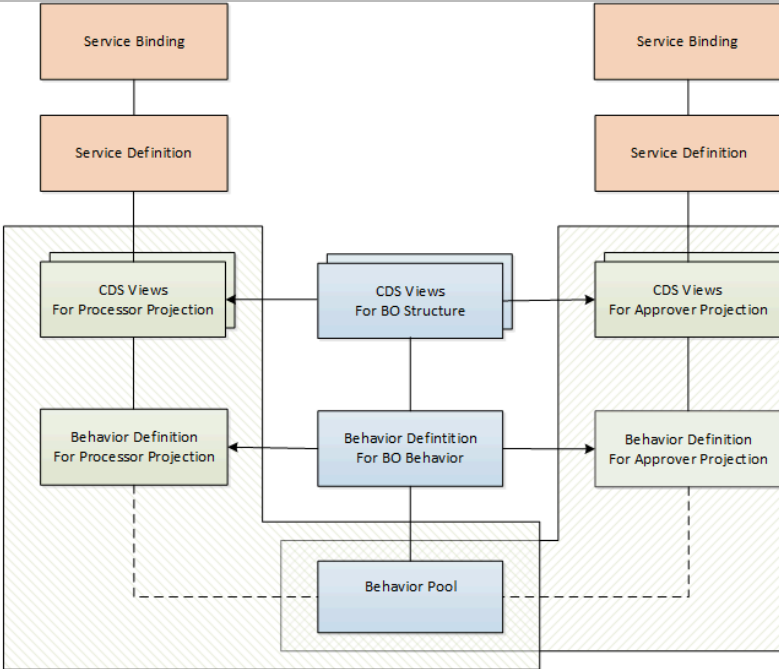
Available as of	Feature	Description
2008	Feature Control	You can now use <code>mandatory:create</code> to assure that a field is filled in during a create operation and <code>readonly:update</code> to assure that the field value isn't changed during modify operation.
2008	Possibly Incompatible Changes for ABAP RESTful Application Programming	You can find an overview of possibly incompatible changes for the ABAP RESTful Application Programming in SAP Note 2943761  .
2005	Implementing Own Locking Logic for Managed Business Objects	You can now define an unmanaged lock in a managed scenario. The unmanaged lock method can be defined in the behavior definition with the syntax <code>lock master unmanaged</code> and implemented in the behavior pool in the method <code>FOR_LOCK</code> , just like in an unmanaged scenario. The method is invoked during runtime.
2005	Using <code>dependent by _Association</code> in Behavior Definition	You can now use the syntax <code>dependent by _Association</code> for lock, ETag, or authorization dependent entities. The association to the respective master entity must be explicitly specified in the behavior definition and implemented when using the unmanaged scenario.
2005	Using Client-Independent Tables in Managed Scenarios	The RAP managed BO runtime now supports client-independent database tables.
2005	Dynamic Operation Control for Create By Association	You can now dynamically control the create by association operation. The syntax for defining feature control is the following: <pre>_association {create (features: instance); }</pre> Like other feature controls, the control for the create by association must be implemented in the behavior pool with the method <code>FOR_FEATURES</code>.
2005	New Read-Only Associations in CDS Projection Views	It is now possible to define new read-only associations in the projection view. These associations can be reused to display additional information in a UI service, for example analytical data which is related to the transactional basic business object. With associations in projection views, you can manage service-specific relationships. The syntax for the association definition is the same as in CDS views: <pre>association [min..max] to TargetEntity [as _Alias] on OnCondition</pre>
2005	Filtering for Null Values in OData V2 Services	OData does not foresee initial values for the following EDM data types: • Boolean • GUID • Time • DateTime • DateTimeOffset. Fiori Elements UIs offer the <code>empty</code> operator in filter bars for these fields. Although they are empty in the UIs, the actual value sent to the client is <code>null</code>.

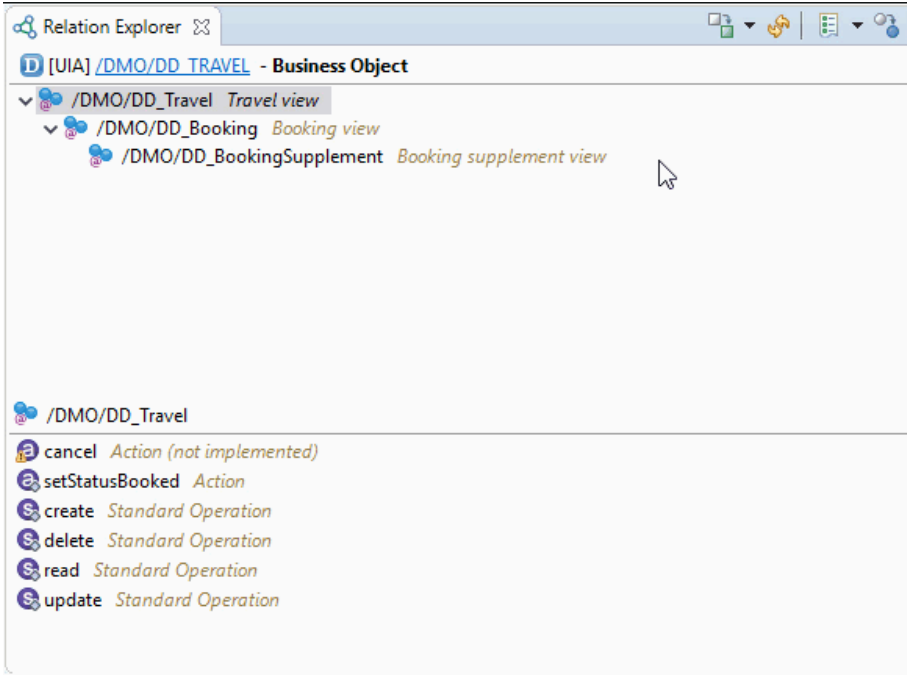
Available as of	Feature	Description
		To be able to select records with empty values for these data types, the client filters null values. The filter is transformed to filter for initial values for the backend. Real null values cannot be represented in the database by filtering for them in OData V2.
2005	Understanding Runtime Processes	The concepts section in the ABAP RESTful Application Programming Model has been enhanced with interactive diagrams so you can get a better understanding of the runtime processes for OData.
2005	Control Structure for Import Parameter in CREATE Operations	The control structure <code>%control</code> for import parameters of <code>CREATE</code> and <code>CREATE BY</code> operations is now filled according to the elements that were sent by the OData client or with the EML call. In particular this means that the application developer knows which elements were provided for the create. ❖ Example In the following example the elements <code>agencyid</code>, <code>customerid</code>, <code>begindate</code>, and <code>enddate</code> are filled by the OData client and thus are marked in the control structure, while the other elements are not. <pre> METHOD create_travel. DATA lt_messages TYPE /dmo/t_message. DATA ls_travel_in TYPE /dmo/travel. DATA ls_travel_out TYPE /dmo/travel. LOOP AT it_travel_create ASSIGNING FIELD-SYMBOL(<fs_travel_create>). ls_travel_in = <fs_travel_create>. CALL FUNCTION 'EXPOSE_IMPORT_ENTRIES' EXPORTING IMPORT_ENTRIES = ls_travel_in IMPORTING EXPORT_ENTRIES = ls_travel_out. ENDLOOP. IF lt_messages IS NOT INITIAL. MESSAGE 'Import failed' TYPE 'E'. ELSE. /dmo/travel_create = ls_travel_out. ENDIF. ENDMETHOD. </pre> Provided Elements in Create for Travel

Available as of	Feature	Description
2002	Automatically Drawing Primary Key Values for Managed Business Objects	The managed runtime framework can now automatically generate key values in scenario: if managed numbering is defined in the behavior definition. Syntax for defining managed numbering in the behavior definition: <pre> [implementation] managed [implementation in class ABAP_CLASS [unique define behavior for Entity [alias AliasedName] lock (master dependent()) ... { ... field ([read only,] numbering:managed) KeyField1, [KeyField2]; ... } </pre>
2002	New Options for Action and Function Results	The BO runtime framework now supports results for actions and functions other than \$S can be entities of the same BO the action is defined for, entities of other BOs, or result str differentiate between result entities and result structures the syntax element entity ha introduced. In addition, you can now create actions, for which the action consumer can decide wheth shall be returned completely or only parts of it, for example the keys only. Such an action marked with the keyword selective in the behavior definition. <pre> define behavior for CDSEntity ... ** Action with result entity action ActionName result [cardinality] entity OutputEntity ** Action with result structure action ActionName result [cardinality] OutputStructure ** Action with selective result action ActionName result selective [cardinality] entity OutputEnt ... </pre>
2002	Reporting Messages in ADJUST_NUMBERS and SAVE	The implicit returning parameter REPORTED is now available for the methods adjust_n save. By filling this parameter you can report information or success messages after the return in the save sequence.
2002	Documenting Behavior Definitions	Now you can document behavior definitions in the Knowledge Transfer Document editor From the Project Explorer, select the behavior definition you want to document. Use the create a knowledge transfer document.

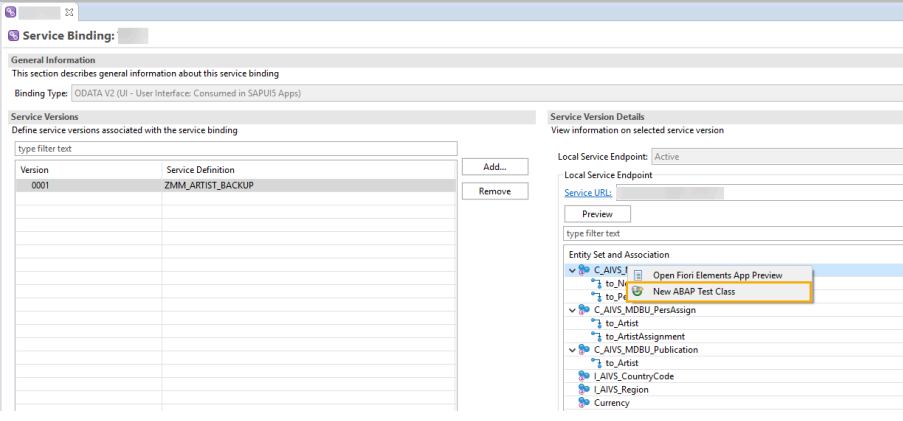
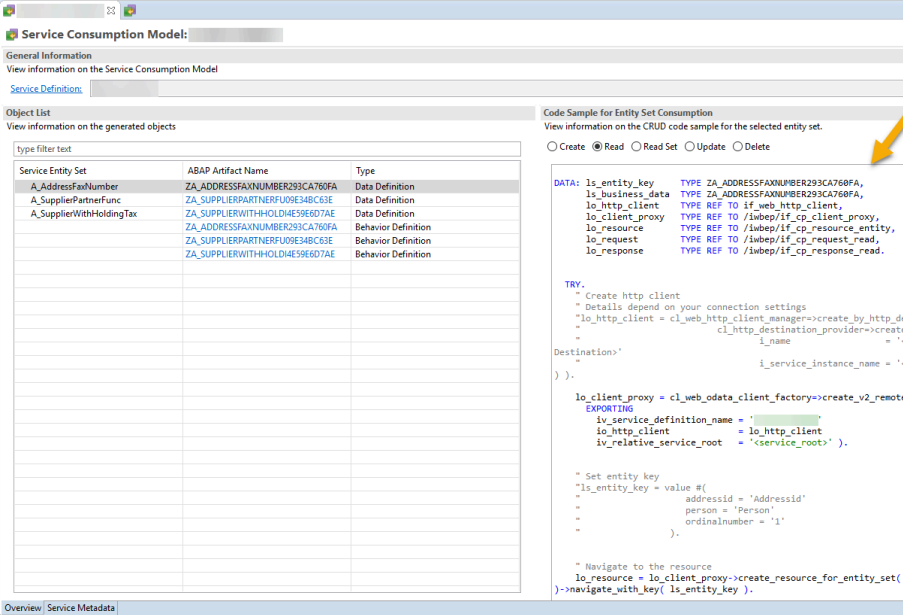
Available as of	Feature	Description
		Knowledge Transfer Document Editor
1911	Defining the Service Namespace for OData Services	You can now define the OData service namespace in service definitions with the annotation <code>@OData.schema.name</code> .
1911	Filling the Result Parameter for Actions	A runtime check is introduced to enforce the proper assignment of result values in action implementations when result parameters are declared in the corresponding action definition. Actions that returned the input entity if the result parameter was not filled in the action implementation before 1911, they now return the value of the result parameter as specified in the action definition exactly according to its implementation. To avoid that a Fiori UI shows initial values, if nothing is returned, you must fill the result parameters in actions.
1911	Navigating More Than One Step in OData Requests	It is now possible to navigate to associated entities using more than one navigation for both read and write services. ❁ Example To request the booking supplements for the booking entity with BookingID 5 of the travel entity with TravelID 1, use the following syntax. <pre><service_URL>/Travel('1')/to_Booking('5')/to_BookingSupplement</pre> You can link an unlimited number of entities in one OData request.
1911	Outsourcing UI Metadata in Metadata Extensions	You can now use metadata extensions to define CDS annotations outside of the corresponding definition of the CDS projection view. The use of metadata extensions allows the separation of the data model from domain-specific semantics, such as UI-related information and user interface consumption.
1911	Using Type and Control Mapping	You can now define a mapping contract for applications that include unmanaged or managed objects based on CDS entities on the one hand, and legacy data types that are generally not supported by SAP on the other. This allows you to implement the behavior or parts of it. The general syntax for field and control (information) mapping in a behavior definition is as follows:

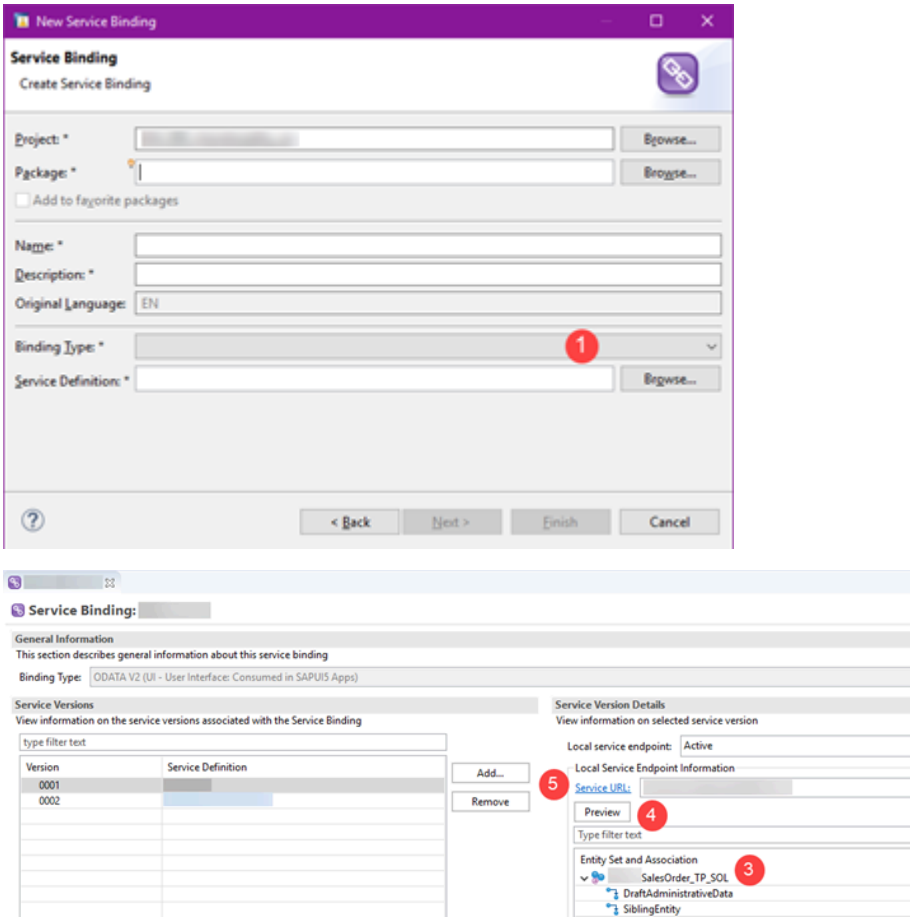
Available as of	Feature	Description
		mapping for LegacyType control ControlType corresponding <pre> { EntityField1 = LegacyField1; EntityField2 = LegacyField2; ... }</pre>
1911	Integrating Additional Save in Managed Business Objects	You can now integrate Additional Save for each entity of a given business object with managed implementation type. This is done in the behavior definition of the business object by adding <code>with additional save</code> . The actual implementation of Additional Save takes place in class as part of the behavior pool.
1911	Integrating Unmanaged Save in Managed Business Objects	You can now integrate Unmanaged Save within the transactional life cycle of managed business objects. In order to integrate Unmanaged Save into the save sequence as a part of the managed implementation, you must first add the corresponding syntax (<code>with unmanaged save</code>) to the behavior definition and then implement the saver handler method as a part of the behavior pool.
1911	Using Groups for Large Implementations	Until now, the implementation of business object entity's operations and actions had to be implemented in a single Local Type include of the behavior pool associated with that entity. Since the Local Type can only be changed by one developer at a time, the efficiency of development would be significantly reduced in case of larger implementations. As a solution, the groups can now be used to divide operations and other implementation-relevant parts of the business logic into several groups for better management and implementation.
1911	Using Virtual Elements	You can now define elements in CDS projection views that are not persisted on the database. Virtual elements are defined with the new syntax <code>element virtual</code> . Their calculation is implemented in a separate ABAP class, which is called during runtime via an ABAP code exit for virtual elements.
1911	Using Service Binding Editor	In the Service Binding editor, the preview of the Fiori elements app now automatically opens in the Fiori language.
1908	New URL for the Documentation of the ABAP RESTful Programming Model in SAP Cloud Platform ABAP Environment	The documentation of the ABAP RESTful Programming Model for SAP Cloud Platform ABAP Environment on the SAP Help Portal has been moved.
1908	Developing New Managed Transactional Apps	The ABAP RESTful Programming Model now supports the managed implementation type for developing new transactional apps. This scenario aims at use cases to develop new transactional applications. All required standard operations must only be specified in the behavior definition to obtain a business object. The business logic is implemented using actions, validations and determinations.
1908	Business Object Projection	You can now use the ABAP-native approach to project and to alias a subset of the business object as a specific business service. The projection enables flexible service consumption as well as reusable service designs. With projections, it is possible to project one business object for different role-based UIs. For example, given in Developing a Projection Layer for Flexible Service Consumption. The travel business object is exposed for a processor Fiori UI and for an approver Fiori UI.

Available as of	Feature	Description
		
1908	Implementing an Unmanaged Query	A new API is available to implement an unmanaged query in a query implementation class. When an unmanaged query is requested, the request is delegated to the query implementation class, which implements the <code>select</code> method of the interface <code>IF_RAP_QUERY_PROVIDER</code>. The new interface provides query capabilities, such as paging, filtering, sorting, or counting. It replaces the <code>IF_A4C_RAP_QUERY_PROVIDER</code>, which is deprecated as of 1908. The implementation of an unmanaged query with the interface <code>IF_RAP_QUERY_PROVIDER</code> is a new topic in the common task section.
1908	Server-Side Paging	Server-side paging has now been implemented to improve performance for OData calls. If a client does not provide paging query options, the default paging restricts the response to 5000 records. If <code>\$top</code> is set to values greater than 5000, the response is reduced to 5000 data records. Following from this, unmanaged queries require at least the implementation for paging. If corresponding methods are not implemented and the unmanaged query does not return information, there will be a dump during runtime.
1908	Additions to the Unmanaged Reference Scenario	The sources for the unmanaged reference scenario have been updated with the latest features. Explore the new features in the sources that you can download from GitHub and in the related guide on how to develop unmanaged transactional apps based on existing applications. Mapping CDS names to database field names By defining a mapping specification in the behavior definition, you can use the mapping in the behavior implementation to write records to the database that have a description of the names of the database table fields and the CDS view field names. Message handling The behavior processing framework provides a message object that can be used for handling in the behavior pool. New message handling has been added to the unmanaged reference scenario. The corresponding methods are implemented in an auxiliary class that implements <code>cl_abap_behv</code> and called from the behavior handler.
1908	Authorizations Checks for Modifying Operations in Managed Business Objects	To protect data from unauthorized read access, the ABAP CDS provides its own authorization checks based on a data control language (DCL). The authorization checks for read operations apply to all the results returned by an entity to those results you authorize a user to see.

Available as of	Feature	Description
		For business objects that are implemented for managed contract, also modifying operational standard operations create, update, delete, create by associations, and actions must be controlled to prevent unauthorized access. With the current release, the instance-based authorization control is implemented.
1908	Adding Dynamic Feature Control	Apart from static feature control, you can now also use dynamic feature control. In this case, you can control the availability of a state of the node instance if certain elements or actions are available. Dynamic feature control is defined in the behavior definition: <pre> ... define behavior for /DMO/I_TRAVEL_M field (features : instance) travel_id; action (features: instance) acceptedTravel result [1] \$self; ... </pre> The implementation for the control must then be implemented in the respective method pool. For example, for this behavior definition, you can implement that the field <code>travel_id</code> is mandatory on create, but read-only on the update operation. The action can be implemented if the travel entity is already set to accepted, but enabled if it is not yet accepted.
1908	Exploring Business Objects	A business object consists of hierarchical connected entities. The behavior for each entity is defined in the behavior definition object and implemented in the behavior classes. In the Relation Explorer, you can see the structure and behavior of a certain business object independent of the technical location. You can navigate to all the entities and the corresponding behavior (definition and implementation). Sometimes you might be interested in more CDS-specific aspects and want to see access control and test classes. You can achieve this by switching the context from Business Object to Core context, as you can see in the following animation. Switching the context in the Relation Explorer  Relation Explorer provides much more features such as further contexts, for example, to view the objects for a certain class.
1908	Service Consumption Model Wizard	In the service consumption model creation wizard: ETag support can now be selected for any entity set. If ETag support is marked in the wizard, the ETag support checkbox is selected by default.

Available as of	Feature	Description																																																																																																																			
		2. An entity set can now be selected for the generation. You can only edit the ABAP artifact name of the entity set that you've selected for generation. 3. Issues in an entity set are displayed and these entity sets cannot be selected for generation. New Service Consumption Model Define Entity Set Select entity sets for generation, edit the ABAP artifact name and select ETag support. Entity sets with issues are not generated. type filter text <table><thead><tr><th></th><th>Service Entity Set</th><th>ABAP Artifact Name</th><th>ETag Support</th><th>Issue</th></tr></thead><tbody><tr><td>2</td><td>☒ A_BusinessPartner</td><td>ZA_BUSINESSPARTNER73911184C</td><td>☒ 1</td><td></td></tr><tr><td></td><td>☒ A_BusinessPartnerAddress</td><td>ZA_BUSINESSPARTNERADDRESS</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_BusinessPartnerBank</td><td>ZA_BUSINESSPARTNERBA10EF53F5...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_BusinessPartnerRole</td><td>ZA_BUSINESSPARTNERRO84E9222...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_BusinessPartnerTaxNumber</td><td>ZA_BUSINESSPARTNERTAB0E4B215...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustSalesPartnerFunction</td><td>ZA_CUSTSALESPARTNERF159B283...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_Customer</td><td>ZA_CUSTOMERD6C7ECF4B8</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerCompany</td><td>ZA_CUSTOMERCOMPANY3D79525...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerDunning</td><td>ZA_CUSTOMERDUNNING55ECBD...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerSalesArea</td><td>ZA_CUSTOMERSALESAREA96C421...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerSalesAreaTax</td><td>ZA_CUSTOMERSALESAREA7A7938...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerWithHoldingCompany</td><td>ZA_CUSTOMERWITHHOLDI5302D...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_Supplier</td><td>ZA_SUPPLIER617002ED75</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_SupplierCompany</td><td>ZA_SUPPLIERCOMPANY1BBDA39...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_SupplierDunning</td><td>ZA_SUPPLIERDUNNING8194E0C2D6</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_SupplierPartnerFunction</td><td>ZA_SUPPLIERPARTNERFUB236BA0...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_SupplierPurchasingOrg</td><td>ZA_SUPPLIERPURCHASIN8CD8D1...</td><td>☐</td><td>3</td></tr><tr><td></td><td>☒ A_SupplierWithHoldingTax</td><td>ZA_SUPPLIERWITHHOLDIACA225...</td><td>☐</td><td></td></tr><tr><td></td><td>☐ A_BPContactToAddress</td><td>ZA_BPCONTACTTOADDRESS</td><td>☐</td><td>Type Edm.DateTime of element</td></tr><tr><td></td><td>☐ A_BPContactToFunctionAndDataObject</td><td>ZA_BPCONTACTTOFUNCANDDEPT</td><td>☐</td><td>Type Edm.DateTime of element</td></tr><tr><td></td><td>☐ A_BusinessPartnerContact</td><td>ZA_BUSINESSPARTNERCONTACT</td><td>☐</td><td>Type Edm.DateTime of element</td></tr><tr><td></td><td>☐ A_AddressHomePageURL</td><td>ZA_ADDRESSHOMEPAGEURL</td><td>☐</td><td>Type Edm.DateTime of element</td></tr></tbody></table> Select All Deselect All ? < Back Next > Finish		Service Entity Set	ABAP Artifact Name	ETag Support	Issue	2	☒ A_BusinessPartner	ZA_BUSINESSPARTNER73911184C	☒ 1			☒ A_BusinessPartnerAddress	ZA_BUSINESSPARTNERADDRESS	☐			☒ A_BusinessPartnerBank	ZA_BUSINESSPARTNERBA10EF53F5...	☐			☒ A_BusinessPartnerRole	ZA_BUSINESSPARTNERRO84E9222...	☐			☒ A_BusinessPartnerTaxNumber	ZA_BUSINESSPARTNERTAB0E4B215...	☐			☒ A_CustSalesPartnerFunction	ZA_CUSTSALESPARTNERF159B283...	☐			☒ A_Customer	ZA_CUSTOMERD6C7ECF4B8	☐			☒ A_CustomerCompany	ZA_CUSTOMERCOMPANY3D79525...	☐			☒ A_CustomerDunning	ZA_CUSTOMERDUNNING55ECBD...	☐			☒ A_CustomerSalesArea	ZA_CUSTOMERSALESAREA96C421...	☐			☒ A_CustomerSalesAreaTax	ZA_CUSTOMERSALESAREA7A7938...	☐			☒ A_CustomerWithHoldingCompany	ZA_CUSTOMERWITHHOLDI5302D...	☐			☒ A_Supplier	ZA_SUPPLIER617002ED75	☐			☒ A_SupplierCompany	ZA_SUPPLIERCOMPANY1BBDA39...	☐			☒ A_SupplierDunning	ZA_SUPPLIERDUNNING8194E0C2D6	☐			☒ A_SupplierPartnerFunction	ZA_SUPPLIERPARTNERFUB236BA0...	☐			☒ A_SupplierPurchasingOrg	ZA_SUPPLIERPURCHASIN8CD8D1...	☐	3		☒ A_SupplierWithHoldingTax	ZA_SUPPLIERWITHHOLDIACA225...	☐			☐ A_BPContactToAddress	ZA_BPCONTACTTOADDRESS	☐	Type Edm.DateTime of element		☐ A_BPContactToFunctionAndDataObject	ZA_BPCONTACTTOFUNCANDDEPT	☐	Type Edm.DateTime of element		☐ A_BusinessPartnerContact	ZA_BUSINESSPARTNERCONTACT	☐	Type Edm.DateTime of element		☐ A_AddressHomePageURL	ZA_ADDRESSHOMEPAGEURL	☐	Type Edm.DateTime of element
	Service Entity Set	ABAP Artifact Name	ETag Support	Issue																																																																																																																	
2	☒ A_BusinessPartner	ZA_BUSINESSPARTNER73911184C	☒ 1																																																																																																																		
	☒ A_BusinessPartnerAddress	ZA_BUSINESSPARTNERADDRESS	☐																																																																																																																		
	☒ A_BusinessPartnerBank	ZA_BUSINESSPARTNERBA10EF53F5...	☐																																																																																																																		
	☒ A_BusinessPartnerRole	ZA_BUSINESSPARTNERRO84E9222...	☐																																																																																																																		
	☒ A_BusinessPartnerTaxNumber	ZA_BUSINESSPARTNERTAB0E4B215...	☐																																																																																																																		
	☒ A_CustSalesPartnerFunction	ZA_CUSTSALESPARTNERF159B283...	☐																																																																																																																		
	☒ A_Customer	ZA_CUSTOMERD6C7ECF4B8	☐																																																																																																																		
	☒ A_CustomerCompany	ZA_CUSTOMERCOMPANY3D79525...	☐																																																																																																																		
	☒ A_CustomerDunning	ZA_CUSTOMERDUNNING55ECBD...	☐																																																																																																																		
	☒ A_CustomerSalesArea	ZA_CUSTOMERSALESAREA96C421...	☐																																																																																																																		
	☒ A_CustomerSalesAreaTax	ZA_CUSTOMERSALESAREA7A7938...	☐																																																																																																																		
	☒ A_CustomerWithHoldingCompany	ZA_CUSTOMERWITHHOLDI5302D...	☐																																																																																																																		
	☒ A_Supplier	ZA_SUPPLIER617002ED75	☐																																																																																																																		
	☒ A_SupplierCompany	ZA_SUPPLIERCOMPANY1BBDA39...	☐																																																																																																																		
	☒ A_SupplierDunning	ZA_SUPPLIERDUNNING8194E0C2D6	☐																																																																																																																		
	☒ A_SupplierPartnerFunction	ZA_SUPPLIERPARTNERFUB236BA0...	☐																																																																																																																		
	☒ A_SupplierPurchasingOrg	ZA_SUPPLIERPURCHASIN8CD8D1...	☐	3																																																																																																																	
	☒ A_SupplierWithHoldingTax	ZA_SUPPLIERWITHHOLDIACA225...	☐																																																																																																																		
	☐ A_BPContactToAddress	ZA_BPCONTACTTOADDRESS	☐	Type Edm.DateTime of element																																																																																																																	
	☐ A_BPContactToFunctionAndDataObject	ZA_BPCONTACTTOFUNCANDDEPT	☐	Type Edm.DateTime of element																																																																																																																	
	☐ A_BusinessPartnerContact	ZA_BUSINESSPARTNERCONTACT	☐	Type Edm.DateTime of element																																																																																																																	
	☐ A_AddressHomePageURL	ZA_ADDRESSHOMEPAGEURL	☐	Type Edm.DateTime of element																																																																																																																	
1905	Understanding Concepts	As an application developer, you may not only be interested on how you can implement different scenarios and use cases for your business applications by following the stateless programming model of the ABAP RESTful programming model. It may also be important for you to understand the concepts behind it. The concepts section provides you with background information of ABAP RESTful programming model and helps you to understand main concepts from both, the design time, and runtime perspectives.																																																																																																																			
1905	Test Class Generation for Service Binding	You can now generate automated tests for an OData service that you've created using service binding. The test class provides guidance on how to access the OData service using ABAP Units and test code for performing CRUD operations on an entity set.																																																																																																																			

Available as of	Feature	Description
		
1905	Remote OData Access for Service Consumption Model	You can now view the code sample for performing the CRUD operations on an entity set to remote OData service. 
1905	Defining Names for OData Entity Sets and Entity Types	You can now define external names for OData entity sets and entity types that are then used in service metadata. The annotations <code>@OData.entitySet.name</code> and <code>@OData.entityType.name</code> can now be used in any CDS entity. CDS Entity <pre> @OData.entitySet.name: 'New_EntitySet_Name' @OData.entityType.name: 'New_EntityType_Name' define view /DMO/I_Customer_OData as select from /dmo/customer { key customer_id, first_name, last_name } </pre> OData Metadata <pre> <EntityType Name="New_EntityType_Name" sap:label="Test for Renaming" sap:content="..." <Key> <PropertyRef Name="customer_id"/> </Key> <Property Name="customer_id" Type="Edm.String" Nullable="false" MaxLength="..." </Property> <Property Name="first_name" Type="Edm.String" MaxLength="40" sap:label="First Name" </Property> <Property Name="last_name" Type="Edm.String" MaxLength="40" sap:label="Last Name" </Property> </EntityType> <EntityContainer Name="cds_xdmocustomer_odata_entities" m:IsDefaultEntityContainer="true" <EntitySet Name="New_EntitySet_Name" EntityType="cds_xdmocustomer_odata_entities" sap:creatable="false" sap:updatable="false" sap:deletable="false" sap:content="..." </EntitySet> </EntityContainer> </pre>
1902	Downloading the ABAP Flight Reference Scenario	You can now download the relevant development objects that are used in the course of the guides via GitHub. This enables you to import a complete OData service into your system use and reuse for learning purposes. Limitation: The import of a service binding artifact is not possible. To complete the OData service, you need to create service binding in your own package.
1902	Entity Manipulation Language (EML)	Entity Manipulation Language (in short: EML) is a part of the ABAP language that is used to define the business object's behavior in the context of ABAP RESTful programming model. It provides read and modifying access to data in transactional development scenarios.

Available as of	Feature	Description
1902	Extension of Transactional Development Guide	The development guide Developing Unmanaged Transactional Apps was extended with a hierarchy. Booking Supplements are now part of the business object. In addition, the travel business object can now be consumed using EML syntax.
1902	Service Binding UI	The service binding tools come with a new UI design: 1. The binding type does not have a value populated by default. 2. The Publish and Unpublish buttons have been renamed to Activate and Deactivate. 3. Entity Set and Association was displayed with the service URL as the root node. Nodes are separate and it is displayed only if the local service endpoint has been activated. 4. The Preview button is now available on the interface for previewing the SAP Fiori UI. This is applicable for OData V2 UI service only. 5. The local service endpoint information now displays the service URL when it is in state. 
1902	Consuming Services	The service consumption model replaces the OData client proxy to generate service artifacts. It comes with a new wizard and an editor to work on the generated artifact. The consumption model artifact provides an overview of all abstract entities and generated business service definitions that belong to the imported service.
1902	Transactional Behavior for the Service Consumption Scenario	The guide on how to consume a remote service has been extended with transactional capabilities. The use case of maintaining discount data in a local database is exemplified in the scenario.

Available as of	Feature	Description
1902	Using Aggregate Data in SAP Fiori Apps	Aggregate functions, such as sum, maximum, minimum and average, as well as a count are now available in the ABAP environment to be implemented in CDS and displayed in your SAP Fiori Apps. Annotations are used to mark the elements as measures, whose values can be aggregated.
1902	Adding Static Feature Control in SAP Fiori Apps	In a typical transactional scenario, you have to specify which operations should be provided for an entity or you must specify which fields of an entity have specific access restrictions (read-only, mandatory fields).
1902	Freely Selectable Name for Handler Methods in Behavior Pools	The method name in handler classes is now freely selectable. What kind of method it is, is determined by the <code>FOR</code> clause. The old syntax <code>METHODS modify FOR BEHAVIOR ...</code> becomes now: <code>METHODS FreeModify FOR MODIFY</code> The old syntax <code>METHODS read FOR BEHAVIOR ...</code> becomes now: <code>METHODS FreeMethod READ</code> The old syntax <code>METHODS lock FOR BEHAVIOR ...</code> becomes now: <code>METHODS FreeMethod LOCK</code> Note that the old syntax remains valid but is no longer recommended!
1811	Defining Service Versions	The service binding tools come with a new UI design and some additional functions for versioning (business) services.
1811	Support for Compositions	The current version of the ABAP RESTful programming model supports compositions: A composition consists of a tree of nodes where the nodes are linked by means of a special kind of associations. A composition is a specialized association that defines a whole-part relationship. A composite part only exists together with its parent entity (whole).
1811	Developing an A2X Service	It is possible to publish an OData service as an application-to-cross application (A2X) service. This means the service is published without information relevant for a UI service (for example, a UI annotation to define a UI). An A2X service facilitates the exchange of business information between an application and any client, including from a different system or server.
1811	Developing a UI Service with Access to a Remote Service	With the help of the service consumption model it is now possible to consume a Web API service and build a new SAP Fiori application by consuming the remote service. This development goes beyond the definition of a custom entity and implementing a custom query.

SAP S/4HANA Cloud

Available as of	Feature	Description
2605	Support of CDS Table Entities as Draft Persistence in RAP	CDS table entities can now be used as a draft persistency within RAP, allowing access control (DCLs) directly on the draft table.

Available as of	Feature	Description
2602	RAP Collaborative Draft	Collaborative draft now also works in cross-business object scenarios if both business objects are collaborative draft-enabled, and if the cross-business object scenario is fully draft scope-enabled.
2602	RAP Recommendations	Previously, business users had to fill out input fields manually and choose the right values. Recommendations standardize and ease this process: In RAP applications on an SAP Fiori, recommendations provide input assistance by proposing field values for input fields based on input and interaction history. RAP recommendations can be implemented using deterministic rules or using generative AI. Generative AI predicts the most likely values for each input field based on the context of the data. By using the ABAP AI SDK powered by Intelligent Scenario Lifecycle Management, you can embed AI capabilities into the application, enriching traditional value help with context-aware suggestions.
2605	Support of CDS Table Entities as Draft Persistence in RAP	CDS table entities can now be used as a draft persistency within RAP, allowing access to draft table entities (DCLs) directly on the draft table.
2602	RAP Change Operations for Data Records	With the introduction of the new syntax <code>add read entity MyEntity with change fields fields (...)) with ... result data(rdc)</code> , an application developer can now access the corresponding change operations of a data record processed within the table.
2602	Fiori Analytical Table - Read-Only	You miss the ABAP List Viewer (ALV) in Fiori with simple analytical capabilities? The Fiori Analytical Table will be introduced to combine the transactional and light-weight analytical world. RAP now provides a further OData URL to ABAP SQL mapping with analytical capabilities. For advanced analytical capabilities and dimensional analytical reporting please check the analytical clients.
2602	RAP Global Side Effects	RAP side effects are an optional part of a RAP behavior definition that define interdependent business object properties that trigger a reload of the affected properties on the user interface. Global side effects are those side effects that are defined without any source properties or source entities and can be triggered by an end user with a dedicated key stroke.
2508	Privileged Access for Cross-BO Associations	In cross-BO scenarios, when association-related ABAP EML requests (such as read-by-association or create-by-association) are performed on the association source and specify IN LOCAL , the association target's implementation of the operation is accessed in privileged mode (not bypassing the association target's authorization checks. This applies only if the association is specified with a privileged context.
2508	Derivation Functions	Derivation functions derive context-specific values that can be used to restrict data in CD views. The output of a derivation function can be further influenced by the use of parameters and filters.
2508	Review Booklet Application Generator wizard	You need to quickly build an analytical Fiori application, that puts multiple analytical queries in a single UI? All you need to do is to provide the queries, bundle them in an InA model, and you are good to just start the brand new Review Booklet Application Generator wizard to create the application. Afterwards, you can refine the configuration of your Review Booklet in an easy-to-use Fiori UI.
2508	Background Processing Framework - Final Version	Background processing is very important for business applications. A lot of logic can be executed asynchronously to improve performance. In scenarios where changes of the current LUW are communicated remotely, the asynchronous execution is even mandatory to be compliant with the LUW (due to implicit database commits). We want to simplify and improve the background processing by covering all required quality aspects: "exactly once", "in order", testability and an integration into the ADT cross trace for better debugging. In the next step of bgPF the AIF integration and its monitoring capabilities are provided. In addition, "in order" capabilities are supported.
2508	Fiori-based Multi-Dimensional Reporting - Cloud-compliant	You need to quickly build an analytical Fiori application, but simple list reporting doesn't meet your needs? Instead, you already built an analytical model with the new embedded analytics capabilities.

Available as of	Feature	Description
		developer extensibility and now want to unleash its full potential? In that case, we have the answer for you: Without writing a single line of additional code, you can simply expose your model as a fledged analytical application with a modern Fiori-based UI and real multi-dimensional "star" capabilities. ABAP Cloud-compliant creation of UIAD and App Variant (based on "MDA Generator")
2508	Outbound SQL via CDS External Entities : Static Access	External Entities allow to access external databases with the means of ABAP CDS & ABAP consumption External Entities directly in ASQL via the "dynamic access", we will now also call it "static access": The "static access" will allow modelling of these external data sources via CDS External Entities, External Schemata and consumption of those CDS External Entities via CDS View Entities. You can create view entity stacks on top of external data sources allowing to for example realize analytics scenarios combining local and remote data sources within one star schema.
2508	Generation Wizard: Outbound SQL via CDS External Entities	The ADT Generation Wizard allows to develop Outbound SQL scenarios more efficiently by generating the required CDS External Entities based on an imported CSN definition of the Remote SQL
2508	Fiori-based Multi-Report Applications	You need to quickly build an analytical Fiori application, that puts multiple analytical queries in context in a single UI? All you need to do is to provide the queries, bundle them in an InA Service and you are good to go. With the brand new "Review Booklet Application" wizard to generate the application. Afterwards, you can refine the configuration of your Review Booklet in an easy to use Fiori UI.
2508	CDS Hierarchy Generation Wizard	Defining hierarchies for the usage in a CDS view can become a complex task with the need to define the CDS hierarchy definition itself, but also for a directory, nodes, text views and more. With the hierarchy generation wizard, their creation becomes easier, quicker and less error-prone. You can now derive potential hierarchy nodes already from the start (on the master data view) and guide you through the creation process.
2508	Collaborative Draft	You can now add collaborative capabilities to your business object. The collaborative draft allows multiple users to work concurrently on the same draft instance.
2508	Cross-BO Transactional Enablement	Cross-BO scenarios can now be transactionally enabled and make use of an improved model implementation with additional syntax checks and error handling.
2508	Improved CDS Buffering - Buffer Delegation	Optimize the ABAP memory consumption for scenarios which make use of CDS Entity Buffering. Provide support for "delegated buffers" to allow buffering on CDS view entity-level by reusing buffers on table-level at runtime.
2508	RAP Factory Actions	RAP factory actions now return the http status 201 instead of 200 in case of a successful response.
2508	OData UI Service from Scratch	You can create all RAP-service related repository objects from scratch by using the OData UI Service from Scratch wizard. See: OData UI Service from Scratch
2508	Table Entities as Active Persistence	The RAP managed BO runtime now supports CDS table entities as the active persistence scenarios without draft.
2502	Nonstandard Operations for Associations	You can now define nonstandard operations for associations. These are link action, unlink action and inverse function.
2502	ABAP SQL Service - (read-only) Database Procedures	Have you already used the ABAP SQL Service / ODBC Driver for ABAP to expose ABAP-based data via SQL-APIs to external SQL consumers? We are now increasing the feature scope for the ABAP SQL Service. Besides exposing CDS view entities for read-only scenarios, it will be possible to

Available as of	Feature	Description
		"read-only" AMDP-based database procedures to be used by external ODBC clients. By default, both expose SQL and SQLScript-based business logic to external ODBC clients.
2502	CDS Updatable View Entities	With CDS Writeable View Entities, you can wrap existing DDIC Tables (or CDS Table Entities) with a native CDS signature that you can use with ABAP SQL (SELECT, INSERT, UPDATE, DELETE) in combination with DCL-based access control. CDS Writeable View Entities are projections of the underlying DDIC Table / CDS Table Entity. CDS Writeable View Entities will allow the following scenarios: • Enabling of "Cloudy-ready" READ/WRITE AMDPs, as they ensure "client safe" native SQLScript coding by preventing cross-client access • Exposure of a stable CDS signature for full SQL DML access but still allowing characteristics of the underlying persistence model • Enabling the usage of Dynamic View Caches based on released SAP content • Enabling of WRITE scenarios via the ABAP SQL Service / ABAP ODBC Driver
2502	Event-Driven Side Effects	You can now define an event to trigger a side effect in your RAP business object. Whenever the event is raised by the application server, the side effect is triggered and the defined target is executed. For more information, see Event-Driven Side Effects .
2502	Communication Management: Local APIs	The communication management will provide local APIs for the creation and update of communication arrangements, communication systems, communication users, and the upload of client certificates. A REST API will be provided for the usage in Developer Extensibility.
2502	Navigation From SAP Fiori Launchpad Towards Event Broker UI	It is now possible to navigate from the SAP Fiori launchpad of SAP S/4HANA Public Cloud towards the UI of the connected SAP Cloud Application Event Hub.
2502	Generic Audit Filter	You can now define generic audit filter(s) and assign auditor users accordingly to maintain audit logs or time-based restrictions. The key benefits: • Simple: Using only few but central dimensions, audit filters can be maintained and applied easily to restrict data access. • Powerful: Also advanced scenarios with multiple dimensions and intervals (organizational units, date-based) are supported. • Efficient: Time restriction filter logic is directly applied at the data selection level where the filter has been assigned. • Modern: Using the state of the art design concepts and technology. • Modular: Implemented on application-level complementing Z1 requirements with external tax auditor. See Generic Audit Filter.
2502	Editable Treeview with Draft Capabilities	In RAP, a treeview visualizes data modeled by a CDS hierarchy, using an SAP Fiori Element UI. Previously, you could model hierarchical structures and expose them as read-only treeviews in the SAP Fiori UI. Now you can also expose hierarchical structures as editable treeviews with draft capabilities.
2408	Annotation: ObjectModel.supportedCapabilities	You can use this annotation to check whether or not the UI use case is supported. If supported, either the creation of UI projections or alternatively the direct consumption in an UI service is possible. Otherwise, warnings or errors occur indicating that the UI usage is not supported and may lead to unexpected errors. For more information, see the annotation documentation of the domain in KTD.

Available as of	Feature	Description
2408	Annotation: Semantics.valueRange	You can use this annotation to inform consumers about the value range of a CDS element: specify minimal and/or maximal values and indicate whether these are exclusive or inclusive. The annotation can be applied to all fully ordered data types, i.e. Characters, Strings, Number. For more information, see the annotation documentation of the Semantics domain in KTI.
2408	Annotation: UI.dataPoint.title	You can use this annotation to define the title of a data point. For more information, see the annotation documentation of the UI domain in KTD.
2408	Using Dynamic Topics in Advanced Event Mesh	You can use dynamic topics for routing and filtering outbound events in the Advanced Event Mesh. Dynamic topics make use of the context attribute annotations to extend the corresponding event with further dynamic segments, derived from the corresponding annotated payload field. This enables detailed event filtering and routing, ensuring that events are delivered only to the interested components, improving efficiency and reducing unnecessary processing.
2408	Direct Push-Based Integration	With direct push-based integration, you can connect neighboring ABAP Systems with each other. In this scenario, the connected systems exchange data (like business events) by calling each other's RFC.
2408	Communication Targets for HTTP Outbound Communication	The usage of the new application destinations via outbound services of type communication is recommended when developing a new application with HTTP outbound communication. This allows developers to define and control which applications can use a particular application destination.
2408	Hello, ABAP Cloud! Page	Are you wondering where to find information about ABAP Cloud like documentation, release notes, roadmap or where to find examples with different levels of complexity? The ABAP Cloud landing page in the ABAP Development Tools for Eclipse shows all of this information and everything about ABAP Cloud in one place.
2402	Static and instance feature control for key user field extensions	In behavior definition extensions on the base layer, you can now define static and instance features for fields that have been added by means of key user extensibility.
2402	Generator for custom OData UI services based on RAP BO interfaces	You can now generate custom OData UI services based on RAP BO interfaces using a generator (ADT).
2402	Changed runtime behavior for determinations and validations	For performance reasons, determinations and validations that are assigned to a determination action (including prepare) are skipped during this determine action, if these determinations and validations have already been executed as part of a previous determine action, that has been executed on the same instance (draft or active) in the same transaction. Determinations and validations are skipped in the save sequence, if these determinations and validations have already been executed as part of a previous determine action that has been executed on the same instance in the same transaction. The performance optimizations outlined above are not applied in the following cases:

Available as of	Feature	Description
		• A modification performed after the first determine action triggers a determination. • A validation in the first determine action reports a failed key. • A determination/validation has been assigned using the addition always. Ensure that the determinations and validations of your business object follow the respective guidelines described in Determination and Validation Modelling. Neglecting these guidelines can lead to unexpected runtime behavior.
2402	CDS Simple Types	Simple types can be used for data typing on CDS level.
2402	Service Consumption Model	The Service Consumption Model now generates all data type information necessary for consuming remote services in one single ABAP proxy model class.
2402	RAP Default Values Function	You can now default fields and input parameters for operations in OData Services using Default Value Functions.
2402	RAP Treeview	You can now use Treeviews to define read-only hierarchical entity structures in OData V4.
2402	Large Objects	You can now define static feature control for large object fields in OData V2.
2402	Ancestor Associations Binding Fields	Ancestor associations binding fields now are filled automatically by the RAP framework.
2308	Local Consumption of Business Events	You can now consume business events in the same system as they are raised (local consumption).
2308	Strict Mode	It is now best practice to use strict(2) for all RAP BOs, except for: • Business Configuration (BC) BOs • BOs that consume a BC API or one of the following:

Available as of	Feature	Description
		◦ CL_EXCHANGE_RATES ◦ CL_NUMBERRANGE_INTERVALS ◦ CL_UOM_DIM_MAINTENANCE ◦ CL_UOM_MAINTENANCE ◦ XCO I18N APIs ◦ XCO Transport APIs
2308	SAP Object Types	You can now use SAP Object Types to semantically relate and group development artifact
2308	Verifying the Business Event Payload	You can now verify the payload of RAP Business Events using the Event Test Double Fram
2308	Derived Events	You can now define a custom payload for already existing RAP Business Events.
2308	Background Processing Framework: Initial Version	Background processing is important for business applications. Much logic can be executed asynchronously to improve performance. In scenarios where changes of the current LUW communicated remotely, the asynchronous execution is even mandatory to be compliant LUW (because implicit database commits). We want to simplify and improve the background processing by covering all required qual “exactly once”, “in order”, testability and an integration into the ADT cross trace for better In the first version, only limited monitoring capabilities are supported (bgRFC monitoring
2308	Extensibility Enablement of Service Definitions	You can now enable service definitions for extensibility by defining respective provider co
2308	Transactional Enablement of WHERE Clauses in CDS Projection Views	WHERE clauses in CDS Projection Views can now be used transactionally.
2308	Authorization Control for Actions	You can now define authorization for actions that overrule the authorization definition in authorization master. In BDEF extension, authorization control must be defined for every

Available as of	Feature	Description
		action.
2308	Extension Layering	You can now define behavior definition extensions that extend the behavior which is defined by behavior definition extensions.
2308	Defining a Leading Entity in RAP BOs	You can now define a leading entity in the service definition. In the service binding this entity is marked.
2302	Side Effects	You can now use side effects in your RAP behavior definition to trigger reloads based on update-enabled UI services.
2302	Integrating SAP Workflow Management	You can now integrate SAP Workflow Management in your RAP BO to use the workflow capabilities for process automation.
2302	Static and Dynamic Feature Control for Large Objects	You can now use dynamic and static feature control for large objects in OData V2.
2302	Define Own Authorization Context and Privileged Access	You can use the own authorization context to define the Identity and Access signature for own authorization context lists all authorization objects required during the RAP BO consumption runtime. In combination a RAP BO can offer privileged access for consumers to allow them to access certain authorization objects.
2302	Static Feature Controls for Fields in RAP BO Interfaces	You can define static feature control for fields in a RAP BO interface.
2302	abapGit Plugin	All RAP related development objects can be pushed and pulled with the abapGit plugin.
2302	Modeling Parameters for Non-Standard Operations	There is now documentation about how to model parameters and use their representation in RAP.
2208	Large Objects	You can enable your RAP application for maintaining large objects (LOBs). By doing so, you give users the option to incorporate external binary files or text files when editing entity instances.
2208	Business Events	You can now define and raise business events in a RAP BO or in a RAP BO behavior extension. Business events provide the opportunity of light-weight, decoupled process integration based on standard APIs and they are now a native part of the ABAP RESTful Application Programming Model. Using the RAP Business Event Bindings Editor, you can create RAP Event Bindings which

Available as of	Feature	Description
		provide a mapping between the definition of RAP Events via behavior definition (BDEF) and representation of Business Events.
2208	Extensibility-Enabling for RAP Business Objects	RAP extensibility is based on an opt-in approach to provide you with fine-granular control parts of your business object can be extended. You can easily enable your RAP BO and de use case extensions can be created. Using the respective CDS annotations, the BDEF syn Extend release contract , you can decide if you want to enable your BO for data model ext behavior extensions or node extensions depending on your business requirements.
2208	Extending RAP Business Objects	RAP enables you to build upgrade-safe and lifecycle-stable extensions using well-defined anchors of the original business object. You can extend the data model and behavior of th BO behavior and build data model extensions, behavior extensions or node extensions to to your business requirements.
2208	Repeatable Actions and Functions	You can now define actions and functions that can be executed multiply on the same inst same request.
2208	More Flexible Usage of Parameters	In Analytics with CDS, parameters are restricted to single values. More general input is of analytics. <code>@AnalyticsDetails.variable</code> can be used to create any variable in the a from a parameter.
2208	Business Object Interfaces	You can now define business object interfaces to introduce an additional abstraction laye BO and its business service projection layer With RAP BO interfaces, you can technically , requirements for stable consumption from the behavior and data model of the underlying object. You can release RAP BO interfaces for C1 with visibility Use in Cloud Develop enable EML-consumption between different software components.
2208	Unmanaged/additional save with full data	It is now possible to use the option <code>WITH FULL DATA</code> for an unmanaged/additional save BO. With this option enabled, when an instance is modified, not only the changed fields b provided into the <code>save_modified</code> method.
2202	Filter Tree API for Unmanaged Queries	You can now use the method <code>get_as_tree</code> of the interface <code>if_rap_query_filter</code> i query to retrieve the filter conditions in an easily parsable expression tree.
2202	Business Object Interfaces	You can now use business object interfaces to introduce an additional abstraction layer b base BO and its business service projection layer. With RAP BO interfaces, you can techni requirements for stable consumption from the behavior and data model of the underlying
2202	Add Dimension to Query at Runtime (New)	With 2021 support package 1, some BI-tools like WebDynpro Grid offer the possibility to a from the cube view with the "free"-axis at runtime. With this, it isn't necessary to include : dimensions into the CDS analytical query view that should be available at runtime. Only t common ones can be add to the query view. With this the UI view becomes clearer and pe slightly better. Since the cube may contain sensible dimensions that not all users are allo query has to be enabled for this feature with the header annotation <code>@Analytics.Settings.runtimeExtensibility.allowed: true</code> . Dimensions be exposed at runtime can be excluded with the annotation <code>@Analytics.Setting.excludedFromRuntimeExtensibility</code> on field level in th cube view.

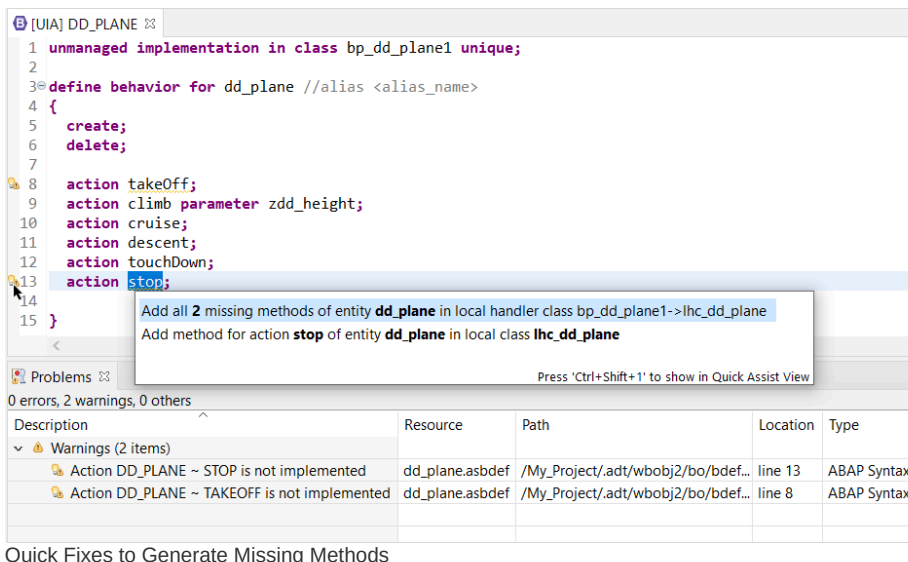
Available as of	Feature	Description
2202	Changes in the Consumption or Provisioning of RAP BOs	You can find an overview of changes in the RAP framework in SAP Note 2943761  .
2111	Using Client-Independent Tables in Transactional Apps with Draft Capabilities	You can now use client-independent tables as data sources for managed transactional applications.
2111	Multiple Message Targets in OData V4 RAP Services	OData V4 now supports multiple message targets. In the reported structure of RAP hand can define multiple elements in the derived type component <code>%element</code> . These elements by OData and are highlighted on a Fiori V4 UI.
2111	Using <code>UI.hidden</code> Annotation for Action Input Parameters	You can now use <code>UI.hidden</code> with action input parameters fields to prevent the input field displayed on the UI, while leaving the input parameter fields available for a WebAPI client
2111	Read Access Logging (RAL)	You can now use Read Access Logging for your RAP services. With this feature you can manage read accesses.
2111	Service Binding Errors in ATC Checks	ATC checks now also expose errors that are related to a RAP service. These errors are displayed in the service binding. They can now also be retrieved by running ATC checks for the service.
2111	Late Numbering for Managed Implementation Type	Late numbering can now also be used for the managed implementation type.
2111	Consumption of released Business Objects	You can now consume released Business Objects via C1 released RAP BO Interfaces.
2111	Instance Factory Actions	You can now use instance factory actions to model actions that create new instances based on an existing instance. Instance factory actions can be useful if you want to copy specific values of an existing instance.
2111	Operations in Reported Response Parameter	When providing the reported response parameter in methods of the RAP behavior pool, you can specify the operation the provided message belongs to. The derived type component <code>%operation</code> provides the operations. The relevant one must be marked with <code>if_abap_behv=>mark-on</code> .
2111	Generate ABAP Repository Objects	With the Generate ABAP Repository Objects wizard, you can now generate RAP artifacts for a managed transactional application based on a database table.

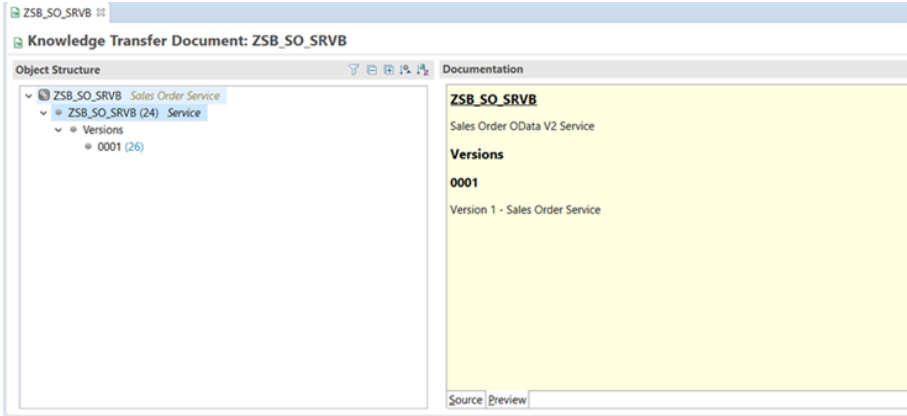
Available as of	Feature	Description
2111	Semantic Versioning for OData V4 Service	In the service binding editor, you can now specify semantic versioning in the service versioning service by adding minor and patch version.
2111	Changes in the Consumption or Provisioning of RAP BOs	You can find an overview of changes in the RAP framework in SAP Note 3107113  .
2108	Authorization and Feature Control Checks in Unmanaged Business Objects	The implementation for authorization and feature control in the corresponding RAP handler now also called during every modify operation in business objects with unmanaged implementation.
2108	Delegating Authorization Checks	You can now use the syntax <code>authorization: update</code> in the behavior definition to delegate the authorization check for delete, create-by-association, or action operation to the authorization check of the update operation.
2108	Additional Implementation for Draft Actions	You can now add an additional implementation for all draft actions to extend the RAP handler cycle. The additional implementation is defined in the behavior definition with <code>with additional implementation</code> and implemented in the behavior pool.
2108	Runtime Changes for Cleanup Execution	The Cleanup is now called after a <code>ROLLBACK/COMMIT ENTITIES</code> statement also for asynchronous instance access including lock operation, precheck, instance authorization, and instance deletion.
2108	Idempotency for OData V4 services	Idempotent behavior is now supported for OData V4 services. With this feature, it is guaranteed that synchronous messages are delivered exactly once.
2108	RAP Reuse Data Elements	You can now use reuse data elements for the administrative fields in RAP applications.
2108	Unit and Currency Types as Built-In Entity Sets	In OData V2 services, unit and currency types are automatically added to the service metadata entity sets. With this feature, unit and currency values are always displayed correctly and regarding decimals on a UI.
2108	Fail Causes for failed in RAP Handler Methods	The fail causes <code>disabled</code> , <code>readonly</code> and <code>dependency</code> can now be provided in the response parameter <code>failed</code> of RAP handler methods.

Available as of	Feature	Description
2108	Semantic Versioning for OData V2 Service	In the service binding editor, you can now specify semantic versioning in the service versioning service by adding minor and patch version.
2108	Using Service Binding Editor for SQL Service	You can now expose a RAP business service as an SQL service. The binding type must be set to SQL when creating the service binding for the service.
2108	Save Actions for Managed BOs	You can now use actions that are executed only during the save sequence. On-save action behavior definition with the following syntax: <code>[static] save action actionName [parameter parType] [result card]</code>
2108	Changes in the Consumption or Provisioning of RAP BOs	You can find an overview of changes in the RAP framework in SAP Note 3070134 
2105	Adding Actions and Functions in a Projection Behavior Definition	You can now add actions and functions to a projection behavior definition. You can also add annotations for the new actions and functions.
2105	Using Strict Mode	You can now implement your RAP business object in strict mode to ensure that your EML and implemented in accordance with the RAP best practices.
2105	RAP BO Contract	Rules on how to implement a RAP BO are now available in the RAP documentation.
2105	Developing a RAP BO that Supports Multi-Inline-Editing on a Fiori UI	You can now implement a RAP business object with a singleton mechanism to support multi-inline editing on a Fiori UI.
2105	Using AUTO_FILL_CID Option	In EML statements for static operations, you can now use the option AUTO_FILL_CID. This ensures that %cid is filled with a value automatically generated by the framework.
2105	Flag always for Determinations/Validations on Subentities	You can now use the always flag in determine actions also for determinations or validations on subentities.
2105	COMMIT_ENTITIES in Simulation Mode	You can now use the COMMIT_ENTITIES statement in simulation mode. This triggers the commit of RAP BOs without actually saving data.

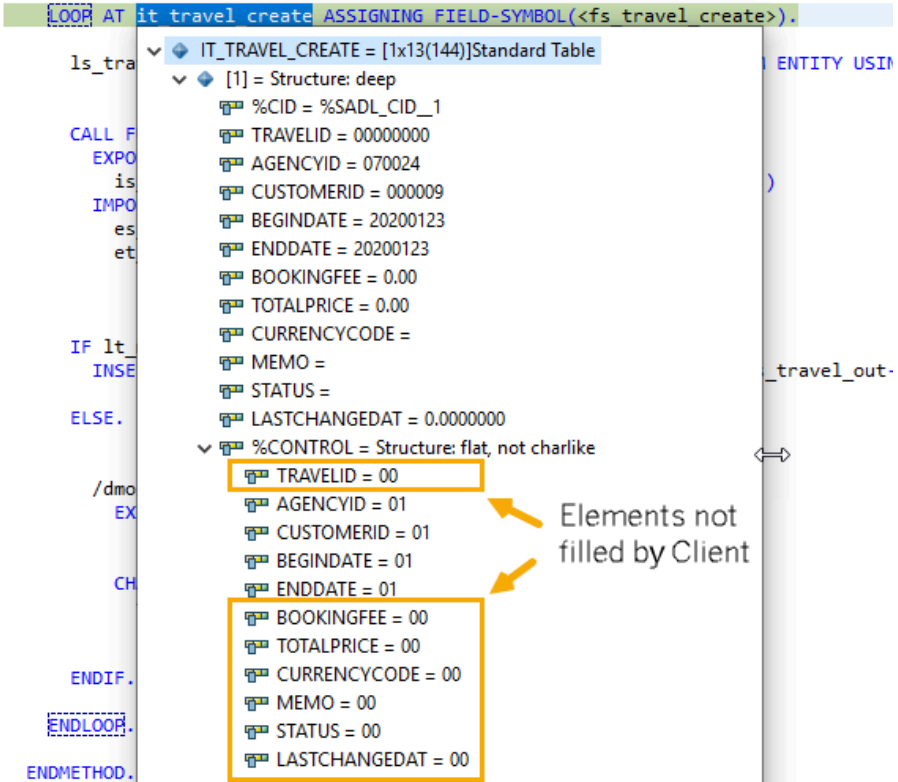
Available as of	Feature	Description
2105	Mapping for Action and Function Parameters	You can now map names of action and function parameters that are represented as abstract ABAP dictionary tables and structures.
2105	Suppress Fields in Transactional Handling	You can now remove fields that are only added to a RAP BO data model for technical reasons and do not have a required appearance in all behavior-definition-related generated components, for example in derived EML fields. The annotation <code>@Consumption.hidden:true</code> is a prerequisite for this. Syntax in CDS projection or abstract behavior definition: <pre>projection abstract; ... define behavior for BO_Entity { field (suppress) TechnicalField; ... }</pre>
2105	Flexible Association Modeling	It is now possible to define associations to CDS projection view entities in CDS view entity definitions. This enables model relationships between consumption / projection layers. It is not supported to retrieve fields or other associations via such association.
2105	Field Control for Modify-Enabled Fields	Virtual or denormalized fields are <code>readonly</code> by default. You can modify-enable them in a behavior definition with the syntax <code>field (modify)</code> . Now, you can also add field characteristics to such fields.
2105	Early Unmanaged Numbering in Unmanaged Business Objects with Draft	You can now use early unmanaged numbering in unmanaged business objects with managed drafts.
2105	Harmonization of Commit Work and Commit Entities	<code>Commit Work</code> and <code>Commit Entities</code> have been harmonized, so that <code>Commit Work</code> and <code>Commit Entities</code> can be used in the same RAP save sequence.
2105	Possibly Incompatible Changes for ABAP RESTful Application Programming Model	You can find an overview of possibly incompatible changes for the ABAP RESTful Application Programming Model in SAP Note 3050328  .
2102	Implementing Unmanaged Early Numbering in Managed Business Objects	<code>CREATE</code> .
2102	Retrieving Permission Information from Business Objects	You can now use the EML statement <code>Get Permissions</code> to retrieve information about field and authorization permissions from business objects on global and instance level.

Available as of	Feature	Description
2102	Implementing Cleanup in Managed Business Objects	You can now implement the cleanup method in managed BOs if an additional or an unma defined in the behavior definition. In this implementation you can handle legacy coding au objects. The cleanup for managed BOs must be defined in the behavior definition and imp redefined method in the local saver class.
2102	Possibly Incompatible Changes for ABAP RESTful Application Programming	You can find an overview of possibly incompatible changes for the ABAP RESTful Applcal Programming in SAP Note 3017989  .
2011	Using Service Binding Editor for OData V4 Service	You can now expose a RAP business service as an OData V4 service. The binding type mu creating the service binding for the service.
2011	Using Service Binding Editor for InA Service	You can now expose CDS Analytical Queries via Information Access (InA) service using se The binding type must be chosen while creating the service binding for the service. To exp Analytical Query as an InA service, the CDS view must be added to the service definition. definition must be used to create the service binding.
2011	Setting Default Parameter Values in OData V4 UI Services	It is now possible to model a default value for action input parameters in OData V4 UI ser default value is defined with the annotation <code>@Consumption.defaultValue: '<val</code> abstract CDS entity that defines the input parameter. This feature is only available for UI services with OData V4.
2011	Nested Determinations on modify	It is now possible to trigger a determination on modify by another determination on modi
2011	Always Flag in Determine Actions	You can now use the flag <code>always</code> for determinations and validations that are assigned to action. When the determine action is called, determinations and validations with this flag regardless of their trigger conditions.
2011	Global Feature Control	You can now define global feature control for feature control that is independent of a busi instance.
2011	Global Authorization Control	You can now define global authorization control to check authorization for incoming requ particular for static operations.
2011	Augmenting Incoming Requests	You can now define augmentation in your projection behavior definition to enhance incon with custom implementation, for example with default values.
2011	Non-Locking Actions	You can now suppress the locking mechanism for action execution with the syntax <code>(lock</code> action definitions in the behavior definition.

Available as of	Feature	Description
2011	Possibly Incompatible Changes for ABAP RESTful Application Programming	You can find an overview of possibly incompatible changes for the ABAP RESTful Application Programming in SAP Note 2943761 .
2011	Quick Fixes For Creating Behavior Implementations	In case of missing implementation classes or missing methods, corresponding warnings displayed in the Behavior Definition Editor and in the Problems View. You can apply Quick Fixes to generate the entire implementation class or missing methods.  The screenshot shows the Behavior Definition Editor for entity <code>dd_plane</code>. A warning is displayed: "Add all 2 missing methods of entity <code>dd_plane</code> in local handler class <code>bp_dd_plane1</code> -> <code>lhc_dd_plane</code>". The quick fix popup suggests: "Add method for action <code>stop</code> of entity <code>dd_plane</code> in local class <code>lhc_dd_plane</code>". Below the editor, the Problems View shows two warnings: "Action DD_PLANE ~ STOP is not implemented" and "Action DD_PLANE ~ TAKEOFF is not implemented". Quick Fixes to Generate Missing Methods
2008	Draft Support for RAP Business Services	The ABAP RESTful Application Programming Model now provides draft support for your business services. Draft-enabled business objects ensure that transactional data does not get lost interrupted or continued from a different device. You draft-enable your business object by using the keyword <code>with draft</code> in the behavior definition. Draft capabilities can be included for both business object implementation types, managed and unmanaged.
2008	Draft Scenario on GitHub	You can now download a RAP managed business service with draft from GitHub. The development objects are now available as part of the ABAP Flight Reference Scenario that you can download into your ABAP system.
2008	Controlling Active Data Access in Draft Business Objects	You can now define a designated field, the <code>total ETag</code> field, that controls concurrent access to data in draft business objects. The RAP runtime framework checks its value on every transaction draft to active data to prevent concurrent BO consumers from overwriting each other with new information. The <code>total ETag</code> is defined in the behavior definition and is updated by the RAP runtime framework automatically if annotated with the annotation <code>@Semantics.systemDateTime.lastUpdated = true</code> in the corresponding CDS view.
2008	Working with Determinations and Validations	Determinations and validations are now processed by the determinations-validations-management framework. You can now define determinations and validations with all three standard operation triggers: <code>create</code>, <code>update</code>, and <code>delete</code>, and in combination with field triggers. In addition, the method definition in the behavior pool uses different syntax: <pre> METHODS method_name FOR VALIDATE ON SAVE IMPORTING keys FOR AliasedEntityName~ValidationName. </pre>

Available as of	Feature	Description
		<code>METHODS method_name FOR DETERMINE ON MODIFY SAVE IMPORTING keys FOR AliasedEntityName~ValidationName.</code>
2008	Executing Determinations and Validations on Request	New determine actions allow the business object consumer to execute determinations without fulfilling the trigger conditions. You can now assign determinations and validator determine action that is called by a consumer's trigger, just like any other actions.
2008	Prechecking Modify Operations	You can now prevent illegal changes from reaching the application buffer by prechecking operations. The precheck method is called during runtime before the assigned modify operation removes all input from the modifying request for which the condition in the precheck is not met.
2008	Authorization Control	Consumer hints in the OData service metadata now include authorization control by a path description for the respective operation. That means, if you define authorization control for your business authorization-relevant entity sets receive a path description instead of a static information about the ability to be updated or deleted. In addition, the corresponding properties are included in the entity type.
2008	RAP Runtime Checks	Stricter RAP runtime checks are introduced to help you make your RAP implementation more friendly, and compatible to further RAP features. These runtime checks affect - creating and updating read only fields via EML, - executing modify calls in a late save phase, - consuming actions or functions with parameters exceeding the defined length.
2008	Documenting Service Binding	You can now provide documentation using knowledge transfer documents (KTD) for service bindings. 
2008	Feature Control	You can now use <code>mandatory: create</code> to assure that a field is filled in during a create operation and <code>readonly: update</code> to assure that the field value isn't changed during modify operation.
2008	Possibly Incompatible Changes for ABAP RESTful Application Programming	You can find an overview of possibly incompatible changes for the ABAP RESTful Application Programming in SAP Note 2943761 .
2005	Implementing Own Locking Logic for Managed Business Objects	You can now define an unmanaged lock in a managed scenario. The unmanaged lock mechanism can be defined in the behavior definition with the syntax <code>lock master unmanaged</code> and in the behavior pool in the method <code>FOR LOCK</code> , just like in an unmanaged scenario. The method is invoked during runtime.

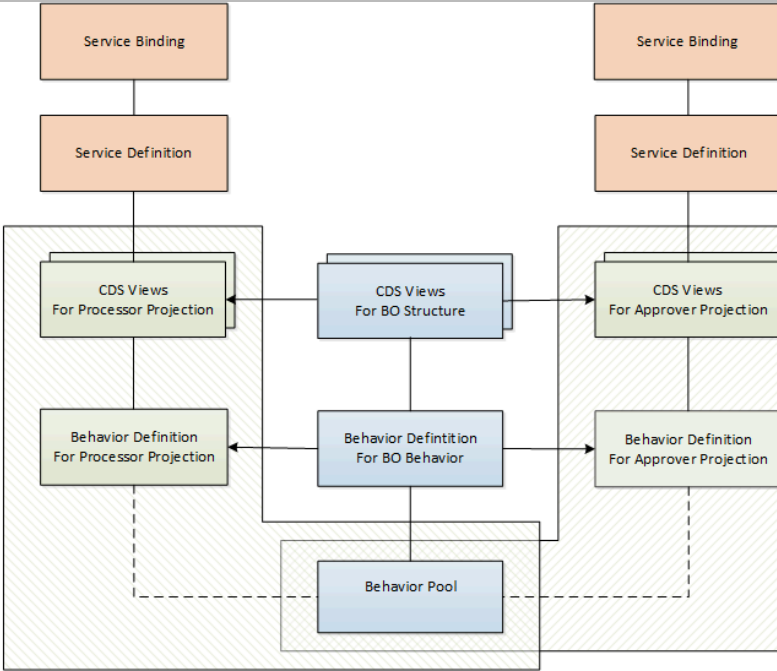
Available as of	Feature	Description
2005	Using <code>dependent by _Association</code> in Behavior Definition	You can now use the syntax <code>dependent by _Association</code> for lock, ETag, or authorization dependent entities. The association to the respective master entity must be explicitly specified in the behavior definition and implemented when using the unmanaged scenario.
2005	Using Client-Independent Tables in Managed Scenarios	The RAP managed BO runtime now supports client-independent database tables.
2005	Dynamic Operation Control for Create By Association	You can now dynamically control the create by association operation. The syntax for defining feature control is the following: <pre>_association {create (features: instance); }</pre> Like other feature controls, the control for the create by association must be implemented in the behavior pool with the method <code>FOR FEATURES</code>.
2005	New Read-Only Associations in CDS Projection Views	It is now possible to define new read-only associations in the projection view. These associations are reused to display additional information in a UI service, for example analytical data which is related to the transactional basic business object. With associations in projection views, you can manage service-specific relationships. The syntax for the association definition is the same as in CDS views: <pre>association [min..max] to TargetEntity [as _Alias] on OnCondition</pre>
2005	Filtering for Null Values in OData V2 Services	OData does not foresee initial values for the following EDM data types: • Boolean • GUID • Time • DateTime • DateTimeOffset. Fiori Elements UIs offer the <code>empty</code> operator in filter bars for these fields. Although they are empty in the UIs, the actual value sent to the client is <code>null</code>. To be able to select records with empty values for these data types, the client filters null values. The filter is transformed to filter for initial values for the backend. Real null values cannot be stored in the database by filtering for them in OData V2.

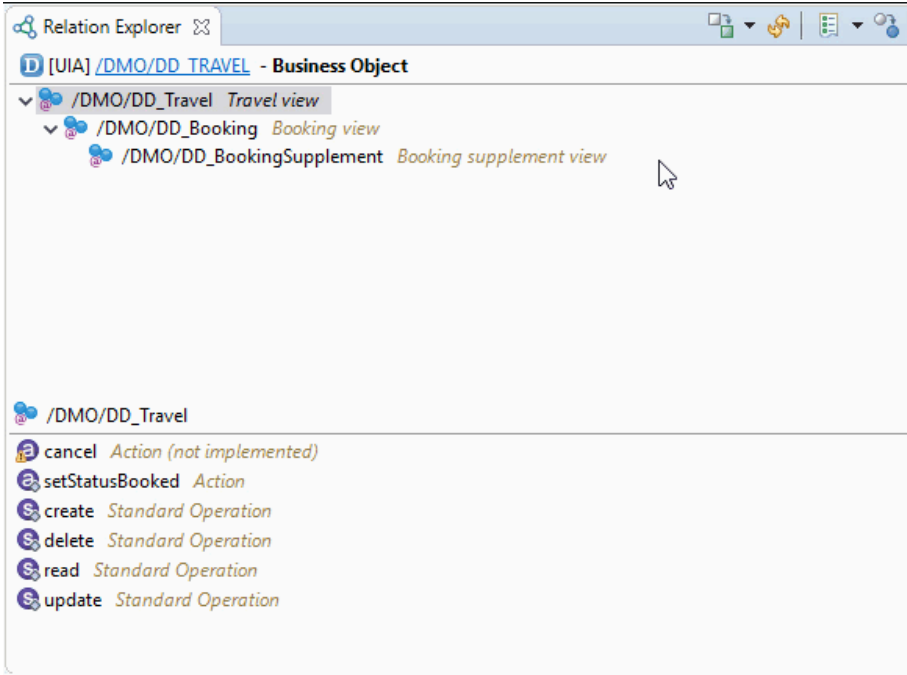
Available as of	Feature	Description
2005	Understanding Runtime Processes	The concepts section in the ABAP RESTful Application Programming Model has been enhanced with interactive diagrams so you can get a better understanding of the runtime processes for
2005	Control Structure for Import Parameter in CREATE Operations	The control structure <code>%control</code> for import parameters of <code>CREATE</code> and <code>CREATE BY</code> As operations is now filled according to the elements that were sent by the OData client or via the EML call. In particular this means that the application developer knows which elements are filled for the create. ❖ Example In the following example the elements <code>agencyid</code>, <code>customerid</code>, <code>begindate</code>, and <code>enddate</code> are filled by the OData client and thus are marked in the control structure, while the others are not. <pre> METHOD create_travel. DATA lt_messages TYPE /dmo/t_message. DATA ls_travel_in TYPE /dmo/travel. DATA ls_travel_out TYPE /dmo/travel. LOOP AT it_travel_create ASSIGNING FIELD-SYMBOL(<fs_travel_create>). ls_travel_in = <fs_travel_create> CALL FUNCTION 'EXPONENTIAL_IMPORT' EXPORTING is_travel_in = ls_travel_in IMPORTING is_travel_out = ls_travel_out EXCEPTIONS ... IF lt_messages IS NOT INITIAL. INSE ELSE. /dmo EX CH ENDIF. ENDLOOP. ENDMETHOD. </pre>  Provided Elements in Create for Travel
2002	Automatically Drawing Primary Key Values for Managed Business Objects	The managed runtime framework can now automatically generate key values in scenarios if managed numbering is defined in the behavior definition. Syntax for defining managed numbering in the behavior definition:

Available as of	Feature	Description
		<pre> [implementation] managed [implementation in class ABAP_CLASS [unique define behavior for Entity [alias AliasedName] lock (master dependent()) ... { ... field ([read only,] numbering:managed) KeyField1, [KeyField2]; ... } </pre>
2002	New Options for Action and Function Results	The BO runtime framework now supports results for actions and functions other than \$S can be entities of the same BO the action is defined for, entities of other BOs, or result str differentiate between result entities and result structures the syntax element entity has introduced. In addition, you can now create actions, for which the action consumer can decide whether shall be returned completely or only parts of it, for example the keys only. Such an action marked with the keyword selective in the behavior definition. <pre> define behavior for CDSEntity ... ** Action with result entity action ActionName result [cardinality] entity OutputEntity ** Action with result structure action ActionName result [cardinality] OutputStructure ** Action with selective result action ActionName result selective [cardinality] entity OutputEnt ... </pre>
2002	Reporting Messages in ADJUST_NUMBERS and SAVE	The implicit returning parameter REPORTED is now available for the methods adjust_n save. By filling this parameter you can report information or success messages after the return in the save sequence.
2002	Documenting Behavior Definitions	Now you can document behavior definitions in the Knowledge Transfer Document editor From the Project Explorer, select the behavior definition you want to document. Use the create a knowledge transfer document.

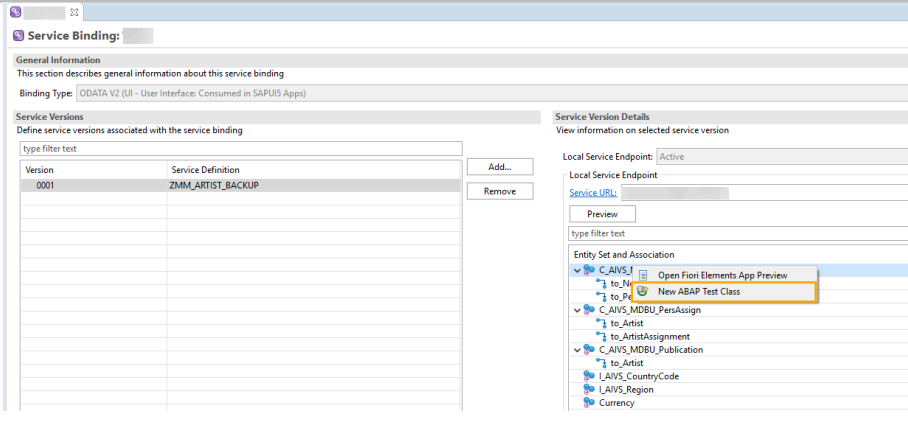
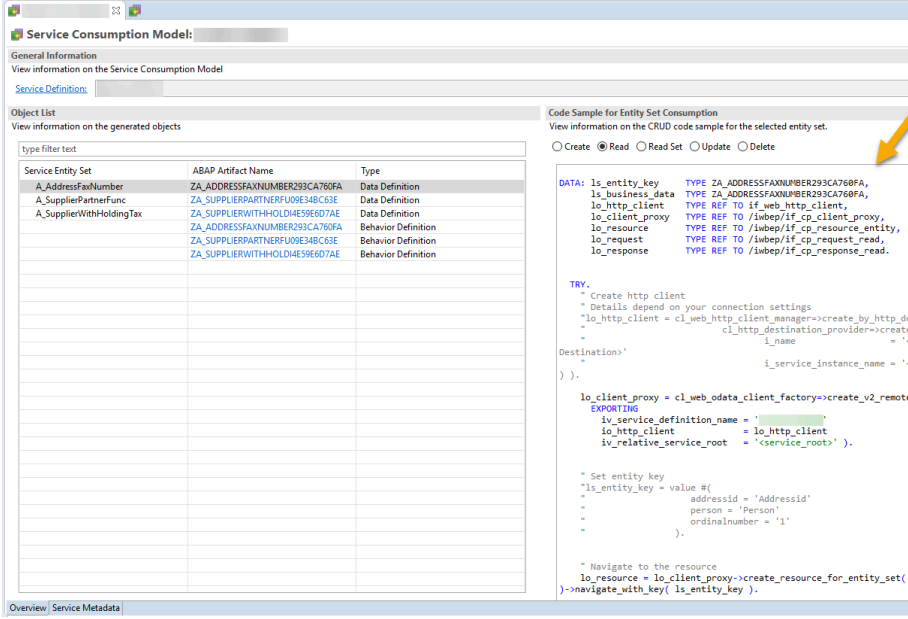
Available as of	Feature	Description
		Knowledge Transfer Document Editor
1911	Defining the Service Namespace for OData Services	You can now define the OData service namespace in service definitions with the annotation <code>@OData.schema.name</code> .
1911	Filling the Result Parameter for Actions	A runtime check is introduced to enforce the proper assignment of result values in action implementations when result parameters are declared in the corresponding action definition. Actions that returned the input entity if the result parameter was not filled in the action implementation before 1911, they now return the value of the result parameter as specified in the action definition exactly according to its implementation. To avoid that a Fiori UI shows initial values, if nothing is returned, you must fill the result parameters in actions.
1911	Navigating More Than One Step in OData Requests	It is now possible to navigate to associated entities using more than one navigation for both read and write services. ❁ Example To request the booking supplements for the booking entity with BookingID 5 of the travel entity with TravelID 1, use the following syntax. <pre><service_URL>/Travel('1')/to_Booking('5')/to_BookingSupplement</pre> You can link an unlimited number of entities in one OData request.
1911	Outsourcing UI Metadata in Metadata Extensions	You can now use metadata extensions to define CDS annotations outside of the corresponding definition of the CDS projection view. The use of metadata extensions allows the separation of the data model from domain-specific semantics, such as UI-related information and user interface consumption.
1911	Using Type and Control Mapping	You can now define a mapping contract for applications that include unmanaged or managed objects based on CDS entities on the one hand, and legacy data types that are generally not supported by SAP on the other. This allows you to implement the behavior or parts of it. The general syntax for field and control (information) mapping in a behavior definition is as follows:

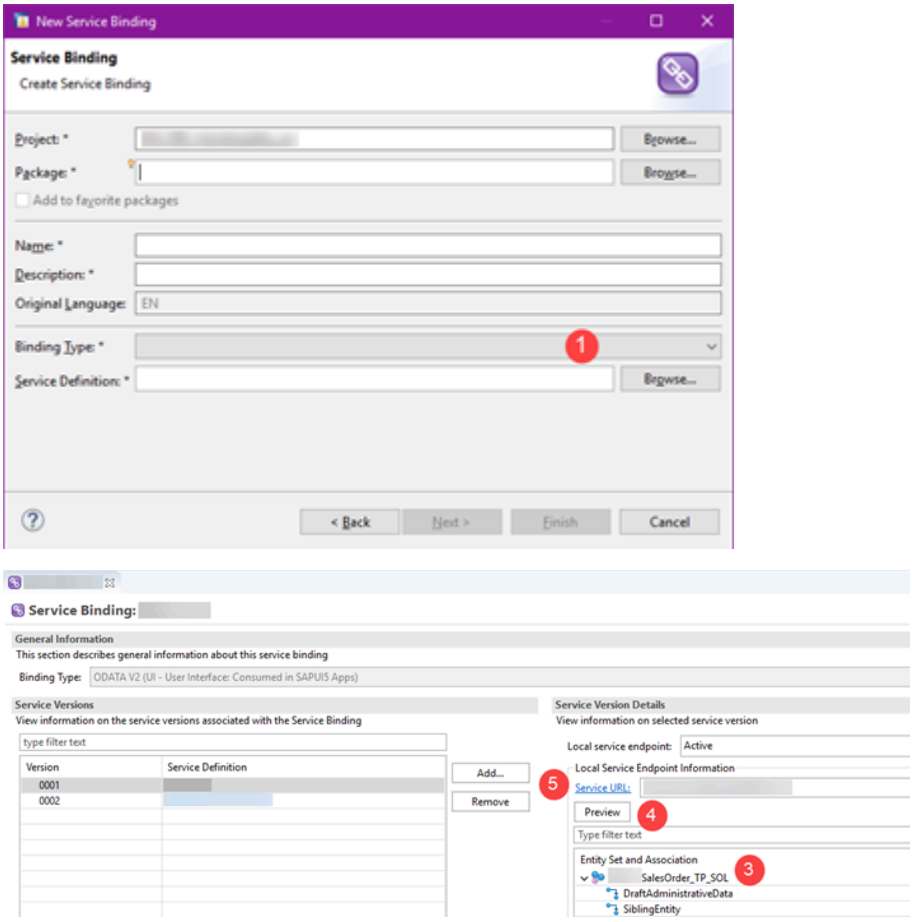
Available as of	Feature	Description
		mapping for LegacyType control ControlType corresponding <pre> { EntityField1 = LegacyField1; EntityField2 = LegacyField2; ... }</pre>
1911	Integrating Additional Save in Managed Business Objects	You can now integrate Additional Save for each entity of a given business object with managed implementation type. This is done in the behavior definition of the business object by adding <code>with additional save</code> . The actual implementation of Additional Save takes place in class as part of the behavior pool.
1911	Integrating Unmanaged Save in Managed Business Objects	You can now integrate Unmanaged Save within the transactional life cycle of managed business objects. In order to integrate Unmanaged Save into the save sequence as a part of the managed implementation, you must first add the corresponding syntax (<code>with unmanaged save</code>) to the behavior definition and then implement the saver handler method as a part of the behavior pool.
1911	Using Groups for Large Implementations	Until now, the implementation of business object entity's operations and actions had to be implemented in a single Local Type include of the behavior pool associated with that entity. Since the Local Type can only be changed by one developer at a time, the efficiency of development would be significantly reduced in case of larger implementations. As a solution, the groups can now be used to divide operations and other implementation-relevant parts of the business logic into several groups for better management and implementation.
1911	Using Virtual Elements	You can now define elements in CDS projection views that are not persisted on the database. Virtual elements are defined with the new syntax <code>virtual</code> . Their calculation is implemented in a separate ABAP class, which is called during runtime via an ABAP code exit for virtual elements.
1911	Using Service Binding Editor	In the Service Binding editor, the preview of the Fiori elements app now automatically opens in the Fiori language.
1908	New URL for the Documentation of the ABAP RESTful Programming Model in SAP Cloud Platform ABAP Environment	The documentation of the ABAP RESTful Programming Model for SAP Cloud Platform ABAP Environment on the SAP Help Portal has been moved.
1908	Developing New Managed Transactional Apps	The ABAP RESTful Programming Model now supports the managed implementation type for developing new transactional apps. This scenario aims at use cases to develop new transactional applications. All required standard operations must only be specified in the behavior definition to obtain a business object. The business logic is implemented using actions, validations and determinations.
1908	Business Object Projection	You can now use the ABAP-native approach to project and to alias a subset of the business object as a specific business service. The projection enables flexible service consumption as well as the creation of service designs. With projections, it is possible to project one business object for different role-based UIs. For more information, see Developing a Projection Layer for Flexible Service Consumption. The travel business object is exposed for a processor Fiori UI and for an approver Fiori UI.

Available as of	Feature	Description
		 BO Projection for Processor
1908	Implementing an Unmanaged Query	A new API is available to implement an unmanaged query in a query implementation class. When an unmanaged query is requested, the request is delegated to the query implementation class, which implements the <code>select</code> method of the interface <code>IF_RAP_QUERY_PROVIDER</code>. The new interface provides query capabilities, such as paging, filtering, sorting, or counting. It replaces the <code>IF_A4C_RAP_QUERY_PROVIDER</code>, which is deprecated as of 1908. The implementation of an unmanaged query with the interface <code>IF_RAP_QUERY_PROVIDER</code> is a new topic in the common task section.
1908	Server-Side Paging	Server-side paging has now been implemented to improve performance for OData calls. If a client does not provide paging query options, the default paging restricts the response to 5000 records. If <code>\$top</code> is set to values greater than 5000, the response is reduced to 5000 data records. Following from this, unmanaged queries require at least the implementation for paging. If corresponding methods are not implemented and the unmanaged query does not return information, there will be a dump during runtime.
1908	Additions to the Unmanaged Reference Scenario	The sources for the unmanaged reference scenario have been updated with the latest features. Explore the new features in the sources that you can download from GitHub and in the related guide on how to develop unmanaged transactional apps based on existing applications. Mapping CDS names to database field names By defining a mapping specification in the behavior definition, you can use the mapping in the behavior implementation to write records to the database that have a description of the names of the database table fields and the CDS view field names. Message handling The behavior processing framework provides a message object that can be used for handling in the behavior pool. New message handling has been added to the unmanaged reference scenario. The corresponding methods are implemented in an auxiliary class that implements <code>cl_abap_behv</code> and called from the behavior handler.
1908	Authorizations Checks for Modifying Operations in Managed Business Objects	To protect data from unauthorized read access, the ABAP CDS provides its own authorization checks based on a data control language (DCL). The authorization checks for read operations apply to all the results returned by an entity to those results you authorize a user to see.

Available as of	Feature	Description
		For business objects that are implemented for managed contract, also modifying operational standard operations create, update, delete, create by associations, and actions must be controlled to prevent unauthorized access. With the current release, the instance-based authorization control is implemented.
1908	Adding Dynamic Feature Control	Apart from static feature control, you can now also use dynamic feature control. In this case, you can control the availability of a state of the node instance if certain elements or actions are available. Dynamic feature control is defined in the behavior definition: <pre> ... define behavior for /DMO/I_TRAVEL_M field (features : instance) travel_id; action (features: instance) acceptedTravel result [1] \$self; ... </pre> The implementation for the control must then be implemented in the respective method pool. For example, for this behavior definition, you can implement that the field <code>travel_id</code> is mandatory on create, but read-only on the update operation. The action can be implemented if the travel entity is already set to accepted, but enabled if it is not yet accepted.
1908	Exploring Business Objects	A business object consists of hierarchical connected entities. The behavior for each entity is defined in the behavior definition object and implemented in the behavior classes. In the Relation Explorer, you can see the structure and behavior of a certain business object independent of the technical location. You can navigate to all the entities and the corresponding behavior (definition and implementation). Sometimes you might be interested in more CDS-specific aspects and want to see access control and test classes. You can achieve this by switching the context from Business Object to Core context, as you can see in the following animation. Switching the context in the Relation Explorer  Relation Explorer provides much more features such as further contexts, for example, to use objects for a certain class.
1908	Service Consumption Model Wizard	In the service consumption model creation wizard: ETag support can now be selected for any entity set. If ETag support is marked in the wizard, the ETag support checkbox is selected by default.

Available as of	Feature	Description																																																																																																																			
		2. An entity set can now be selected for the generation. You can only edit the ABAP artifact name of the entity set that you've selected for generation. 3. Issues in an entity set are displayed and these entity sets cannot be selected for generation. New Service Consumption Model Define Entity Set Select entity sets for generation, edit the ABAP artifact name and select ETag support. Entity sets with issues are not generated. type filter text <table><thead><tr><th></th><th>Service Entity Set</th><th>ABAP Artifact Name</th><th>ETag Support</th><th>Issue</th></tr></thead><tbody><tr><td>2</td><td>☒ A_BusinessPartner</td><td>ZA_BUSINESSPARTNER73911184C</td><td>☒ 1</td><td></td></tr><tr><td></td><td>☒ A_BusinessPartnerAddress</td><td>ZA_BUSINESSPARTNERADDRESS</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_BusinessPartnerBank</td><td>ZA_BUSINESSPARTNERBA10EF53F5...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_BusinessPartnerRole</td><td>ZA_BUSINESSPARTNERRO84E9222...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_BusinessPartnerTaxNumber</td><td>ZA_BUSINESSPARTNERTAB0E4B215...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustSalesPartnerFunction</td><td>ZA_CUSTSALESPARTNERF159B283...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_Customer</td><td>ZA_CUSTOMERD6C7ECF4B8</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerCompany</td><td>ZA_CUSTOMERCOMPANY3D79525...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerDunning</td><td>ZA_CUSTOMERDUNNING55ECBD...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerSalesArea</td><td>ZA_CUSTOMERSALESAREA96C421...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerSalesAreaTax</td><td>ZA_CUSTOMERSALESAREA7A7938...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_CustomerWithHoldingCompany</td><td>ZA_CUSTOMERWITHHOLDI5302D...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_Supplier</td><td>ZA_SUPPLIER617002ED75</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_SupplierCompany</td><td>ZA_SUPPLIERCOMPANY1BBDA39...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_SupplierDunning</td><td>ZA_SUPPLIERDUNNING8194E0C2D6</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_SupplierPartnerFunction</td><td>ZA_SUPPLIERPARTNERFUB236BA0...</td><td>☐</td><td></td></tr><tr><td></td><td>☒ A_SupplierPurchasingOrg</td><td>ZA_SUPPLIERPURCHASIN8CD8D1...</td><td>☐</td><td>3</td></tr><tr><td></td><td>☒ A_SupplierWithHoldingTax</td><td>ZA_SUPPLIERWITHHOLDIACA225...</td><td>☐</td><td></td></tr><tr><td></td><td>☐ A_BPContactToAddress</td><td>ZA_BPCONTACTTOADDRESS</td><td>☐</td><td>Type Edm.DateTime of element</td></tr><tr><td></td><td>☐ A_BPContactToFunctionAndData</td><td>ZA_BPCONTACTTOFUNCANDDEPT</td><td>☐</td><td>Type Edm.DateTime of element</td></tr><tr><td></td><td>☐ A_BusinessPartnerContact</td><td>ZA_BUSINESSPARTNERCONTACT</td><td>☐</td><td>Type Edm.DateTime of element</td></tr><tr><td></td><td>☐ A_AddressHomePageURL</td><td>ZA_ADDRESSHOMEPAGEURL</td><td>☐</td><td>Type Edm.DateTime of element</td></tr></tbody></table> Select All Deselect All ? < Back Next > Finish		Service Entity Set	ABAP Artifact Name	ETag Support	Issue	2	☒ A_BusinessPartner	ZA_BUSINESSPARTNER73911184C	☒ 1			☒ A_BusinessPartnerAddress	ZA_BUSINESSPARTNERADDRESS	☐			☒ A_BusinessPartnerBank	ZA_BUSINESSPARTNERBA10EF53F5...	☐			☒ A_BusinessPartnerRole	ZA_BUSINESSPARTNERRO84E9222...	☐			☒ A_BusinessPartnerTaxNumber	ZA_BUSINESSPARTNERTAB0E4B215...	☐			☒ A_CustSalesPartnerFunction	ZA_CUSTSALESPARTNERF159B283...	☐			☒ A_Customer	ZA_CUSTOMERD6C7ECF4B8	☐			☒ A_CustomerCompany	ZA_CUSTOMERCOMPANY3D79525...	☐			☒ A_CustomerDunning	ZA_CUSTOMERDUNNING55ECBD...	☐			☒ A_CustomerSalesArea	ZA_CUSTOMERSALESAREA96C421...	☐			☒ A_CustomerSalesAreaTax	ZA_CUSTOMERSALESAREA7A7938...	☐			☒ A_CustomerWithHoldingCompany	ZA_CUSTOMERWITHHOLDI5302D...	☐			☒ A_Supplier	ZA_SUPPLIER617002ED75	☐			☒ A_SupplierCompany	ZA_SUPPLIERCOMPANY1BBDA39...	☐			☒ A_SupplierDunning	ZA_SUPPLIERDUNNING8194E0C2D6	☐			☒ A_SupplierPartnerFunction	ZA_SUPPLIERPARTNERFUB236BA0...	☐			☒ A_SupplierPurchasingOrg	ZA_SUPPLIERPURCHASIN8CD8D1...	☐	3		☒ A_SupplierWithHoldingTax	ZA_SUPPLIERWITHHOLDIACA225...	☐			☐ A_BPContactToAddress	ZA_BPCONTACTTOADDRESS	☐	Type Edm.DateTime of element		☐ A_BPContactToFunctionAndData	ZA_BPCONTACTTOFUNCANDDEPT	☐	Type Edm.DateTime of element		☐ A_BusinessPartnerContact	ZA_BUSINESSPARTNERCONTACT	☐	Type Edm.DateTime of element		☐ A_AddressHomePageURL	ZA_ADDRESSHOMEPAGEURL	☐	Type Edm.DateTime of element
	Service Entity Set	ABAP Artifact Name	ETag Support	Issue																																																																																																																	
2	☒ A_BusinessPartner	ZA_BUSINESSPARTNER73911184C	☒ 1																																																																																																																		
	☒ A_BusinessPartnerAddress	ZA_BUSINESSPARTNERADDRESS	☐																																																																																																																		
	☒ A_BusinessPartnerBank	ZA_BUSINESSPARTNERBA10EF53F5...	☐																																																																																																																		
	☒ A_BusinessPartnerRole	ZA_BUSINESSPARTNERRO84E9222...	☐																																																																																																																		
	☒ A_BusinessPartnerTaxNumber	ZA_BUSINESSPARTNERTAB0E4B215...	☐																																																																																																																		
	☒ A_CustSalesPartnerFunction	ZA_CUSTSALESPARTNERF159B283...	☐																																																																																																																		
	☒ A_Customer	ZA_CUSTOMERD6C7ECF4B8	☐																																																																																																																		
	☒ A_CustomerCompany	ZA_CUSTOMERCOMPANY3D79525...	☐																																																																																																																		
	☒ A_CustomerDunning	ZA_CUSTOMERDUNNING55ECBD...	☐																																																																																																																		
	☒ A_CustomerSalesArea	ZA_CUSTOMERSALESAREA96C421...	☐																																																																																																																		
	☒ A_CustomerSalesAreaTax	ZA_CUSTOMERSALESAREA7A7938...	☐																																																																																																																		
	☒ A_CustomerWithHoldingCompany	ZA_CUSTOMERWITHHOLDI5302D...	☐																																																																																																																		
	☒ A_Supplier	ZA_SUPPLIER617002ED75	☐																																																																																																																		
	☒ A_SupplierCompany	ZA_SUPPLIERCOMPANY1BBDA39...	☐																																																																																																																		
	☒ A_SupplierDunning	ZA_SUPPLIERDUNNING8194E0C2D6	☐																																																																																																																		
	☒ A_SupplierPartnerFunction	ZA_SUPPLIERPARTNERFUB236BA0...	☐																																																																																																																		
	☒ A_SupplierPurchasingOrg	ZA_SUPPLIERPURCHASIN8CD8D1...	☐	3																																																																																																																	
	☒ A_SupplierWithHoldingTax	ZA_SUPPLIERWITHHOLDIACA225...	☐																																																																																																																		
	☐ A_BPContactToAddress	ZA_BPCONTACTTOADDRESS	☐	Type Edm.DateTime of element																																																																																																																	
	☐ A_BPContactToFunctionAndData	ZA_BPCONTACTTOFUNCANDDEPT	☐	Type Edm.DateTime of element																																																																																																																	
	☐ A_BusinessPartnerContact	ZA_BUSINESSPARTNERCONTACT	☐	Type Edm.DateTime of element																																																																																																																	
	☐ A_AddressHomePageURL	ZA_ADDRESSHOMEPAGEURL	☐	Type Edm.DateTime of element																																																																																																																	
1905	Understanding Concepts	As an application developer, you may not only be interested on how you can implement different scenarios and use cases for your business applications by following the stateless programming model of the ABAP RESTful programming model. It may also be important for you to understand the concepts behind it. The concepts section provides you with background information of ABAP RESTful programming model and helps you to understand main concepts from both, the design time, and runtime perspectives.																																																																																																																			
1905	Test Class Generation for Service Binding	You can now generate automated tests for an OData service that you've created using service binding. The test class provides guidance on how to access the OData service using ABAP Units and test code for performing CRUD operations on an entity set.																																																																																																																			

Available as of	Feature	Description
		
1905	Remote OData Access for Service Consumption Model	You can now view the code sample for performing the CRUD operations on an entity set to remote OData service. 
1905	Defining Names for OData Entity Sets and Entity Types	You can now define external names for OData entity sets and entity types that are then used in service metadata. The annotations <code>@OData.entitySet.name</code> and <code>@OData.entityType.name</code> can now be used in any CDS entity. CDS Entity <pre> @OData.entitySet.name: 'New_EntitySet_Name' @OData.entityType.name: 'New_EntityType_Name' define view /DMO/I_Customer_OData as select from /dmo/customer { key customer_id, first_name, last_name } </pre> OData Metadata <pre> <EntityType Name="New_EntityType_Name" sap:label="Test for Renaming" sap:content="..." <Key> <PropertyRef Name="customer_id"/> </Key> <Property Name="customer_id" Type="Edm.String" Nullable="false" MaxLength="..." </Property> <Property Name="first_name" Type="Edm.String" MaxLength="40" sap:label="First Name" </Property> <Property Name="last_name" Type="Edm.String" MaxLength="40" sap:label="Last Name" </Property> </EntityType> <EntityContainer Name="cds_xdmocustomer_odata_entities" m:IsDefaultEntityContainer="true" <EntitySet Name="New_EntitySet_Name" EntityType="cds_xdmocustomer_odata_entities/New_EntityType_Name" </EntitySet> </EntityContainer> </pre>
1902	Downloading the ABAP Flight Reference Scenario	You can now download the relevant development objects that are used in the course of the guides via GitHub. This enables you to import a complete OData service into your system use and reuse for learning purposes. Limitation: The import of a service binding artifact is not possible. To complete the OData service, you need to create service binding in your own package.
1902	Entity Manipulation Language (EML)	Entity Manipulation Language (in short: EML) is a part of the ABAP language that is used to define the business object's behavior in the context of ABAP RESTful programming model. It provides read and modifying access to data in transactional development scenarios.

Available as of	Feature	Description
1902	Extension of Transactional Development Guide	The development guide Developing Unmanaged Transactional Apps was extended with a hierarchy. Booking Supplements are now part of the business object. In addition, the travel business object can now be consumed using EML syntax.
1902	Service Binding UI	The service binding tools come with a new UI design: 1. The binding type does not have a value populated by default. 2. The Publish and Unpublish buttons have been renamed to Activate and Deactivate. 3. Entity Set and Association was displayed with the service URL as the root node. They are separate and it is displayed only if the local service endpoint has been activated. 4. The Preview button is now available on the interface for previewing the SAP Fiori UI. This is applicable for OData V2 UI service only. 5. The local service endpoint information now displays the service URL when it is in state. 
1902	Consuming Services	The service consumption model replaces the OData client proxy to generate service artifacts. It comes with a new wizard and an editor to work on the generated artifact. The consumption model artifact provides an overview of all abstract entities and generated business service definitions that belong to the imported service.
1902	Transactional Behavior for the Service Consumption Scenario	The guide on how to consume a remote service has been extended with transactional capabilities. The use case of maintaining discount data in a local database is exemplified in the scenario.

Available as of	Feature	Description
1902	Using Aggregate Data in SAP Fiori Apps	Aggregate functions, such as sum, maximum, minimum and average, as well as a countin now available in the ABAP environment to be implemented in CDS and displayed in your Annotations are used to mark the elements as measures, whose values can be aggregate
1902	Adding Static Feature Control in SAP Fiori Apps	In a typical transactional scenario, you have to specify which operations should be provid entity or you must specify which fields of an entity have specific access restrictions (reac mandatory fields).
1902	Freely Selectable Name for Handler Methods in Behavior Pools	The method name in handler classes is now freely selectable. What kind of method it is, is the FOR clause. The old syntax <code>METHODS modify FOR BEHAVIOR ...</code> becomes now: <code>METHODS FreeMe FOR MODIFY</code> The old syntax <code>METHODS read FOR BEHAVIOR ...</code> becomes now: <code>METHODS FreeMe READ</code> The old syntax <code>METHODS lock FOR BEHAVIOR ...</code> becomes now: <code>METHODS FreeMe LOCK</code> Note that the old syntax remains valid but is no longer recommended!
1811	Defining Service Versions	The service binding tools come with a new UI design and some additional functions for ve (business) services.
1811	Support for Compositions	The current version of the ABAP RESTful programming model supports compositions: A l consists of a tree of nodes where the nodes are linked by means of a special kind of assoc compositions. A composition is specialized association that defines a whole-part relation composite part only exists together with its parent entity (whole).
1811	Developing an A2X Service	It is possible to publish an OData service as an application-to-cross application (A2X) ser means the service is published without information relevant for a UI service (for example annotation to define a UI). An A2X service facilitates the exchange of business informatio application and any client, including from a different system or server.
1811	Developing a UI Service with Access to a Remote Service	With the help of the service consumption model it is now possible to consume a Web API build a new SAP Fiori application by consuming the remote service. This development gu definition of a custom entity and implementing an custom query.

SAP S/4HANA and SAP S/4HANA Cloud, private edition

Available as of	Application Server Version	Feature	Description
2025 FPS 00	8.16	Privileged Access for Cross-BO Associations	In cross-BO scenarios, when association-related ABAP EML requests (such as create-by-association) are performed on the association source and speci association target's implementation of the operation is accessed in privile

Available as of	Application Server Version	Feature	Description
			bypassing the association target's authorization checks. This applies only specified with a privileged context.
2025 FPS 00	8.16	Derivation Functions	Derivation functions derive context-specific values that can be used to res output of a derivation function can be further influenced by the use of para
2025 FPS 00	8.16	Background Processing Framework - Final Version	Background processing is very important for business applications. A lot o asynchronously to improve performance. In scenarios where changes of th communicated remotely, the asynchronous execution is even mandatory t LUW (due to implicit database commits). We want to simplify and improve the background processing by covering a "exactly once", "in order", testability and an integration into the ADT cross t In the next step of bgPF the AIF integration and its monitoring capabilities order" capabilities are supported.
2025 FPS 00	8.16	Fiori-Based Multi-Dimensional Reporting – Cloud-Compliant	You need to quickly build an analytical Fiori application, but simple list rep needs? Instead, you already built an analytical model with the new embed developer extensibility and now want to unleash its full potential? In that c for you: Without writing a single line of additional code, you can simply exp fledged analytical application with a modern Fiori-based UI and real multi- capabilities.
2025 FPS 00	8.16	Generation Wizard: Outbound SQL via CDS External Entities	The ADT generation wizard allows you to develop outbound SQL scenarios the required CDS external entities based on an imported CSN definition of
2025 FPS 00	8.16	Fiori-Based Multi-Report Applications	You need to quickly build an analytical Fiori application, that puts multiple context in a single UI? All you need to do is to provide the queries, bundle them in an InA Service the brand new "Review Booklet Application" wizard to generate the applica refine the configuration of your Review Booklet in an easy to use Fiori UI.
2025 FPS 00	8.16	CDS Hierarchy Generation Wizard	Defining hierarchies for the use in a CDS view can become a complex task, CDS hierarchy definition itself, but also for a directory, nodes, text views, a hierarchy generation wizard, their creation becomes easier, quicker and le derive potential hierarchy nodes already from the start (on the master dat the creation process.
2025 FPS 00	8.16	Collaborative Draft	You can now add collaborative capabilities to your business object. The col users to work concurrently on the same draft instance.
2025 FPS 00	8.16	Cross-BO Transactional Enablement	Cross-BO scenarios can now be transactionally enabled by defining a cross BDEFs.
2025 FPS 00	8.16	Improved CDS Buffering - Buffer Delegation	Optimize the ABAP memory consumption for scenarios which make use of Provide support for "delegated buffers" to allow buffering on CDS view ent on table-level at runtime.
2025 FPS 00	8.16	RAP Factory Actions	RAP factory actions now return the http status 201 instead of 200 in case response.

Available as of	Application Server Version	Feature	Description
2025 FPS 00	8.16	OData UI Service from Scratch	You can create all RAP-service related repository objects from scratch by using the from Scratch wizard.
2025 FPS 00	8.16	Null Value Handling	You can now find an overview of the differences in the behavior of null values in V2 and V4 with regard to sorting and filtering on the Fiori Elements UI as well as the OData Web API request.
2025 FPS 00	8.16	CDS Table Entities	A new type of CDS entity is available: the CDS table entity. A CDS table entity is created in ABAP CDS as an ABAP-managed database object. Table entities provide a new type of applications that allows you to define and manage SAP HANA database table objects. The initial shipment of table entities supports CDS associations, CDS views, and CDS with both CDS simple types and CDS enumerated types.
2025 FPS 00	8.16	Nonstandard Operations for Associations	You can now define nonstandard operations for associations. These are like the inverse function.
2025 FPS 00	8.16	ABAP SQL Service - (read-only) Database Procedures	Have you already used the ABAP SQL Service / ODBC Driver for ABAP to expose data via SQL-APIs to external SQL consumers? We are now increasing the number of scenarios. Besides exposing CDS view entities for read-only scenarios, it will now expose "read-only" AMDP-based database procedures to be used by external ODBC clients. Both expose SQL and SQLScript-based business logic to external ODBC clients.
2025 FPS 00	8.16	CDS Updatable View Entities	With CDS Writeable View Entities, you can wrap existing DDIC Tables (or CDS Table Entities) with a native CDS signature that you can use with ABAP SQL (SELECT, INSERT, UPDATE, DELETE) in combination with DCL-based access control. CDS Writeable View Entities are based on the underlying DDIC Table / CDS Table Entity. CDS Writeable View Entities will allow the following scenarios: • Enabling of "Cloud-ready" READ/WRITE AMDPs, as they ensure "cloud-ready" SQLScript coding by preventing cross-client access • Exposure of a stable CDS signature for full SQL DML access but still maintaining the underlying persistence model • Enabling the usage of Dynamic View Caches based on released SAP HANA Cloud • Enabling of WRITE scenarios via the ABAP SQL Service / ABAP ODBC Driver
2025 FPS 00	8.16	Event-Driven Side Effects	You can now define an event to trigger a side effect in your RAP business object. When an event is raised by the application server, the side effect is triggered and the corresponding actions are executed.
2025 FPS 00	8.16	Communication Management: Local APIs	The communication management will provide local APIs for the creation and management of communication arrangements, communication systems, communication users, and the uplink API will be provided for the usage in Developer Extensibility.
2025 FPS 00	8.16	Navigation From SAP Fiori Launchpad Towards Event Broker UI	It is now possible to navigate from the SAP Fiori launchpad of SAP S/4HANA Cloud towards the UI of the connected SAP Cloud Application Event Hub. For more information, see the SAP Fiori launchpad documentation.
2025 FPS 00	8.16	Generic Audit Filter	You can now define generic audit filter(s) and assign auditor users accordingly. This allows you to define generic audit filter(s) and assign auditor users accordingly or time-based restrictions. The key benefits: • Simple: Using only few but central dimensions, audit filters can be easily to restrict data access. • Powerful: Also advanced scenarios with multiple dimensions and time-based (e.g. date-based) are supported.

Available as of	Application Server Version	Feature	Description
			• Efficient: Time restriction filter logic is directly applied at the data has been assigned. • Modern: Using the state of the art design concepts and technology • Modular: Implemented on application-level complementing Z1 rec external tax auditor.
2025 FPS 00	8.16	Editable Treeview with Draft Capabilities	In RAP, a treeview visualizes data modeled by a CDS hierarchy, using an SAP Fiori UI. Previously, you could model hierarchical structures and expose them as reusable CDS views. Now you can also expose hierarchical structures as editable treeview.
2025 FPS 00	8.16	Transport Layer Security (TLS 1.3)	We now support transport layer security 1.3 for outbound HTTP, WebSockets. You can define for each target host, whether you want to use transport layer security.
2025 FPS 00	8.16	CDS Reusable Measures & Filters	Managing and aligning analytical reporting might be quite challenging for customers. With the introduction of reusable measures and filters, you can define key figures, numbers, and many more features centrally, based on your needs. Those reusable measures and filters are utilized by CDS view entities by referencing those reusable artifacts. This enables a new definition of complex calculations and consolidates existing coherences of CDS views.
2025 FPS 00	8.16	Improved CDS Buffering - Buffer Propagation	Extend the reach of the CDS/ABAP buffering approach and optimize the runtime by buffer propagation to enable the ASQL runtime to read denormalized elements directly from the ABAP buffer instead of pushing the joins to the database.
2025 FPS 00	8.16	Creation of Local ABAP CDS Models Based on imported CDS Models	Do you have an existing CDS Model that was created via CAP Core Data Services or BAS or One Domain Model and you want to transfer it into your ABAP Cloud environment? You can now import external CDS models via the CSN-import and create a local CSN object model. This creates a local CDS data model based on the imported CSN model.
2025 FPS 00	8.16	Annotation: ObjectModel.supportedCapabilities	You can use this annotation to check whether or not the UI use case is supported. It indicates either the creation of UI projections or alternatively the direct consumption of data is possible. Otherwise, warnings or errors occur indicating that the UI usage is not supported, which lead to unexpected errors. For more information, see the annotation documentation of the ObjectModel.
2025 FPS 00	8.16	Annotation: Semantics.valueRange	You can use this annotation to inform consumers about the value range of a data point. You can specify minimal and/or maximal values and indicate whether these are exclusive or inclusive. This annotation can be applied to all fully ordered data types, i.e. Characters, Strings, and Numbers. For more information, see the annotation documentation of the Semantics.
2025 FPS 00	8.16	Annotation: UI.dataPoint.title	You can use this annotation to define the title of a data point. For more information, see the annotation documentation of the UI domain.
2025 FPS 00	8.16	Hello, ABAP Cloud! Page	Are you wondering where to find information about ABAP Cloud like documentation, roadmap or where to find examples with different levels of complexity? The new landing page in the ABAP Development Tools for Eclipse shows all of this information in one place. Everything about ABAP Cloud in one place.
2025 FPS 00	8.16	Result Cardinality for Factory Actions	You can now define the result cardinality [1..*] for factory actions.
2025 FPS 00	8.16	Entity Assignment of RAP Business Events	You can now assign RAP business events to entities that are not defined as business objects.

Available as of	Application Server Version	Feature	Description
2025 FPS 00	8.16	Subentities as Authorization Master	You can now define entities as authorization master that are not defined as
2025 FPS 00	8.16	Generator for Custom OData UI Services Based on RAP BO Interfaces	You can now generate custom OData UI services based on RAP BO interface ADT.
2025 FPS 00	8.16	Changed Runtime Behavior for Determinations and Validations	For performance reasons, determinations and validations that are assigned (including prepare) are skipped during this determine action, if these determinations have already been executed as part of a previous determine action, that has an active instance (draft or active) in the same transaction. Determinations and validations are skipped in the save sequence, if these determinations have already been executed as part of a previous determine action that has an active instance in the same transaction. The performance optimizations outlined above are not applied in the following scenarios: • A modification performed after the first determine action triggers a determination. • A validation in the first determine action reports a failed key. • A determination/validation has been assigned using the addition of a new determination. Ensure that the determinations and validations of your business object follow the guidelines described in Determination and Validation Modelling. Neglecting these guidelines can lead to unexpected behavior.
2025 FPS 00	8.16	CDS Simple Types	Simple types can be used for data typing on CDS level.
2025 FPS 00	8.16	Service Consumption Model	The Service Consumption Model now generates all data type information for remote services in one single ABAP proxy model class.
2025 FPS 00	8.16	RAP Default Values Function	You can now default fields and input parameters for operations in OData Services using Functions.
2025 FPS 00	8.16	RAP Treeview	You can now use Treeviews to define read-only hierarchical entity structure for RAP.
2025 FPS 00	8.16	Large Objects	You can now define static feature control for large object fields in OData V2 services.
2025 FPS 00	8.16	Ancestor Association Binding Fields	Ancestor associations binding fields now are filled automatically by the RAP framework.
2025 FPS 00	8.16	Cross-BO Transactional Enablement	Cross-BO scenarios can now be transactionally enabled and make use of a new implementation with additional syntax checks and error handling.
2025 FPS 00	8.16	Table Entities as Active Persistence	The RAP managed BO runtime now supports CDS table entities as the active persistence scenarios without draft.
2023 FPS 00	7.58	Migration Tool	You can now use the Migration Tool in ABAP Development Tools (ADT) to pre-validate the migration of CDS-based BOPF business objects to BOPF-based

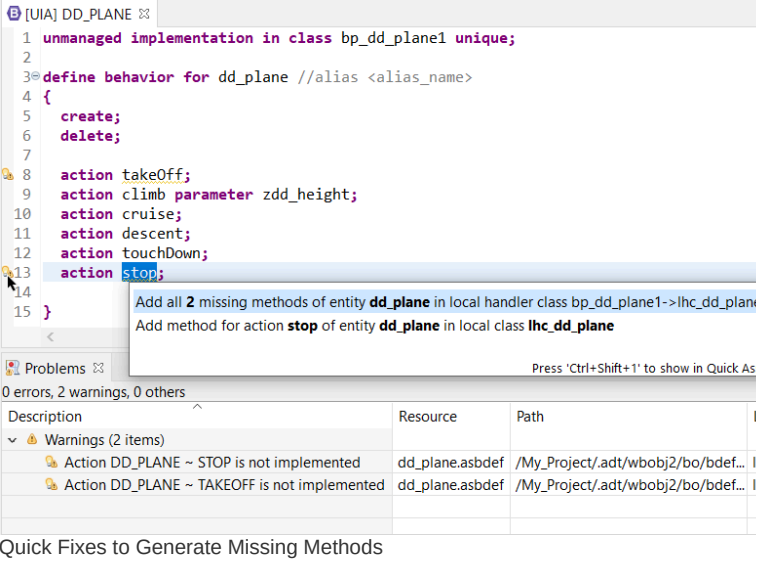
Available as of	Application Server Version	Feature	Description
2023 FPS 00	7.58	Local Consumption of Business Events	You can now consume business events in the same system as they are raised.
2023 FPS 00	7.58	Strict Mode	It is now best practice to use strict(2) for all RAP BOs, except for: • Business Configuration (BC) BOs • BOs that consume a BC API or one of the following: ◦ CL_EXCHANGE_RATES ◦ CL_NUMBERRANGE_INTERVALS ◦ CL_UOM_DIM_MAINTENANCE ◦ CL_UOM_MAINTENANCE ◦ XCO I18N APIs ◦ XCO Transport APIs
2023 FPS 00	7.58	SAP Object Types	You can now use SAP Object Types to semantically relate and group development objects.
2023 FPS 00	7.58	Verifying the Business Event Payload	You can now verify the payload of RAP Business Events using the Event Test Framework.
2023 FPS 00	7.58	Derived Events	You can now define a custom payload for already existing RAP Business Events.
2023 FPS 00	7.58	Background Processing Framework: Initial Version	Background processing is important for business applications. Much logic is executed asynchronously to improve performance. In scenarios where changes of data are communicated remotely, the asynchronous execution is even mandatory to avoid long-running user waits (because implicit database commits). We want to simplify and improve the background processing by covering a "exactly once", "in order", testability and an integration into the ADT cross layer. In the first version, only limited monitoring capabilities are supported (background processing framework).
2023 FPS 00	7.58	Extensibility Enablement of Service Definitions	You can now enable service definitions for extensibility by defining respect

Available as of	Application Server Version	Feature	Description
2023 FPS 00	7.58	Transactional Enablement of WHERE Clauses in CDS Projection Views	WHERE clauses in CDS Projection Views can now be used transactionally.
2023 FPS 00	7.58	Authoriation Control for Actions	You can now define authorization for actions that overrule the authorizatic authorization master. In BDEF extension, authorization control must be de action.
2023 FPS 00	7.58	Extension Layering	You can now define behavior definition extensions that extend the behavior behavior definition extensions.
2023 FPS 00	7.58	Defining a Leading Entity in RAP BOs	You can now define a leading entity in the service definition. In the service marked.
2023 FPS 00	7.58	Side Effects	You can now use side effects in your RAP behavior definition to trigger relo draft-enabled UI services.
2023 FPS 00	7.58	Integrating SAP Workflow Management	You can now integrate SAP Workflow Management in your RAP BO to use t process automation.
2023 FPS 00	7.58	Static and Dynamic Feature Control for Large Objects	You can now use dynamic and static feature control for large objects in OD
2023 FPS 00	7.58	Define Own Authorization Context and Priviledged Access	You can use the own authorization context to define the Identity and Acces own authorization context lists all authorization objects required during th runtime. In combination a RAP BO can offer priviledged access for consur certain authorization objects.
2023 FPS 00	7.58	Static Feature Controls for Fields in RAP BO Interfaces	You can define static feature control for fields in a RAP BO interface.
2023 FPS 00	7.58	abapGit Plugin	All RAP related development objects can be pushed and pulled with the ab
2023 FPS 00	7.58	Modeling Parameters for Non-Standard Operations	There is now documentation about how to model parameters and use thei
2022 FPS 00	7.57	Using Client-Independent Tables in Transactional Apps with Draft Capabilities	You can now use client-independent tables as data sources for managed tr capabilities.
2022 FPS 00	7.57	Multiple Message Targets in OData V4 RAP Services	OData V4 now supports multiple message targets. In the reported structur can define multiple elements in the derived type component %element. T by OData and are highlighted on a Fiori V4 UI.
2022 FPS 00	7.57	Using UI .hidden Annotation for Action Input Parameters	You can now use UI .hidden with action input parameters fields to preven displayed on the UI , while leaving the input parameter fields available for a

Available as of	Application Server Version	Feature	Description
2022 FPS 00	7.57	Read Access Logging (RAL)	You can now use Read Access Logging for your RAP services. With this feature, you can log all read accesses.
2022 FPS 00	7.57	Service Binding Errors in ATC Checks	ATC checks now also expose errors that are related to a RAP service. These errors can now be retrieved by running ATC checks for the service binding. They can now also be retrieved by running ATC checks for the service binding.
2022 FPS 00	7.57	Late Numbering for Managed Implementation Type	Late numbering can now also be used for the managed implementation type.
2022 FPS 00	7.57	Consumption of released Business Objects	You can now consume released Business Objects via C1 released RAP BOs.
2022 FPS 00	7.57	Instance Factory Actions	You can now use instance factory actions to model actions that create new instances. Instance factory actions can be useful if you want to copy specific data from one instance to another.
2022 FPS 00	7.57	Operations in Reported Response Parameter	When providing the reported response parameter in methods of the RAP service, you can now specify the operation the provided message belongs to. The derived type component can be used to specify the operation. The relevant one must be marked with <code>if_abap_behv=>mk-op</code> .
2022 FPS 00	7.57	Generate ABAP Repository Objects	With the Generate ABAP Repository Objects wizard, you can now generate end transactional application based on a database table.
2022 FPS 00	7.57	Semantic Versioning for OData V4 Service	In the service binding editor, you can now specify semantic versioning in the service by adding minor and patch version.
2022 FPS 00	7.57	Changes in the Consumption or Provisioning of RAP BOs	You can find an overview of changes in the RAP framework in SAP Note 310 .
2022 FPS 00	7.57	Filter Tree API for Unmanaged Queries	You can now use the method <code>get_as_tree</code> of the interface <code>if_rap_query</code> to retrieve the filter conditions in an easily parsable expression tree.

Available as of	Application Server Version	Feature	Description
2022 FPS 00	7.57	Business Object Interfaces	You can now use business object interfaces to introduce an additional abstract base BO and its business service projection layer. With RAP BO interfaces, you can define requirements for stable consumption from the behavior and data model of the object.
2022 FPS 00	7.57	Add Dimension to Query at Runtime (New)	With 2021 support package 1, some BI-tools like WebDynpro Grid offer the possibility to add dimensions from the cube view with the "free"-axis at runtime. With this, it isn't necessary to add dimensions into the CDS analytical query view that should be available at runtime. Common ones can be added to the query view. With this the UI view becomes slightly better. Since the cube may contain sensible dimensions that not all queries need, a query has to be enabled for this feature with the header annotation <code>@Analytics.Settings.runtimeExtensibility.allowed: true</code> . Dimensions that should not be exposed at runtime can be excluded with the annotation <code>@Analytics.Setting.excludedFromRuntimeExtensibility</code> or the <code>excludeFromRuntime</code> property in the cube view.
2022 FPS 00	7.57	Business Object Interfaces	You can now define business object interfaces to introduce an additional abstract base BO and its business service projection layer. With RAP BO interfaces, you can define requirements for stable consumption from the behavior and data model of the object. You can release RAP BO interfaces for C1 with visibility <code>Use in C1</code> to enable EML-consumption between different software components.
2022 FPS 00	7.57	Unmanaged/additional save with full data	It is now possible to use the option <code>WITH FULL DATA</code> for an unmanaged BO. With this option enabled, when an instance is modified, not only the changes but also the full data is provided into the <code>save_modified</code> method.
2022 FPS 00	7.57	Large Objects	You can enable your RAP application for maintaining large objects (LOBs). Users can use the option to incorporate external binary files or text files when editing data.
2022 FPS 00	7.57	Business Events	You can now define and raise business events in a RAP BO or in a RAP BO I. Business events provide the opportunity of light-weight, decoupled process integration. They are now a native part of the ABAP RESTful Application Programming Model (RAP). In the <code>RAP Business Event Bindings Editor</code> , you can create RAP Event Bindings. They provide a mapping between the definition of RAP Events via behavior definition and the runtime representation of Business Events.
2022 FPS 00	7.57	Extensibility-Enabling for RAP Business Objects	RAP extensibility is based on an opt-in approach to provide you with fine-grained control over which parts of your business object can be extended. You can easily enable your business object for extensions. Use case extensions can be created. Using the respective CDS annotations <code>extendable</code> and <code>extendable_behaviors</code> , you can decide if you want to enable your BO for behavior extensions or node extensions depending on your business requirements.
2022 FPS 00	7.57	Extending RAP Business Objects	RAP enables you to build upgrade-safe and lifecycle-stable extensions using the <code>extendable</code> and <code>extendable_behaviors</code> anchors of the original business object. You can extend the data model and the behavior of the BO and build data model extensions, behavior extensions or node extensions.

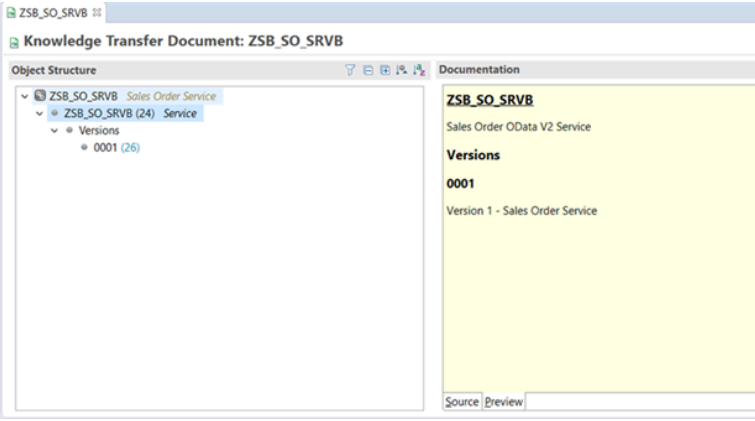
Available as of	Application Server Version	Feature	Description
2022 FPS 00	7.57	Repeatable Actions and Functions	You can now define actions and functions that can be executed multiply or same request.
2022 FPS 00	7.57	More Flexible Usage of Parameters	In Analytics with CDS, parameters are restricted to single values. More ger analytics. <code>@AnalyticsDetails.variable</code> can be used to create any v from a parameter.
2021 FPS 00	7.56	Using Service Binding Editor for InA Service	You can now expose CDS Analytical Queries via Information Access (InA) s The binding type must be chosen while creating the service binding for the Analytical Query as an InA service, the CDS view must be added to the ser definition must be used to create the service binding.
2021 FPS 00	7.56	Creating Service Consumption Model for RFC and Web Service	You can now create a service consumption model to generate proxies for n Web Service.
2021 FPS 00	7.56	Nested Determinations on modify	It is now possible to trigger a determination on modify by another determi
2021 FPS 00	7.56	Always Flag in Determine Actions	You can now use the flag <code>always</code> for determinations and validations that i action. When the determine action is called, determinations and validation regardless of their trigger conditions.
2021 FPS 00	7.56	Global Feature Control	You can now define global feature control for feature control that is indepe instance.
2021 FPS 00	7.56	Global Authorization Control	You can now define global authorization control to check authorization for particular for static operations.
2021 FPS 00	7.56	Augmenting Incoming Requests	You can now define augmentation in your projection behavior definition to with custom implementation, for example with default values.
2021 FPS 00	7.56	Quick Fixes For Creating Behavior Implementations	In case of missing implementation classes or missing methods, correspon displayed in the Behavior Definition Editor and in the Problems View You c incompatible changes for the ABAP RESTful Application Programming in S to generate the entire implementation class or missing methods.

Available as of	Application Server Version	Feature	Description
			 Quick Fixes to Generate Missing Methods
2021 FPS 00	7.56	Implementing Unmanaged Early Numbering in Managed Business Objects	You can now use unmanaged early numbering to automatically draw keys in the CREATE.
2021 FPS 00	7.56	Retrieving Permission Information from Business Objects	You can now use the EML statement <code>Get Permissions</code> to retrieve information and authorization permissions from business objects on global and instance levels.
2021 FPS 00	7.56	Implementing Cleanup in Managed Business Objects	You can now implement the cleanup method in managed BOs if an additional cleanup method is defined in the behavior definition. In this implementation you can handle local and global objects. The cleanup for managed BOs must be defined in the behavior definition or the redefined method in the local saver class.
2021 FPS 00	7.56	Adding Actions and Functions in a Projection Behavior Definition	You can now add actions and functions to a projection behavior definition. This allows you to define actions and functions for the new actions and functions.
2021 FPS 00	7.56	Using Strict Mode	You can now implement your RAP business object in strict mode to ensure consistency and implemented in accordance with the RAP best practices.
2021 FPS 00	7.56	RAP BO Contract	Rules on how to implement a RAP BO are now available in the RAP documentation.
2021 FPS 00	7.56	Developing a RAP BO that Supports Multi-Inline-Editing on a Fiori UI	You can now implement a RAP business object with a singleton mechanism for multi-inline editing on a Fiori UI.

Available as of	Application Server Version	Feature	Description
2021 FPS 00	7.56	Flag <code>always</code> for Determinations/Validations on Subentities	You can now use the <code>always</code> flag in determine actions also for determinat subentities.
2021 FPS 00	7.56	Flexible Association Modeling	It is now possible to define associations to CDS projection view entities in model relationships between consumption / projection layers. It is not supported to retrieve fields or other associations via such associat
2021 FPS 00	7.56	Using <code>AUTO_FILL_CID</code> Option	In EML statements for static operations, you can now use the option <code>AUTO</code> ensures that <code>%cid</code> is filled with a value automatically generated by the fra
2021 FPS 00	7.56	Harmonization of <code>Commit Work</code> and <code>Commit Entities</code>	<code>Commit Work</code> and <code>Commit Entities</code> have been harmonized. so that C RAP save sequence.
2021 FPS 00	7.56	Early Unmanaged Numbering in Unmanaged Business Objects with Draft	You can now use early unmanaged numbering in unmanaged business obje
2021 FPS 00	7.56	Field Control for Modify-Enabled Fields	Virtual or denormalized fields are <code>readonly</code> by default. You can modify-e behavior definition with the syntax <code>field (modify)</code> . Now, you can also such fields.
2021 FPS 00	7.56	Mapping for Action and Function Parameters	You can now map names of action and function parameters that are repres ABAP dictionary tables and structures.
2021 FPS 00	7.56	<code>COMMIT ENTITIES</code> in Simulation Mode	You can now use the <code>COMMIT ENTITIES</code> statement in simulation mode. 1 of RAP BOs without actually saving data.
2021 FPS 00	7.56	Suppress Fields in Transactional Handling	You can now remove fields that are only added to a RAP BO data model for appearance in all behavior-definition-related generated components, for e: EML fields. The annotation <code>@Consumption.hidden:true</code> is a prerequis
2021 FPS 00	7.56	Authorization and Feature Control Checks in Unmanaged Business Objects	The implementation for authorization and feature control in the correspon now also called during every modify operation in business objects with unr Objects
2021 FPS 00	7.56	Delegating Authorization Checks	You can now use the syntax <code>authorization: update</code> in the behavior c authorization check for delete, create-by-association, or action operation t the update operation.

Available as of	Application Server Version	Feature	Description
2021 FPS 00	7.56	Additional Implementation for Draft Actions	You can now add an additional implementation for all draft actions to external cycle. The additional implementation is defined in the behavior definition <code>additionalImplementation</code> and implemented in the behavior pool.
2021 FPS 00	7.56	Runtime Changes for Cleanup Execution	The Cleanup is now called after a <code>ROLLBACK/COMMIT ENTITIES</code> statement instance access including lock operation, precheck, instance authorization
2021 FPS 00	7.56	Idempotency for OData V4 services	Idempotent behavior is now supported for OData V4 services. With this feature, synchronous messages are delivered exactly once.
2021 FPS 00	7.56	RAP Reuse Data Elements	You can now use reuse data elements for the administrative fields in RAP endpoints
2021 FPS 00	7.56	Unit and Currency Types as Built-In Entity Sets	In OData V2 services, unit and currency types are automatically added to the entity sets. With this feature, unit and currency values are always displayed regarding decimals on a UI.
2021 FPS 00	7.56	Fail Causes for failed in RAP Handler Methods	The fail causes <code>disabled</code> , <code>readonly</code> and <code>dependency</code> can now be provided as parameter <code>failed</code> of RAP handler methods.
2021 FPS 00	7.56	Semantic Versioning for OData V2 Service	In the service binding editor, you can now specify semantic versioning in the service by adding minor and patch version.
2021 FPS 00	7.56	Using Service Binding Editor for SQL Service	You can now expose a RAP business service as an SQL service. The binding editor creates the service binding for the service.
2021 FPS 00	7.56	Save Actions for Managed BOs	You can now use actions that are executed only during the save sequence. Define the behavior definition with the following syntax: <code>[static] save action actionName [parameter parType] [...]</code>
2021 FPS 00	7.56	Changes in the Consumption or Provisioning of RAP BOs	You can find an overview of changes in the RAP framework in SAP Note 309

Available as of	Application Server Version	Feature	Description
2020 FPS 01	7.55	Using Service Binding Editor for OData V4 Service	You can now expose a RAP business service as an OData V4 service. The b creating the service binding for the service.
2020 FPS 00	7.55	Draft Support for RAP Business Services	The ABAP RESTful Application Programming Model now provides draft sup services. Draft-enabled business objects ensure that transactional data dc interrupted or continued from a different device. You draft-enable your business object by using the keyword <code>with draft</code>. Draft capabilities can be included for both business object implementation <code>unmanaged</code>.
2020 FPS 00	7.55	Draft Scenario on GitHub	You can now download a RAP managed business service with draft from G objects are now available as part of the ABAP Flight Reference Scenario th into your ABAP system.
2020 FPS 00	7.55	Controlling Active Data Access in Draft Business Objects	You can now define a designated field, the <code>total ETag</code> field, that control data in draft business objects. The RAP runtime framework checks its valu draft to active data to prevent concurrent BO consumers from overwriting information. The total ETag is defined in the behavior definition and is updated by the R automatically if annotated with the annotation <code>@Semantics.systemData true</code> in the corresponding CDS view.
2020 FPS 00	7.55	Working with Determinations and Validations	Determinations and validations are now processed by the determinations- You can now define determinations and validations with all three standard <code> update delete</code>}, and in combination with field triggers. In addition behavior pool uses different syntax: <pre> METHODS method_name FOR VALIDATE ON SAVE IMPORTING keys FOR AliasedEntityName~ValidationName METHODS method_name FOR DETERMINE ON MODIFY SAVE IMPORTING keys FOR AliasedEntityName~ValidationName </pre>
2020 FPS 00	7.55	Executing Determinations and Validations on Request	New determine actions allow the business object consumer to execute de without fulfilling the trigger conditions. You can now assign determinations: determine action that is called by a consumer's trigger, just like any other :
2020 FPS 00	7.55	Prechecking Modify Operations	You can now prevent illegal changes from reaching the application buffer b operations. The precheck method is called during runtime before the assign removes all input from the modifying request for which the condition in the
2020 FPS 00	7.55	Authorization Control	Consumer hints in the OData service metadata now include authorization the respective operation. That means, if you define authorization control fo authorization-relevant entity sets receive a path description instead of a st ability to be updated or deleted. In addition, the corresponding properties of the entity type.

Available as of	Application Server Version	Feature	Description
2020 FPS 00	7.55	RAP Runtime Checks	Stricter RAP runtime checks are introduced to help you make your RAP im friendly, and compatible to further RAP features. These runtime checks aff • creating and updating read only fields via EML, • executing modify calls in a late save phase, • consuming actions or functions with parameters exceeding the de
2020 FPS 00	7.55	Documenting Service Binding	You can now provide documentation using knowledge transfer documents 
2020 FPS 00	7.55	Feature Control	You can now use <code>mandatory:create</code> to assure that a field is filled in dur <code>readonly:update</code> to assure that the field value isn't changed during mc
2020 FPS 00	7.55	Possibly Incompatible Changes for ABAP RESTful Application Programming	You can find an overview of possibly incompatible changes for the ABAP R Programming in SAP Note 2943761 🌐.
2020 FPS 00	7.55	Implementing Own Locking Logic for Managed Business Objects	You can now define an unmanaged lock in a managed scenario. The unma be defined in the behavior definition with the syntax <code>lock master unma</code> the behavior pool in the method <code>FOR LOCK</code>, just like in an unmanaged sc invoked during runtime.
2020 FPS 00	7.55	Using dependent by _Association in Behavior Definition	You can now use the syntax <code>dependent by _Association</code> for lock, E dependent entities. The association to the respective master entity must b behavior definition and implemented when using the unmanaged scenario
2020 FPS 00	7.55	Using Client-Independent Tables in Managed Scenarios	The RAP managed BO runtime now supports client-independent database
2020 FPS 00	7.55	Dynamic Operation Control for Create By Association	You can now dynamically control the create by association operation. The : feature control is the following: <pre>_association {create (features: instance); }</pre> Like other feature controls, the control for the create by association must l behavior pool with the method <code>FOR FEATURES</code>.

Available as of	Application Server Version	Feature	Description
2020 FPS 00	7.55	New Read-Only Associations in CDS Projection Views	It is now possible to define new read-only associations in the projection view reused to display additional information in a UI service, for example analytical the transactional basic business object. With associations in projection view service-specific relationships. The syntax for the association definition is the same as in CDS views: <pre>association [min..max] to TargetEntity [as _Alias] on OData</pre>
2020 FPS 00	7.55	Filtering for Null Values in OData V2 Services	OData does not foresee initial values for the following EDM data types: • Boolean • GUID • Time • DateTime • DateTimeOffset. Fiori Elements UIs offer the empty operator in filter bars for these fields. As empty in the UIs, the actual value sent to the client is null. To be able to select records with empty values for these data types, the client filter is transformed to filter for initial values for the backend. Real null values in the database by filtering for them in OData V2.
2020 FPS 00	7.55	Understanding Runtime Processes	The concepts section in the ABAP RESTful Application Programming Model contains interactive diagrams so you can get a better understanding of the runtime
2020 FPS 00	7.55	Control Structure for Import Parameter in CREATE Operations	The control structure %control for import parameters of CREATE and CI operations is now filled according to the elements that were sent by the OData client in the EML call. In particular this means that the application developer knows for the create. ❖ Example In the following example the elements agencyid, customerid, begin are filled by the OData client and thus are marked in the control structure, while not.

Available as of	Application Server Version	Feature	Description
			<pre> METHOD create_travel. DATA lt_messages TYPE /dmo/t_message. DATA ls_travel_in TYPE /dmo/travel. DATA ls_travel_out TYPE /dmo/travel. LOOP AT it_travel_create ASSIGNING FIELD-SYMBOL(<fs_travel_create> ls_travel_in CALL F EXPO is IMPO es et IF lt_ INSE ELSE. /dmo EX CH ENDIF. ENDLOOP. ENDMETHOD. </pre> Provided Elements in Create for Travel
2020 FPS 00	7.55	Automatically Drawing Primary Key Values for Managed Business Objects	The managed runtime framework can now automatically generate key values if managed numbering is defined in the behavior definition. Syntax for defining managed numbering in the behavior definition: <pre> [implementation] managed [implementation in class ABAP_] define behavior for Entity [alias AliasedName] lock (master dependent()) ... { ... field ([read only,] numbering:managed) KeyField1, [I ... } </pre>
2020 FPS 00	7.55	New Options for Action and Function Results	The BO runtime framework now supports results for actions and functions can be entities of the same BO the action is defined for, entities of other BO differentiate between result entities and result structures the syntax elements introduced. In addition, you can now create actions, for which the action consumer can shall be returned completely or only parts of it, for example the keys only. This is marked with the keyword <code>selective</code> in the behavior definition. <pre> define behavior for CDSEntity ... </pre>

Available as of	Application Server Version	Feature	Description
			** Action with result entity action ActionName result [cardinality] entity OutputEntity ** Action with result structure action ActionName result [cardinality] OutputStructure ** Action with selective result action ActionName result selective [cardinality] entity ...
2020 FPS 00	7.55	Reporting Messages in ADJUST_NUMBERS and SAVE	The implicit returning parameter REPORTED is now available for the method save. By filling this parameter you can report information or success messages in the save sequence.
2020 FPS 00	7.55	Documenting Behavior Definitions	Now you can document behavior definitions in the Knowledge Transfer Document Editor. From the Project Explorer, select the behavior definition you want to document and create a knowledge transfer document.  Knowledge Transfer Document Editor
2020 FPS 00	7.55	Defining the Service Namespace for OData Services	You can now define the OData service namespace in service definitions with <code>@OData.schema.name</code> .
2020 FPS 00	7.55	Filling the Result Parameter for Actions	A runtime check is introduced to enforce the proper assignment of result values in action implementations when result parameters are declared in the corresponding actions returned the input entity if the result parameter was not filled in the past, they now return the value of the result parameter as specified exactly according to its implementation. To avoid that a Fiori UI shows initial values, if nothing is returned, you must fill the result parameter in the action implementations.
2020 FPS 00	7.55	Navigating More Than One Step in OData Requests	It is now possible to navigate to associated entities using more than one step in OData requests.  Example

Available as of	Application Server Version	Feature	Description
			To request the booking supplements for the booking entity with <code>BookinTravelID1</code>, use the following syntax. <pre><service_URL>/Travel('1')/to_Booking('5')/to_BookingSu</pre> You can link an unlimited number of entities in one OData request.
2020 FPS 00	7.55	Using Type and Control Mapping	You can now define a mapping contract for applications that include unmanaged objects based on CDS entities on the one hand, and legacy data types that implement the behavior or parts of it. The general syntax for field and control (information) mapping in a behavior definition is as follows: <pre>mapping for LegacyType control ControlType corresponding { EntityField1 = LegacyField1; EntityField2 = LegacyField2; ... }</pre>
2020 FPS 00	7.55	Integrating Additional Save in Managed Business Objects	You can now integrate Additional Save for each entity of a given business class implementation type. This is done in the behavior definition of the business class with <code>additional save</code>. The actual implementation of Additional Save is provided by the <code>AdditionalSave</code> class as part of the behavior pool.
2020 FPS 00	7.55	Integrating Unmanaged Save in Managed Business Objects	You can now integrate Unmanaged Save within the transactional life cycle of a business class. In order to integrate Unmanaged Save into the save sequence as a part of the behavior pool, you must first add the corresponding syntax (<code>with unmanaged save</code>) to the behavior definition and implement the saver handler method as a part of the behavior pool.
2020 FPS 00	7.55	Using Groups for Large Implementations	Until now, the implementation of business object entity's operations and Local Types included in the behavior pool associated with that entity. Since only one developer can change the behavior pool at a time, the efficiency of development in case of larger implementations is low. As a solution, the groups can be now used to split the implementation and other implementation-relevant parts of the business logic into several smaller groups for parallel implementation.
2020 FPS 00	7.55	Using Virtual Elements	You can now define elements in CDS projection views that are not persisted in the database. Virtual elements are defined with the new syntax element <code>virtual</code>. Their implementation is provided by a separate ABAP class, which is called during runtime via an ABAP code exit.
1909 FPS 00	7.54	First Shipment of ABAP RESTful Application Programming Model	Starting with Application Server ABAP 7.54 SP00, the ABAP RESTful Application Programming Model (RAP) is available to you as one of the major investment areas in the ABAP platform. RAP provides a set of extensions of the ABAP language, development tools and frameworks and enables the efficient end-to-end development of intrinsically SAP HANA-optimized OD apps in Application Server ABAP. The focus of this shipment is on the use of queries (ready-only apps) and transactions (transactional apps based on existing application logic).

Glossary and Feature Tables

7/6/26, 6:31 PM

You can look up all RAP-related terms in the RAP Glossary of the ABAP Keyword Documentation.

For a full list of RAP terms and their definitions, see [RAP Glossary \(ABAP Keyword Documentation\)](#).

You can look up all available RAP features and their release dates in the RAP Feature Tables of the ABAP Keyword Documentation:
[RAP Feature Table \(ABAP Keyword Documentation\)](#)